

MODEL · · DRIVEN · PRIVACY

Nguyễn Phước Bảo Hoàng

DISS. ETH NO. 32311

Model-driven Privacy

A thesis submitted to attain the degree of

DOCTOR OF SCIENCES
(Dr. sc. ETH Zurich)

presented by

HOANG NGUYEN

born on 14.06.1996

accepted on the recommendation of

Prof. Dr. David Basin, examiner
Assoc. Prof. Dr. Srđan Krstić, co-examiner
Assoc. Prof. Dr. Raúl Pardo, co-examiner

2026

Abstract

Ensuring that software systems, especially web applications, comply with security and privacy requirements is a complex and critical challenge. On the one hand, it is complex because national and supranational laws impose strict requirements on how organizations handle (personal) data, but offer limited guidance on concrete technical solutions. On the other hand, it is critical because failing to comply with these requirements can result in substantial financial penalties.

Model-driven security (MDS) is a software development methodology in which security requirements are specified as models, and enforcement mechanisms are generated automatically via model transformations. In this thesis, we extend MDS to support privacy. Our approach enables developers to specify three interconnected declarative models: a *data model* describing the state space of the system, a *security model* capturing fine-grained access control requirements, and a *privacy model* expressing data protection requirements, a particularly challenging class of privacy requirements. From these models, our methodology automatically generates a web application together with enforcement mechanisms that make it security- and privacy-aware by design.

We realize our methodology through two complementary tools that enforce these requirements at the application layer and the data layer, respectively. *vActionGUI* is a model-driven development tool that takes the three models as input and generates a complete web application with fully configured enforcement mechanisms at the application layer, including authentication, access control, consent collection, and purpose tracking. At the data layer, *SQLSPI* is a security interceptor that rewrites database queries into security-aware stored procedures that enforce access control and privacy requirements.

We evaluate our methodology along three dimensions. We demonstrate its practicality by applying our tools to develop non-trivial case studies across different application domains. We assess its efficiency by measuring the performance overhead of the generated enforcement mechanisms, which remains modest for typical workloads. Finally, we demonstrate its effectiveness through a longitudinal empirical study conducted over four editions of a graduate security engineering course, comparing model-driven and code-centric approaches for implementing security and privacy requirements. Our results show that the model-driven approach yields significantly fewer security and privacy violations, while also reducing developer effort and improving maintainability under evolving requirements.

Zusammenfassung

Die Entwicklung von Softwaresystemen, die Sicherheits- und Datenschutzanforderungen erfüllen müssen, ist oft komplex und kritisch. Dies trifft insbesondere auf Webanwendungen zu. Deren Entwicklung ist komplex, weil nationale und supranationale Gesetze strenge Anforderungen an den Umgang von Organisationen mit personenbezogenen Daten stellen, aber nur begrenzte Anleitungen zu konkreten technischen Lösungen bieten. Sie ist kritisch, weil die Nichteinhaltung dieser Anforderungen zu erheblichen finanziellen Strafen führen kann.

Model-driven security (MDS) ist eine Softwareentwicklungsmethodik, bei der Sicherheitsanforderungen als Modelle spezifiziert und Durchsetzungsmechanismen automatisch über Modelltransformationen generiert werden. In dieser Arbeit erweitern wir MDS, um Datenschutzanforderungen zu formulieren und durchzusetzen. Unser Ansatz ermöglicht es Entwicklern, drei miteinander verbundene deklarative Modelle zu spezifizieren: ein Datenmodell, das den Zustand des Systems beschreibt, ein Sicherheitsmodell, das detaillierte Anforderungen an die Zugriffskontrolle erfasst, und ein Datenschutzmodell, das eine besonders anspruchsvolle Klasse von Datenschutzanforderungen unterstützt. Aus diesen Modellen generiert unsere Methodik automatisch eine Webanwendung zusammen mit Durchsetzungsmechanismen, die Sicherheit und Datenschutz garantieren.

Auf der Grundlage dieser Methodik entwickeln wir zwei Werkzeuge, die diese Anforderungen jeweils auf der Anwendungs- und der Datenebene durchsetzen. *vActionGUI* ist ein modellgetriebenes Entwicklungstool, das die drei Modelle als Eingabe verwendet und eine komplette Webanwendung mit vollständig konfigurierten Durchsetzungsmechanismen generiert, einschliesslich Authentifizierung, Zugriffskontrolle, Einholung der Einwilligung und Zweckverfolgung. *SQLSPI* ist ein Sicherheits-Interceptor, der Datenbankabfragen zu sicheren Stored Procedures umschreibt, die Zugriffskontroll- und Datenschutzanforderungen durchsetzen.

Wir bewerten unsere Methodik über drei Dimensionen. Wir demonstrieren zunächst ihre Praxistauglichkeit, indem wir unsere Werkzeuge zur Entwicklung nicht trivialer Fallstudien in verschiedenen Anwendungsbereichen einsetzen. Wir bewerten ausserdem ihre Effizienz, indem wir den Leistungsaufwand der generierten Durchsetzungsmechanismen messen, der in den untersuchten Szenarien moderat bleibt. Schliesslich demonstrieren wir ihre Wirksamkeit durch eine longitudinale empirische Studie, die über vier Durchgänge eines Sicherheitstechnikurses durchgeführt wurde und modellgetriebene sowie codezentrierte Ansätze zur Umsetzung von Sicherheits- und Datenschutzanforderungen vergleicht. Unsere Ergebnisse zeigen, dass der modellgetriebene Ansatz deutlich weniger Sicherheits- und Datenschutzverstösse verursacht, den Entwicklungsaufwand reduziert und bei sich ändernden Anforderungen die Wartbarkeit verbessert.

Tóm tắt

Hệ thống phần mềm phải đảm bảo tuân thủ các quy định về an toàn thông tin và quyền riêng tư. Đó là một yêu cầu quan trọng nhưng không hề đơn giản. Các quy định pháp luật và tiêu chuẩn hiện hành yêu cầu hệ thống phải có những biện pháp phù hợp để bảo vệ quyền riêng tư, nhưng lại cung cấp rất ít hướng dẫn về cách thức thực hiện chúng. Do đó, việc tuân thủ các quy định này thường được lập trình trực tiếp trên mã nguồn, dẫn đến việc dễ xảy ra lỗi, khó bảo trì và mở rộng, cũng như không đảm bảo được tính chính xác của các quy định và tiêu chuẩn trên.

Phương pháp phát triển hệ thống theo hướng mô hình về an toàn thông tin (Model-driven Security – MDS) là một phương pháp kỹ thuật tập trung vào việc định nghĩa các mô hình an toàn thông tin và dựa vào các phép biến đổi mô hình để tự động sinh ra một phần mã nguồn của hệ thống. Luận án này mở rộng phương pháp MDS theo hướng mô hình tập trung vào những quy định và tiêu chuẩn mới về quyền riêng tư. Phương pháp này không chỉ bao gồm yêu cầu kiểm soát truy cập như trên mà còn tích hợp các yêu cầu về quyền riêng tư, chẳng hạn như yêu cầu hệ thống sử dụng dữ liệu đúng mục đích và có sự đồng ý của người dùng theo quy định pháp luật.

Chúng tôi hiện thực hóa phương pháp luận của mình thông qua hai công cụ hỗ trợ lẫn nhau. Đầu tiên, *vActionGUI* là một công cụ phát triển hướng mô hình nhận ba mô hình làm đầu vào và tự động sinh ra một ứng dụng hoàn chỉnh với các cơ chế an toàn thông tin đã được cấu hình đầy đủ, bao gồm cơ chế xác thực, kiểm soát truy cập, thu thập sự đồng ý và theo dõi mục đích sử dụng dữ liệu. Thứ hai, chúng tôi mở rộng ứng dụng SQLSPI với khả năng hỗ trợ các lớp liên kết, thực thi cơ chế giới hạn mục đích và tự động theo dõi sự đồng ý ở tầng cơ sở dữ liệu, qua đó bảo đảm rằng các yêu cầu về quyền riêng tư được thực thi một cách minh bạch đối với mọi dữ liệu truy vấn.

Chúng tôi kiểm chứng phương pháp này trên ba khía cạnh. Thứ nhất, chúng tôi chứng minh tính thực tiễn bằng cách phát triển các phép biến đổi cho hai ngôn ngữ lập trình phổ biến và sử dụng chúng để phát triển các hệ thống thuộc nhiều lĩnh vực ứng dụng khác nhau. Thứ hai, chúng tôi đánh giá hiệu năng của các cơ chế thực thi được sinh ra, cho thấy chúng tạo ra độ trễ không đáng kể khi vận hành. Thứ ba, chúng tôi thí nghiệm tính hiệu quả của phương pháp thông qua nghiên cứu thực nghiệm so sánh phương pháp hướng mô hình của chúng tôi với phương pháp lập trình truyền thống trong việc triển khai các yêu cầu về an toàn thông tin và quyền riêng tư. Kết quả cho thấy phương pháp của chúng tôi giúp giảm đáng kể công sức phát triển hệ thống, cải thiện khả năng bảo trì và phát triển, đồng thời giảm thiểu tỷ lệ vi phạm về an toàn thông tin và quyền riêng tư của hệ thống được lập trình ra.

Acknowledgements

First of all, I would like to thank my supervisor, *David Basin*, for giving me the opportunity to do research at the place I had long dreamed of. I am especially grateful for the trust you gave me to explore the topics I am most excited about and for the many thoughtful discussions we had along the way.

I would like to thank my co-supervisor, *Srdan Krstić*, for his patience and kindness, and for always taking the time to carefully discuss ideas and tackle challenges with me. Rushing to meet deadlines is not always ideal, but I have to admit that going into *hero mode* with you was often a lot of fun!

To Assoc. Prof. Dr. *Raúl Pardo*, thank you for being on my examination committee and for taking the time to carefully review this thesis.

To my former supervisor, *Manuel Clavel*, thank you for opening the door that has led me to this journey and for your guidance and friendship, both in academic work and in life.

To my InfSec and SysSec colleagues, thank you all for making ETH Zurich such a great place to work. To the GDPR task force, *François Hublet*, *Daniel Galán*, *Ahmed Bouhoula*, and *Karel Kubíček*, thank you for the many pleasant discussions and for helping to keep personal data safe!

To the students at ETH whom I had the privilege of supervising and teaching, thank you for your curiosity, dedication, and the many interesting discussions we shared. Mentoring you was probably one of the most interesting parts of my time here, and I learned a lot from working with you.

To bé *Thi*, chị *Yến*, *Khánh*, anh *Quang* and *Linh*, *Nhật* and *Khanh*, *Minh*, *Graciana Aad*, *Desmond Liu*, and my goodminton teammates, thank you all for giving me a reason to stay away from the office on weekends and holidays.

To *Dhruv Nevatia* and *Jesus Solano*, thank you for being such an important part of my life during these years: gym buddies, badminton, tennis, and ping-pong partners, labmates, flatmates, and above all great friends. I treasure every moment we shared.

To ba Bảo, mẹ Hương, ba Tuấn, mẹ Trang, hai ngoại, Minh và Nhân, cảm ơn mọi người vì luôn hết lòng ủng hộ con. Những phần đồ ăn mọi người chuẩn bị và đóng gói mang theo luôn làm con ấm bụng, kể cả trong những đêm Zürich lạnh nhất.

Thảo, cảm ơn em đã đến.

_____ ◇ _____

The following acknowledgements are dedicated to the collaborators and contributors who directly supported the tools and studies presented in this thesis.

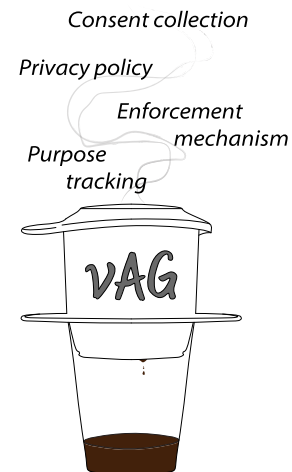
Srđan Krstić implemented the initial version of the model transformation for generating security- and privacy-aware Flask web applications, as well as the Python version of the OCL parser and evaluator, which are now used internally in the model-driven security and privacy tool. *Andrei Cotor* later extended the tool’s underlying security and privacy modeling languages.

Manuel Clavel helped extend the characterization of enforcing fine-grained access control policies on SQL queries. *Karlo Piskor* developed the first version of the SQL database interceptor to enforce these security policies in C#. *Sofie Daniëls* later extended this tool to support the characterization and enforcement of privacy requirements.

Felix Linker contributed to the design of the empirical case study presented in this thesis. *Juan Guarnizo*, *Martín Ochoa*, and *Daniel Galán* contributed to the study in subsequent iterations. *Krzysztof Cybulski* helped improve statistical reporting. The Security Engineering course staff at ETH Zurich over the years also contributed to the training material.

I would also like to thank the students enrolled in the Security Engineering Master’s course during the academic years 2022–2026 for their active participation in the course and the empirical study. I also thank the anonymous reviewers of the papers on which this thesis is based for their constructive feedback.

This research was supported by the Swiss National Science Foundation grant “Model-driven Security & Privacy” (Grant No. 204796) and the “Vingroup Science and Technology Scholarship Program for Overseas Study for Master’s and Doctoral Degrees”.



Contents

I	Prologue	1
1	Introduction	3
1.1	Motivation	3
1.2	Scope and Problem Statement	4
1.3	Solution Overview	6
1.4	Publications and Dissemination	7
1.5	Thesis Organization	8
2	Background	11
2.1	Model-driven Engineering	11
2.2	Modeling Foundations	12
2.3	ActionGUI: Security-aware GUIs for Web Applications	26
2.4	SQLSI: SQL Security Interceptor	32
II	Model-driven Security and Privacy	43
3	Related Work	45
4	Modeling Security and Privacy Requirements	51
4.1	Extending Data with Application Endpoints	51
4.2	Privacy Model	52
4.3	Security Permissions for Personal Data	58
5	ActionGUI: Privacy-aware Flask Application Generator	61
5.1	Modeling Languages	63
5.2	Generated Artifacts	70
5.3	Evaluation	80
6	Security and Privacy-aware Object-Relational Mappers	87
6.1	Extending the Model-based Characterization	87
6.2	From SQLSI to SQLSPI	93
6.3	Evaluation	103
III	Empirical Evaluation	113
7	Literature Review	115
8	Study Design	121
8.1	Code-Centric Approach	122

8.2	Study Organization	137
8.3	Research Questions	139
8.4	Evaluation Methodology	141
8.5	Ethical Considerations	144
9	Results and Analysis	147
9.1	Participation and Attrition	147
9.2	Collected and Derived Data	149
9.3	Answers to Research Questions	152
9.4	General Remarks	187
9.5	Threats to Validity	188
IV	Epilogue	193
10	Conclusions	195
	Appendix	197
A	Model-based characterization of enforcing FGAC policies on SQL queries	199
B	Model-driven Privacy: Formal Semantics	205
C	ASP.NET C# Model Transformation	211
D	vActionGUI Tool Usage	221
D.1	Usage Setup	222
D.2	Developer Setup	225
E	Case Study Requirements	227
E.1	Conference Management System	227
E.2	Social Networking Application	230
E.3	Event Management Platform	232
F	Empirical Study Artifacts	239
F.1	Variations Across Study Editions	239
F.2	Test Generation	244
F.3	Survey Questionnaire	262
F.4	Codebook and Qualitative Analysis	265
F.5	Reference Solution	268
	Bibliography	275
	Curriculum Vitae	289

List of Figures

Chapter 2 — Background

2.1	UML class diagram of a simplified EventPlatform	11
2.2	UML object diagrams of a simplified EventPlatform	15
2.3	UML class diagram of a simplified EventPlatform with data invariants .	16
2.4	UML class diagram of a simplified EventPlatform with security roles . .	21
2.5	Data action hierarchy semi-lattice	23
2.6	Overview of the ActionGUI development methodology	26
2.7	Overview of the SQLSI development methodology	32

Chapter 4 — Modeling Security and Privacy Requirements

4.1	Overview of model definitions and extensions	51
4.2	UML class diagram of EventPlatform with security & privacy concepts .	57
4.3	Extended role hierarchy with the base role	60

Chapter 5 — `vActionGUI`: Privacy-aware Flask Application Generator

5.1	<code>vActionGUI</code> generated privacy notice	75
5.2	<code>vActionGUI</code> generated privacy dashboard	76
5.3	Purpose stack during <code>view_event</code> execution	77
5.4	Runtime performance of different MiniTweet implementations	84
5.5	Runtime performance of Hipaa implementations	86

Chapter 6 — Security and Privacy-aware Object-Relational Mappers

6.1	UML class diagram of EventPlatform with association classes	88
6.2	Runtime workflow of the SQLSPI ORM interceptor	103
6.3	SQLSPI generation time as the data model grows	106
6.4	SQLSPI generation time as the security model grows	106
6.5	SQLSPI generation time as the privacy model grows	107
6.6	SQLSPI execution time of Query #1	109
6.7	SQLSPI execution time of Query #2	110
6.8	SQLSPI execution time of Query #3	111

Chapter 9 — Results and Analysis

9.1	Average percentage of failed security tests (by edition, approach)	153
9.2	Paired failure rate differences for Ed'22 and Ed'25	156
9.3	Average number of complete mediation violations in Flask	159

9.4	Distribution of failed privacy tests (by approach, privacy scenario)	162
9.5	Average percentage of failed privacy tests (by edition, approach)	164
9.6	Number of over-declared purposes (by submission)	166
9.7	Distribution of LoC per passed test	170
9.8	Distribution of self-reported time per phase	171
9.9	Average percentage of failed tests in the Development phases (by group)	173
9.10	Average percentage of failed security tests (by security role)	177
9.11	Most violated privacy policies in the Development phases	179
9.12	Average security regression rate in the Evolution phase (by security role)	181
9.13	Most violated Development privacy policies in the Evolution phase . . .	183
9.14	Average percentage of failed Flask security tests (by approach)	184
9.15	Participant background knowledge across editions	187

Appendix E — Case Study Requirements

E.1	UML class diagram of ConfMS	227
E.2	UML class diagram of MiniTweet	230
E.3	UML class diagram of EventPlatform in the Development phases	233
E.4	Purpose hierarchy of EventPlatform in the Development phases	236
E.5	UML class diagram of EventPlatform in the Evolution phase	237
E.6	Purpose hierarchy of EventPlatform in the Evolution phase	238

Appendix F — Empirical Study Artifacts

F.1	UML class diagram of EventPlatform in the Development phases (Ed'25)	242
F.2	Overview of the security and privacy test generator	244

List of Tables

Chapter 2 — Background

2.1	Semantics of OCL expressions	19
-----	--	----

Chapter 5 — ν ActionGUI: Privacy-aware Flask Application Generator

5.1	Model-driven privacy methodology via ν ActionGUI	61
5.2	Mapping from data actions to Python operations	73
5.3	Overview of the case study implementations	81
5.4	Model sizes and # LoC of the case study implementations	82

Chapter 6 — Security and Privacy-aware Object-Relational Mappers

6.1	SQLSPI generation experiment configurations	104
6.2	SQLSPI runtime experiment configurations	108

Chapter 7 — Literature Review

7.1	Literature review on empirical studies	119
-----	--	-----

Chapter 8 — Study Design

8.1	Empirical study requirements and approaches	121
8.2	Empirical study timeline and structure	137
8.3	Empirical study evaluation artifacts and related research questions	141
8.4	Empirical study security and privacy unit tests	142

Chapter 9 — Results and Analysis

9.1	Empirical study participation and attrition summary	148
9.2	Notation used to identify sets of student submissions	149
9.3	Notation used to identify sets of survey responses	150
9.4	Notation used to identify subsets of security tests	151
9.5	Notation used to identify subsets of privacy tests	151
9.6	Test sets and paired submission sets used to evaluate RQ1	154
9.7	WSR test results for RQ1	156
9.8	Test sets and paired submission sets used to evaluate RQ2	157
9.9	Library usage in code-centric (Flask) submissions	158
9.10	Test sets and paired submission sets used to evaluate RQ3	163
9.11	WSR test results for RQ3	165

9.12	Average number of failed privacy tests per submission	168
9.13	WSR test results for RQ5	169
9.14	Test sets and paired submission sets used to evaluate RQ8	172
9.15	RQ8 prior-knowledge effect per edition and requirements	174
9.16	RQ9 failed security tests by expected authorization verdict	178
9.17	Average percentage of passed Evolution tests (by edition)	186

Appendix B — Model-driven Privacy: Formal Semantics

B.1	Operational semantics of the system	210
-----	---	-----

Appendix C — ASP.NET C# Model Transformation

C.1	Model-driven privacy methodology targeting ASP.NET C#	211
-----	---	-----

Appendix F — Empirical Study Artifacts

F.1	Security tests summary in the Development phases	246
F.2	Security tests summary in the Evolution phase	254
F.3	Privacy tests summary in all phases	257
F.4	Survey response code frequency counts by edition	267

Part I

Prologue

Introduction

1.1 Motivation

Software Engineering is the science of engineering software systems. Over the years, new programming languages, frameworks, tools, and development methodologies have been introduced with the goal of improving the reliability, maintainability, and cost-effectiveness of software. A characteristic of well-engineered software that one cannot take for granted is *security*.

Ensuring security is critical because failures can have severe economic and social consequences. For example, due to an incorrectly implemented access control mechanism, the 2019 First American data breach exposed 885 million sensitive customer records, including bank account numbers, social security numbers, and driver's license information [139], potentially leading to identity theft, financial fraud, and other forms of misuse of personal data. With respect to web applications, the most recent report from the 2025 OWASP Top Ten Project [101], a widely recognized reference on web application security, identifies *broken access control* as the most prevalent vulnerability.

In addition to security, *privacy* has received increasing public attention in recent years, especially in light of data protection regulations that have become well established over the past decade. Privacy regulations, such as the European Union's General Data Protection Regulation (GDPR) [105], impose strict requirements on how organizations collect, process, and store personal data. Failing to comply with these regulations can result in substantial financial penalties. As of June 2026, the most common GDPR fines involve *insufficient legal basis for data processing* (972 fines; €3.12 billion) and *non-compliance with data processing principles* (788 fines; €1.81 billion), with both categories involving violations of user consent and purpose limitation requirements. In total, nearly 3,200 fines totaling €6.31 billion have been issued since the regulation came into effect in 2018 [73].

These statistics motivate the following overarching research question.

How to develop secure and privacy-compliant software systems?

A typical development approach is to implement security and privacy requirements directly in the application code, along with functional requirements. In the remainder of this thesis, we refer to this development paradigm as the *code-centric approach*. While

the code-centric approach is straightforward and arguably among the most widely used, it presents several challenges. First, building and maintaining complex systems in which the application code remains consistent with security and privacy requirements is prone to errors.¹ Second, since these requirements are implemented manually, it is difficult to verify that such systems correctly enforce the specified requirements.

For security, while existing frameworks *inherently* support specifying and enforcing typical role-based access control, more complex policies, such as fine-grained access control, must be implemented *manually*. More specifically, developers must implement these policies directly in the application code and ensure that all appropriate access checks are placed at every enforcement point. In the case of privacy, existing regulations and standards require appropriate technical measures to be taken to ensure compliance, but they offer limited guidance on concrete implementation solutions. This lack of framework support and technical guidance can lead to inconsistent interpretations and implementations of these requirements in software systems. For example, the GDPR mandates that *personal data be collected only for specified, explicit, and legitimate purposes* (Art. 5 §1 (b) GDPR) and that *data processing be grounded in a valid legal basis, of which user consent is one* (Art. 6 §1 (a) GDPR). However, in practice, user consent is often implicit. By using a website, users tacitly agree to the (often difficult to find and understand) text of the website's *privacy notice* [1].

1.2 Scope and Problem Statement

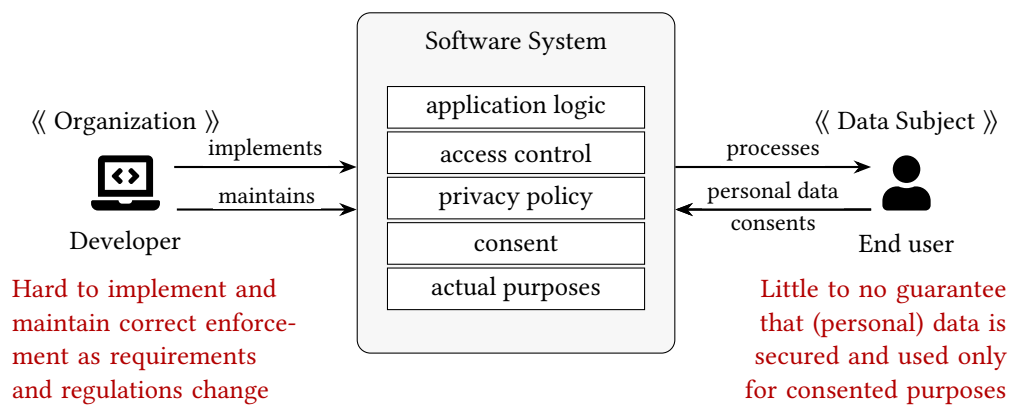
Security and privacy are broad terms that cover many different aspects. In the context of this thesis, we restrict the scope of security to *access control*, i.e., the specification and enforcement of requirements that determine which users may perform which actions on which data. For privacy, we focus on the specification and enforcement of *purpose limitation* and *user consent* requirements, as commonly mandated by data protection regulations. Throughout the thesis, we interpret privacy and the legal obligation of data protection as the enforcement of appropriate data-usage policies. In the remainder of this thesis, we therefore use *security requirements* to refer to access control requirements and *privacy requirements* to refer to purpose limitation and consent requirements.

The central problem is that security and privacy requirements are often specified informally and then implemented *ad hoc*. In other words, there is no methodology that provides an end-to-end mechanism connecting what an organization intends to enforce with what the deployed system actually enforces. This gap manifests itself differently for the two stakeholders involved.

¹Software evolution is generally a challenging task that carries significant risk. For example, the July 2024 CrowdStrike outage [31], caused by a faulty software security update, disrupted critical services worldwide, including airport and airline operations.

Organization’s perspective. For organizations that develop and operate software systems, correctly specifying and enforcing security and privacy requirements is difficult, and maintaining this correctness over time is even harder.

In a code-centric approach, developers implement security and privacy checks directly in the application code, which is inherently error-prone: a missing, misplaced, or incorrectly implemented authorization check can either expose sensitive data and permit unauthorized actions, or wrongly block legitimate users from performing allowed actions. For more complex security policies, such as fine-grained access control (FGAC) policies, the complexity of maintaining correct enforcement grows rapidly with the size of the application. Moreover, when requirements or regulations change, as they regularly do, these policies must be updated across the entire codebase.



End-user’s perspective. For end users whose personal data is collected and processed by these systems, there are no by-design guarantees that their data is protected as expected. Data protection regulations, such as the GDPR, grant users the right to consent selectively to specific processing purposes and to withdraw that consent at any time.

In practice, however, systems implement consent collection as an *all-or-nothing* mechanism, i.e., users either accept or deny the entire privacy policy. Even when consent is collected at a finer granularity, no technical mechanism prevents a developer from inadvertently querying personal data for an unauthorized purpose, and there is typically no assurance that the system actually respects the user’s choices during execution.

1.3 Solution Overview

Prior work. Basin et al. [18] have proposed model-driven security (MDS), an approach in which access control requirements are specified as a security model, and the enforcement mechanism is generated automatically via model transformations. Two existing tools realize this approach at different layers of a web application. First, ActionGUI [19] generates a functional web application where access control requirements are automatically enforced at the application layer. Second, SQLSI [93] intercepts the SQL queries issued by the application, rewrites them into security-aware stored procedures, and then executes them at the data layer. Both tools are limited to fine-grained access control requirements and do not support privacy requirements.

Our solution. In this thesis, we extend this prior work by improving how access control requirements are specified and enforced, and by supporting privacy requirements such as purpose limitation and user consent. In our approach, the software developer first creates a design model that captures the application’s data, expressed in a standard modeling language (e.g., a UML class diagram). The security developer then specifies an access control model. Finally, the privacy developer defines the privacy requirements of the system in a privacy model. From these three models, model transformations automatically produce an implementation that enforces both the security and privacy requirements by construction.

Threat model and assumptions. Our approach relies on minimal assumptions about the system. In particular, developers who *specify* an application’s security requirements (i.e., who define a security model) and privacy requirements (i.e., who define a privacy model) must know the structure of the application’s state (i.e., the data model). Having a data model of an application at the design phase is a realistic assumption. We trust these developers to correctly specify the application’s well-formed states and its security and privacy requirements. The problem of enforcing these requirements becomes extremely challenging if application developers are malicious, and it would require some form of trusted platform technology to ensure that a correct version of the enforcement mechanism code runs [108]. We therefore consider application developers to be trustworthy. Additional design validation techniques, such as model checking or manual model inspection, can help justify this assumption.

In contrast, developers who *implement* the application code, while not malicious, may unintentionally deviate from the design by making implementation errors. Our methodology and tools help such well-intentioned developers systematically and consistently enforce security and privacy policies. As these policies have a cross-cutting effect on

system execution, generating enforcement mechanism code and instrumenting relevant actions effectively connects privacy-relevant design and implementation artifacts, thereby enabling security- and privacy-by-design.

In general, our approach helps organizations mitigate the risk of privacy violations caused by accidental implementation errors. We do not aim to provide a general defense against common web vulnerabilities such as cross-site scripting, cross-site request forgery, or SQL injection.

Contributions. In summary, the main contributions of our thesis are as follows:

1. We revisit and extend the existing model-driven security approach and its tools to improve their expressiveness.
2. We propose a privacy modeling language that enables developers to specify privacy policies incorporating purpose limitation and user consent requirements.
3. We extend ActionGUI into ν ActionGUI, a model-driven development tool that takes data, security, and privacy models as input and automatically generates a web application with fully configured enforcement mechanisms.
4. We extend SQLSI into SQLSPI with support for association classes, purpose limitation enforcement, and dynamic consent collection at the database layer.
5. We evaluate our tools in multiple case studies with respect to their usability, generality, and performance.
6. We report on a longitudinal empirical study conducted with more than 300 student participants over four academic years that evaluates the effectiveness of different model-driven approaches to implementing security and privacy requirements compared to the current state of practice.

1.4 Publications and Dissemination

The research presented in this thesis is based on work that has been published or is currently under review at peer-reviewed international conferences and in journals. The following works are listed chronologically. Parts of their text, figures, and results are incorporated into this thesis with the permission of the coauthors and publishers.

[P1] Hoàng Nguyễn and Manuel Clavel
An Extended Model-Based Characterization of FGAC for SQL queries
In *Proceedings of the 26th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MoDELS 2023)*.

[P2] David Basin, Juan Guarnizo, Srđan Krstić, Hoàng Nguyễn, and Martín Ochoa
Is Modeling Access Control Worth It?
In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*.

[P3] Srđan Krstić, Hoàng Nguyễn, and David Basin
Model-driven Privacy
In *Proceedings on Privacy Enhancing Technologies (PoPETs)*, volume 2024, number 1.

[P4] David Basin, François Hublet, Srđan Krstić, and Hoàng Nguyễn
Mechanizing Privacy by Design
In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*.

[P5] Hoàng Nguyễn, Srđan Krstić, and David Basin
Models Trump Code: An Empirical Analysis of GDPR Compliance
In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS 2026)*.

1.5 Thesis Organization

Part I: Prologue

This part provides the technical foundations required for the remainder of the thesis. In particular, Chapter 2 introduces the model-driven development methodology and two approaches to enforcing security requirements, namely ActionGUI and SQLSI.

Part II: Model-driven Security and Privacy

Chapter 3 reviews the current state of the art in security and privacy specification, modeling, and enforcement approaches. Chapter 4 introduces our privacy modeling language to specify purpose limitation and user consent requirements.

Chapter 5 presents *vActionGUI*, our model-driven development tool that generates privacy-aware web applications from the specified models via a semantics-preserving model transformation. Chapter 6 describes our security improvements and privacy extensions for the SQLSI tool. We evaluate the tools presented in Chapters 5 and 6 on different case studies to demonstrate their generality, effectiveness, and modest overhead for typical workloads.

Part III: Empirical Evaluation

Chapter 7 reviews the literature on empirical studies of model-driven approaches and current practices for implementing security and privacy requirements. Chapter 8 gives an overview of our longitudinal study design and organization. Chapter 9 reports and analyzes the results of the study, including correctness, developer effort, and maintainability.

Part IV: Epilogue

Chapter 10 summarizes the contributions and findings of this thesis, discusses limitations, and outlines some potential directions for future work.

The appendices provide the model-based characterization of enforcing FGAC policies on SQL queries (Appendix A), the formal semantics of the model-driven privacy approach (Appendix B), the ASP.NET C# model transformation (Appendix C), the *vActionGUI* usage (Appendix D), the case study requirements (Appendix E), and the empirical study artifacts (Appendix F).

Background

2.1 Model-driven Engineering

Model-driven engineering (MDE) is a software development methodology that promotes models, rather than code, to first-class artifacts of the development process [51]. A *model* in MDE is a formal or semi-formal representation of certain aspects of a software system that abstracts away implementation details while preserving the properties relevant to the aspect being addressed. Models are expressed using a *modeling language* whose syntax and semantics are defined by a *metamodel*. A metamodel specifies which concepts may appear in models of that language, how those concepts relate to one another, and what constraints they must satisfy.

The Unified Modeling Language (UML) is an industry standard for software and system modeling [99]. It is a widely adopted general-purpose modeling language in the software development process. UML provides a family of diagrams for capturing the structural aspects of a system (e.g., class diagrams and object diagrams) as well as the behavioral aspects (e.g., sequence diagrams and state machines).

In this thesis, we introduce several data modeling languages and their corresponding models. Although these languages differ in certain aspects, they are considered dialects of the UML class diagram. For clarity, we therefore use UML class diagrams as a visual notation for representing these models.

We start by introducing a sample application and its functional, security, and privacy requirements. This application will serve as a running example to illustrate the concepts discussed throughout this thesis. We rely on common software engineering and modeling concepts, such as UML notation, without introducing them in detail.

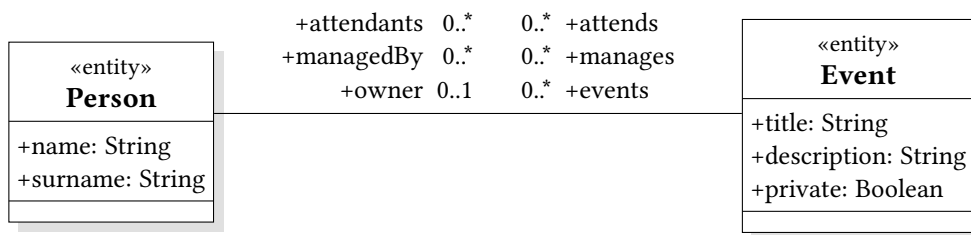


Figure 2.1: UML class diagram of the Event Management Platform.

Example 2.1. Typical event (management) platforms, such as Meetup or Facebook Events, allow users to create and manage collections of events. Consider the following simplified version of a platform where users may own, help manage, or attend events.

Figure 2.1 shows the UML class diagram of the described system. Users, modeled by the Person entity, have attributes name and surname. Events, modeled by the Event entity, have a title and a description and may be marked as private.

There are three associations between Person and Event: a person can create (own), manage, or attend events. Each person has the $0..*$ association ends events, manages, and attends. An event is owned by exactly one person but can be managed or attended by multiple users. This information is stored in the $0..1$ owner and $0..*$ managedBy and attendants association ends of Event.

The example above shows how models can document the structure of a system's data. In MDE, however, models are not merely used for documentation. Since they are formally defined objects, they can be used by tools to analyze system properties and generate other artifacts. The use of models to generate such artifacts is known as *model transformation*. A model transformation takes one or more models as input and produces system artifacts, such as source code, configuration files, or test scripts, as output. Model-to-model transformations refine or translate models across abstraction levels, while model-to-code transformations directly generate executable textual artifacts.

MDE is particularly useful in security-related domains, where security properties can be expressed formally in *models* at the design level, and model transformations can be used to automatically generate enforcement mechanisms. Model-driven security (MDS) [18] is a specialization of MDE that focuses on modeling security requirements. MDS integrates security into the software design process at a high level of abstraction, promoting security-by-design.

In this work, we focus on securing data through access control. We adopt an access control semantics in which a system action is permitted only if the executing user is authorized to perform that action on the relevant data resource. When the resource contains personal data, access additionally requires that the action be performed for a declared purpose and that the data subject has given the required consent. SecureUML [77] is one of the most prominent languages for defining such access control policies.

2.2 Modeling Foundations

In this section, we introduce the formal modeling and constraint languages that form the foundations of this thesis, namely, the data modeling language ComponentUML (Section 2.2.1), the Object Constraint Language (Section 2.2.2), and the security modeling

language SecureUML (Section 2.2.3). These models will be extended with security and privacy concepts in Chapter 4.

2.2.1 Data Model: ComponentUML

A *data model* is a formalization of a UML class diagram that defines the structure of the system states. We begin with the modeling language ComponentUML, which is used to define the data model. As mentioned earlier, for ease of presentation, we use the UML syntax to visualize the data models. Nevertheless, the ideas presented in this subsection on ComponentUML, as well as in the later extensions of this model in our approach, are independent of the specific UML syntax.

First, we introduce a straightforward set-theoretic representation of our models. Let ID be a countable set of names and $S \subset ID$ be a set of sorts. Sorts can be *primitive sorts* from $PS \subset S$, consisting of natural numbers (N), integers (Z), floats (F), strings ($String$) and Boolean values ($Bool$); *enumeration sorts* from $ES \subset S$; or *custom* (i.e., class sorts defined by a data model). Formally, an *enumeration* $s \in ES$ is a sort together with a finite set of *literals* $Lit(s) = \{l_1, \dots, l_k\} \subset ID$. The literals are the only values that an enumeration sort can take.

We use $\times$, $+$, $\rightarrow$, and $\rightharpoonup$ to denote the type constructors for the product, sum, function, and partial function, respectively. For a set A , we write A^* (resp. A^+) to denote a finite (resp. nonempty) sequence of A , and $()$ to denote the empty sequence. Starting from a set of sorts, we use these operations to build more complex *types*. For a function f , we denote its domain and range by $dom(f)$ and $rng(f)$, respectively. Finally, we write $a \mapsto b$ to denote a pair (a, b) that belongs to a function f , indicating that $f(a) = b$.

Definition 2.1 (Data Model).

A *data model* $\mathcal{D}$ is a tuple $(CS, \preceq_{\mathcal{D}}, attr, meth, as)$ of:

- a finite set of *class sorts* $CS \subset S$, with $\preceq_{\mathcal{D}}$ a partial order on CS representing class inheritance;
- $attr : CS \rightarrow (ID \rightharpoonup S)$, assigning to each class sort a partial function from attribute names to their sorts;
- $meth : CS \rightarrow (ID \rightharpoonup S^+ \rightarrow S)$, assigning to each class sort a partial function from method names to their types, where each method takes the owning class sort as its first argument; and
- $as : ID \rightharpoonup (ID \times Mult \times CS) \times (ID \times Mult \times CS)$, a partial function mapping each binary association name to a pair of association ends, where each end is a triple of end-name, multiplicity, and class sort.

Here, $Mult$ denotes the set of multiplicities, i.e., $Mult = N \cup \{*\}$.

For simplicity and without loss of generality, we consider only binary associations. We say that a relation is $as(n)$ -typed, where $as(n) = ((a, m_a, A), (b, m_b, B))$, if the relation has type $A \times B$.

Example 2.2. Consider the event platform described in Example 2.1 and its UML class diagram shown in Figure 2.1. The diagram corresponds to the following data model $(\{Person, Event\}, \emptyset, attr, meth, as)$, where

```

attr(Person) = {name ↦ String, surname ↦ String}
attr(Event) = {title ↦ String, description ↦ String, private ↦ Bool}
meth(Person) = ∅
meth(Event) = ∅
as(ownership) = ((owner, 1, Person), (events, *, Event))
as(management) = ((managedBy, *, Person), (manages, *, Event))
as(attendance) = ((attendants, *, Person), (attends, *, Event))

```

Object Model

An instance of a data model is called an *object model*, which interprets sorts with finite sets of objects and associations as relations over these objects. Each object provides an interpretation for all attributes by its sort, whereas the methods have a fixed interpretation shared by all objects of that same sort.

Two objects are (structurally) equal if they provide the same interpretation to all of their attributes. To distinguish structurally equal objects, each object is associated with a unique name $o \in ID$. Hence, an object of sort C is a pair $(o \mapsto (C, ats))$, where ats denotes the interpretation of the attributes of sort C . In other words, an object binds its unique name o to a concrete interpretation ats of attributes of the sort C .

Definition 2.2 (Object Model).

An *object model* O of a data model $\mathcal{D}$ is a triple $(\mathbb{O}, \mathbb{M}, \mathbb{R})$, where $\mathbb{O}$ is a finite set of objects, $\mathbb{M}$ maps each method name $n \in \text{dom}(meth)$ to a $meth(n)$ -typed function, and $\mathbb{R}$ maps each association name $n \in \text{dom}(as)$ to an $as(n)$ -typed relation over the objects in $\mathbb{O}$.

Since object names are unique, any finite collection of objects can be represented as a (partial) function from object names to their interpretations. Intuitively, an object model corresponds to a UML object diagram representing a concrete state of the system. For conciseness, we use the object names instead of the objects themselves in $\mathbb{R}$.

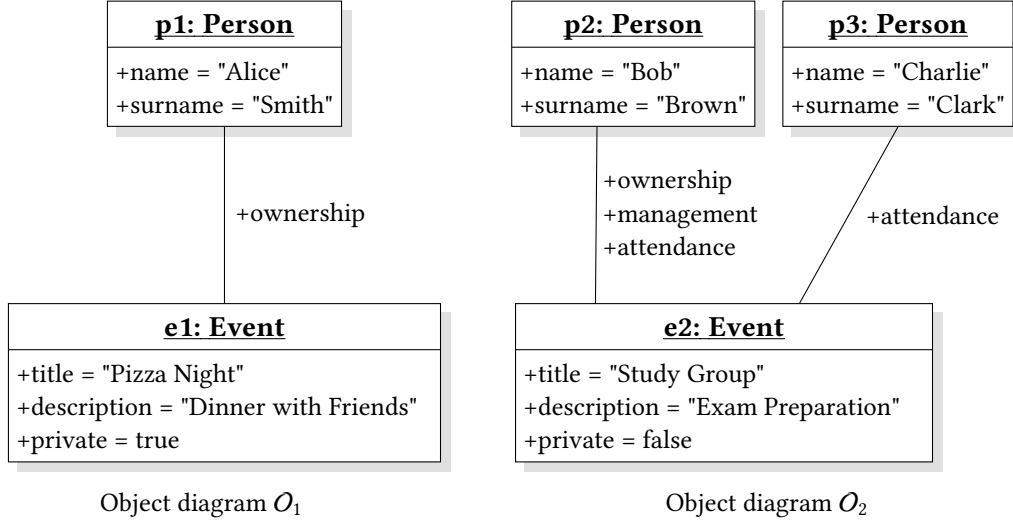


Figure 2.2: Simplified UML object diagrams of the Event Management Platform.

Example 2.3. Consider two concrete states of the event platform from Example 2.2. The first state corresponds to the left UML object diagram shown in Figure 2.2, which consists of a user, “Alice”, and an event “Pizza Night”. Let p_1 represent “Alice Smith” and let e_1 represent the event “Pizza Night”. The corresponding object model $O_1 = (\mathbb{O}_1, \mathbb{M}_1, \mathbb{R}_1)$ is

$$\begin{aligned}
 \mathbb{O}_1(\text{Person}) &= \{p_1 \mapsto (\text{Person}, (\text{“Alice”}, \text{“Smith”}))\} \\
 \mathbb{O}_1(\text{Event}) &= \{e_1 \mapsto (\text{Event}, (\text{“Pizza Night”}, \text{“Dinner with Friends”}, \top))\} \\
 \mathbb{R}_1(\text{ownership}) &= \{(p_1, e_1)\} \\
 \mathbb{R}_1(\text{management}) &= \emptyset \\
 \mathbb{R}_1(\text{attendance}) &= \emptyset
 \end{aligned}$$

where $\top$ denotes true and $\perp$ denotes false; here the event is private.

The second state corresponds to the right UML object diagram shown in Figure 2.2, which consists of two users and one event. Let p_2 represent “Bob Brown” and p_3 represent “Charlie Clark”, and let e_2 represent the public event “Study Group”. The corresponding object model $O_2 = (\mathbb{O}_2, \mathbb{M}_2, \mathbb{R}_2)$ is

$$\begin{aligned}
 \mathbb{O}_2(\text{Person}) &= \{p_2 \mapsto (\text{Person}, (\text{“Bob”}, \text{“Brown”})), \\
 &\quad p_3 \mapsto (\text{Person}, (\text{“Charlie”}, \text{“Clark”}))\} \\
 \mathbb{O}_2(\text{Event}) &= \{e_2 \mapsto (\text{Event}, (\text{“Study Group”}, \text{“Exam Preparation”}, \perp))\} \\
 \mathbb{R}_2(\text{ownership}) &= \{(p_2, e_2)\} \\
 \mathbb{R}_2(\text{management}) &= \{(p_2, e_2)\} \\
 \mathbb{R}_2(\text{attendance}) &= \{(p_2, e_2), (p_3, e_2)\}
 \end{aligned}$$

indicating that the event is public.

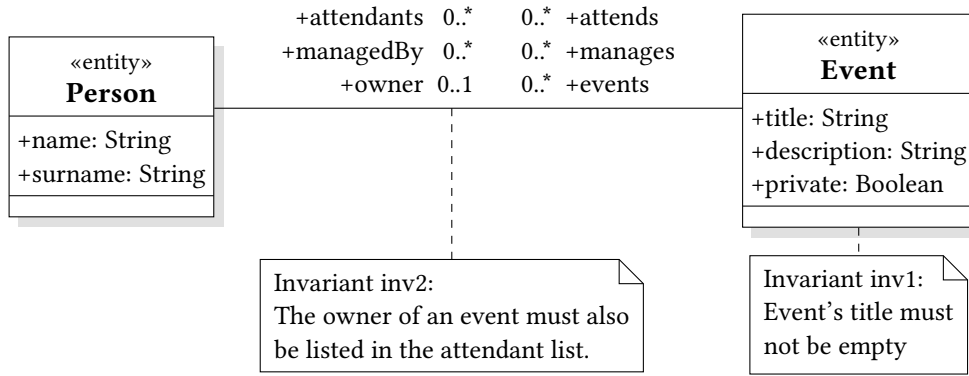


Figure 2.3: Simplified UML class diagram of the Event Platform with data invariants.

Data Invariants

An invariant defined over a data model specifies a property that must hold for every well-formed system state. Together with the data model, a set of data invariants defines the *well-formed* system states, i.e., the object models that represent well-formed configurations of the system.

Example 2.4. Consider the event platform from Example 2.2. The following data invariants can be defined over its data model:

- Every event must have a non-empty title.
- The owner of an event must also be listed among its attendants.

The object model O_1 shown in Example 2.3 violates the second invariant, as the owner of the event does not appear among its attendants. In contrast, the object model O_2 satisfies all invariants: e_2 has the non-empty title “Study Group” and p_2 (the owner of e_2) appears in $\mathbb{R}_2$ (attendance).

While some invariants can be expressed directly in UML (for example, multiplicity and uniqueness constraints on the association ends), many application-specific properties cannot. Such constraints are often documented informally in notes attached to the model, e.g., as shown in Figure 2.3. To formally specify these properties, additional constraint languages such as the Object Constraint Language (OCL) are used.

2.2.2 Object Constraint Language

The Object Constraint Language (OCL) [112] is a formal language used to specify constraints and queries over object models using a textual notation. As part of the UML Specification in Version 1.1, OCL serves as the standard for querying object models or describing properties (e.g., invariants) of data models. However, as of Version 2.0,

OCL has been adopted for broader use, including the definition of specific domain metamodels, model transformation, testing, and validation [99].

OCL is a strongly-typed language: each expression has either a primitive type, a class type, a tuple type, or a collection type. Collections can be sets, bags, ordered sets, and sequences, and can be parametrized by any type, including other collection types. The language provides standard operators on primitive types, tuples, and collections. Importantly, OCL expressions do not have side effects, i.e., the object model does not change as a consequence of evaluating OCL expressions.

An OCL expression can be written in the context of a data model. For objects, OCL provides a notational style similar to that of object-oriented languages: a dot-operator to access the value of an attribute of the object, or the collection of objects linked with another object at the end of an association. For example, suppose that the data model includes a class C with an attribute at and an association end . Then, if o is an object of the class C in the given data model instance, the expression $o.at$ refers to the value of the attribute at of the object o , and $o.end$ refers to the collection of objects related to the object o at the association end .

For collections, OCL provides an **allInstances** operator to collect all objects of a specific class and an arrow-operator “ $\rightarrow$ ” to access a property of the collection or to iterate over the collection and perform some actions. For example, suppose that the data model includes a class C . Then $C.allInstances()$ represents the collection of all objects in class C . Now, suppose that $source$ represents a collection. Then, $source \rightarrow size()$ returns the size of this collection, $source \rightarrow isEmpty()$ returns whether this collection is empty, $source \rightarrow forAll(v \mid body)$ iterates over this collection and checks whether all elements v in this collection satisfy the property stated in $body$, $source \rightarrow exists(v \mid body)$ iterates over this collection and checks whether there exists at least one element v in this collection that satisfies the property stated in $body$, and $source \rightarrow includes(o)$ iterates over this collection and checks whether object o is included.

We next provide an overview of the OCL subset used throughout this thesis.

Syntax. Let a family of sets of variables V_s indexed by a sort $s \in S$ be given. The following grammar θ_s^Σ defines OCL expressions that evaluate to an element of type s over the signature Σ (possibly extended with nullary function symbols representing constants). In the following, we fix the signature Σ and simply write θ_s .

$\theta_s ::=$	self	
	x_s	if $x_s \in V_s$
	$\omega(\theta_{s'_1}, \dots, \theta_{s'_n})$	if $\omega : s_1 \times \dots \times s_n \rightarrow s$ and $s'_i \preceq s_i$
	if θ_{Bool} then θ_t else θ_f fi	if $t \preceq s$ and $f \preceq s$
	$\theta_{s'} \text{ as } s$	if $s \preceq s'$
	$\theta_{Collection(s')} \rightarrow \text{iterate}(x_{s'}; x_a = \theta_a \mid \theta_a)$	if $x_{s'} \in V_{s'}$ and $x_a \in V_a$

Intuitively, the expression **self** refers to the object of sort s that forms the context in which the OCL expression is written. The variables $x_s \in V_s$ denote elements of sort s . The function application applies the function symbol ω to a tuple of OCL expressions $\varphi_{s_1}, \dots, \varphi_{s_n}$ with appropriate types.

The expression **if-then-else** is evaluated to either of the two alternative expressions of a subsort of s depending on the evaluation of the OCL constraint θ_{Bool} . The expression **as** casts an expression of sort s' to s , a subsort of s' . Finally, the expression **iterate** is a *fold* operation on a collection, where the variable $x_{s'}$ binds to each element of the collection, while the variable x_s serves as an accumulator that is initially set to the value of the first OCL expression φ_s and then iteratively updated for each element of the collection, based on the second OCL expression θ_s .

We denote the set of free variables of an OCL expression φ by $\text{fv}(\varphi)$, $\text{fv}(\varphi) \subseteq \bigcup_{s \in S} V_s$. Given $x_s \in \text{fv}(\varphi)$ and an OCL expression $t \in \theta_s$, $\varphi[x_s/t]$ is an OCL expression obtained from φ by substituting all instances of x_s with t in a capture-avoiding way.

Semantics. Let σ be the structure induced by an object model that interprets the signature Σ . A valuation $v : V_s \rightarrow \llbracket s \rrbracket$ is a family of variable assignments, each for variables of the appropriate sort. The semantics of an OCL expression $\varphi \in \theta_s$ is defined with respect to a valuation v and a context object c of the type $C \in S$ using the interpretation function

$$I_\sigma : \theta_s \rightarrow V \times C \rightarrow \llbracket s \rrbracket_\sigma$$

where V is a set of all valuations.

$$\begin{aligned} I_\sigma(\mathbf{self})(v, c) &= c \\ I_\sigma(x)(v, c) &= v(x) \\ I_\sigma(\omega(\varphi_1, \dots, \varphi_n))(v, c) &= \llbracket \omega \rrbracket_\sigma(I_\sigma(\varphi_1)(v, c), \dots, I_\sigma(\varphi_n)(v, c)) \\ I_\sigma(\mathbf{if} \varphi \mathbf{then} \varphi_1 \mathbf{else} \varphi_2 \mathbf{fi})(v, c) &= \begin{cases} I_\sigma(\varphi_1)(v, c) & \text{if } I_\sigma(\varphi)(v, c) = \top \\ I_\sigma(\varphi_2)(v, c) & \text{otherwise} \end{cases} \\ I_\sigma(\varphi \mathbf{as} s)(v, c) &= \begin{cases} I_\sigma(\varphi)(v, c) & \text{if } I_\sigma(\varphi)(v, c) \in \llbracket s \rrbracket_\sigma \\ \text{invalid} & \text{otherwise} \end{cases} \\ I_\sigma(\varphi \rightarrow \mathbf{iterate}(e; a = \varphi_a \mid \varphi_u))(v, c) &= I_\sigma(\text{fold}(I_\sigma(\varphi)(v, c), \varphi_u))(v[a/I_\sigma(\varphi_a)(v, c)], c) \end{aligned}$$

where, writing $\cdot$ for string concatenation and $|s|$ for the length of a string s ,

$$\begin{aligned} I_\sigma(\text{fold}(\emptyset, \varphi_u))(v, c) &= I_\sigma(a)(v, c) \\ I_\sigma(\text{fold}(f \cdot \varphi, \varphi_u))(v, c) &= I_\sigma(\text{fold}(\varphi, \varphi_u))(v[a/I_\sigma(\varphi_u)(v[e/f], c)], c) \end{aligned}$$

For completeness, Table 2.1 defines the semantics of the OCL expressions used in this thesis.

Table 2.1: Semantics of OCL expressions.

Expression	Description	Semantics
$\varphi_1 = \varphi_2$	Equality	$I_\sigma(\varphi_1 = \varphi_2)(v, c) = (I_\sigma(\varphi_1)(v, c) = I_\sigma(\varphi_2)(v, c))$
$\varphi_1 < > \varphi_2$	Inequality	$I_\sigma(\varphi_1 < > \varphi_2)(v, c) = \neg (I_\sigma(\varphi_1)(v, c) = I_\sigma(\varphi_2)(v, c))$
$\varphi_1 < \varphi_2$	Less than	$I_\sigma(\varphi_1 < \varphi_2)(v, c) = (I_\sigma(\varphi_1)(v, c) < I_\sigma(\varphi_2)(v, c))$
$\varphi_1 > \varphi_2$	Greater than	$I_\sigma(\varphi_1 > \varphi_2)(v, c) = (I_\sigma(\varphi_1)(v, c) > I_\sigma(\varphi_2)(v, c))$
not φ	Logical negation	$I_\sigma(\text{not } \varphi)(v, c) = \neg I_\sigma(\varphi)(v, c)$
φ_1 and φ_2	Logical conjunction	$I_\sigma(\varphi_1 \text{ and } \varphi_2)(v, c) = I_\sigma(\varphi_1)(v, c) \wedge I_\sigma(\varphi_2)(v, c)$
φ_1 or φ_2	Logical disjunction	$I_\sigma(\varphi_1 \text{ or } \varphi_2)(v, c) = I_\sigma(\varphi_1)(v, c) \vee I_\sigma(\varphi_2)(v, c)$
φ_1 implies φ_2	Logical implication	$I_\sigma(\varphi_1 \text{ implies } \varphi_2)(v, c) = \neg I_\sigma(\varphi_1)(v, c) \vee I_\sigma(\varphi_2)(v, c)$
$\varphi_1.\text{concat}(\varphi_2)$	Concatenates two strings	$I_\sigma(\varphi_1.\text{concat}(\varphi_2))(v, c) = I_\sigma(\varphi_1)(v, c) \cdot I_\sigma(\varphi_2)(v, c)$
$\varphi.\text{size}()$	Returns the length of a string	$I_\sigma(\varphi.\text{size}())(v, c) = I_\sigma(\varphi)(v, c) $
$C.\text{allInstances}()$	Returns all instances of class C	$I_\sigma(C.\text{allInstances}())(v, c) = \{o \mid (o \mapsto (C, _)) \in \mathbb{O}_\sigma\}$
$\varphi.\text{ocllsUndefined}()$	Checks whether a value is defined	$I_\sigma(\varphi.\text{ocllsUndefined}())(v, c) = \neg (I_\sigma(\varphi)(v, c) = \text{invalid})$
$\varphi \rightarrow \text{forAll}(x \mid \varphi')$	True if φ' holds for every element	$I_\sigma(\varphi \rightarrow \text{forAll}(x \mid \varphi'))(v, c) = I_\sigma(\varphi \rightarrow \text{iterate}(x; a = \top \mid a \text{ and } \varphi'))(v, c)$
$\varphi \rightarrow \text{exists}(x \mid \varphi')$	True if φ' holds for some element	$I_\sigma(\varphi \rightarrow \text{exists}(x \mid \varphi'))(v, c) = I_\sigma(\varphi \rightarrow \text{iterate}(x; a = \perp \mid a \text{ or } \varphi'))(v, c)$
$\varphi \rightarrow \text{includes}(x)$	True if x is in the collection	$I_\sigma(\varphi \rightarrow \text{includes}(x))(v, c) = I_\sigma(\varphi \rightarrow \text{exists}(x' \mid x = x'))(v, c)$
$\varphi \rightarrow \text{excludes}(x)$	True if x is not in the collection	$I_\sigma(\varphi \rightarrow \text{excludes}(x))(v, c) = \neg I_\sigma(\varphi \rightarrow \text{includes}(x))(v, c)$
$\varphi \rightarrow \text{select}(x \mid \varphi')$	Elements satisfying φ'	$I_\sigma(\varphi \rightarrow \text{select}(x \mid \varphi'))(v, c) = \{e \in I_\sigma(\varphi)(v, c) \mid I_\sigma(\varphi')(v[x/e], c)\}$
$\varphi \rightarrow \text{reject}(x \mid \varphi')$	Elements not satisfying φ'	$I_\sigma(\varphi \rightarrow \text{reject}(x \mid \varphi'))(v, c) = \{e \in I_\sigma(\varphi)(v, c) \mid \neg I_\sigma(\varphi')(v[x/e], c)\}$
$\varphi \rightarrow \text{collect}(x \mid \varphi')$	Evaluates φ' for each element	$I_\sigma(\varphi \rightarrow \text{collect}(x \mid \varphi'))(v, c) = \{I_\sigma(\varphi')(v[x/e], c) \mid e \in I_\sigma(\varphi)(v, c)\}$
$\varphi \rightarrow \text{size}()$	Returns the number of elements	$I_\sigma(\varphi \rightarrow \text{size}())(v, c) = I_\sigma(\varphi \rightarrow \text{iterate}(x; a = 0 \mid a + 1))(v, c)$
$\varphi \rightarrow \text{isEmpty}()$	True if the collection is empty	$I_\sigma(\varphi \rightarrow \text{isEmpty}())(v, c) = (I_\sigma(\varphi \rightarrow \text{size}())(v, c) = 0)$
$\varphi \rightarrow \text{notEmpty}()$	True if the collection is not empty	$I_\sigma(\varphi \rightarrow \text{notEmpty}())(v, c) = (I_\sigma(\varphi \rightarrow \text{size}())(v, c) < > 0)$

The free variables in an OCL expression are those not bound by any collection function, e.g., $\text{fv}(\text{self.events} \rightarrow \text{forAll}(b \mid a.\text{concat}(b.\text{title}).\text{size}() < c)) = \{a_{\text{String}}, c_Z\}$. Here, variable b_{Event} is bound by the **forAll** function. Note that we subscript variables with their sorts for clarity.

Example 2.5. Consider the first invariant from Example 2.4. In OCL, it can be written in the context of Event as

self.title.size() > 0

over the signature Σ extended with the constant 0 of type Z . It is desugared to the expression $>(\text{size}(\text{title}(\text{self})), 0)$ defined by the above grammar.

The second invariant from Example 2.4 can be specified by the following OCL expression:

Event.allInstances() → forAll(e | e.attendants → includes(e.owner))

Unlike the invariant written above, this expression is context-free, as it quantifies over the set of all events rather than being evaluated in the context of a particular event.

2.2.3 Security Model: SecureUML and ComponentUML

Within the scope of this thesis, we consider security in terms of access control. We distinguish between an access control policy, which defines user privileges, and an access control mechanism, which enforces that policy.

When access control policies are sufficiently simple, it is possible to formalize them declaratively and independently of the application's business logic. For example, role-based access control (RBAC) policies [119] are those where decisions depend only on the roles assigned to users. In this case, the policy can be expressed as two simple relations, one mapping users to roles and another mapping roles to permissions. It can be enforced using standard, off-the-shelf access control mechanisms [90, 86].

However, in practice, access control requirements are often more complex. In these cases, an RBAC policy will no longer suffice, and one needs a policy with finer granularity. Such an access control policy is considered a fine-grained access control (FGAC) policy. In addition to the permission assignment to roles and the role assignment to users in RBAC, these FGAC policies define access conditions (also known as authorization constraints) based on specific values of attributes and associations of users, resources, and the given application context.

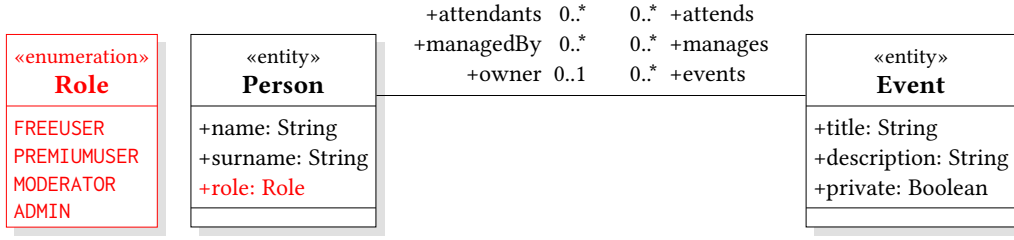


Figure 2.4: Simplified UML class diagram of the Event Management Platform, extended with security roles. Newly added elements ($\mathcal{R}$, and the role attribute) are highlighted in red.

SecureUML [77] is one of the most prominent modeling languages for specifying fine-grained access control policies. Using SecureUML, one can model access control decisions that depend on two types of information:

- *static information*, namely, the assignments of users and permissions to roles, and the role hierarchy; and
- *dynamic information*, namely, the satisfaction of authorization constraints in the given system state.

To integrate SecureUML with our setting, we extend the data model introduced earlier with the concepts necessary to represent roles and application users.

Definition 2.3 (Security extension of data model).

Given a data model $\mathcal{D} = (CS, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$, a security extension of $\mathcal{D}$ is a data model $\mathcal{D}_S = (\text{user}, CS', \preceq_{\mathcal{D}_S}, \text{attr}', \text{meth}, \text{as})$ defined as follows:

- $\text{user} : CS \rightarrow \text{Bool}$ is a predicate representing application user sort;
- $CS' = CS \cup \{\text{User}, \mathcal{R}\}$, where User and $\mathcal{R}$ are the user and role sorts;
- $\preceq_{\mathcal{D}_S} = \preceq_{\mathcal{D}} \cup \{(\text{User}, C) \mid C \in CS \text{ and } \text{user}(C) = \top\}$; and
- $\text{attr}' = \text{attr} \cup \{(\text{User}, \{\text{role} \mapsto \mathcal{R}\})\}$.

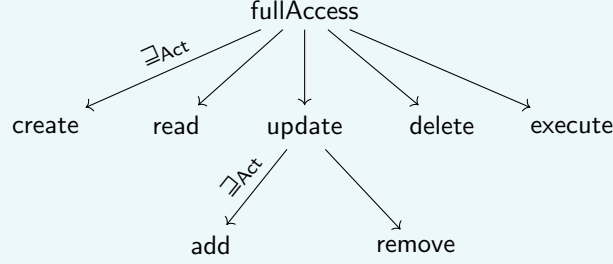
Intuitively, the predicate user specifies which sorts represent application users. Instances of these sorts inherit from the User sort, which defines a role attribute whose value is an element in $\mathcal{R}$.

Example 2.6. Consider the event platform introduced in Example 2.1 and the data model $\mathcal{D}$ in Example 2.2. We extend $\mathcal{D}$ to a new data model $\mathcal{D}_S$ with the entity User such that $\preceq_{\mathcal{D}_S}$ is the same as $\preceq_{\mathcal{D}}$ except for $\text{Person} \preceq_{\mathcal{D}_S} \text{User}$. Figure 2.4 displays the extended data model. We define four roles $\mathcal{R} = \{\text{FREEUSER}, \text{PREMIUMUSER}, \text{MODERATOR}, \text{ADMIN}\}$.

On the next page, we describe the actions that can be performed on the data model.

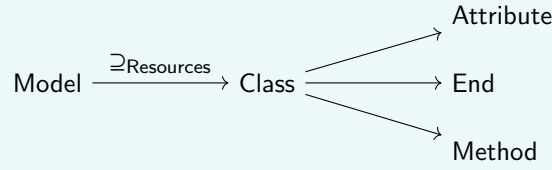
Data actions. Let $\text{Act} = \{\text{fullAccess}, \text{create}, \text{read}, \text{update}, \text{delete}, \text{add}, \text{remove}, \text{execute}\}$ be our action vocabulary. We first introduce the action hierarchy.

Definition 2.4 (Action hierarchy). We denote by $\supseteq_{\text{Act}}$ the partial order on Act that represents the action hierarchy. $(\supseteq_{\text{Act}}, \text{Act})$ forms the following *semi-lattice* (where $x \rightarrow y$ means $(x, y) \in \supseteq_{\text{Act}}$, or equivalently, $x \supseteq_{\text{Act}} y$).



Let $\text{Resources} = \{\text{Model}, \text{Class}, \text{Attribute}, \text{End}, \text{Method}\}$ be our resource vocabulary. Next, we define the subresource relation in our data model.

Definition 2.5 (Subresource relation). We denote by $\supseteq_{\text{Resources}}$ the subresource binary relation. $(\supseteq_{\text{Resources}}, \text{Resources})$ forms the following *semi-lattice*.



Given a data model $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$, we denote by $\text{Ends} = \{e \mid (e, _) \in \text{as} \vee (_, e) \in \text{as}\}$ the set of all association ends. For each $C \in \text{CS}$, $\text{Ends}(C) = \{e \mid e \in \text{Ends} \wedge e = (n, _, C)\}$. Then, a data action is then a pair of an action and a resource. Not all pairs are valid; for example, executing an attribute or reading a method are not considered valid data actions. In what follows, we define precisely the set of data actions.

Definition 2.6 (Data actions).

$\mathcal{A} \subseteq \text{Act} \times \text{Resources}$ is the set of data actions (a, r) where:

- $a \in \text{Act}$, if r is the data model resource itself ($r = \text{Model}$)
- $a \in \text{Act}$, if r is a class resource ($r = \text{Class}$)
- $a \in \{\text{read}, \text{update}\}$, if r is an attribute resource ($r = \text{Attribute}$)
- $a \in \{\text{read}, \text{update}\}$, if r is a to-one association end resource ($r = \text{End}$)
- $a \in \{\text{read}, \text{add}, \text{remove}\}$, if r is a to-many association end resource ($r = \text{End}$)
- $a \in \{\text{execute}\}$, if r is a method resource ($r = \text{Method}$).

We introduce the data action hierarchy as the combined orderings of its parts.

Definition 2.7 (Data action hierarchy). Let Act be the set of actions, and let Resources be the set of resources. Let $\mathcal{A}$ be the set of data actions Act on resources Resources . The data action hierarchy is the tuple $(\mathcal{A}, \geq_{\mathcal{A}})$, where the binary relation $\geq_{\mathcal{A}}$ is defined as:

$$(act_1, res_1) \geq_{\mathcal{A}} (act_2, res_2) \iff act_1 \supseteq_{\text{Act}} act_2 \wedge res_1 \supseteq_{\text{Resources}} res_2.$$

The semi-lattice is described in detail below. We use $\rightarrow$ to represent the action hierarchy and $\dashrightarrow$ to represent the subresource relation.

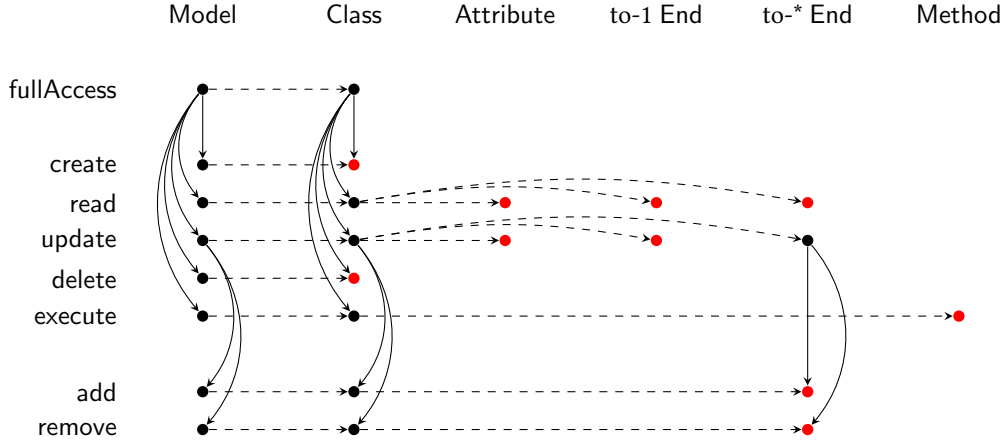


Figure 2.5: Data action hierarchy semi-lattice.

We denote the set of all leaf nodes of the semi-lattice (the red nodes) as the *data atomic actions*.

Definition 2.8 (Data atomic actions).

Let $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model. The set of atomic actions of $\mathcal{D}$, denoted by $\mathcal{A}_{\mathcal{D}}$, that a user can perform, consists of:

- for each $C \in \text{CS}$: $\text{create}(C)$ and $\text{delete}(C)$;
- for each $C \in \text{CS}, n \in \text{dom}(\text{attr}(C))$: $\text{update}(C, n)$ and $\text{read}(C, n)$;
- for each $\alpha \in \text{dom}(\text{as}), n \in \text{ID}, n' \in \text{ID}, C \in \text{CS}, C' \in \text{CS}$ with $\text{as}(\alpha) = ((n, 1, C'), (n', _, C))$ or $((n', _, C), (n, 1, C'))$: $\text{update}(C, n)$ and $\text{read}(C, n)$;
- for each $\alpha \in \text{dom}(\text{as}), n \in \text{ID}, n' \in \text{ID}, C \in \text{CS}, C' \in \text{CS}$ with $\text{as}(\alpha) = ((n, _, C'), (n', *, C))$ or $((n', *, C), (n, _, C'))$: $\text{add}(C, n)$, $\text{remove}(C, n)$, and $\text{read}(C, n)$;
- for each $C \in \text{CS}, n \in \text{dom}(\text{meth}(C))$: $\text{execute}(C, n)$.

These actions correspond to creating and deleting objects, reading and updating attributes, manipulating associations between objects, and executing class methods.

Example 2.7. A user may create an Event object (i.e., the `create(Event)` action), set its title (i.e., the `update(Event, title)` action), and assign themselves as the owner of the event (i.e., the `update(Event, owner)` action or, symmetrically, the `add(Person, events)` action).

A user may read an Event object, which means they can read all of its attributes (e.g., title, description, private) and association ends (e.g., owner, managedBy, attendants).

The actions that are not atomic are referred to as *composite actions*. In the subsequent parts, unless stated otherwise, when we write *actions*, we refer to the set of atomic actions.

Authorization constraints. Authorization constraints are OCL constraints that express conditions under which a user (e.g., Alice) in some role (e.g., FREEUSER) may take some action (e.g., read the title attribute) on some resource (e.g., an object of the Event sort). A constraint is written in the context of the object representing the resource; hence, the **self** OCL expression evaluates to the object. The user object is represented as a free variable `callerUser` in the constraint.

Depending on the action, other factors may need to be considered when making the authorization decision. For example, when updating an attribute or an association end, the new intended value of some sort C can influence the decision, which is captured by the free variable `valueC`. Given a valuation of variables, the OCL constraint evaluates to a Boolean value that encodes whether the user can carry out the action.

Given $\mathcal{D}_S$ the extension of the data model $\mathcal{D}$, let $\text{Con}(V) = \{e \mid e \in \theta_{\text{Bool}}, \text{fv}(e) \subseteq V\}$ be the set of OCL constraints with free variables contained in V . Given an atomic action $a \in \mathcal{A}_{\mathcal{D}}$, the set of authorization constraints $C(a)$ is

$$\begin{aligned} &\text{Con}(\{\text{caller}\}), && \text{if } a \text{ is a create, delete, read, or execute action} \\ &\text{Con}(\{\text{caller}, \text{value}\}), && \text{if } a \text{ is an update action} \\ &\text{Con}(\{\text{caller}, \text{target}\}), && \text{if } a \text{ is an add or remove action.} \end{aligned}$$

The security model defines permissions only on atomic actions. A permission on a composite action A , which dominates atomic actions $\{a_1, a_2, \dots, a_n\}$, is understood as shorthand for permissions on the atomic actions subsumed by A . Since these atomic actions may expose different free variables, the set of admissible constraints for A is computed using only the variables common to all subsumed atomic actions. Formally,

$$C(A) = \left(\bigcap_{a \in \mathcal{A}}^{A \geq_{\mathcal{A}} a} C(a) \right)$$

Definition 2.9 (Security Model).

Given a data model $\mathcal{D}$, a *security model* $\mathcal{S}$ is a triple $(\mathcal{D}_S, \preceq_{\mathcal{R}}, \mathcal{PA})$, where $\mathcal{D}_S$ is a security extension of $\mathcal{D}$, $\preceq_{\mathcal{R}}$ is a partial order on $\mathcal{R}$ representing the role hierarchy, and $\mathcal{PA}$ is a set of permission assignments:

$$\mathcal{PA} \subseteq \bigcup_{a \in \mathcal{A}} \mathcal{R} \times \{a\} \times C(a)$$

Security enforcement mechanism. Given a security model $\mathcal{S} = (\mathcal{D}_S, \preceq_{\mathcal{R}}, \mathcal{PA})$, a user action is allowed if (i) it is associated with a permission containing a satisfied authorization constraint and (ii) the user's role dominates the role in the permission.¹

More precisely, let $\mathcal{O}$ be a valid object model. Then, a security enforcement mechanism allows a user u to take an action a on an object o and (possibly) update the object with the value v if and only if there exists a permission assignment $(r, a, \varphi) \in \mathcal{PA}$ such that $r \preceq_{\mathcal{R}} u.\text{role}$ and $\varphi[\text{self}/o][\text{caller}/u][\text{value}/v]$ evaluates to true in the context of $\mathcal{O}$.

Example 2.8. Consider the event platform from Example 2.1 with the data model $\mathcal{D}$ in Example 2.2, which we extend to $\mathcal{D}_S$ in Example 2.6. Let the role hierarchy be $\preceq_{\mathcal{R}} = \{(\text{PREMIUMUSER}, \text{ADMIN}), (\text{PREMIUMUSER}, \text{MODERATOR}), (\text{FREEUSER}, \text{PREMIUMUSER})\}$.

Suppose that the security model specifies the following requirement: *A FREEUSER can read the title of an event if the event is public or if the user is the event's owner, a manager, or an attendant.*

This requirement is modeled by the triple $(\text{FREEUSER}, \text{read}(\text{Event}, \text{title}), \varphi) \in \mathcal{PA}$, where φ is the following authorization constraint:

not **self.private** or **self.owner** = caller or
self.managedBy → **includes**(caller) or **self.attendants** → **includes**(caller)

Then, consider the object model $\mathcal{O}_2$ in Example 2.3. Suppose that all Person instances in $\mathcal{O}_2$, namely, 'Bob' (p_2) and 'Charlie' (p_3), are users of the FREEUSER role. Since 'Bob' is the owner of event e_2 , the OCL constraint $\varphi[\text{self}/e_2][\text{caller}/p_2]$ holds. Similarly, 'Charlie' is attending e_2 , so the constraint $\varphi[\text{self}/e_2][\text{caller}/p_3]$ also holds. Therefore, both users should be able to read the title of the Event e_2 .

In the subsequent sections, we introduce two model-driven security tools that rely on these models to enforce fine-grained access control policies in applications.

¹A role r dominates a role r' if $r' \preceq_{\mathcal{R}} r$, i.e., r extends r' in the role hierarchy of $\mathcal{S}$.

2.3 ActionGUI: Security-aware GUIs for Web Applications

ActionGUI is a model-driven development tool for building secure-by-design data-centric web applications [15]. Using this tool, software developers define several models that specify different aspects of the system, namely:

- A data model that defines the application’s data domain in terms of its classes, attributes, associations, and methods. In ActionGUI, the data model is defined using ComponentUML (Section 2.2.1).
- A security model that defines the application’s fine-grained access control policies over the actions on the resources provided by the data model. It is specified using a dialect of SecureUML (Section 2.2.3).
- A GUI model that defines both the application’s graphical user interface and the application logic. In other words, it specifies both the presentation and control behavior of the system.

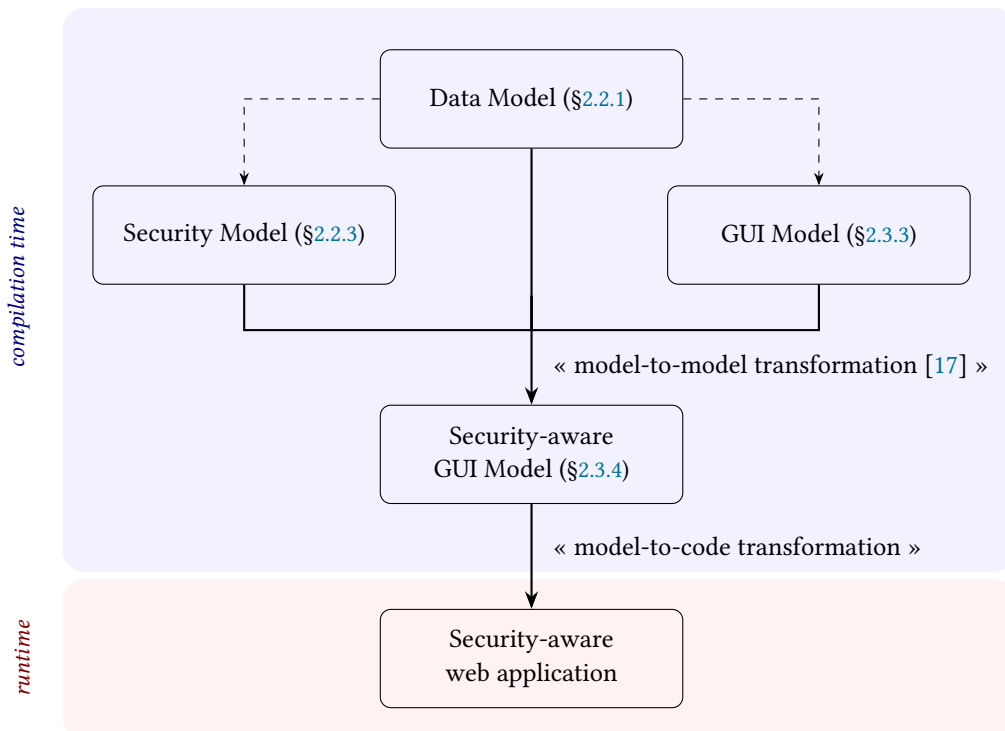


Figure 2.6: Overview of the ActionGUI development methodology. $\longrightarrow$ denotes a model transformation; $- - \rightarrow$ denotes a dependency, where the target model is defined over the source model.

Once the models are defined, ActionGUI first performs a *model-to-model transformation*, generating a so-called *security-aware* GUI model. This model is then used as input

for the *model-to-code transformation* to generate a fully functional data-centric web application. The target application is written in Vaadin [134]. Specifically,

- The underlying database is generated in MySQL.
- The access control mechanism is generated automatically.
- The graphical interface of the web application is generated according to the specifications in the GUI model.

The semantics of the ActionGUI modeling languages and the definition of model-to-model transformation are provided in the technical report in [16].

2.3.1 ActionGUI Data Model

In ActionGUI, developers specify the data model using a domain-specific language. This language allows developers to define entities and their properties. A property is defined as a pair consisting of a name and a type. A type can be primitive (e.g., Boolean, Integer, or String), an entity type, or a collection type (e.g., Set, Bag, OrderedSet, Sequence). The association ends must also specify the name of the opposite end of the association, using the `oppositeTo` keyword.

Example 2.9. Consider the event platform described in Example 2.1 and its UML class diagram shown in Figure 2.1. The data model specified using the ActionGUI data modeling language is as follows:

```

1  entity Person {
2      String name
3      String surname
4
5      Set(Event) events oppositeTo owner
6      Set(Event) manages oppositeTo managedBy
7      Set(Event) attends oppositeTo attendants
8  }
9
10 entity Event {
11     String title
12     String description
13     Boolean private
14
15     Person owner oppositeTo events
16     Set(Person) attendants oppositeTo attends
17     Set(Person) managedBy oppositeTo manages
18 }
```

2.3.2 ActionGUI Security Model

In ActionGUI, the user class is configured in a separate configuration `pom.xml` file. The developers then need to extend the data model to include a role attribute in the user class and enumerate the potential roles.

Example 2.10. Consider the event platform described in Example 2.1 and the data model shown in Example 2.9. Suppose that `Person` is defined as the user class with the potential roles defined in Example 2.6, namely: `FREEUSER`, `PREMIUMUSER`, `MODERATOR`, and `ADMIN`. We extend the data model in Example 2.9 as follows:

```

1  entity Person {
2    String name
3    String surname
4    Role role
5
6    Set(Event) events oppositeTo owner
7    Set(Event) manages oppositeTo managedBy
8    Set(Event) attends oppositeTo attendants
9  }
10
11 entity Event {
12   String title
13   String description
14   Boolean private
15
16   Person owner oppositeTo events
17   Set(Person) attendants oppositeTo attends
18   Set(Person) managedBy oppositeTo manages
19 }
20
21 enum Role {
22   FREEUSER,
23   PREMIUMUSER,
24   MODERATOR,
25   ADMIN
26 }
```

The security modeling language defines the access control policy of the application in terms of the actions allowed on the resources provided by the data model.

In ActionGUI, developers need to define roles and their hierarchy, and for each role, the corresponding access control policies. Each permission consists of an action on a resource and an authorization constraint, which is an OCL expression that determines whether the action is allowed. In the authorization constraints, `self` refers to the resource upon which the action is performed, `caller` refers to the user performing the action, and `value` refers to the value being assigned for update actions (see Section 2.2.3). The atomic actions on a resource correspond to those defined in Definition 2.8.

In addition, ActionGUI supports some composite actions defined in the previous section. For example, a read action on an entity without specifying a property means read access to all its properties, and a fullAccess action on an entity means granting all possible actions on that entity.

Example 2.11. Consider the event platform described in Example 2.1 and the data model shown in Example 2.9. The security model from Example 2.8 is specified as follows:

```

1  role FREEUSER {
2    Event {
3      read(title) constrainedBy [not self.private or self.owner = caller
4                                or self.managedBy→includes(caller)
5                                or self.attendants→includes(caller)]
6      // Other access control policies for actions on Event for FREEUSER.
7    }
8    // Access control policies for actions on other entities for FREEUSER.
9    Person {
10     ...
11   }
12 }
13
14 role PREMIUMUSER extends FREEUSER {
15   // Access control policies for actions for PREMIUMUSER.
16   ...
17 }
18
19 role MODERATOR extends PREMIUMUSER {
20   // Access control policies for actions for MODERATOR.
21   ...
22 }
23
24 role ADMIN extends PREMIUMUSER {
25   // Access control policies for actions for ADMIN.
26   ...
27 }
```

2.3.3 GUI Model

A *GUI model* defines the widgets contained in each page of the web application (e.g., labels, text boxes, buttons, tables, and others) and the behavior of their event handlers (e.g., the functionality executed on a button click). This behavior can be specified using simple control-flow structures (such as assignments, if-then-else statements, and loops) and actions from $\mathcal{A}$. The full description of the GUI modeling language and its formal semantics can be found in the work of Basin et al. [15].

Example 2.12. Consider the GUI model for the EventViewWindow page in the event platform. The page EventViewWindow is used to display event information such as its title, description, and attendants. Here, the page is bound to an instance of the Event class, i.e., the event currently being viewed (line 2).

```

1 Window EventViewWindow {
2   Event entity
3   Label Header_L { String text := ['View Event'] }
4   Label Title_L { String text := ['Title:' + $EventViewWindow.entity$.title] }
5   Table Table_Attendants_T {
6     Set(Person) rows := [$EventViewWindow.entity$.attendants]
7     columns {
8       ['Name'] : Label name {
9         event onView (text) {
10          try {
11            text := [$Table_Attendants_T.row$.name]
12          } catch (SecurityException) {
13            text := ['RESTRICTED']
14          }
15        }
16      }
17    }
18  }
19 }

```

Labels such as Title_L compute their value via an OCL navigation expression using the \$.\$. notation: \$EventViewWindow.entity\$.title navigates from the window's bound entity to its title attribute (line 4). The Table_Attendants_T table renders the event's attendants as a set of Person rows (line 6). Each row uses the onView handler to display the attendant's name (line 11).

When the handler attempts to read the attendant's name, the generated application checks the corresponding authorization constraint from the security model. If the user is not authorized, a `SecurityException` is thrown, and the catch block displays 'RESTRICTED' instead. This `try-catch` pattern is the primary mechanism through which the security model affects the GUI behavior at runtime.

Navigation between windows is specified using the `open` action, which passes the authenticated user and their role to the target window. For example, after a successful login, the action

```
open EventViewWindow(caller:[$loggedUser$], role:[$loggedUser$.role])
```

opens the EventViewWindow and passes the authenticated user as caller and their role as role. This is how the caller variable used in authorization constraints (e.g., `self.owner = caller` in Example 2.11) receives its value at runtime.

2.3.4 Security-aware GUI Model

A *security-aware GUI model* [17] is a generated intermediate model in which the GUI behavior is enriched with authorization checks derived from the security model, ensuring that every data action triggered by the interface is executed only if the corresponding access control policy holds. As this concept is not central to this thesis, we do not discuss it in further detail.

Example 2.13. We proceed with the event platform example. In ActionGUI, the three input models are: the data model $\mathcal{D}$ from Example 2.2 defined in Example 2.9, the security model $\mathcal{S}$ from Example 2.8 defined in Example 2.11, and the GUI model shown in Example 2.12. From these models, ActionGUI generates a web application that contains a *security-aware* GUI interface in which every data action, such as reading an event's title or listing its attendants, is checked against the authorization constraints in $\mathcal{S}$.

Here, the control behavior of the application is already implemented, and for any data actions included in the control logic, the access control policy is automatically enforced (i.e., the application is *security-aware*, as each data action is executed if and only if the authorization check specified in the security model holds).

2.3.5 Strengths and Limitations

ActionGUI is a strong instance of a model-driven development methodology for developing secure-by-design software systems. First, it promotes modularity and separation of concerns. By working with different models of the system, developers can focus independently on the application's data, security, presentation, and behavior. Second, since models are formally defined, they can be analyzed, for example, to verify access control properties, analyze policy consistency, and detect unintended permissions [37].

However, ActionGUI also has some limitations. Although the GUI modeling language is expressive and allows developers to specify both the structure and behavior of the user interface, implementing highly customized functionality within the GUI model may be cumbersome. Ideally, developers should be able to implement application logic freely in a general-purpose programming language, while the access control mechanism remains automatically generated and enforced as a cross-cutting concern for all data actions invoked by the application code.

ActionGUI has been used extensively in both research and teaching for security engineering. In the later part of this thesis, we substantially extend ActionGUI to overcome some of its main limitations and provide additional support for privacy requirements.

2.4 SQLSI: SQL Security Interceptor

In this section, we present SQLSI, another model-driven development tool for enforcing fine-grained access control policies [93]. While ActionGUI enforces the security policy at the presentation and application layers, SQLSI moves the security enforcement to the data layer. This prevents the data from leaving the database unprotected. The mechanism here acts as an independent enforcement point that applies regardless of how the application code accesses the database.

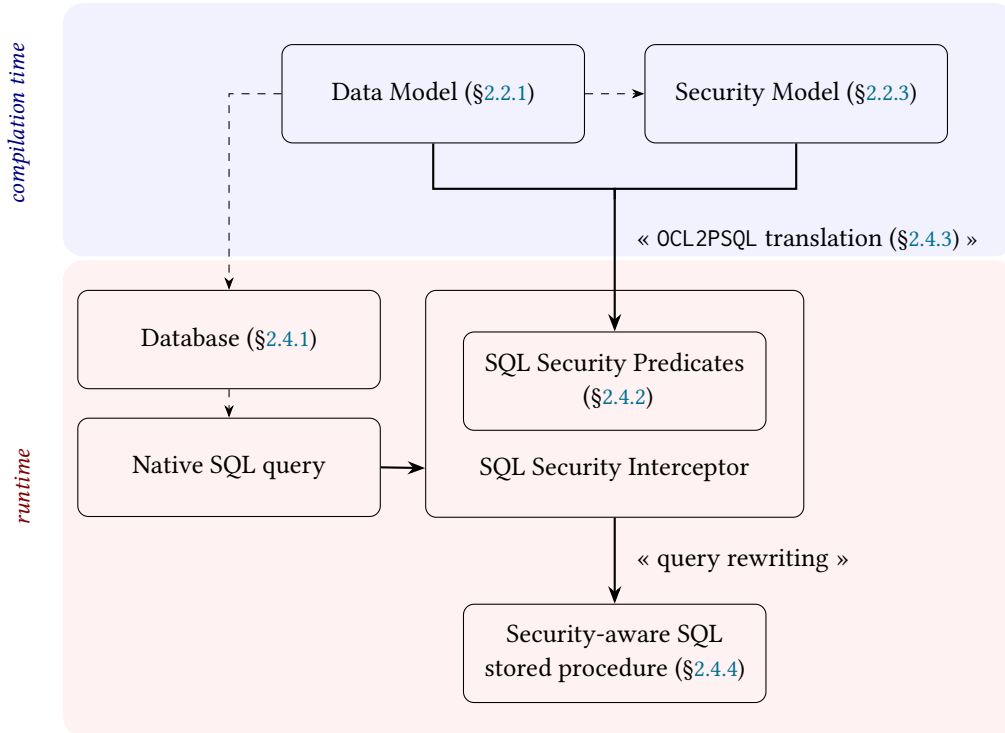


Figure 2.7: Overview of the SQLSI development methodology. $\longrightarrow$ denotes a model transformation or data flow; $--\longrightarrow$ denotes a dependency, where the target is defined over the source.

In a nutshell, SQLSI captures native SQL queries at runtime and rewrites them into security-aware stored procedures. Figure 2.7 illustrates the overall methodology. To use SQLSI, developers must first specify two models:

- A data model that defines the application domain in terms of classes, attributes, and associations. Similar to ActionGUI, the SQLSI data model is defined using ComponentUML (Section 2.2.1).
- A security model that specifies fine-grained access control policies over read operations on the data domain. This model is a dialect of SecureUML (Section 2.2.3),

restricted to read-only data actions. Its authorization constraints are expressed in the Object Constraint Language.

Unlike ActionGUI, SQLSI uses a JSON-based format for specifying both the data and security models. For ease of presentation, we do not show the concrete JSON syntax here (the full syntax is described in [93]). Instead, we rely on the abstract representations of the data model (Definition 2.1) and security model (Definition 2.9).

In the development phase, these models are defined. Once they are available, SQLSI applies model transformations that map the data model to the application's database schema (Section 2.4.1) and the security model to a collection of SQL authorization predicates (Section 2.4.2). These predicates are stored in the database management system and contain SQL statements implementing the OCL authorization constraints, e.g., using the OCL2PSQL mapping (Section 2.4.3) [95].

At runtime, when an application submits an SQL query to the database system, SQLSI captures this query and rewrites it into a call to a security-aware stored procedure. This stored procedure enforces access control policies specified in the given security model. The rewriting rules follow the characterization of fine-grained access control policies for SQL queries introduced in [94]. We provide an example of the characterization in Section 2.4.4. The complete characterization can be found in Appendix A.

SQLSI then invokes the stored procedure on behalf of the current user. If the current user is authorized to execute the original query with respect to the security model, then calling the rewritten stored procedure returns the same result as the original query. Otherwise, if the current user is not authorized, SQLSI rejects the query and returns an unauthorized message instead.

2.4.1 Mapping from SQLSI Data Model to Database Schema

SQLSI maps a data model to a relational database schema as follows.

Definition 2.10 (SQLSI data model to database schema).

Given a data model $\mathcal{D} = (CS, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$, the mapping of $\mathcal{D}$ to a relational database schema, denoted $\bar{\mathcal{D}}$, is defined as follows:

- for every entity $C \in CS$, SQLSI creates a table with a primary key column:

```
CREATE TABLE C (C_id INT PRIMARY KEY);
```

- for every $C \in CS$ and $at \in \text{dom}(\text{attr}(C))$ with $\text{attr}(C)(at) = t$, SQLSI adds a corresponding column to table C :

```
ALTER TABLE C ADD COLUMN at SqlType(at);
```

where $\text{SqlType}(Z) = \text{INT}$, $\text{SqlType}(\text{String}) = \text{VARCHAR}(255)$, and $\text{SqlType}(\text{Bool}) = \text{BOOLEAN}$. If $t \in \text{CS}$, then $\text{SqlType}(at) = \text{INT}$ and the column is additionally declared as a foreign key referencing $t(t_id)$.

- for every $\alpha \in \text{dom}(\text{as})$ with $\text{as}(\alpha) = ((n_l, m_l, C_l), (n_r, m_r, C_r))$, SQLSI creates an association table:

```
CREATE TABLE  $\alpha$  (
   $n_l$  INT REFERENCES  $C_l(C_l\_id)$ ,
   $n_r$  INT REFERENCES  $C_r(C_r\_id)$ ,
  PRIMARY KEY ( $n_l, n_r$ )
);
```

If $m_l = 1$ (respectively, $m_r = 1$), a **UNIQUE** constraint is added on n_r (resp. n_l) to enforce the multiplicity.

Example 2.14. Consider the data model from Example 2.2. The mapping produces the following database schema:

```
1 CREATE TABLE Person (
2   Person_id INT PRIMARY KEY,
3   name VARCHAR(255),
4   surname VARCHAR(255)
5 );
6 CREATE TABLE Event (
7   Event_id INT PRIMARY KEY,
8   title VARCHAR(255),
9   description VARCHAR(255),
10  private BOOLEAN
11 );
12 CREATE TABLE Event_managedBy_manages_Person (
13   managedBy INT REFERENCES Person(Person_id),
14   manages INT REFERENCES Event(Event_id),
15   PRIMARY KEY (managedBy, manages)
16 );
17 CREATE TABLE Event_attends_attendants_Person (
18   attends INT REFERENCES Event(Event_id),
19   attendants INT REFERENCES Person(Person_id),
20   PRIMARY KEY (attends, attendants)
21 );
22 CREATE TABLE Event_owner_events_Person (
23   owner INT REFERENCES Person(Person_id),
24   events INT REFERENCES Event(Event_id),
25   PRIMARY KEY (owner, events),
26   UNIQUE (events)
27 );
```

The ownership association has multiplicity 1 on the owner side, i.e., each Event has a single owner. This is captured by the **UNIQUE** constraint on its events column (line 26).

We then define the mapping from object models to database instances.

Definition 2.11 (SQLSI object model to database instance).

Let $\mathcal{D} = (CS, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model and let $\mathcal{O} = (\mathbb{O}, \mathbb{M}, \mathbb{R})$ be an object model of $\mathcal{D}$. The mapping of $\mathcal{O}$ to a database instance over $\overline{\mathcal{D}}$, denoted $\overline{\mathcal{O}}$, is defined as follows:

- For every $(o \mapsto (C, \text{ats})) \in \mathbb{O}(C)$, SQLSI inserts a corresponding row:

```
INSERT INTO C (C_id) VALUES (o);
```

and updates the row attribute values:

```
UPDATE C SET at = v WHERE C_id = o;
```

where o serves as the primary key and $at \in \text{attr}$, $o.at = v$ provides the attribute value of the attribute at .

- For every $\alpha \in \text{dom}(\text{as})$ and $(o_1, o_2) \in \mathbb{R}(\alpha)$, SQLSI inserts the pair into the association table:

```
INSERT INTO  $\alpha$  VALUES (o1, o2);
```

Example 2.15. Consider the object model $\mathcal{O}_2$ from Example 2.3, where p_2 , p_3 , and e_2 map to `Person_id = 1`, `Person_id = 2`, and `Event_id = 1`, respectively. The corresponding database instance $\overline{\mathcal{O}}_2$ is:

```

1  INSERT INTO Person (Person_id) VALUES (1);
2  UPDATE Person SET name = 'Bob' WHERE Person_id = 1;
3  UPDATE Person SET surname = 'Brown' WHERE Person_id = 1;
4
5  INSERT INTO Person (Person_id) VALUES (2);
6  UPDATE Person SET name = 'Charlie' WHERE Person_id = 2;
7  UPDATE Person SET surname = 'Clark' WHERE Person_id = 2;
8
9  INSERT INTO Event (Event_id) VALUES (1);
10 UPDATE Event SET title = 'Study Group' WHERE Event_id = 1;
11 UPDATE Event SET description = 'Exam Preparation' WHERE Event_id = 1;
12 UPDATE Event SET private = FALSE WHERE Event_id = 1;
13
14 INSERT INTO Event_owner_events_Person (owner, events) VALUES (1, 1);
15 INSERT INTO Event_managedBy_manages_Person (managedBy, manages) VALUES (1, 1);
16 INSERT INTO Event_attends_attendants_Person (attends, attendants) VALUES (1, 1);
17 INSERT INTO Event_attends_attendants_Person (attends, attendants) VALUES (1, 2);
```

In the next section, we describe the mapping from the security model to SQL predicate functions. SQLSI generates these SQL predicate functions that are embedded with the authorization constraints of the relevant access control policies from the security model.

2.4.2 Mapping from SQLSI Security Model to the Database

Representing users and roles. The security model is defined over the security extension $\mathcal{D}_S$ from Definition 2.3, which augments $\mathcal{D}$ with the User sort and the role attribute from the enumeration $\mathcal{R}$. SQLSI represents these concepts in the database as follows.

Definition 2.12 (User and Role in the database).

Let $\mathcal{D} = (\mathcal{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model and let $\mathcal{S} = (\mathcal{D}_S, \preceq_{\mathcal{R}}, \mathcal{PA})$ be a security model for $\mathcal{D}$, where $\mathcal{D}_S = (\text{user}, \mathcal{CS}', \preceq_{\mathcal{D}_S}, \text{attr}', \text{meth}, \text{as})$. SQLSI stores the role sort $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$ in a dedicated Role table.

```
CREATE TABLE Role (
  Role_id INT PRIMARY KEY,
  name VARCHAR(255) UNIQUE
);
ALTER TABLE C_user ADD COLUMN role INT REFERENCES Role(Role_id);
INSERT INTO Role (name) VALUES ('r_1'), ('r_2'), ..., ('r_n');
```

To make the role hierarchy explicit in the database, SQLSI provides the hierarchy $\preceq_{\mathcal{R}}$ in a RoleHierarchy table.

```
CREATE TABLE RoleHierarchy (
  parent INT REFERENCES Role(Role_id),
  child INT REFERENCES Role(Role_id),
  PRIMARY KEY (parent, child)
);
```

The RoleHierarchy table presents the partial order $\preceq_{\mathcal{R}}$. Every (*parent*, *child*) row represents a pair of roles with *child* $\preceq_{\mathcal{R}}$ *parent*, including the reflexive pairs.

Example 2.16. Recall the Event Management Platform security extension in Example 2.6, where Person is the user sort and $\mathcal{R}$ is the set {FREEUSER, PREMIUMUSER, MODERATOR, ADMIN} with the role hierarchy in Example 2.8: FREEUSER $\preceq_{\mathcal{R}}$ PREMIUMUSER, PREMIUMUSER $\preceq_{\mathcal{R}}$ MODERATOR, and PREMIUMUSER $\preceq_{\mathcal{R}}$ ADMIN. Following the definition above, SQLSI populates the new tables.

```
INSERT INTO Role (Role_id, name) VALUES
  (1, 'FREEUSER'), (2, 'PREMIUMUSER'),
  (3, 'MODERATOR'), (4, 'ADMIN');
ALTER TABLE Person ADD COLUMN role INT REFERENCES Role(Role_id);
INSERT INTO RoleHierarchy (parent, child) VALUES
  (1,1),
  (2,1), (2,2),
  (3,1), (3,2), (3,3),
  (4,1), (4,2), (4,4);
```

On the next page, we present a helper function `dominate(parent, child)` that returns whether a *parent* role inherits the permissions granted to a *child* role.


```

CREATE FUNCTION dominate (parent INT, child INT)
RETURNS INT DETERMINISTIC -- 1 = true, 0 = false
BEGIN
  RETURN EXISTS (
    SELECT 1
    FROM RoleHierarchy r
    WHERE r.parent = parent AND r.child = child
  );
END

```

Actions to security predicates. In what follows, we denote by a function map an OCL to SQL translation: given an OCL constraint φ , $\text{map}(\varphi)$ returns an SQL query whose result contains a column `res` representing the truth value of evaluating φ .

Definition 2.13 (Auxiliary attribute authorization functions).

Let $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model. Let $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$ be a security model for $\mathcal{D}$. For each role $r \in \mathcal{R}$, for each entity $C \in \text{CS}$, and for each attribute $at \in \text{attr}(C)$, SQLSI generates the following SQL function:

```

CREATE FUNCTION AuthFuncRole_C_at_r (self INT, caller INT)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 0;
  SELECT res INTO result
  FROM map( $\varphi$ ) AS TEMP;
  RETURN result;
END

```

where $(r, \text{read}(C, at), \varphi) \in \mathcal{PA}$. Given that $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$, these functions are invoked by a wrapper function that combines their results over all roles dominated by the current user's role:

```

CREATE FUNCTION AuthFunc_C_at (self INT, caller INT, role INT)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 0;
  SET result = result OR (dominate(role, r1) AND AuthFuncRole_C_at_r1(self, caller));
  SET result = result OR (dominate(role, r2) AND AuthFuncRole_C_at_r2(self, caller));
  ...
  SET result = result OR (dominate(role, rn) AND AuthFuncRole_C_at_rn(self, caller));
  RETURN result;
END

```

This function grants access to attribute at of class C whenever the caller's role dominates some role r whose permission $(r, \text{read}(C, at), \varphi) \in \mathcal{PA}$ has a satisfied constraint, thereby enforcing the role hierarchy $\preceq_{\mathcal{R}}$ through `dominate`. SQLSI defines analogous auxiliary functions for associations.

Definition 2.14 (Auxiliary association authorization functions).

Let $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model. Let $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$ be a security model for $\mathcal{D}$. For each $r \in \mathcal{R}$ and for each $as \in \text{dom}(\text{as})$, where as connects classes C_l and C_r through association ends ase_l and ase_r , SQLSI generates the following SQL function:

```
CREATE FUNCTION AuthFuncRole_as_r (left INT, right INT, caller INT)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result_left INT DEFAULT 0;
  DECLARE result_right INT DEFAULT 0;
  SELECT res INTO result_left
    FROM map( $\varphi_1$ ) AS TEMP_LEFT;
  SELECT res INTO result_right
    FROM map( $\varphi_2$ ) AS TEMP_RIGHT;
  RETURN result_left OR result_right;
END
```

where $(r, \text{read}(C_r, as_{el}), \varphi_1) \in \mathcal{PA}$ and $(r, \text{read}(C_l, as_{er}), \varphi_2) \in \mathcal{PA}$. Assuming that $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$, these functions are invoked by a wrapper function that combines their results over all roles dominated by the current user's role:

```
CREATE FUNCTION AuthFunc_as (left INT, right INT, caller INT, role INT)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 0;
  SET result = result OR dominate(role,  $r_1$ ) AND AuthFuncRole_as_r1(left, right, caller);
  SET result = result OR dominate(role,  $r_2$ ) AND AuthFuncRole_as_r2(left, right, caller);
  ...
  SET result = result OR dominate(role,  $r_n$ ) AND AuthFuncRole_as_rn(left, right, caller);
  RETURN result;
END
```

This function grants the association as whenever the caller's role dominates a role whose permission has a satisfied constraint, again through `dominate`, so that the role hierarchy $\preceq_{\mathcal{R}}$ is enforced for associations as well.

2.4.3 Mapping from OCL Expressions to SQL Statements

Several OCL-to-SQL translations exist [34, 39]. In SQLSI, we currently rely on the OCL2PSQL translation, which maps OCL expressions to SQL queries [95]. For any OCL expression φ in the supported fragment, OCL2PSQL produces an SQL query q such that, for any valid database state $\overline{O}$ that corresponds to an object model O , φ evaluates to true over O if and only if q returns TRUE when executed in $\overline{O}$.

Example 2.17. Consider the expression from Example 2.5, which restricts the object model to those in which all events have non-empty titles: `self.title.size() > 0` or equivalently, `Event.allInstances() → forAll(e | e.title.size() > 0)`. OCL2PSQL translates the latter Boolean expression into the following SQL query:

```
SELECT COUNT(*) = 0 AS res, 1 AS val
FROM (
  SELECT TEMP_left.res > TEMP_right.res AS res,
        TEMP_left.val * TEMP_right.val AS val,
        TEMP_left.ref_v AS ref_v
  FROM (
    SELECT LEN(TEMP_src.res) AS res, TEMP_src.val AS val, TEMP_src.ref_v AS ref_v
    FROM (
      SELECT Event.title AS res, TEMP_obj.val AS val, TEMP_obj.ref_v AS ref_v
      FROM (
        SELECT TEMP_dmn.res AS res, TEMP_dmn.val AS val, TEMP_dmn.res AS ref_v
        FROM (SELECT Event_id AS res, 1 AS val FROM Event) AS TEMP_dmn
      ) AS TEMP_obj
      LEFT JOIN Event
      ON TEMP_obj.ref_v = Event.Event_id AND TEMP_obj.val = 1
    ) AS TEMP_src
  ) AS TEMP_left
  JOIN (SELECT 0 AS res, 1 AS val) AS TEMP_right
) AS TEMP_body
WHERE TEMP_body.res = 0;
```

Consider the object model O_2 from Example 2.3 and the corresponding database instance $\overline{O}_2$ from Example 2.15. Evaluating the query above over $\overline{O}_2$ returns:

res	val
1	1

The result matches the OCL evaluation, i.e., the invariant holds for O_2 .

2.4.4 Model-based Characterization of Security Enforcement on SQL Queries

First, we define the characterization of enforcing FGAC policies on SQL queries via an authorization predicate `AuthQuery()`. More specifically, `AuthQuery( $S, O, u, r, q$ )` specifies the conditions under which a user u with role r is authorized to execute a given SQL query q on an object model O with respect to the security model S .

`AuthQuery()` is defined recursively over SQL query patterns. For each pattern, the predicate checks that the user is authorized to read every resource accessed during query execution. This includes not only the properties returned in the final result, but also those used in subqueries, WHERE clauses, and ON clauses. We illustrate below

the characterization for the simplest pattern. The informal descriptions and formal definitions of AuthQuery for all patterns are given in Appendix A.

Definition 2.15 (AuthQuery: SELECT from class).

Let $\mathcal{D} = (\mathcal{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model, and let $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$ be a security model for $\mathcal{D}$, where $\mathcal{D}_{\mathcal{S}}$ extends $\mathcal{D}$ to $(\text{user}, \mathcal{CS}', \preceq_{\mathcal{D}_{\mathcal{S}}}, \text{attr}', \text{meth}, \text{as})$. Let $\mathcal{O} = (\mathbb{O}, \mathbb{M}, \mathbb{R})$ be an object model of $\mathcal{D}_{\mathcal{S}}$. Let $r \in \mathcal{R}$ be a role, and $u \in \text{dom}(\mathbb{O}(\text{user}))$ be a user object.

Let $q = \text{SELECT items FROM } C \text{ WHERE } \text{boolexp}$ be an SQL query.

Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if:

- for every object $o \in \text{dom}(\mathbb{O}(C))$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the Boolean expression *boolexp* of q , the user u with role r is allowed to read at of o with respect to the security model $\mathcal{S}$ in the context of $\mathcal{O}$; and
- for every object $o \in \text{dom}(\mathbb{O}(C))$ that satisfies *boolexp*, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the projection clauses *items* of q , the user u with the role r is allowed to read at of o with respect to the security model $\mathcal{S}$ in the context of $\mathcal{O}$.

Example 2.18. Consider a user ‘Bob’ with the role FREEUSER who submits the following query q_1 to the event platform application.

```
SELECT title
FROM Event
WHERE LOWER(description) LIKE '%education%' AND private = TRUE;
```

By Definition 2.15, $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, \text{Bob}, \text{FREEUSER}, q_1)$ holds for this query if and only if:

1. for every Event object, user ‘Bob’ with the FREEUSER role is allowed to perform read(Event, description) and read(Event, private); and
2. for every Event object that satisfies the WHERE clause, ‘Bob’ with the FREEUSER role is also allowed to perform read(Event, title).

2.4.5 From SQL Query to Stored Procedure

SQLSI realizes the characterization defined by AuthQuery by implementing a mapping function SecQuery. In a nutshell, $\text{SecQuery}(\mathcal{S}, q)$ rewrites q into a stored procedure parameterized by a user and a role, which enforces this characterization at the database layer using calls to the auxiliary SQL functions from Definitions 2.13 and 2.14.

At runtime, when an SQL query q is submitted, SQLSI rewrites q into the stored procedure generated by $\text{SecQuery}(\mathcal{S}, q)$ and calls it with the current user caller and their role role

as arguments. The procedure first verifies that the caller actually holds the given role, then proceeds to perform authorization checks. If any authorization predicate fails, the procedure signals an error and aborts. Otherwise, once all (intermediate) authorization checks succeed, the original query q is executed, and its result is returned.

Example 2.19. Continuing from Example 2.18, SQLSI rewrites the query into the following stored procedure.

```
CREATE PROCEDURE SecQuery( $S, q$ ) (IN user INT, IN role INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Unauthorized access';
  END;
  START TRANSACTION;
  -- Verify that the caller holds the supplied role.
  IF NOT EXISTS (
    SELECT 1
    FROM Person p
    WHERE p.Person_id = user AND p.role = role
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Unauthorized access';
  END IF;
  -- Check WHERE clause attributes for all Event rows.
  IF EXISTS (
    SELECT 1
    FROM Event e
    WHERE AuthFunc_Event_description(e.Event_id, user, role) = 0
      OR AuthFunc_Event_private(e.Event_id, user, role) = 0
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Unauthorized access';
  END IF;
  -- Check SELECT clause attributes for matching rows.
  IF EXISTS (
    SELECT 1
    FROM Event e
    WHERE LOWER(e.description) LIKE '%education%' AND e.private = TRUE
      AND AuthFunc_Event_title(e.Event_id, user, role) = 0
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Unauthorized access';
  END IF;
  -- Execute the original query.
  SELECT e.title
  FROM Event e
  WHERE LOWER(e.description) LIKE '%education%' AND e.private = TRUE;
  COMMIT;
END;
```

The stored procedure installs an exception handler. If any authorization check fails, the procedure signals an SQL exception, rolls back the transaction, and returns an unauthorized access error. Only if all authorization checks succeed does the procedure execute the original query and return its result.

2.4.6 Strengths and Limitations

Compared to the previous approach, a key advantage of SQLSI is that the security enforcement is independent of the application code. Since SQLSI rewrites and calls secured SQL stored procedures at the database layer, the enforcement guarantees apply regardless of which application component or code path is used to access the data.

SQLSI also has several notable limitations. First, the OCL2PSQL mapping does not support the full OCL language. Some constraints may fall outside of the supported OCL fragment and require manual translation. Second, the current characterization of SQLSI covers only a subset of the SQL `SELECT` statement. Other SQL operations, such as the `CREATE`, `INSERT`, `UPDATE`, or `DELETE` statements, are not supported. Finally, SQLSI implements the non-Truman model [113], where a query is either fully authorized or rejected entirely. While this provides clear enforcement semantics, it can also restrict the application's querying capabilities. Chapter 6 presents SQLSPI, which lifts several of these restrictions.

Part II

Model-driven Security and Privacy

Related Work

We organize related work on model-driven security and privacy into four parts: access control, privacy specification and design, security and privacy enforcement, and model-driven development for security and privacy.

Access control

The role-based access control (RBAC) model was initially proposed by Ferraiolo and Kuhn [41] and later formalized by Sandhu et al. [119]. Specifically, application users in the RBAC model are assigned to security roles, and each security role is associated with a set of permissions that govern which data actions may be performed on which data resources. To this day, the RBAC model remains a widely adopted standard for access control in data-centric applications [42] and is supported by mainstream database management systems and web frameworks [106, 44, 86, 90].

In many systems, access control decisions depend not only on the user's role but also on additional runtime conditions. The fine-grained access control (FGAC) model generalizes RBAC by allowing access control decisions to depend on additional predicates over users, resources, actions, and system state. In addition to SecureUML [77] (described in Chapter 2), several other languages have been proposed to express FGAC policies. The eXtensible Access Control Markup Language (XACML) [98] is a well-established extension of XML for defining access control policy. XACML supports policies whose decisions depend on attributes of users, resources, actions, and the surrounding environment.

UMLsec [66] extends UML with security stereotypes and tagged values to model security requirements at the design level. Although UMLsec focuses on security properties such as confidentiality and information flow, it can be used together with modeling languages such as SecureUML to strengthen the security design of a system.

More recently, Amazon developed Cedar [5, 32], an open-source access control policy language and authorization engine. Cedar supports schema-based authorization and expresses policies as permit or forbid rules over principals, actions, and resources. In our empirical study (Chapter 9), we asked participants to develop a secure software system and provided the Cedar language, together with its authorization engine, as one of the code-centric practices for enforcing access control requirements.

Basin et al. [14] model tasks as workflows and associate cost and risk scores with access control policies. They formulate the problem of finding a least privilege access control policy that minimizes cost as an optimization problem and prove that it is NP-complete for role-based cost functions.

In terms of querying semantics, Rizvi et al. [113] introduce the distinction between the Truman and non-Truman enforcement models: the former implicitly filters unauthorized data from query results, while the latter rejects the query entirely if any unauthorized data is accessed. Wang et al. [137] identify three correctness criteria for the enforcement of FGAC, namely soundness, security, and maximality. Guarnieri and Basin [52] later proved that no FGAC enforcement algorithm can simultaneously satisfy all three aforementioned properties.

Privacy specification and design

A privacy policy is a statement that specifies how an organization collects, uses, discloses, stores, and maintains personal data about data subjects [79]. Such policies are now heavily anchored in national and international law. They can be domain-specific, such as the Health Insurance Portability and Accountability Act (HIPAA) [57] and the Gramm-Leach-Bliley Act (GLBA) [50]. Alternatively, they can be domain-independent, such as the General Data Protection Regulation (GDPR) [105] and the California Consumer Privacy Act (CCPA) [26].

Lam et al. [72] formalize certain HIPAA regulations that focus on access control rules for the use and disclosure of protected health information, including treatment information, healthcare operations, and payment. They implement their formalization using a fragment of the Datalog programming language [78] and demonstrate its usability in a prototype hospital system.

Similarly, May et al. [81] extend the classical Harrison, Ruzzo, and Ullman access control model [56] to formally analyze and audit HIPAA regulations. They formalize the target regulations and automatically translate them into a Promela model for model checking [58]. DeYoung et al. [36] go beyond the aforementioned HIPAA regulations and formalize regulations on the protection of customer information under the GLBA, which they also automatically audit [27].

Various languages have been proposed to specify privacy policies [75]. The now obsolete P3P [30, 29] and EPAL [10] languages were proposed to declare the purposes of data processing in a machine-readable form. With P3P, users can set their preferences, which the browser then matches against the websites' declared purposes, notifying the user about any mismatch. EPAL goes further by allowing enterprises to declare internal access control policies needed to enforce the declared privacy policies within the system.

As they lack formal semantics, P3P and EPAL do not by themselves provide guarantees that the implemented system complies with the declared policies.

The Ponder language [33, 142] is an access control specification language. It can specify obligations, which is relevant for some common privacy requirements, such as data minimization. The PrimeLife Policy Language (PPL) [8] and its extensions [12] are based on XACML with custom notions of purpose and obligation. They reuse existing XACML implementations and inherit XACML's complexity.

More recent policy languages, such as Pilot [102] and the policy language of Baramshetru et al. [13], target GDPR-specific privacy requirements. Besides purpose limitation and consent, the former can also specify to whom private data may be transferred, while the latter can specify data retention and location-specific processing.

Security and privacy enforcement

Since native support for FGAC policies is limited [89], developers typically combine the RBAC model with manually implemented access control checks. This approach, as argued by Kabra et al. [67], is error-prone and scales poorly.

In the research literature, several systems have addressed FGAC enforcement in relational databases. LeFevre et al. [74] develop the concept of Hippocratic databases, which enforce FGAC by rewriting SQL queries. Similarly, Mehta et al. [82] propose Qapla, a system that associates each table with a policy and rewrites queries to enforce compliance. Nguyen and Clavel [94] formally characterize the enforcement of FGAC policies for a subset of SQL queries.

Guerriero et al. [53] propose a privacy policy language and an enforcement mechanism tailored for data-intensive applications. Application users can specify privacy policies in a fragment of past-time metric temporal logic. A policy is enforced by removing from the data streams all the user-owned data items that violate the user's policy. Although the approach can specify purpose limitation, consent is not taken into account, and the approach is specialized for data-intensive applications.

Karami et al. [68] propose a Data Protection Language (DPL), a programming language with explicit constructs for purpose and consent. Providing purpose and consent constructs directly in the implementation language improves privacy enforcement. However, it also imposes a significant technological restriction: developers must implement both the privacy policy and the business logic in the DPL. In contrast, our model-driven approach separates privacy specifications from implementation technology.

Byun et al. [23, 24] model purpose hierarchies and distinguish between the *intended* purpose defined for each data item and the *access* purpose, which is the purpose for which the data item is accessed. Assuming that the latter is accurately stated by a user,

their enforcement of purpose limitation amounts to checking whether this purpose conforms to the data item's intended purpose, where both may be complex objects.

There are different possible semantics for data usage. In access control semantics, only the initial access to data is considered usage [120]. In data-flow semantics, usage also includes access to data explicitly derived from personal data initially accessed, for example, by assignment [122]. In information-flow semantics, usage also includes access to data implicitly derived from the initially accessed personal data, for example, by branching on private data, or through other explicit or implicit flows [116]. In this thesis, we adopt access control semantics, as stronger data-flow and information-flow semantics substantially complicate the development of typical web applications. We revisit this design decision in the limitations (Chapter 10).

Although information-flow enforcement is outside the scope of this thesis, several works on privacy policies are worth mentioning. Jeeves/Jacqueline [140, 141] use faceted execution to enforce sensitive data policies [11]. More recently, Hublet et al. [62] combine dynamic information-flow control with runtime verification to enforce user-defined temporal privacy policies in web applications.

Enforcing purpose limitation and user consent requires a consent collection mechanism. In practice, consent collection is often handled in an *ad hoc* manner or delegated to commercial platforms, such as OneTrust [100] and Cookiebot [131], which provide cookie banners and consent dashboards. However, these platforms primarily support consent collection and preference management at the user-interface level. By themselves, they do not guarantee that application-level data accesses are mediated according to the collected consent decisions.

Model-driven security and privacy

Model-driven development uses models as primary development artifacts and derives implementation artifacts through model transformations. Privacy by design [25] posits that privacy must be accounted for throughout the whole engineering process. In particular, software must be developed to ensure the enforcement of privacy policies. MDE supports this aspect of privacy by design by making privacy policies explicit design artifacts from which enforcement mechanisms can be generated.

Existing MDE approaches for privacy focus on automatically transforming architectural system designs [7] (e.g., by adding access control mechanisms or integrity-protected logger components) or reasoning about privacy properties of different designs [6]. However, a large gap remains between the designs and the implementations of the concrete system. In contrast, approaches to enforcing data-usage policies [136, 21] focus on the enforcement mechanism, making the policy statement implementation-dependent and, hence, not a design artifact.

Nguyen and Clavel [93] propose a model-based methodology for developing an enforcement mechanism based on the formal characterization in [94]. Subsequently, they extend their methodology along two dimensions, namely performance [P8] and correctness [P7].

There are approaches that discuss privacy by design without providing concrete enforcement mechanisms, but instead report on experiences [54], analyze regulations [4], or exclusively work with system design models [3].

Antignac et al. [7] propose an automatic transformation of a high-level architectural system design into a privacy-preserving design. Their transformation takes a data-flow diagram (DFD; i.e., the initial architectural design) and a privacy policy. The privacy policy classifies each DFD's data-flow based on the data subject to whom the flowing data belongs, the purpose of the flow, and the data's retention time. The model transformation enhances the input DFD with additional processes that check privacy policies before data is used (e.g., checking the purpose and the existence of the user's consent before data processing).

Antignac et al. [6] also support the thesis that privacy should be addressed at the architectural level, and they specifically focus on data minimization policies. They propose two languages: a formal language for modeling architectures for data minimization and a logic for reasoning about the correctness of such architectures. The latter, called privacy logic, is a variant of epistemic logic with a proof system to reason about the knowledge and beliefs of different architectural components. However, their privacy architectures are modeled at a high level, and obtaining a concrete implementation requires substantial manual refinement.

The privacy policies modeled in ModelSec [110] do not consider purpose or consent. According to the authors, privacy policies ensure that information can be read only by authorized principals, which coincides with the standard notion of confidentiality.

Modeling Security and Privacy Requirements

The data model (Section 2.2.1) and the security model (Section 2.2.3) introduced in Chapter 2 provide the foundation for our model-driven approach. We now describe how these models are extended to support the privacy enforcement presented later in this chapter. Figure 4.1 gives an overview of the definition of the models and their extensions.

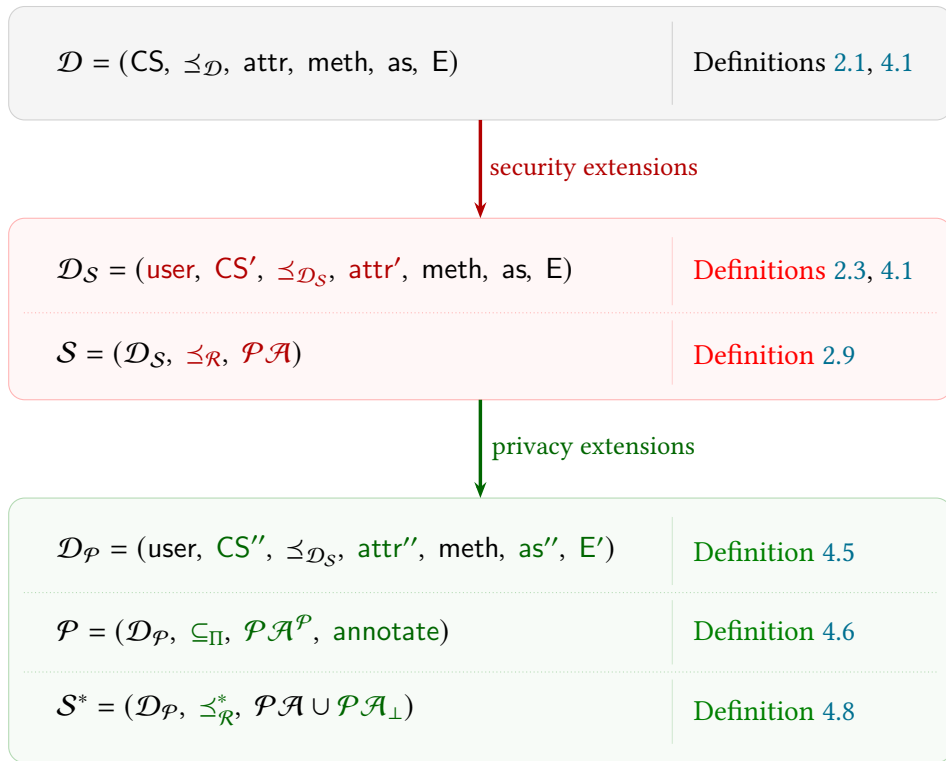


Figure 4.1: Overview of model definitions and extensions introduced in this chapter. Security extensions are highlighted in red; privacy extensions are in green.

4.1 Extending Data with Application Endpoints

In addition to the class methods, we include a notion of application endpoints (or entrypoints) in the data model definition. Unlike class methods, which operate on

objects of a specific class sort, application endpoints define the application's interface. In the context of web applications, application endpoints represent HTTP endpoints that end users access to interact with the web application.

Definition 4.1 (Application Endpoints in Data Model).

A *data model* is now defined as $\mathcal{D} = (CS, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as}, E)$ where CS , $\preceq_{\mathcal{D}}$, attr , meth , and as are defined as in Definition 2.1 and application endpoints $E \subseteq \text{ID} \times \{\text{GET}, \text{POST}, \text{PUT}, \text{DELETE}\}$ are defined as the finite set of pairs of an endpoint name and an HTTP method.

Example 4.1. We extend the event platform described in Example 2.1 by introducing additional class methods and endpoints that can be invoked through HTTP requests.

A Person object can execute `getFullname()` to display its full name, which is a concatenation of its name and surname, and `countAttendingEvents()` to compute the number of events attended. Similarly, an Event object can execute the method `countPureAttendants()` to obtain the number of its attendants, excluding managers and the event owner.

We consider several endpoints related to events, namely: `create_event` (to create an event), `view_event` (to view information about an event), `edit_event` (to edit an event), and `manage_event` (to manage an event). Both `edit_event` and `manage_event` are mapped to PUT since both modify the event resource.

The data model $\mathcal{D}$ in Example 2.2 will then be extended with:

$$\begin{aligned} \text{meth}(\text{Person}) &= \{\text{getFullname} \mapsto \text{Person} \mapsto \text{String}, \\ &\quad \text{countAttendingEvents} \mapsto \text{Person} \mapsto \mathbb{N}\} \\ \text{meth}(\text{Event}) &= \{\text{countPureAttendants} \mapsto \text{Event} \mapsto \mathbb{N}\} \\ E &= \{(\text{create_event}, \text{POST}), (\text{view_event}, \text{GET}), \\ &\quad (\text{edit_event}, \text{PUT}), (\text{manage_event}, \text{PUT})\} \end{aligned}$$

In the remainder of this thesis, we use this definition to refer to a data model $\mathcal{D}$.

4.2 Privacy Model

Purpose limitation is one of the most prominent privacy requirements mandated by the GDPR. It requires that personal data be collected for specified, explicit, and legitimate purposes and not be further processed in a manner incompatible with those purposes. In our setting, we consider the purposes for which personal data, namely, the data provided by the application's users, may be processed. We must first distinguish which data constitute personal data.

4.2.1 Personal Data

In our privacy model, we define personal data at the level of class properties (attributes and association ends), rather than marking entire class sorts as personal. This provides the necessary fine-grained usage control required in practice. For example, a user's name and surname may be personal data, while the information about events they manage may not be, even though all three belong to the Person class sort.

Let $\mathcal{D} = (CS, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as}, E)$ be a data model. We denote by $\text{Props}(C)$ the set of *properties* of a class sort $C \in CS$, defined as the union of its attribute names and its association end names $\text{Props}(C) = \text{dom}(\text{attr}(C)) \cup \{n \mid \exists \alpha \in \text{dom}(\text{as}), n' \in \text{ID}, C' \in CS: \text{as}(\alpha) = ((n, _, C'), (n', _, C)) \vee \text{as}(\alpha) = ((n', _, C), (n, _, C'))\}$.

Definition 4.2 (Personal Data).

Given a data model $\mathcal{D}$, the personal data relation $\text{Props}^{\mathcal{P}}$ identifies, for each class sort, the subset of its properties that constitute personal data.

$$\text{Props}^{\mathcal{P}} \subseteq \bigcup_{C \in CS} \{C\} \times \text{Props}(C)$$

We denote by $\text{Props}^{\mathcal{P}}(C) = \{n \mid (C, n) \in \text{Props}^{\mathcal{P}}\}$ the personal properties of a class sort C . Only actions on personal properties are subject to purpose limitation.

Hence, let $\mathcal{A}^{\mathcal{P}} \subseteq \mathcal{A}$ be the set of actions on personal data defined as:

$$\begin{aligned} \mathcal{A}^{\mathcal{P}} = & \{ \text{read}(C, n), \text{update}(C, n) \mid C \in CS, n \in \text{Props}^{\mathcal{P}}(C) \cap \text{dom}(\text{attr}(C)) \} \\ & \cup \{ \text{read}(C, n), \text{update}(C, n) \mid C, C' \in CS, n \in \text{Props}^{\mathcal{P}}(C), n' \in \text{ID}, \alpha \in \text{dom}(\text{as}): \\ & \quad \text{as}(\alpha) = ((n, 1, C'), (n', _, C)) \vee \text{as}(\alpha) = ((n', _, C), (n, 1, C')) \} \\ & \cup \{ \text{read}(C, n), \text{add}(C, n), \text{remove}(C, n) \mid C, C' \in CS, n \in \text{Props}^{\mathcal{P}}(C), n' \in \text{ID}, \alpha \in \text{dom}(\text{as}): \\ & \quad \text{as}(\alpha) = ((n, *, C'), (n', _, C)) \vee \text{as}(\alpha) = ((n', _, C), (n, *, C')) \}. \end{aligned}$$

Example 4.2. Consider the event platform from Example 4.1. The Person sort has properties name, surname, attends, manages, and events. Suppose that only a subset of these are personal data, e.g., $\text{Props}^{\mathcal{P}} = \{(\text{Person}, \text{name}), (\text{Person}, \text{events})\}$. In other words, we say $\text{Props}^{\mathcal{P}}(\text{Person}) = \{\text{name}, \text{events}\}$, while the other attributes and association ends are not considered personal data.

4.2.2 Purposes

In what follows, we define our notion of purposes. Let Π be a set of basic purpose sorts, which induces the lattice $\subseteq_{\Pi} = (2^{\Pi}, \subseteq)$ of *complex purposes*. A complex purpose is any subset of Π ; the lattice is ordered by set inclusion, so a complex purpose p_1 is more

specific than p_2 whenever $p_1 \subseteq p_2$. For example, *any* purpose is the set Π , *no* purpose is $\emptyset$, and *marketing* purpose is $\{\text{TargetedMarketing}, \text{MassMarketing}\}$.

Example 4.3. Consider again the event platform from Example 4.1. Let the basic purposes be $\Pi = \{\text{CORE}, \text{ANALYTICS}\}$, where additionally ANY includes both CORE and ANALYTICS.

Based on the previous work of Byun et al. [23], we further distinguish two general applications of purposes: declared purpose and actual purpose.

4.2.3 Declared Purposes

A *declared purpose* is a purpose presented to the user in the form of a privacy policy. The set of declared purposes constitutes a purpose-based access control policy that specifies the possible usage of personal data by the application. End-users can control which of these uses they permit, and these choices are enforced during the application's execution, i.e., any access to the personal data must be checked with respect to the declared purposes. In fact, declared purposes, the privacy policy, and the enforcement mechanism must always be synchronized. Therefore, in our model-driven approach, it is sufficient to specify the declared purposes, from which the other two are then automatically generated. Furthermore, whenever the declared purposes change, users can receive a regenerated privacy policy, and the enforcement mechanism consistently changes to enforce the new policy.

To model declared purposes, we emulate the permission assignment relation from the security model (Definition 2.9). The relation $\mathcal{PA}^P$ replaces the role notion in $\mathcal{PA}$ with the purpose:

$$\mathcal{PA}^P \subseteq \bigcup_{a \in \mathcal{A}^P} \Pi \times \{a\} \times C(a).$$

Example 4.4. Consider again the event platform from Example 4.1 with the personal data and purposes defined as in Examples 4.2 and 4.3, respectively. Consider the following privacy policy: *We will read your name and events to display them when you interact with the platform.* The declared purposes for modeling this privacy policy are the tuples $(\text{CORE}, \text{read}(\text{Person}, \text{name}), \top)$ and $(\text{CORE}, \text{read}(\text{Person}, \text{events}), \top)$.

4.2.4 Actual Purposes

A natural question arises: *how should an enforcement mechanism decide whether some data access conforms to the declared purpose?* In contrast to the declared purpose, an *actual purpose* describes how personal data is actually being used. It is difficult to

determine the actual purpose automatically by analyzing the design models we have seen so far.

It must therefore be specified by annotating the application's callable units with purposes. Recall that a data model defines two kinds of callable units: *class methods*, which belong to a specific class sort, and *application endpoints*, which handle HTTP requests and are not bound to any particular class sort. If data access occurs as part of the execution of a callable unit, that unit's annotation is the actual purpose of the data access.

Given a data model $\mathcal{D}$, we denote by $\mathcal{F}$ the application's functions as a union of class methods and endpoints:

$$\mathcal{F} \subseteq \bigcup_{c \in \text{CS}} \text{dom}(\text{meth}(c)) \cup E.$$

Then, we define the actual purposes of the application as a labeling function

$$\text{annotate} : \mathcal{F} \rightarrow 2^\Pi$$

that annotates each element of the application's function for (complex) purposes.

Example 4.5. Consider the event platform from Example 4.1 with the application's function $\mathcal{F} = \{\text{view_event}, \text{edit_event}, \text{countAttendingEvents}, \text{getFullname}, \text{countPureAttendants}\}$. Then, the actual purposes can be the following:

```

annotate(view_event) = {CORE},
annotate(countPureAttendants) = {ANALYTICS},
annotate(edit_event) = {CORE},
annotate(countAttendingEvents) = {ANALYTICS},
annotate(getFullname) = {CORE}.

```

4.2.5 Data Ownership

Even if declared purposes are presented to the user and enforced in the application, data protection regulations further require that personal data be processed only when the data subject has provided the required consent. In this thesis, we use the terms *data subject* and *data owner* interchangeably to refer to the individual to whom the personal data relates and whose consent is required for processing. Before modeling consent, we must therefore capture the relationship between a personal data item and its owner.

Consent is expressed at the data model level: a user consents to have property n of class sort C used for a purpose $\pi \in \Pi$. Ownership, however, operates at the object model level: the concrete value of property n in a particular object o of sort C belongs to a specific user u . To bridge these two levels, we introduce a dedicated class sort `PersonalData` that reifies the notion of a personal data resource. Each instance of `PersonalData` records the

class sort and property name of a personal data item. The ownership is then modeled as an association between `PersonalData` and the `User` sort.

Definition 4.3 (Personal Data sort).

Given $\mathcal{D}_S = (\text{user}, CS', \preceq_{\mathcal{D}_S}, \text{attr}', \text{meth}, \text{as}, E)$ the security extension of the data model, let $\text{Props}^{\mathcal{P}}$ be a personal data relation (Definition 4.2). The `PersonalData` sort is a class sort with

- attributes $\text{attr}(\text{PersonalData}) = \{\text{entity} \mapsto \text{String}, \text{property} \mapsto \text{String}\}$, identifying the class sort and the property name of the personal data item;
- $\text{as}(\text{data_ownership}) = ((\text{owner}, 1, \text{User}), (\text{data}, *, \text{PersonalData}))$ association relating each instance of personal data to its owner; and
- for each sort $C \in CS_{\neq \emptyset}^{\mathcal{P}}$ (where $CS_{\neq \emptyset}^{\mathcal{P}} = \{C \in CS \mid \text{Props}^{\mathcal{P}}(C) \neq \emptyset\}$), an association $\text{as}(\text{pdata}_C) = ((C_source, 1, C), (\text{pdata}, *, \text{PersonalData}))$.

Every object of a sort $C \in CS_{\neq \emptyset}^{\mathcal{P}}$ is linked, via the pdata_C association, to one or more `PersonalData` objects, each of which records the owner of a specific personal property.

4.2.6 Consent

In practice, users' consent is often implicit: by using a website, they tacitly agree to its privacy policy. We aim to make this consent collection step explicit. Users can partially consent to or reject individual declared purposes. Consent can be provided *lazily* (e.g., when a user performs an action that requires it) and can be revoked at any time.

Definition 4.4 (Consent).

The sort `Consent` is a class sort with:

- an attribute $\text{attr}(\text{Consent}) = \{\text{purposes} \mapsto 2^{\Pi}\}$, recording the (complex) purposes to which the user has consented;
- an association to `User`:
 $\text{as}(\text{user_consents}) = ((\text{user}, 1, \text{User}), (\text{consents}, *, \text{Consent}))$; and
- an association to `PersonalData`:
 $\text{as}(\text{data_consents}) = ((\text{data}, 1, \text{PersonalData}), (\text{consents}, *, \text{Consent}))$.

An instance of `Consent` represents the fact that the user identified by `user` has consented to the personal data item identified by `data` being used for the purposes recorded in `purposes`.

Example 4.6. Suppose that Alice has consented to have her name used for `ANALYTICS`. This is represented by a `Consent` object c such that $c.\text{user} = \text{Alice}$, $c.\text{data}.\text{entity} = \text{Person}$, $c.\text{data}.\text{property} = \text{name}$, and $c.\text{purposes} = \{\text{ANALYTICS}\}$. Alice may independently refuse consent for her surname.

4.2.7 Putting Things Together

We now combine the above notions into our extension of the data model.

Definition 4.5 (Privacy extension of data model).

Let $\mathcal{D}_S = (\text{user}, \text{CS}', \preceq_{\mathcal{D}_S}, \text{attr}', \text{meth}, \text{as}, \text{E})$ be the security extension of the data model. The privacy extension of the data model, $\mathcal{D}_P$, is a tuple $(\text{user}, \text{CS}'', \preceq_{\mathcal{D}_S}, \text{attr}'', \text{meth}, \text{as}'', \text{E}')$ where:

- $\text{CS}'' = \text{CS}' \cup \{\Pi, \text{PersonalData}, \text{Consent}\}$, where Π , PersonalData , and Consent are purpose, personal data, and consent sorts, respectively.
- $\text{attr}'' = \text{attr}' \cup \text{attr}(\text{PersonalData})$ (PersonalData's attributes, Def. 4.3)
 $\cup \text{attr}(\text{Consent})$ (Consent's attributes, Def. 4.4)
 $\cup \{(\Pi, \{\text{name} \mapsto \text{String}\})\}$; (purpose name)
- $\text{as}'' = \text{as} \cup \{\text{data_ownership}, \text{user_consents}, \text{data_consents}\} \cup \{\text{pdata}_C \mid C \in \text{CS}'_{\neq \emptyset}\}$, where data_ownership , user_consents , data_consents , and pdata_C are the associations defined in Definitions 4.3 and 4.4;
- $\text{E}' = \text{E} \cup \{(\text{privacy_policy}, \text{GET}), \quad (\text{to view the privacy policy})$
 $(\text{grant_consent}, \text{POST}), \quad (\text{to grant a consent})$
 $(\text{revoke_consent}, \text{DELETE})\}. \quad (\text{to revoke a consent})$

Example 4.7. Consider the Event Management Platform $\mathcal{D}$ in Example 4.1. Figure 4.2 presents the resulting data model extension. The security extension adds the role attribute to Person along with the Role enumeration. The privacy extension marks the personal properties name and events in green and introduces the new privacy class sorts and associations following Definitions 4.3 and 4.4.

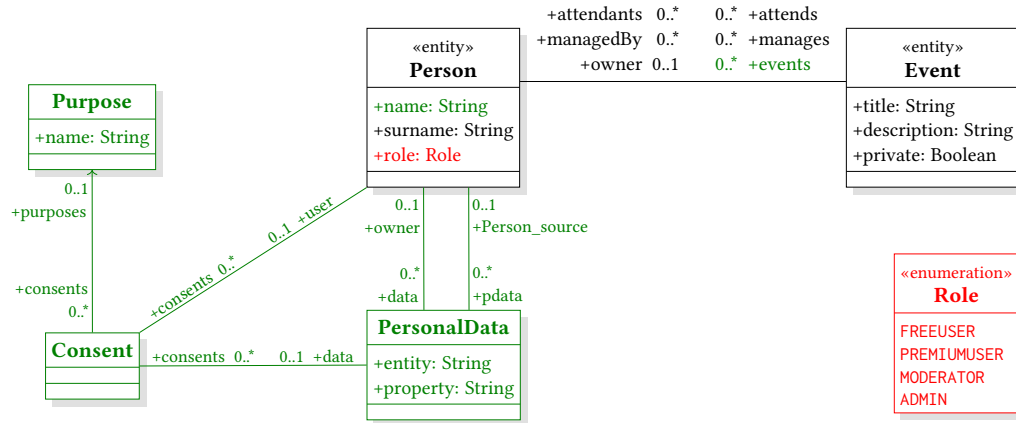


Figure 4.2: UML class diagram of the Event Management Platform, extended with **security** and **privacy** concepts.

Finally, we formally define our privacy model.

Definition 4.6 (Privacy Model).

Given a data model $\mathcal{D}$ and a security model $\mathcal{S}$, the *privacy model* $\mathcal{P}$ is a quadruple

$$(\mathcal{D}_{\mathcal{P}}, \subseteq_{\Pi}, \mathcal{PA}^{\mathcal{P}}, \text{annotate})$$

where $\mathcal{D}_{\mathcal{P}}$ is the privacy extension of the data model (Definition 4.5), $\subseteq_{\Pi} = (2^{\Pi}, \subseteq)$ is the lattice of complex purposes, $\mathcal{PA}^{\mathcal{P}}$ is the declared purpose relation, and $\text{annotate} : \mathcal{F} \rightarrow 2^{\Pi}$ is the actual purpose labeling function.

Privacy enforcement mechanism. Intuitively, an enforcement mechanism for a privacy model ensures that (i) the actual purpose always conforms to the declared purpose and that (ii) the owner of the accessed personal data has consented to every actual purpose. More precisely, given an object model $\mathcal{O}$, the privacy enforcement mechanism allows a user u to perform an action a on a personal property $(C, n) \in \text{Props}^{\mathcal{P}}$ of an object o of sort C , that (possibly) updates o with value v as part of the execution of $f \in \mathcal{F}$, if and only if

- $\text{annotate}(f) \subseteq \{p \mid (p, a, \varphi) \in \mathcal{PA}^{\mathcal{P}} \wedge \varphi[\text{self}/o][\text{caller}/u][\text{value}/v]\}$ in the context of the object model $\mathcal{O}$; and,
- $\exists d \in o.\text{pdata}, \exists c \in d.\text{consents}$ such that $d.\text{entity} = C \wedge d.\text{property} = n \wedge c.\text{user} = d.\text{owner} \wedge \text{annotate}(f) \subseteq c.\text{purposes}$.

4.3 Security Permissions for Personal Data

The GDPR grants data subjects specific access control rights over their personal data. More specifically, the *right of access* (Art. 15) and the *right to rectification* (Art. 16) allow data subjects to read and correct their own personal data.

In the current approach, a developer could define these permissions in the security model. However, this is both error-prone and repetitive. On the one hand, developers may forget to include the required permissions for certain personal properties. On the other hand, the security model becomes cluttered with boilerplate entries in applications with many personal data properties.

We extend our approach to avoid these problems by automatically deriving a set of *base permissions* from the privacy model. These permissions are then merged with the application's security model at the code generation step.

4.3.1 Base Security Model

In a nutshell, the base permissions allow every data owner to perform actions on their personal data. Since ownership is tracked through the `PersonalData` sort (Definition 4.3), the authorization constraint in each base permission navigates from the accessed object to its associated `PersonalData` instance and verifies that the caller is the owner. Concretely, for a personal property $(C, n) \in \text{Props}^{\mathcal{P}}$, the constraint verifies that there exists a `PersonalData` instance linked to the accessed object whose property matches n and whose owner is the caller.

As these permissions must be available to every authenticated role, we assign them to a synthetic role $r_{\perp} \notin \mathcal{R}$ that sits below all existing roles in the hierarchy. Every role inherits the permissions of $r_{\perp}$, ensuring that all users can access their own personal data regardless of their assigned role.

Definition 4.7 (Base Permissions).

Let $\mathcal{P} = (\mathcal{D}_{\mathcal{P}}, \subseteq_{\Pi}, \mathcal{PA}^{\mathcal{P}}, \text{annotate})$ be a privacy model and let $r_{\perp} \notin \mathcal{R}$ be a synthetic role. The *base permissions* are defined as:

$$\mathcal{PA}_{\perp} = \bigcup_{(C,n) \in \text{Props}^{\mathcal{P}}} \{r_{\perp}\} \times \{a \mid a \in \mathcal{A}, a \text{ acts on } (C, n)\} \times \{\varphi_{\text{own}}^{C,n}\},$$

and the authorization constraint $\varphi_{\text{own}}^{C,n}$ is the OCL Boolean expression `self.pdata → exists( $d \mid d.\text{entity} = C$  and  $d.\text{property} = n$  and  $d.\text{owner} = \text{caller}$ ).`

Example 4.8. Consider $\text{Props}^{\mathcal{P}} = \{(\text{Person}, \text{name}), (\text{Person}, \text{events})\}$ from Example 4.2. The base permissions include:

$$\begin{aligned} &(r_{\perp}, \text{read}(\text{Person}, \text{name}), \varphi_{\text{own}}^{\text{Person}, \text{name}}), \\ &(r_{\perp}, \text{update}(\text{Person}, \text{name}), \varphi_{\text{own}}^{\text{Person}, \text{name}}), \\ &(r_{\perp}, \text{read}(\text{Person}, \text{events}), \varphi_{\text{own}}^{\text{Person}, \text{events}}), \\ &(r_{\perp}, \text{add}(\text{Person}, \text{events}), \varphi_{\text{own}}^{\text{Person}, \text{events}}), \\ &(r_{\perp}, \text{remove}(\text{Person}, \text{events}), \varphi_{\text{own}}^{\text{Person}, \text{events}}). \end{aligned}$$

For example, $\varphi_{\text{own}}^{\text{Person}, \text{name}}$ expands to the OCL constraint: `self.pdata → exists( $d \mid d.\text{entity} = \text{"Person"}$  and  $d.\text{property} = \text{"name"}$  and  $d.\text{owner} = \text{caller}$ ).`

4.3.2 Merging Security Models

The base permissions (Definition 4.7) use a synthetic role $r_{\perp} \notin \mathcal{R}$. To incorporate these permissions into the application's security model $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$, we extend the

role hierarchy $\preceq_{\mathcal{R}}$ to a partial order $\preceq_{\mathcal{R}}^*$ on $\mathcal{R}^* = \mathcal{R} \cup \{r_{\perp}\}$ by adding $r_{\perp}$ as the minimum element: $r_1 \preceq_{\mathcal{R}}^* r_2$ if and only if $r_1 \preceq_{\mathcal{R}} r_2$ or $r_1 = r_{\perp}$.

In the Hasse diagram (Figure 4.3, left side), this amounts to placing $r_{\perp}$ directly below the leaf roles, i.e., the minimal elements $\text{Leaves}(\preceq_{\mathcal{R}}) = \{r \in \mathcal{R} \mid \nexists r' \in \mathcal{R}. r' \preceq_{\mathcal{R}} r \wedge r' \neq r\}$. Since $r_{\perp}$ is the minimum of $\preceq_{\mathcal{R}}^*$, the base permissions assigned to $r_{\perp}$ are inherited by every role, ensuring that all users, regardless of their assigned role, are authorized to access their own personal data.

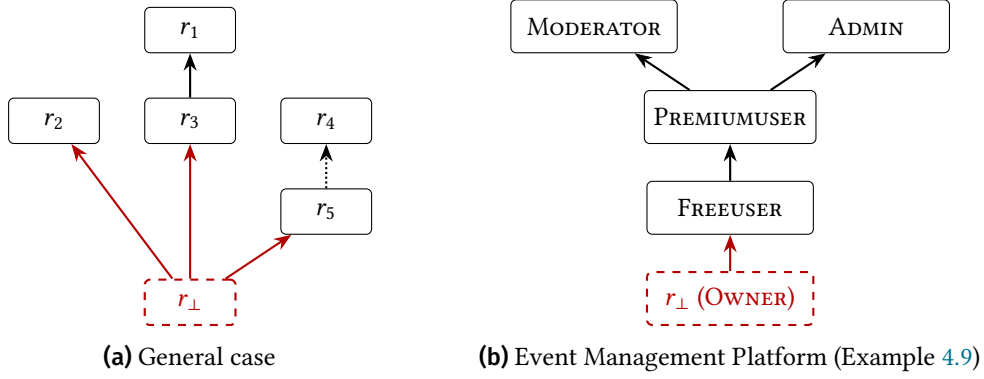


Figure 4.3: Extended role hierarchy $\preceq_{\mathcal{R}}^*$ with the base role $r_{\perp}$. The dotted arrow in (a) indicates that intermediate roles may exist between r_4 and r_5 .

Definition 4.8 (Combined Security Model).

Given a security model $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$ and the base permissions $\mathcal{PA}_{\perp}$ for a synthetic role $r_{\perp} \notin \mathcal{R}$ (Definition 4.7), the *combined security model* is the triple

$$\mathcal{S}^* = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}^*, \mathcal{PA} \cup \mathcal{PA}_{\perp}),$$

where $r_1 \preceq_{\mathcal{R}}^* r_2$ if and only if $r_1 \preceq_{\mathcal{R}} r_2$ or $r_1 = r_{\perp}$.

Example 4.9. Consider the event platform with the role hierarchy from Example 2.8: $\text{FREEUSER} \preceq_{\mathcal{R}} \text{PREMIUMUSER}$, $\text{PREMIUMUSER} \preceq_{\mathcal{R}} \text{MODERATOR}$, and $\text{PREMIUMUSER} \preceq_{\mathcal{R}} \text{ADMIN}$. The only leaf role is FREEUSER , so $r_{\perp}$ (OWNER) is placed directly below it in the Hasse diagram (Figure 4.3, right side).

Since $r_{\perp}$ is the minimum of $\preceq_{\mathcal{R}}^*$, every role inherits the base permissions. The combined security model $\mathcal{S}^*$ then includes both the developer-defined permissions $\mathcal{PA}$ (e.g., allowing a FREEUSER to read a name) and the base permissions $\mathcal{PA}_{\perp}$ (e.g., allowing any user to read their own personal data).

vActionGUI: Privacy-aware Flask Application Generator

We now introduce the vActionGUI (vAG, for short) tool and describe our development methodology. vActionGUI [96] implements a concrete instance of the model-driven privacy methodology presented in this thesis. More specifically, it takes data, security, and privacy models as input and automatically generates a data-centric web application that enforces fine-grained access control, purpose limitation, and user consent.

vActionGUI is distributed as a Docker container and is publicly available at [96]. A detailed description of the usage and development workflow is provided in Appendix D.

Table 5.1: Model-driven privacy methodology via vActionGUI.

Developers define	vActionGUI generates
Step 1 Data model (§5.1.1)	Data classes (§5.2.1) & method stubs (§5.2.2)
Step 2 Security model (§5.1.2) – user class	Security enforcement mechanism (§5.2.3) Authentication (§5.2.1)
Step 3 Privacy model (§5.1.3) – personal data denotation – purposes and their hierarchy – declared purposes – actual purpose mapping	Privacy enforcement mechanism (§5.2.5) Privacy entity (§5.2.1) & helper methods (§5.2.2) Database configuration (§5.2.1) Privacy notice & dashboard (§5.2.4) Consent collection mechanism (§5.2.4) Purpose tracking mechanism (§5.2.2)
Step 4 Developers implement method stubs & presentation layers	

In general, our model-driven approach involves two main activities: *designing models* in which developers specify different aspects of the application as models and *implementing functions* in which developers implement the application’s business logic. In practice, the same person may perform both activities. Table 5.1 presents the main development activities of our methodology using this tool.

Step 1 Data modeling. The developers start by specifying the *data model* that captures the entities, attributes, associations, and operations of the application. Entity methods define business logic operations, and application endpoints define the HTTP entry points that users can invoke.

Example 5.1. Consider the event platform from Example 4.1. The data model defines two class sorts, Person and Event, with attributes such as title and private, and associations such as owner linking events to their creators. Moreover, the developer defines several endpoints, e.g., view_event, and entity methods for the application. The formal data model is given in Example 4.1.

Step 2 Security modeling. The developers then specify the *security model* that declares (i) the user entity, (ii) the security roles that users may hold, and (iii) the fine-grained access control policies associated with those roles. Each policy is an authorization constraint expressed as an OCL boolean expression.

Example 5.2. The developers may define roles such as FREEUSER and PREMIUMUSER and specify access control policies for these roles. For instance, a user with the FREEUSER role may read the title of an event only if the event is public or the user is the event's owner. The formal security model is given in Example 2.8.

Step 3 Privacy modeling. The developers specify the *privacy model*, which declares (i) which entity properties constitute personal data, (ii) the purposes for which personal data may be processed, (iii) the privacy policy, and (iv) the actual purposes for each endpoint and entity method.

Example 5.3. The developer may denote certain properties (e.g., a person's name) as personal data and declare that they may be read for the purpose CORE. Each endpoint is annotated with its actual purpose, e.g., annotate(view_event) = {CORE}. The formal privacy model is given in Examples 4.4 and 4.5.

vActionGUI takes all three aforementioned models as input and generates a Flask [43] web application. The generated application includes a database schema, the routes and methods of the application, a functional authentication mechanism, a privacy notice, a privacy dashboard, a consent management interface, and the enforcement mechanisms for both security and privacy policies. These artifacts are automatically generated and correspond to the characterization of the specified models. The implementation bodies of the routes and methods of the application are intentionally left empty.

Example 5.4. The code generator produces the view_event endpoint route in app.py, guarded by the generated @secure decorator that triggers the security and privacy enforcement mechanisms

```
1 @app.route('/view_event', methods=['POST', 'GET'])
2 @secure(db, P(['CORE']))
3 def view_event():
4     from project import view_event
5     return view_event(request)
```

that calls an implementation method with a method stub in `project.py`

```
1 def view_event(request):
2     # TODO: implement the method stub
3     return "TODO: Implement the method stub"
```

Note that the code generator emits both GET and POST methods for every endpoint route by default, so that a single endpoint can serve both the initial page request and subsequent form submissions. The generated route in `app.py` is not intended to be modified by the developer. Instead, the developer completes the corresponding method stub in `project.py` by implementing the application-specific business logic.

Step 4 Implementation. Finally, the developers implement the stubs with the application-specific logic. Here, it is worth noting that developers do not need to write any enforcement code because the policy enforcement is already generated and will be enforced as a cross-cutting concern when the application runs. The developers can further implement the application presentation layer. During execution, calls to the generated route are still mediated by the `@secure` decorator, which records the actual purpose and triggers the generated security and privacy enforcement mechanisms.

Example 5.5. The developer implements the `view_event` endpoint by querying the event and rendering it.

```
1 def view_event(request):
2     # TODO: implement the method stub
3     event_id = request.args.get('event_id')
4     event = Event.query.get(event_id)
5     etitle = event.title
6     eowner = event.owner.getFullname()
7     return "TODO: Implement the method stub"
8     return render_template("view_event.html", title=etitle, owner=eowner)
```

which will then be displayed on the implemented page `view_event.html`.

We now define the modeling languages used to specify `vActionGUI` models.

5.1 Modeling Languages

`vActionGUI` provides developers with modeling languages for specifying different aspects of the system under development, namely the state space of the application (via a data modeling language), fine-grained access control (via a security modeling language), and privacy policies (via a privacy modeling language). In the remainder of this section, we define each of these languages in detail.

5.1.1 Data Modeling Language

The data modeling language defines the structure and operations of the application domain. At an abstract level, it is an instance of the metamodel described in Section 2.2.1. We present the grammar that defines the textual syntax of the data modeling language.

In this language, application entities and endpoints are treated as first-class constructs. An entity is declared using the `entity` keyword and models application data in an object-oriented manner, similar to languages such as Java or C++. The properties of an entity include attributes and association ends. Both are declared with a name and a type, which may be a primitive type (e.g., Boolean, Integer, Real, String), an entity type, or a collection type (e.g., Set, Bag, OrderedSet, Sequence). Association ends must additionally specify either the name of the opposite end or the name of the association. An entity may also define methods that are invoked during the execution of the application. Application endpoints are methods that handle HTTP requests. They are declared as a collection of names under the `endpoints` keyword.

vActionGUI Data modeling language

```

<DataModel> ::= <Component>*
<Component> ::= <Entity> | <Enum> | <Endpoints>
  <Entity> ::= entity <EntityName> { <Property>* }
  <Enum> ::= enum <EnumName> { <EnumLiteral>* }
<Endpoints> ::= endpoints { <Function>* }
  <Property> ::= <Attribute> | <AssocEnd> | <Method>
  <Function> ::= <FuncName>()
  <Attribute> ::= <Type> <AttrName>
  <AssocEnd> ::= <Type> <EndName> (oppositeTo <EndName> | in <AssocName>)
  <Method> ::= <MethodName>()
  <Type> ::= <PrimitiveType> | <EntityName> | <CollectionType>

<CollectionType> ::= (Set | Bag | OrderedSet | Sequence) ( <Type> )
<PrimitiveType> ::= String | Integer | Real | Boolean
  <Identifier> ::= [a-zA-Z_][a-zA-Z0-9_]*
  <EnumLiteral> ::= [A-Z]+
  <EntityName>, <EnumName>, <AttrName>, <EndName>,
  <AssocName>, <FuncName>, <MethodName> are <Identifier>s.

```

Example 5.6. The following listing presents the textual representation of the extended data model described in Example 4.1. The `Person` entity defines three association ends. The first, `events`, is declared through the `Ownership` association (line 5), while the other two are specified via their corresponding opposite association ends (lines 6–7).

Unlike in Definition 2.1, our data modeling language simplifies the declaration of these methods and endpoints. For example, `getFullname()` (line 9) and `countAttendingEvents()` (line 10) only contain the method names. Similarly, the application endpoints only contain their name but not their accompanying HTTP methods (lines 25–30).

```

1  entity Person {
2    String name
3    String surname
4
5    Set(Event) events in Ownership
6    Set(Event) manages oppositeTo managedBy
7    Set(Event) attends oppositeTo attendants
8
9    getFullname()
10   countAttendingEvents()
11 }
12
13 entity Event {
14   String title
15   String description
16   Boolean private
17
18   Person owner in Ownership
19   Set(Person) attendants oppositeTo attends
20   Set(Person) managedBy oppositeTo manages
21
22   countPureAttendants()
23 }
24
25 endpoints {
26   create_event()
27   view_event()
28   edit_event()
29   manage_event()
30 }

```

5.1.2 Security Modeling Language

The security modeling language defines the access control policy of the application in terms of the allowed actions on the resources provided by the data model. The concrete syntax of the security modeling language is governed by the grammar shown below.

The user entity represents the instance type that interacts with the application. In the security model, at most one entity from the data model is designated as the user class.

Roles model jobs or functions within the system. Each role groups the permissions required to fulfill a specific function. Users of the system are assigned one of these

vActionGUI Security modeling language

```

<SecurityModel> ::= <UserDecl> <AnonRoleDecl> <DefaultRoleDecl> <RoleDecl>*
  <UserDecl> ::= USER <EntityName>
  <AnonRoleDecl> ::= anonymous <RoleDecl>
  <DefaultRoleDecl> ::= default <RoleDecl>
  <RoleDecl> ::= Role <RoleName> (extends <RoleName>)? { <Resrc>* }
  <Resrc> ::= <EntityName> { (<ResrcPerm> | <SubResrcPerm>)* }
  <ResrcPerm> ::= <ActionType> ([ <OCLBExpr> ])?
  <SubResrcPerm> ::= <SubResrcPermActions> ([ <OCLBExpr> ])?
  <SubResrcPermActions> ::= <SubResrcPermAction> (, <SubResrcPermAction>)*
  <SubResrcPermAction> ::= <SubResrcAction> ( <PropertyName> )
  <ActionType> ::= <ResrcAction> | <SubResrcAction>
  <ResrcAction> ::= fullAccess | create | delete
  <SubResrcAction> ::= read | update | add | remove | execute
  <Identifier> ::= [a-zA-Z_][a-zA-Z0-9_]*
  <RoleName> ::= [A-Z]+
  <EntityName>, <PropertyName> are all <Identifier>s.
  <OCLBExpr> denotes an OCL Boolean expression.

```

roles. The roles are declared using the **Role** keyword, followed by the role's name and its permissions.

For every security model defined by our language, developers must assign two special roles. The anonymous role applies to unauthenticated visitors and typically grants read access to public resources. The **default** role applies to users once they register for the application. Roles may extend one another via the **extends** keyword, inheriting the permissions of the parent role in addition to their own.

For each entity in the underlying data model, at most one permission may be declared per role. Each permission represents the authorization to execute operations on a subset of objects in the system. Permissions are grouped by the entities defined in the data model. A permission has the form *action(subresource) [constraint]*, where the *subresource* may refer to an attribute or association end of the entity under which the permission is defined, and the *constraint* is an Object Constraint Language (OCL) Boolean expression. If the *subresource* is omitted, the action is interpreted as a composite action that applies to all subresources of the entity. If the *constraint* is omitted, it is assumed to be the OCL expression **true**.

Example 5.7. The following listing presents a sample security model for the EventPlatform.

- The anonymous role permits unauthenticated visitors to read the title and description of public events.
- Authenticated users, represented by the default FREEUSER role, inherit the anonymous role's permissions. They may additionally create up to five events and delete their own events.
- The PREMIUMUSER role extends FREEUSER and allows users to create an unlimited number of events, including private ones.
- The MODERATOR role inherits the permissions of PREMIUMUSER.
- The ADMIN role extends the PREMIUMUSER role and has full access to Event instances.

Compared with the running example from Chapter 2, this model introduces the anonymous and default FREEUSER roles required by the language.

```

1  USER Person
2
3  anonymous Role VISITOR {
4    Event {
5      read(title), read(description) [not self.private]
6    }
7  }
8
9  default Role FREEUSER extends VISITOR {
10   Event {
11     create [caller.events->size() < 5]
12     delete [self.owner = caller]
13   }
14 }
15
16 Role PREMIUMUSER extends FREEUSER {
17   Event {
18     create
19     read
20   }
21 }
22
23 Role MODERATOR extends PREMIUMUSER {
24 }
25
26 Role ADMIN extends PREMIUMUSER {
27   Event {
28     fullAccess
29   }
30 }

```

5.1.3 Privacy Modeling Language

The privacy modeling language specifies the data-usage policy governing how personal data may be processed. A privacy model identifies the application's personal data and purposes, declares the purposes for which personal data may be used and under what conditions, and associates application methods with purposes. The grammar of the privacy modeling language is shown below. The privacy model is structured into four blocks.

Personal data. This block enumerates the class attributes that constitute personal data subject to the policy.

Purposes. This block defines the purpose vocabulary and its inclusion hierarchy. A purpose that *includes* another is considered more general; thus, consenting to a more general purpose implies consent to all purposes it includes.

Actual purposes. This block maps each application method, either the entity method or the application endpoint, to the purpose it serves at runtime.

Declared purposes. This block defines the consent items presented to the user, pairing each item with an OCL consent condition, an end-user description for the consent interface, and the corresponding purpose.

vActionGUI Privacy modeling language

```

<PrivacyModel> ::= <PdataBlock> <PurpBlock> <ActualBlock> <DeclaredBlock>
  <PdataBlock> ::= Personal data { <PdataDecl>* }
  <PurpBlock> ::= Purposes { <PurposeDecl>* }
  <ActualBlock> ::= Actual purposes { <ActualPurpDecl>* }
  <DeclaredBlock> ::= Declared purposes { <DeclaredPurp>* }
  <PdataDecl> ::= <EntityName> | <EntityName>.<PropertyName>
  <PurposeDecl> ::= <Purpose> | <Purpose> includes <Purpose>+
  <ActualPurpDecl> ::= <FunctionDecl> <Purpose>
  <FunctionDecl> ::= <EntityName>.<MethodName> | <EndpointName>
  <DeclaredPurp> ::= <PdataDecl> <Purpose> ([ <OCLBExpr> ] <Description> >)?
  <Identifier> ::= [a-zA-Z_][a-zA-Z0-9_]*
  <Purpose> ::= [A-Z]+
  <EntityName>, <PropertyName>, <MethodName>, <EndpointName> are all <Identifier>s.
  <OCLBExpr> denotes an OCL Boolean expression.
  <Description> denotes a free-text string.

```

Example 5.8. Extending the personal data of Example 4.2 (the name attribute and events association end), we additionally designate the surname attribute and the attends association end of a Person object as personal data.

We extend Example 4.4 to the following declared purpose policy:

- *We will read your name, surname, and events to display them when you interact with the platform.*
- *We will read your event's attendance information for analytical purposes if you are a user with the FREEUSER role.*

Three purposes are declared in the hierarchy: ANY, CORE, and ANALYTICS, where ANY includes the other two.

By Example 4.5, the Actual purposes block records that countAttendingEvents() and countPureAttendants() access data for the purpose of ANALYTICS, while endpoints such as view_event and edit_event belong to the core functionality of the application and, during page rendering, display the full name of the event's owner via the getFullname method.

The following listing presents a sample privacy model for the EventPlatform.

```

1  Personal data {
2    Person.name
3    Person.events
4    Person.surname
5    Person.attends
6  }
7
8  Purposes {
9    CORE
10   ANALYTICS
11   ANY includes CORE ANALYTICS
12 }
13
14 Actual purposes {
15   Person.countAttendingEvents ANALYTICS
16   Person.getFullname CORE
17   Event.countPureAttendants ANALYTICS
18   view_event CORE
19   edit_event CORE
20 }
21
22 Declared purposes {
23   Person.name CORE
24   Person.events CORE
25   Person.surname CORE
26   Person.attends ANALYTICS [self.role = Role::FREEUSER]
27                               <you are a user with the FREEUSER role>
28 }

```

5.2 Generated Artifacts

We now describe each category of the generated artifacts.

5.2.1 Database Schema and Entity Classes

From the extended data model, *vActionGUI* generates SQLAlchemy [126] model classes. SQLAlchemy then maps these classes to the corresponding relational database schema.

Authentication. From the USER declaration and the role definitions in the security model, *vActionGUI* extends the user entity class with authentication fields (such as username and password), and integrates the Flask-User [130] class for registration, login, and session management. In addition, each generated entity class is decorated with a `@Secure` decorator, which intercepts attribute and association end accesses to enforce the security and privacy policies, as described in Sections 5.2.3 and 5.2.5.

Example 5.9. The following listing shows an excerpt of the generated Person class. Each property maps either to a database column or to a property that returns a collection. Since Person is the user entity, it is additionally extended with Flask-User's UserMixin class and includes role and authentication fields.

```

1  @Secure(securityModel,privacyModel)
2  class Person(db.Model,UserMixin):
3      id      = db.Column(db.Integer, primary_key=True)
4      username = db.Column(db.String(100), nullable=False, unique=True)
5      password = db.Column(db.String(255), nullable=False)
6      name     = db.Column(db.String(100))
7      surname  = db.Column(db.String(100))
8      roles    = db.relationship('Role', secondary='user_roles')
9      @property
10     def events(self):
11         return eventsSet(self, {x.events for x in self.association_events_owner})
12     @property
13     def role(self):
14         return self.roles[0] if self.roles else None

```

Privacy classes. From the privacy model, *vActionGUI* also generates classes for purposes, personal data, and consent. The purposes defined in the privacy model are inserted into the corresponding database table whenever the database is initialized. A PersonalData entry represents a pair of entity and property declared as personal in the privacy model. In the generated class, the resource and subresource columns correspond to the entity and property of Definition 4.3, and type records the type of the

subresource. Finally, a Consent record links a user to a personal data item and to the set of purposes for which the user has given consent.

Example 5.10. The following listing shows the generated privacy classes.

```

1  class Purpose(db.Model):
2      id          = db.Column(db.Integer(), primary_key=True)
3      name        = db.Column(db.String(50), unique=True)
4      parent_id   = db.Column(db.Integer, db.ForeignKey('purpose.id'))
5      subpurposes = db.relationship('Purpose')
6
7  class PersonalData(db.Model):
8      id          = db.Column(db.Integer(), primary_key=True)
9      resource     = db.Column(db.String(100))
10     subresource  = db.Column(db.String(100))
11     type         = db.Column(db.String(100))
12
13 class Consent(db.Model):
14     id          = db.Column(db.Integer(), primary_key=True)
15     user_id     = db.Column(db.Integer, db.ForeignKey('person.id'))
16     user        = db.relationship('Person', foreign_keys=user_id)
17     data_id     = db.Column(db.Integer, db.ForeignKey('personal_data.id'))
18     data        = db.relationship('PersonalData', foreign_keys=data_id)
19     purposes    = db.relationship('Purpose', secondary=consentedpurposes)

```

5.2.2 Application Routes and Method Stubs

vAG generates a Flask route for each application endpoint. Each route is wrapped with a `@secure` decorator that declares the endpoint's actual purposes, corresponding to the label function in the privacy model. The generated routes are placed in `app.py`, which is intended to remain unmodified. Each route delegates to a corresponding method stub in `project.py`, where the developer implements the business logic.

The same pattern is used for entity methods. Each generated entity method is wrapped with a `@secure` decorator declaring its actual purpose or purposes. The method itself is placed in the corresponding data class in `dtm.py`, and the method calls a generated method stub in `project_aux.py`. Here, the developer replaces the stub with the method's business logic implementation.

vActionGUI generates four important files. The files `app.py`, which contains endpoint routes, and `dtm.py`, which contains data classes and their methods, are generated infrastructure and are not intended to be modified by the developer. The files `project.py`, containing endpoint stubs, and `project_aux.py`, containing method stubs, are completed by the developer with the application's business logic.

Example 5.11. The following shows the generated code for the `view_event` endpoint. In the `app.py` file, the route is generated with a `@secure` decorator that is labeled with the CORE purpose. It calls a function with the same name inside `project.py`, where the developer implements the business logic.

Generated route (`app.py`)

```
1 @app.route('/view_event', methods=['POST', 'GET'])
2 @secure(db, P(['CORE']))
3 def view_event():
4     from project import view_event
5     return view_event(request)
```

Method stub to be filled in by the developer (`project.py`)

```
1 def view_event(request):
2     # TODO: implement the method stub
3     return "TODO: Implement the method stub"
```

The method `countAttendingEvents` is generated as a method in `dtm.py` that delegates to a stub in `project_aux.py`.

Generated class method (`dtm.py`)

```
1 class Person(db.Model, UserMixin):
2     ...
3     @secure(db, P(['ANALYTICS']))
4     def countAttendingEvents(self):
5         from project_aux import countAttendingEvents
6         return countAttendingEvents(self)
```

Method stub to be filled in by the developer (`project_aux.py`)

```
1 def countAttendingEvents(self):
2     # TODO: implement the method stub
3     return "TODO: Implement the method stub"
```

5.2.3 Security Enforcement

Data action mapping. *vActionGUI* maps the data actions defined in Definition 2.8 to concrete operations in the generated Python classes. Table 5.2 summarizes this mapping. These operations are the points at which the generated application accesses or modifies protected data. To enforce security and privacy policies, *vActionGUI* instruments the corresponding methods so that they invoke the enforcement mechanisms implementing the respective models' semantics.

For example, for each sort $C \in \text{CS}$ and attribute $at \in \text{dom}(\text{attr}(C))$, the getter and setter methods for the property at in the class C correspond to actions $\text{read}(C, at)$ and $\text{update}(C, at)$, respectively.

Table 5.2: Mapping from data actions (Definition 2.8) to Python operations.

Action	Applies to	Python operation	Example (Person class)
create(C)	Entity class	$C()$	$\text{Person}()$
delete(C)	Entity class	$\text{db.session.delete}(o)$	$\text{db.session.delete}(p)$
read(C, n)	Attribute / assoc.	$o.n$	$p.name$
update(C, n)	Attribute / to-one	$o.n = v$	$p.name = \text{"Alice"}$
add(C, n)	To-many assoc.	$o.n.append(e)$	$p.events.append(e)$
remove(C, n)	To-many assoc.	$o.n.remove(e)$	$p.events.remove(e)$
execute(C, n)	Entity method	$o.n(request)$	$p.countAttendingEvents()$

Enforcement mechanism. $\nu\text{ActionGUI}$ instruments these operations using two decorators: `@Secure`, a class decorator applied to generated entity classes, and `@secure`, a method decorator applied to generated routes and entity methods. The two decorators are distinguished only by capitalization: the class decorator `@Secure` (capitalized) and the method decorator `@secure` (lowercase).

For security enforcement, each intercepted operation is mapped to the corresponding data action. If the action is permitted by the security model, the operation proceeds; otherwise, a `SecurityException` is raised. By default, when an exception is raised, the read actions will result in a `RESTRICTED` value. For other actions, the developers can choose how to handle the exception. If they do not handle this exception explicitly, the generated default exception handler reports which permission was denied.

Algorithm 5.1: Security enforcement mechanism

```

Function securityPermit( $r, C, p, a, o, u, v$ ):
    foreach role  $r' \in \mathcal{R}$  such that  $r' \preceq_{\mathcal{R}} r$  do
        if securityPermit( $r', C, p, a, o, u, v$ ) then
            return true
        end
    end
    foreach  $(r, a(C, p), \varphi) \in \mathcal{PA}$  do
        if eval( $\varphi, \text{self} \mapsto o, \text{caller} \mapsto u, \text{value} \mapsto v$ ) then
            return true
        end
    end
    return false

```

Algorithm 5.1 presents an overview of the security enforcement logic. The function `securityPermit` takes as input the current role r , the accessed attribute or association end p of class C , the action type a , the target object o , the caller u , and an optional value

v that is used for update actions. It returns the verdict of whether u with the role r is allowed to perform action a on object property $o.p$ (possibly updating it to v). Since Flask-User does not provide built-in support for role hierarchies, *vActionGUI* encodes the role hierarchy directly in the authorization checks.

The function first checks whether access is granted through any inherited role. It then considers all permission entries matching the current role, action, and accessed feature. For each matching permission, the corresponding OCL authorization constraint is evaluated with the appropriate bindings for self, caller, and, where applicable, value. If no matching permission grants access, the function returns `False`.

For example, the `@Secure` class decorator intercepts all attribute accesses by replacing each property with a `SecuredAttribute` descriptor. Algorithm 5.2 illustrates the getter (`__get__`) case. When application code accesses an attribute of a secured entity, such as `person.name` or `event.owner.surname`, the `SecuredAttribute` descriptor is triggered. It retrieves the current user and role, reads the underlying attribute value, and then calls `securityPermit`. If the authorization check fails, i.e., the `securityPermit` function returns `False`, a default `RESTRICTED` value is returned.

Algorithm 5.2: Security enforcement mechanism for attribute read access

```
Function SecuredAttribute.__get__( $at, o$ ):
     $caller \leftarrow \text{GetCurrentUser}()$ 
     $role \leftarrow \text{GetCurrentRole}()$ 
     $C \leftarrow \text{GetClass}(at)$ 
    if securityPermit( $role, C, at, \text{read}, o, caller, \perp$ ) then
         $value \leftarrow o.at$ 
        return  $value$ 
    end
    return RESTRICTED
```

5.2.4 Privacy Notice and Dashboard

From the Declared purposes block of the privacy model, *vAG* generates two privacy-related interfaces: a *privacy notice* for consent collection and a *privacy dashboard* for exercising data subject rights.

Privacy notice and consent collection. The privacy notice presents the application's declared purpose policies in a human-readable form. Users can grant or revoke consent for each declared purpose policy individually.

Each declared purpose policy is shown as a panel item. Inside the panel, the user sees what personal data is associated with that purpose, together with the description of the additional policy condition specified in the privacy model. If a purpose is complex, i.e., it includes other purposes, the interface renders this as a collapsible nested panel. Each declared purpose policy includes a toggle button that allows the user to grant or revoke consent. These functions are also automatically generated.

Figure 5.1: vActionGUI generated privacy notice.

Example 5.12. Figure 5.1 shows the generated privacy notice corresponding to Example 5.8. The listing below shows the generated endpoint for granting consent. The endpoint receives the purpose identifier as a URL parameter and the personal data information from the request body. It then creates a new Consent object for the current user and links it to the selected personal data and purpose.

```

1  @app.route('/add_consent/<int:purposeid>', methods=['POST'])
2  def add_consent(purposeid):
3      resource = request.form['resource']
4      subresource = request.form['subresource']
5      pd = find_personal_data(resource, subresource)
6      purpose = Purpose.query.filter_by(id=purposeid).first()
7      c = Consent(data=pd, user=current_user)
8      c.purposes.append(purpose)
9      db.session.add(c)
10     db.session.commit()

```

Privacy dashboard. *vActionGUI* also generates a privacy dashboard that allows users to manage their personal data. For each personal data item owned by the current user, the dashboard displays the current value and provides actions to rectify the value, erase it by setting it to a null or default value, or restrict its processing.

Example 5.13. Figure 5.2 shows the privacy dashboard for Bob Brown, user p_2 in Example 2.3. The dashboard displays Bob’s personal data: his name (“Bob”), surname (“Brown”), attends association end (Event 1, which he attends), and events association end (Event 1, which he owns). Association ends are shown by the referenced object’s identifier rather than its attribute values, since those values are themselves subject to access control. For each item, the dashboard provides buttons to rectify the value, erase it, or restrict its processing, allowing Bob to exercise his data-subject rights directly within the application.

Privacy Dashboard	
Back to Main Page Policy	
Personal data	Action
Person	
name Bob	Rectify Erase Restrict
surname Brown	Rectify Erase Restrict
attends {Event 1}	Erase Restrict
events {Event 1}	Erase Restrict

Figure 5.2: *vActionGUI* generated privacy dashboard.

5.2.5 Privacy Enforcement

Intuitively, privacy enforcement must ensure two properties. First, every access to personal data must occur for (actual) purposes that conform to the declared purposes. Second, the data subject must have consented to the use of that personal data for those purposes.

Purpose tracking mechanism. To enforce the first property, `vActionGUI` generates a purpose tracking mechanism. This mechanism maintains a stack of actual purposes associated with the functions currently on the execution stack. We implement it by extending the `@secure` function decorator so that it carries the actual purpose declaration specified in the Actual purposes block of the privacy model.

When a function is invoked, its decorator pushes the endpoint's actual purposes onto the session's purpose stack. These purposes remain active for the duration of the endpoint execution. If the function internally invokes another instrumented function, the new actual purposes are pushed onto the stack as well. When an instrumented function returns, its purposes are popped from the stack.¹

Example 5.14. Consider the following `view_event` implementation.

```
1 def view_event(request):
2     event_id = request.args.get('event_id')
3     event = Event.query.get(event_id)
4     owner = event.owner.getFullname()
5     num_attendants = event.countPureAttendants()
6     return render_template("view_event.html",
7                           title = event.title, owner = owner, attendants = num_attendants)
```

Consider the actual purposes from Example 4.5. Suppose the user invokes the `view_event` endpoint, which sequentially calls the entity methods `getFullname` and `countPureAttendants`.

Figure 5.3 shows the purpose stack at four points during execution. The purpose stack starts as an empty stack, denoted by `[]`. Then, the `@secure` decorator pushes `CORE` as `view_event` begins (line 1); when `getFullname` is called, its `CORE` purpose is pushed on top (line 4); when `getFullname` returns, its purpose is popped, and when `countPureAttendants` is called, `ANALYTICS` is pushed and the actual set becomes `[CORE, ANALYTICS]` (line 5).

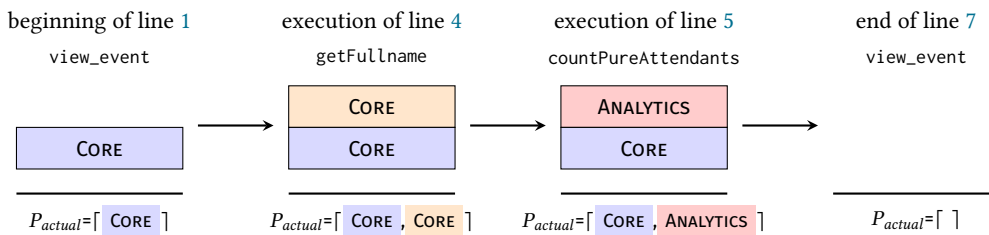


Figure 5.3: Purpose stack during `view_event` execution.

¹The stack is a sequence and may therefore contain the same purpose multiple times. The purpose limitation check operates on sets ($\text{set}(P_{actual})$), so duplicate entries do not affect the result.

Enforcement mechanism. As with security enforcement, *vActionGUI* instruments operations on personal data using the generated decorators. After a data access passes the security check, the privacy enforcement mechanism invokes a `privacyPermit` function to check whether the access also satisfies the privacy policy. Algorithm 5.3 presents the resulting privacy enforcement logic.

Algorithm 5.3: Privacy enforcement mechanism

```

Function privacyPermit( $P_{actual}$ ,  $C$ ,  $n$ ,  $a$ ,  $o$ ,  $u$ ,  $v$ ):
  if  $(C, n) \notin \text{Props}^P$  then
    | return true
  end
   $owner \leftarrow o.get\_owner(C, n)$ 
   $act \leftarrow a(C, n)$ 
   $dp \leftarrow \{p \mid (p, act, \varphi) \in \mathcal{PA}^P \wedge \text{eval}(\varphi, \text{self} \mapsto o, \text{caller} \mapsto u, \text{value} \mapsto v)\}$ 
   $dps \leftarrow dp \cup \bigcup_{p \in dp} \text{subpurposes}(p)$ 
   $cp \leftarrow \{p \mid c \in owner.consent, c.data = (C, n), p \in c.purposes\}$ 
   $cps \leftarrow cp \cup \bigcup_{p \in cp} \text{subpurposes}(p)$ 
  return  $\text{set}(P_{actual}) \subseteq cps \wedge \text{set}(P_{actual}) \subseteq dps$ 

```

The function `privacyPermit` takes as input the stack of actual purposes P_{actual} , collected from the session’s purpose stack, the class C , the property n being accessed, the action type a , the target object o , the caller u , and an optional updated value v . It first checks whether (C, n) is declared as personal data. If not, the access is allowed. Otherwise, it retrieves the owner of the personal data item. The function then performs two checks.

The purpose limitation check collects all declared purposes whose action matches the current access and whose OCL condition evaluates to true, including their sub-purposes. If the actual purposes are not covered by this set of declared purposes, access is denied.

The consent check collects all purposes for which the data owner has given consent on the accessed personal data item, again expanded with sub-purposes. Access is granted only if the actual purposes are a subset of the consented purposes. Consenting to a complex purpose implies consent to all its sub-purposes.

If either check fails, the interceptor raises a `PrivacyException`, which can be handled differently depending on the type of action, as shown with `SecurityException` (Section 5.2.3).

Example 5.15. Consider the code snippet in Example 5.14 and suppose that all security checks have been satisfied. Consider the object model $\mathcal{O}_2$ from Example 2.3, where Bob Brown (p_2) attempts to view his event “Study Group” (e_2).

When line 4 is executed, the method `getFullname` reads the name and surname attributes of Bob, the event owner. At this point, the purpose stack contains CORE twice, i.e., $P_{actual} = [\text{CORE}, \text{CORE}]$ (see Figure 5.3). For the access `read(Person, name)`, the algorithm collects the matching declared purpose policy $dp = \{\text{CORE}\}$ since $(\text{CORE}, \text{read}(\text{Person}, \text{name}), \top) \in \mathcal{PA}^P$ and the condition $\top$ holds trivially. Thus, $dps = \{\text{CORE}\}$ and $\text{set}(P_{actual}) = \{\text{CORE}\} \subseteq dps$, so the purpose limitation check succeeds. If Bob has consented to CORE for name, the consent check also succeeds, and the access is allowed. The same reasoning applies to `read(Person, surname)`.

When line 5 is executed, the method `countPureAttendants` is invoked and the purpose ANALYTICS is pushed onto the stack, i.e., $P_{actual} = [\text{CORE}, \text{ANALYTICS}]$. The result of the privacy check depends on how this function is implemented.

In the first implementation, `countPureAttendants` reads the attendants and managedBy association ends of the current Event object and returns the number of attendants who are not managers. Since $(\text{Event}, \text{attendants})$ and $(\text{Event}, \text{managedBy})$ are not declared as personal data, these accesses do not trigger privacy checks. The method is therefore allowed to execute.

```
1 def countPureAttendants(self):
2     return len(self.attendants) - len(self.managedBy)
```

In the second implementation, `countPureAttendants` computes the same value differently. It first queries all Person objects and then checks, for each person, whether the current event appears in the person's attends association end but not in their manages association end.

```
1 def countPureAttendants(self):
2     persons = Person.query.all()
3     attendants = 0
4     for person in persons:
5         if self in person.attends and self not in person.manages:
6             attendants = attendants + 1
7     return attendants
```

This implementation triggers a privacy check because it reads the resource $(\text{Person}, \text{attends})$. For each person, the access `read(Person, attends)` is intercepted. The privacy enforcement algorithm then collects the matching declared purpose policies. In this example, the only matching declared purpose is ANALYTICS, under the condition `self.role = Role :: FREEUSER`. For Bob, whose role is FREEUSER, this condition holds, giving $dps = dp = \{\text{ANALYTICS}\}$.

However, the active purpose stack contains both CORE and ANALYTICS, because the enclosing endpoint `view_event` is executed under the CORE purpose, and the current method `countPureAttendants` is executed under ANALYTICS.

Hence, $P_{actual} = [\text{CORE}, \text{ANALYTICS}]$ is not covered by the declared purposes, $\text{set}(P_{actual}) \not\subseteq \{\text{ANALYTICS}\}$. The purpose limitation check fails, and a default `RESTRICTED` value is returned.

5.3 Evaluation

We evaluate our approach and tool through the following research questions (RQs):

- vAG-RQ1** Is our model-driven privacy approach applicable across different implementation technologies and application domains?
- vAG-RQ2** How much developer effort does our model-driven approach require compared to a manual implementation?
- vAG-RQ3** How much runtime overhead does our model-driven approach incur?

vAG-RQ1 Is our model-driven privacy approach applicable across different implementation technologies and application domains?

Our model-driven privacy approach can support different implementation technologies. Besides the *vActionGUI* tool for Flask/Python applications described in this chapter, we have also implemented a prototype model transformation targeting ASP.NET and C# applications. We describe this transformation and its usage in Appendix C.

More generally, the approach can be realized for programming languages and frameworks that support cross-cutting instrumentation mechanisms, such as aspect-oriented programming [45, 47] or function decorators [63]. Existing authentication and authorization libraries can further simplify the transformation, because the generated code can integrate with these libraries instead of reimplementing the corresponding infrastructure.

To assess the generality of the approach with respect to the application domains, we implement four realistic web applications from different domains: (i) a social networking site *MiniTweet*, (ii) a conference management system *ConfMS*, (iii) a health record manager *Hipaa*, and (iv) an event platform *EventPlatform*. These case studies provide the basis for answering this research question, as well as **vAG-RQ2** and **vAG-RQ3**.

Table 5.3 summarizes the implementations used in our case studies, including their application name, description, domain, source, and the research question in which they are evaluated.

By using *vActionGUI* to implement these four case study applications, we show that our modeling languages are sufficiently expressive to develop non-trivial security and privacy policies across a range of application domains.

Table 5.3: Overview of the case study implementations.

Name	Description	Application	Source	RQ
ConfMS- ν AG	Secure, generated	ConfMS	Ours	ν AG-RQ2
EventPlatform- ν AG	Secure, generated	EventPlatform	Ours	ν AG-RQ2
EventPlatform-evol- ν AG	Secure, generated	EventPlatform	Ours	ν AG-RQ2
MiniTweet-baseline	Baseline	MiniTweet	[124]	ν AG-RQ3
MiniTweet-secured	Secure, manual	MiniTweet	Ours	ν AG-RQ3
MiniTweet- ν AG	Secure, generated	MiniTweet	Ours	ν AG-RQ3
Hipaa-jacqueline	Secure, Jacqueline	Hipaa	[140]	ν AG-RQ3
Hipaa- ν AG	Secure, generated	Hipaa	Ours	ν AG-RQ3

We introduce each application in the section corresponding to the research question in which it is evaluated.

ν AG-RQ2 How much developer effort does our model-driven approach require compared to a manual implementation?

To assess developer effort, we measure how much application code developers must write manually when using ν ActionGUI, compared with how much code is generated automatically from the models. We report these measurements for two case study applications, ConfMS and EventPlatform. For EventPlatform, we additionally evolve the application with new functional, security, and privacy requirements to measure the effort required to accommodate policy changes.

Conference management system. ConfMS is a web application for hosting and managing a scientific conference. Registered users may have one of three roles: regular user, program committee member, or chair. Regular users can edit their profile information, search for and view accepted papers, and submit new papers. Program committee members can delegate paper reviews to users, while chairs can accept papers for publication. Detailed functional, security, and privacy requirements are included in Appendix E.1. Here, we refer to the implementation generated using ν AG as ConfMS- ν AG.

Event Management Platform. EventPlatform is a web application designed for managing events, similar to Meetup or Facebook Events. Anyone can browse the platform, but registering provides access to additional functionality. Registered users can create events, attend or request to attend events, and help manage them. Events are organized into categories, each managed by moderators, and users can subscribe to categories of interest. Administrators have extra management capabilities, such as managing user profiles and viewing event statistics. The platform also displays advertisements and

uses personal information to send marketing messages and recommend relevant events to users.

This application is designed as part of the empirical study in Part III. We describe in detail the functional, security, and privacy requirements of the web application in Appendix E.3. In this section, we refer to the implementation generated using *vActionGUI* as *EventPlatform-vAG*.

Table 5.4 reports implementation statistics for *ConfMS-vAG* and *EventPlatform-vAG*. For *EventPlatform-vAG*, we additionally report the effort required to evolve the application when new requirements are introduced.

Table 5.4: Model sizes and # LoC of the case study implementations.

	ConfMS-vAG		EventPlatform-vAG (+ new reqs.)	
	LoC	%	LoC	%
Model sizes (# of elements)				
Data model	13		29 + 5	
Security model	17		75 + 12	
Privacy model	20		57 + 5	
Generated application (in # LoC of .py + .html)				
Total	2,048	100%	3,143 + 279	100%
a) Generated artifacts	1,794	87.6%	2,556 + 198	80.5%
a.1) Classes & method stubs	534	26.1%	1,111 + 178	37.7%
a.2) Authentication	-	-	-	-
a.3) Enforcement mechanism (Security)	131	6.4%	134 + 0	3.9%
a.4) Enforcement mechanism (Privacy)	168	8.2%	200 + 0	5.8%
a.5) Enforcement mechanism (Shared)	443	21.6%	444 + 0	13%
a.6) Consent collection	243	11.9%	225 + 0	6.6%
a.7) Others	275	13.4%	442 + 20	13.5%
b) Manually implemented	254	12.4%	587 + 81	19.5%

The upper part of the table reports the size of the three input models. The size of the data model is computed as the total number of sorts, attributes, methods, and associations. The size of a security model is the number of permission assignment entries, whereas the size of a privacy model is the total number of declared and actual purposes.

The lower part of the table reports the generated lines of code (LoC), excluding comments and blank lines, for both Python and HTML files. It also reports the amount of written code to complete the generated method stubs. We do not report separate numbers for authentication, since it is delegated to existing libraries and requires only a few configuration lines.

The results show that developers need to write only a small fraction of the final application manually: approximately 12% of the ConfMS-*vAG* codebase and 19% of the EventPlatform-*vAG* codebase. The remaining code is automatically generated. A substantial part of the generated Python code implements security and privacy enforcement: 10–14% of the generated code implements these enforcement mechanisms separately, with an additional 14–22% for the shared enforcement mechanism that intercepts attribute access and method calls.

The “+ new reqs.” of Table 5.4 quantifies the effort of evolving EventPlatform-*vAG* after new functional, security, and privacy requirements are introduced. These requirements add a total of 198 generated LoC. This increase is concentrated in generated classes and method stubs, which account for 178 LoC. The generated enforcement mechanisms, in contrast, do not grow in this evolution step. The manually implemented business logic adds 81 LoC.

These results indicate that our model-driven approach substantially reduces the amount of developer code required to obtain a functional, security- and privacy-aware web application. The benefit is particularly important when policies evolve. When the security or privacy policies change, our model transformation can regenerate the corresponding enforcement mechanisms automatically. A manual implementation would require developers to locate and update the affected checks throughout the codebase.

***vAG*-RQ3** How much runtime overhead does our model-driven approach incur?

To assess runtime overhead, we measure the response time of applications developed using *vActionGUI* and compare it with reference implementations. We use two case study applications. For MiniTweet, we compare the *vAG* implementation against the baseline (i.e., the application without security and privacy enforcement) and a code-centric version (i.e., the application with manual security and privacy enforcement). For Hipaa, we compare it against an implementation built with the state-of-the-art Jacqueline framework [140].

In each experiment, we vary the size of the application state to evaluate how the overhead scales as the application grows. All experiments are conducted on a machine with an Intel Core i5-11300H processor and 16 GB of RAM. Each reported measurement is averaged over 100 independent executions.

Social networking site. MiniTweet is an improved version of the simple Twitter open-source implementation [124]. Registered users can post messages, follow or unfollow other users, and view their own timeline. To assess privacy policy enforcement, we extend the functionality to support in-application advertisements displayed on user

timelines. The detailed data, security, and privacy models of MiniTweet are presented in Appendix E.2.

We denote by MiniTweet-baseline the baseline implementation. It implements only its business logic and forgoes security and privacy policy enforcement. We refer to the two secured implementations as MiniTweet-secured and MiniTweet-*vAG*. The former extends the baseline application using a code-centric approach that directly enforces the policies, whereas the latter follows our model-driven approach using *vActionGUI*.

The benchmark experiment consists of a registered user logging in and viewing their own timeline by invoking the `view_timeline` endpoint. Each MiniTweet instance contains 4 users and a variable number of messages, ranging from 10 to 100. Message contents are randomly sampled, and messages are assigned uniformly to users. The session user follows all other users. The user’s follow list is treated as personal data. By varying the number of messages in the application state, we assess how the enforcement overhead scales with state size.

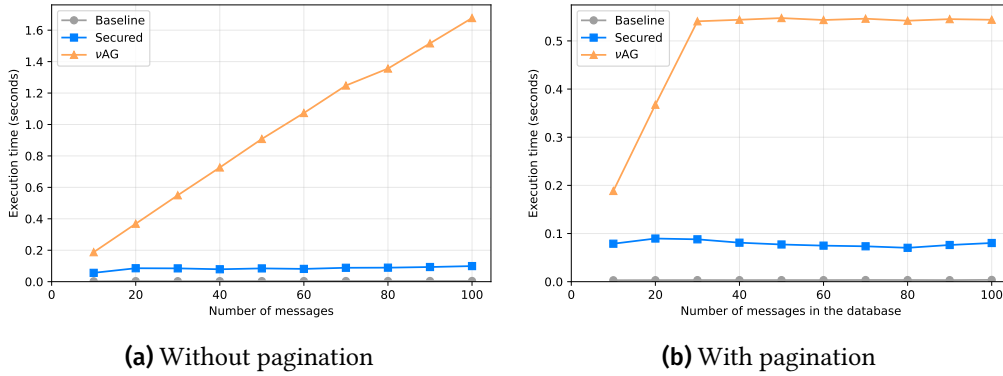


Figure 5.4: Runtime performance of MiniTweet implementations.

Figure 5.4a shows the execution time of the three MiniTweet implementations on this workload. All implementations scale linearly with the number of messages. The *vAG* implementation incurs higher overhead than the manually secured implementation. This is expected, as the manually secured implementation performs only two checks, one for security and one for privacy, before rendering the list of messages on the timeline. This is prone to errors, as it assumes that the subsequent code correctly queries and outputs the right messages. In contrast, our approach performs both checks whenever a message attribute is accessed, ensuring that the developer only manipulates data in compliance with the policies.

Although the added overhead is comparatively high, it remains manageable for typical usage. With 100 messages, the MiniTweet-*vAG* implementation responds in approximately 1.7 seconds. The baseline remains below 5 milliseconds, and the manually secured

implementation stays below 100 milliseconds for the same workload. We present an approach to mitigate the overhead next.

Mitigating overhead with pagination. A common strategy to reduce the number of messages processed per request is *pagination*. Here, only a fixed number of items is shown on each page, and the user navigates to subsequent pages as needed. We therefore repeat the experiment with 30-item pagination enabled for the `view_timeline` endpoint in all three implementations.

Figure 5.4b reports the results. With pagination, the execution time of the *vAG* implementation stabilizes below 0.6 seconds once the application contains more than 30 messages because the database query returns at most 30 messages per page. For workloads with fewer than 30 messages, all messages fit on the first page, so the behavior is identical to the non-paginated version. Note that pagination does not require any changes to the models. This demonstrates that standard web development practices, such as pagination, can effectively bound the overhead introduced by our enforcement mechanism.



Health record manager. Hipaa is a web application that allows users with different roles to view personal medical records. The requirements of the Hipaa application can be found in [140]. We refer to the implementation of Yang et al. [140] as *Hipaa-jacqueline* and to the implementation generated using *vActionGUI* as *Hipaa-vAG*. We deploy and measure the response time of *Hipaa-vAG* and *Hipaa-jacqueline* implementations [140].

For each Hipaa implementation, we initialize the application state with one doctor and a variable number of patients (ranging from 1 to 1,024). The experiment consists of the doctor logging in and opening the initial page of the application (by invoking the `index` method). This page displays all patients assigned to the doctor and the hospitals where the doctor works. In this benchmark, only patient information, namely name, date of birth, and social security number, is considered personal data. Furthermore, the security model is designed to grant full access to all roles, so the measured overhead is due only to privacy enforcement.

Figure 5.5 shows the execution time of the two Hipaa implementations. Both approaches scale linearly with the number of patients, with *Jacqueline* requiring longer to respond. At 1,024 patients, *vAG* responds in 15.6 ms, while *Jacqueline* requires 1,309 ms (an 83.8× speedup). The significant performance difference between the two approaches is caused by the more expensive faceted execution employed by *Jacqueline*. Faceted execution provides stronger information-flow guarantees, but in this benchmark it requires evaluating protected data through a substantially more expensive enforcement mechanism.

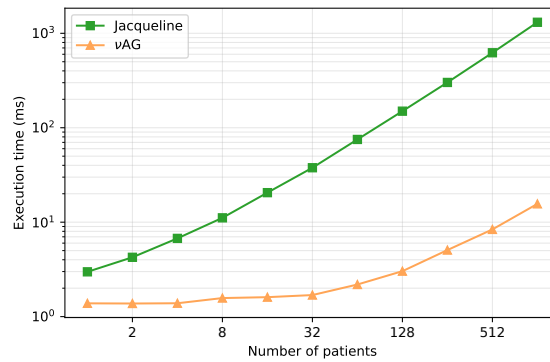


Figure 5.5: Runtime performance of different Hipaa implementations.

Threats to Validity

External validity. Our case studies cover four applications from different domains (social networking, conference management, health records, and event management). While they exercise a range of security and privacy policies, they may not be representative of the full complexity found in large-scale industrial systems.

Internal validity. The code-centric baseline implementations were developed by the thesis author, which may introduce bias in the effort comparison ([vAG-RQ2](#)). To mitigate this, the baseline follows standard Flask development practices. Nevertheless, this threat cannot be eliminated entirely.

Both threats are partly mitigated by the multi-year empirical study in Part [III](#) of this thesis, where independent developers implement the same application requirements using both approaches.

Security and Privacy-aware Object-Relational Mappers

In this chapter, we present our contributions to enforcing security and privacy at the database layer. First, we extend the model-based characterization of fine-grained access control (FGAC) policies for SQL queries (Section 2.4.4) with support for association classes [P1] and privacy enforcement [S4]. Second, we extend SQLSI into SQLSPI, a secure and privacy-aware interceptor that generates stored procedures enforcing both security and privacy requirements [S4]. Third, we develop an object-relational mapper (ORM) interceptor that transparently rewrites ORM-generated queries through SQLSPI to enforce access control and purpose limitation at the database layer [S1, S4].

6.1 Extending the Model-based Characterization

We refine the characterization from Section 2.4.4 and Appendix A in two ways. First, we extend the data model with association classes and the security model with permission assignments over them. Second, we add support for privacy requirements at the database layer.

6.1.1 Extending the Data Model with Association Classes

Association classes store additional information about the relationship between associated classes. In modeling languages such as UML, an *association class* is an association that has its own attributes and operations.

Definition 6.1 (Data model with association class).

A data model with association classes extends the data model of Definition 4.1 with a finite set of association classes:

$$\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as}, \text{E}, \text{asc}),$$

where CS , $\preceq_{\mathcal{D}}$, attr , meth , as , and E are defined as in Definition 4.1, and $\text{asc} \subseteq \text{CS} \times \text{dom}(\text{as})$ is a finite set of *association classes*. Each association class $(C_{as}, as) \in \text{asc}$ pairs a class sort $C_{as} \in \text{CS}$ with a binary association $as \in \text{as}$, $\text{as}(as) = ((ase_l, m_l, C_l), (ase_r, m_r, C_r))$. The class C_{as} carries the attributes and methods recorded about the association and induces two implicit associations linking C_{as} to the two classes in each association end of as , namely, $asi_l = ((ase_l, 1, C_l), (as, m_r, C_{as}))$ and $asi_r = ((as, m_l, C_{as}), (ase_r, 1, C_r))$.

Example 6.1. We extend the Event Management Platform described in Example 4.1 with an association class on the many-to-many attendance association. More specifically, $(\text{Attendance}, \text{attendance}) \in \text{asc}$, where the class sort Attendance records information about this attendance:

$$\text{attr}(\text{Attendance}) = \{ \text{registeredAt} \mapsto \mathbb{Z}, \text{ticketType} \mapsto \text{String} \}.$$

The registeredAt attribute records the registration timestamp, and the ticketType attribute records the attendee type. Figure 6.1 shows the extended data model.

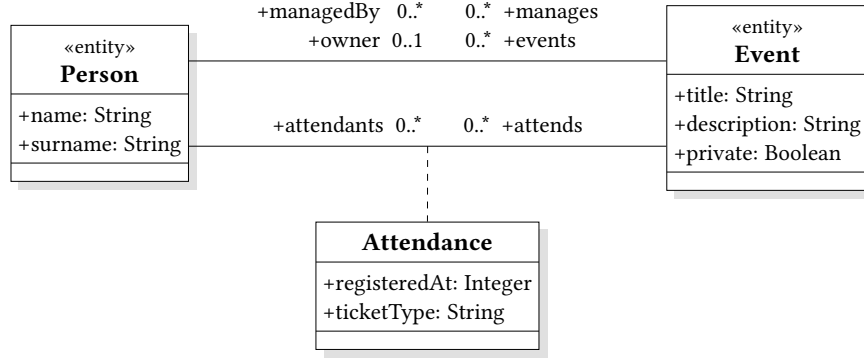


Figure 6.1: UML class diagram of the Event Management Platform with an association class.

6.1.2 Extending the Security Model with Actions on Association Classes

We extend the security modeling language by treating association classes as protected resources. This includes both the class introduced by the association class and the explicit and implicit associations linked to it.

Definition 6.2 (Data actions with association classes).

Let $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as}, \text{E}, \text{asc})$ be a data model. For each association class $(C_{as}, \text{as}) \in \text{asc}$, where $\text{as}(\text{as}) = ((\text{ase}_l, m_l, C_l), (\text{ase}_r, m_r, C_r))$, the set of resources Resources is extended with

- the class sort C_{as} ;
- each attribute in $\text{dom}(\text{attr}(C_{as}))$;
- each method in $\text{dom}(\text{meth}(C_{as}))$;
- the association ends in the explicit association as ; and
- the association ends in the two implicit associations asi_l and asi_r , linking C_{as} to C_l and C_r , respectively.

The set of data actions $\mathcal{A}$ is extended accordingly.

A permission assignment involving an association class is then a triple $(r, a, \varphi) \in \mathcal{PA}$, where the definition of $\varphi \in C(a)$ remains unchanged. Then, the definition of the security model (Definition 2.9) carries over without further modification.

6.1.3 Extending the Authorization Clauses for SQL Queries

We extend the *AuthQuery()* definition with new query patterns for association classes, continuing the numbering of the seven patterns (1)–(7) defined in Appendix A. These patterns cover queries that read from or join with an association class. Following the notation of Section 2.4 and Appendix A, let $(C_{as}, as) \in asc$ be an association class where $as(as) = ((ase_l, m_l, C_l), (ase_r, m_r, C_r))$, and let asi_l and asi_r be its two implicit associations linking C_{as} to C_l and C_r , respectively.

Pattern (8): `SELECT items FROM  $C_{as}$  WHERE  $exp$` , where C_{as} is the table of an association class $(C_{as}, as) \in asc$. Reading from the association class table may touch the attributes of C_{as} , the explicit association as , and the two implicit associations asi_l and asi_r . The user must be authorized to read each of them whenever the corresponding column or end is accessed by the query.

Definition 6.3 (AuthQuery: SELECT from association class).

Let $q = \text{SELECT items FROM } C_{as} \text{ WHERE } exp$ be an SQL query, where C_{as} is the table of an association class $(C_{as}, as) \in asc$. Then *AuthQuery()* holds if and only if:

- for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$, and every attribute $at \in \text{dom}(\text{attr}(C_{as}))$ that appears in the filter expression exp , the user u with role r is allowed to read at of o ;
- for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$ that satisfies exp , and for every attribute $at \in \text{dom}(\text{attr}(C_{as}))$ that appears in the projection clauses *items* of q , the user u with role r is allowed to read at of o ;
- whenever the query accesses ase_l or ase_r via *items* or exp , then for every pair (o_l, o_r) in the Cartesian product $\text{dom}(\mathbb{O}(C_l)) \times \text{dom}(\mathbb{O}(C_r))$ that satisfies exp , the user u with role r is allowed to read the association link (o_l, o_r) of as ; and
- whenever the query accesses the association class object, i.e., accesses C_{as_id} or any attribute of C_{as} appears via *items* or exp , then for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$ that satisfies exp : if one of the association ends of as (e.g., ase_l) appears in *items* or exp , the user u with role r is allowed to read the corresponding implicit association link connecting the accessed association class object to its endpoint object (e.g., (o_l, o)).

Pattern (9): `SELECT items FROM C JOIN Cas ON exp`, where C is a class and C_{as} is the table of an association class $(C_{as}, as) \in asc$. The user must be authorized to read the attributes of both C and C_{as} used in the join and the projection, together with the explicit and implicit associations of the association class, as in Definition 6.3.

Definition 6.4 (AuthQuery: SELECT from class JOIN association class). Let $q = \text{SELECT items FROM } C \text{ JOIN } C_{as} \text{ ON } exp$ be an SQL query, where C is a class and C_{as} is the table of an association class $(C_{as}, as) \in asc$. Then $\text{AuthQuery}()$ holds if and only if:

- for every object $o \in \text{dom}(\mathbb{O}(C))$ (resp. $o \in \text{dom}(\mathbb{O}(C_{as}))$), and for every attribute $at \in \text{dom}(\text{attr}(C))$ (resp. $at \in \text{dom}(\text{attr}(C_{as}))$) that appears in the join expression exp , the user u with role r is allowed to read at of o ;
- for every object $o \in \text{dom}(\mathbb{O}(C))$ (resp. $o \in \text{dom}(\mathbb{O}(C_{as}))$) in the result of $C \text{ JOIN } C_{as} \text{ ON } exp$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ (resp. $at \in \text{dom}(\text{attr}(C_{as}))$) in the projection clauses $items$, the user u with role r is allowed to read at of o ;
- if at least one association end appears in $items$ or exp , then for every pair (o_l, o_r) in the Cartesian product $\text{dom}(\mathbb{O}(C_l)) \times \text{dom}(\mathbb{O}(C_r))$, the user u with role r is allowed to read the association link (o_l, o_r) of as ; and
- if C_{as_id} or any attribute of C_{as} appears in $items$ or exp , then for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$: if one of the association ends (e.g., ase_l) appears, the user is allowed to read the corresponding implicit association link (e.g., (o_l, o)).

The symmetric pattern `SELECT items FROM Cas JOIN C ON exp` is handled analogously.

Pattern (10): `SELECT items FROM Cas JOIN subselect ON exp`, where C_{as} is the table of an association class. The user must be authorized to execute the subquery and, as in Definition 6.3, to read the attributes and the explicit and implicit associations of the association class that the query accesses.

Definition 6.5 (AuthQuery: SELECT from association class JOIN subquery). Let $q = \text{SELECT items FROM } C_{as} \text{ JOIN subselect ON } exp$ be an SQL query, where C_{as} is the table of an association class $(C_{as}, as) \in asc$. Then $\text{AuthQuery}()$ holds if and only if $\text{AuthQuery}()$ holds for the subquery $subselect$, and:

- for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$, and every attribute $at \in \text{dom}(\text{attr}(C_{as}))$ that appears in the join expression exp , the user u with role r is allowed to read at of o ;

- for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$ in the result of $C_{as} \text{ JOIN subselect ON } exp$, and for every attribute $at \in \text{dom}(\text{attr}(C_{as}))$ in the projection clauses $items$, the user u with role r is allowed to read at of o ;
- if at least one association end appears in $items$ or exp , then for every pair (o_l, o_r) in the Cartesian product $\text{dom}(\mathbb{O}(C_l)) \times \text{dom}(\mathbb{O}(C_r))$, the user u with role r is allowed to read the association link (o_l, o_r) of as ; and
- if C_{as_id} or any attribute of C_{as} appears in $items$ or exp , then for every object $o \in \text{dom}(\mathbb{O}(C_{as}))$ in the result of $C_{as} \text{ JOIN subselect ON } exp$: if one of the association ends (e.g., ase_l) appears, the user is allowed to read the corresponding implicit association link (e.g., (o_l, o)).

The symmetric pattern $\text{SELECT items FROM subselect JOIN } C_{as} \text{ ON } exp$ is handled analogously.

6.1.4 Model-based Characterization of Privacy Enforcement on SQL Queries

Recall that the authorization predicate $\text{AuthQuery}()$ in Section 6.1.3 determines when a query may be executed by checking whether the caller is allowed to read each resource accessed by the query. We extend this predicate to account for privacy enforcement. If an accessed resource is personal data, $\text{AuthQuery}()$ additionally checks that the data is read for a valid actual purpose and that the data owner has consented to that purpose.

More specifically, we extend the definition of $\text{AuthQuery}()$ with two additional parameters: the privacy model $\mathcal{P}$ and the actual purpose p_{actual} . In what follows, we illustrate the extended definition (in red text) for one of the AuthQuery definitions.

Definition 6.6 (AuthQuery **extended**: **SELECT from class**).

Let $\mathcal{D} = (\mathcal{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model, and let $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$ be a security model for $\mathcal{D}$, where $\mathcal{D}_{\mathcal{S}}$ extends $\mathcal{D}$ to $(\text{user}, \mathcal{CS}', \preceq_{\mathcal{D}_{\mathcal{S}}}, \text{attr}', \text{meth}, \text{as})$. Let $\mathcal{O} = (\mathbb{O}, \mathbb{M}, \mathbb{R})$ be an object model of $\mathcal{D}_{\mathcal{S}}$. Let $r \in \mathcal{R}$ be a role, and $u \in \text{dom}(\mathbb{O}(\text{user}))$ be a user object. **Let $\mathcal{P} = (\mathcal{D}_{\mathcal{P}}, \subseteq_{\Pi}, \mathcal{PA}^{\mathcal{P}}, \text{annotate})$ be a privacy model for $\mathcal{D}$, where $\mathcal{D}_{\mathcal{P}}$ is the privacy extension of the data model. Let p_{actual} be an actual purpose of the current query execution.**

Let $q = \text{SELECT items FROM } C \text{ WHERE } boolexp$ be an SQL query.

Then $\text{AuthQuery}(\mathcal{P}, \mathcal{S}, \mathcal{O}, u, r, q, p_{\text{actual}})$ holds if and only if:

- for every object $o \in \text{dom}(\mathbb{O}(C))$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the Boolean expression $boolexp$ of q , the user u with role r is allowed to read at of o with respect to the security model $\mathcal{S}$ in the context of $\mathcal{O}$.

- Moreover, if at of o is personal data, then p_{actual} must be covered by a declared purpose in $(p, \text{read}(C, at), \varphi) \in \mathcal{PA}^{\mathcal{P}}$ whose condition φ holds, and the owner of the personal data at of o must have consented to that purpose.
- for every object $o \in \text{dom}(\mathbb{O}(C))$ that satisfies boolexp , and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the projection clauses items of q , the user u with the role r is allowed to read at of o with respect to the security model S in the context of O .
 - Moreover, if at of o is personal data, then p_{actual} must be covered by a declared purpose in $(p, \text{read}(C, at), \varphi) \in \mathcal{PA}^{\mathcal{P}}$ whose condition φ holds, and the owner of the personal data at of o must have consented to that purpose.

Example 6.2. Consider a user ‘Bob’ with the role `FREEUSER` who invokes the `view_event` endpoint of the Event Management Platform. The endpoint has an actual purpose `CORE` and submits the following query q_1 :

```
SELECT title
FROM Event
WHERE LOWER(description) LIKE '%education%' AND private = TRUE
```

By Definition 6.6, $\text{AuthQuery}(\mathcal{P}, S, O, \text{Bob}, \text{FREEUSER}, q_1, \text{CORE})$ holds for this query if and only if:

1. for every Event object, user ‘Bob’ with the `FREEUSER` role is allowed to perform `read(Event, description)` and `read(Event, private)`.
 - If either `description` or `private` is declared as personal data, then the read access must also be covered by a declared purpose policy for the actual purpose `CORE`, and the corresponding data owner must have consented to `CORE`.
2. for every Event object that satisfies the `WHERE` clause, ‘Bob’ with the `FREEUSER` role is also allowed to perform `read(Event, title)`.
 - If `title` is declared as personal data, then the read access must also be covered by a declared purpose policy for the actual purpose `CORE`, and the corresponding data owner must have consented to `CORE`.

The other query patterns in Appendix A are extended analogously. In the next section, we extend SQLSI with privacy components to support privacy enforcement at the database layer.

6.2 From SQLSI to SQLSPI

The mappings from the SQLSPI data and security models to the database schema (Section 2.4.1) and security predicates (Section 2.4.2) remain unchanged. SQLSPI additionally maps the privacy model to database components used for privacy enforcement. In what follows, we first present the generated privacy tables.

6.2.1 Mapping from SQLSPI Privacy Model to Database Tables

Recall the privacy model $\mathcal{P} = (\mathcal{D}_{\mathcal{P}}, \subseteq_{\Pi}, \mathcal{P}\mathcal{A}^{\mathcal{P}}, \text{annotate})$ from Definition 4.6. To track personal data, ownership, purposes, and consent, SQLSPI extends the database schema $\overline{\mathcal{D}}$ from Definition 2.10 to $\overline{\mathcal{D}_{\mathcal{P}}}$, with tables that reify the sorts and associations introduced by the privacy extension $\mathcal{D}_{\mathcal{P}}$.

Definition 6.7 (SQLSPI privacy model to database tables).

Given a privacy model $\mathcal{P} = (\mathcal{D}_{\mathcal{P}}, \subseteq_{\Pi}, \mathcal{P}\mathcal{A}^{\mathcal{P}}, \text{annotate})$ over a data model $\mathcal{D}$ with user sort User, SQLSPI adds the following tables to $\overline{\mathcal{D}}$:

- a PersonalData table reifying the PersonalData sort. Its resource and subresource columns represent the personal data entity and property, and its owner foreign key represents the data_ownership association:

```
CREATE TABLE PersonalData (
  PersonalData_id INT PRIMARY KEY,
  resource VARCHAR(255) NOT NULL,
  subresource VARCHAR(255),
  owner INT REFERENCES User(User_id)
);
```

- for every personal class sort $C \in \text{CS}_{\neq \emptyset}^{\mathcal{P}}$, a link table C_PD which relates objects of C to their personal data items:

```
CREATE TABLE C_PD (
  C_id INT REFERENCES C(C_id),
  PersonalData_id INT REFERENCES PersonalData(PersonalData_id),
  PRIMARY KEY (C_id, PersonalData_id)
);
```

- for every association as that has association ends declared as personal data, $as(as) = ((n, _ C), (n', _ C'))$, SQLSPI analogously creates a table as_PD :

```
CREATE TABLE as_PD (
  n INT, n' INT,
  PersonalData_id INT REFERENCES PersonalData(PersonalData_id),
  PRIMARY KEY (n, n', PersonalData_id),
  FOREIGN KEY (n, n') REFERENCES as(n, n')
);
```

- analogously to roles, a Purpose table enumerates the purposes in Π , and a PurposeHierarchy table records the hierarchy $\subseteq_{\Pi}$.

```
CREATE TABLE Purpose (
  Purpose_id INT PRIMARY KEY,
  name VARCHAR(255) UNIQUE
);
CREATE TABLE PurposeHierarchy (
  parent INT REFERENCES Purpose(Purpose_id),
  child INT REFERENCES Purpose(Purpose_id),
  PRIMARY KEY (parent, child)
);
```

- a Consent table reifying the Consent sort. Since consent is recorded per purpose, each row links a personal data item data and a purpose purpose:

```
CREATE TABLE Consent (
  Consent_id INT PRIMARY KEY,
  purpose INT REFERENCES Purpose(Purpose_id),
  data INT REFERENCES PersonalData(PersonalData_id)
);
```

Example 6.3. Consider the Event Management Platform with personal data resources $\text{Props}^{\mathcal{P}} = \{(\text{Person}, \text{name}), (\text{Person}, \text{events})\}$ from Example 4.2 and the basic purpose set $\Pi = \{\text{CORE}, \text{ANALYTICS}\}$ with ANY as the complex purpose from Example 4.3. In addition to the tables generated from the data model in Example 2.14, and the generic privacy tables for purposes, their hierarchy, and consent, SQLSPI generates the following tables:

```
CREATE TABLE PersonalData (
  PersonalData_id INT PRIMARY KEY,
  resource VARCHAR(255) NOT NULL, subresource VARCHAR(255),
  owner INT REFERENCES Person(Person_id)
);

CREATE TABLE PersonPD (
  Person_id INT REFERENCES Person(Person_id),
  PersonalData_id INT REFERENCES PersonalData(PersonalData_id),
  PRIMARY KEY (Person_id, PersonalData_id)
);

CREATE TABLE Event_owner_events_Person_PD (
  owner INT, events INT,
  PersonalData_id INT REFERENCES PersonalData(PersonalData_id),
  PRIMARY KEY (owner, events, PersonalData_id),
  FOREIGN KEY (owner, events)
    REFERENCES Event_owner_events_Person(owner, events)
);
```

The PersonPD table connects each Person object to the PersonalData rows recording its personal attributes, such as name. Since the association end events is also declared as personal data, a table Event_owner_events_Person_PD is also generated by the tool. This table connects each Person–Event association link to the corresponding PersonalData row. SQLSPI then instantiates the purposes and their hierarchy as follows:

```
INSERT INTO Purpose (Purpose_id, name) VALUES
  (1, 'ANY'), (2, 'ANALYTICS'), (3, 'CORE');
INSERT INTO PurposeHierarchy (parent, child) VALUES
  (1, 1), (1, 2), (1, 3),
  (2, 2),
  (3, 3);
```

6.2.2 Mapping from SQLSPI Privacy Model to SQL Predicates

We now give an overview of the auxiliary SQL functions used for privacy enforcement based on the database schema introduced in the previous section.

Purpose hierarchy. The purpose hierarchy $\subseteq_{\Pi}$ is implemented in the PurposeHierarchy table, analogously to how the role hierarchy is implemented in Section 2.4.2. SQLSPI generates a helper function `cover(parent, child)` that returns whether the (complex) purpose *parent* covers the purpose *child*:

```
CREATE FUNCTION cover (parent VARCHAR(255), child VARCHAR(255))
RETURNS INT DETERMINISTIC
BEGIN
  RETURN EXISTS (
    SELECT 1
    FROM PurposeHierarchy h
    JOIN Purpose par ON par.Purpose_id = h.parent
    JOIN Purpose ch ON ch.Purpose_id = h.child
    WHERE par.name = parent AND ch.name = child
  );
END
```

Consent checking. For each personal property $(C, n) \in \text{Props}^{\mathcal{P}}$, SQLSPI generates a `CheckConsent_C_n` predicate. This predicate takes a personal data object and a purpose, and holds if and only if the owner has consented to the use of that personal data item for that purpose or a more general one. More concretely, it checks whether there exists a Consent row referring to the personal data resource on the given object whose consented purpose *covers* the given purpose, so that consenting to a more general purpose implies consent to its sub-purposes.

The generated predicate depends on the type of resource. If n is an attribute, SQLSPI generates the following SQL predicate:

```
CREATE FUNCTION CheckConsent_C_n (o INT, purpose VARCHAR(255))
RETURNS INT DETERMINISTIC
BEGIN
  RETURN EXISTS (
    SELECT 1
    FROM Consent c
      JOIN PersonalData pd ON pd.PersonalData_id = c.data
      JOIN Purpose pu ON pu.Purpose_id = c.purpose
      JOIN C_PD link ON link.PersonalData_id = c.data
    WHERE pd.resource = 'C' AND pd.subresource = 'n'
      AND link.C_id = o AND cover(pu.name, purpose) = 1
  );
END
```

If n is an association end with $as(as) = ((n, _C), (n', _C'))$, SQLSPI generates the following SQL predicate:

```
CREATE FUNCTION CheckConsent_C_n (o INT, purpose VARCHAR(255))
RETURNS INT DETERMINISTIC
BEGIN
  RETURN EXISTS (
    SELECT 1
    FROM Consent c
      JOIN PersonalData pd ON pd.PersonalData_id = c.data
      JOIN Purpose pu ON pu.Purpose_id = c.purpose
      JOIN as_PD link ON link.PersonalData_id = c.data
    WHERE pd.resource = 'C' AND pd.subresource = 'n'
      AND link.n = o AND cover(pu.name, purpose) = 1
  );
END
```

Finally, for each declared purpose policy $(p, \text{read}(C, n), \varphi) \in \mathcal{PA}^P$, SQLSPI generates an SQL function CheckCondition_C_n_p that embeds the OCL-to-SQL translation $\text{map}(\varphi)$, checking whether the policy condition holds:

```
CREATE FUNCTION CheckCondition_C_n_p (self INT, caller INT)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 0;
  SELECT res INTO result FROM map( $\varphi$ ) AS TEMP;
  RETURN result;
END
```

If the condition is trivially true, the OCL constraint is expressed as **true** and is translated into the following **SELECT** query:

```
SELECT TRUE as res; -- TRUE is stored as integer 1
```

The condition functions above evaluate the constraints of declared purpose policies, but they do not by themselves prevent an access under an undeclared (actual) purpose. Purpose limitation additionally requires that the accessed personal data item (C, n) is used only for purposes covered by the declared purpose policies.

Purpose limitation. SQLSPI captures this principle with the following predicate. Given an actual purpose, the predicate `CheckPurposeLimitation_C_n` checks whether at least one declared purpose policy for (C, n) covers that actual purpose and has a condition that holds. If no such declared policy exists, the predicate returns 0:

```
CREATE FUNCTION CheckPurposeLimitation_C_n (self INT, caller INT, purpose VARCHAR(255))
RETURNS INT DETERMINISTIC
BEGIN
  IF (cover('p1', purpose) = 1 AND CheckCondition_C_n_p1(self, caller) = 1)
    THEN RETURN 1;
  END IF;
  IF (cover('p2', purpose) = 1 AND CheckCondition_C_n_p2(self, caller) = 1)
    THEN RETURN 1;
  END IF;
  ...
  IF (cover('pk', purpose) = 1 AND CheckCondition_C_n_pk(self, caller) = 1)
    THEN RETURN 1;
  END IF;
  RETURN 0;
END
```

where $p_1, p_2, \dots, p_k$ are the declared purposes for which (C, n) has a declared policy, i.e., $(p_i, \text{read}(C, n), \varphi_i) \in \mathcal{PA}^{\mathcal{P}}$.

6.2.3 Mapping SQL Statements to Stored Procedures

SQLSPI implements two rewriting functions. `PrivInsert()` rewrites SQL insertions that introduce personal data, while `SecPrivQuery()` rewrites SQL queries into stored procedures that invoke the security and privacy predicates defined in the previous sections.

INSERT statements. Since the application tracks personal data ownership at runtime, inserting an object of a class with personal properties must also create the corresponding personal data items and record their owner. SQLSPI therefore rewrites such `INSERT` statements into stored procedures that execute the original insertion and create the corresponding privacy metadata in the same transaction.

By default, calling `PrivInsert( $\mathcal{P}, i$ )` does not result in Consent objects being created. Consent may be granted later by the data owner. Until then, the enforcement mechanism blocks access to the inserted personal data.

Definition 6.8 (INSERT statement to stored procedure).

Let $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model, and let $\mathcal{P} = (\mathcal{D}_{\mathcal{P}}, \subseteq_{\Pi}, \mathcal{PA}^{\mathcal{P}}, \text{annotate})$ be a privacy model for $\mathcal{D}$, where $\mathcal{D}_{\mathcal{P}}$ is the privacy extension of the data model. Let i be an **INSERT** statement over the database schema $\overline{\mathcal{D}_{\mathcal{P}}}$.

If i inserts an object into a personal class $C \in \text{CS}_{\neq \emptyset}^{\mathcal{P}}$,

```
INSERT INTO C (...) VALUES (...);
```

then SQLSPI rewrites i into $\text{PrivInsert}(\mathcal{P}, i)$:

```
CREATE PROCEDURE PrivInsert( $\mathcal{P}, i$ ) (IN caller INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insert failed';
  END;
  START TRANSACTION;
  INSERT INTO C (...) VALUES (...);
  SET @o = LAST_INSERT_ID();
  -- for every personal attribute (C,n)
  INSERT INTO PersonalData (resource, subresource, owner) VALUES ('C', 'n', caller);
  SET @pd = LAST_INSERT_ID();
  INSERT INTO C_PD (C_id, PersonalData_id) VALUES (@o, @pd);
  COMMIT;
END;
```

When C has multiple personal data subresources, the link statements are generated once for each subresource within the same transaction.

If i inserts a link into an association as , where $as(as) = ((n, _C), (n', _C'))$ and one association end, say (C, n) , is personal data,

```
INSERT INTO as (n, n') VALUES (v, v');
```

then SQLSPI rewrites i into $\text{PrivInsert}(\mathcal{P}, i)$:

```
CREATE PROCEDURE PrivInsert( $\mathcal{P}, i$ ) (IN caller INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insert failed';
  END;
  START TRANSACTION;
  INSERT INTO as (n, n') VALUES (v, v');
  INSERT INTO PersonalData (resource, subresource, owner) VALUES ('C', 'n', caller);
  SET @pd = LAST_INSERT_ID();
  INSERT INTO as_PD (n, n', PersonalData_id) VALUES (v, v', @pd);
  COMMIT;
END;
```

Example 6.4. Suppose a user with identifier u creates a new Event with identifier e and assigns themselves as its owner.

```
INSERT INTO Event_owner_events_Person (owner, events) VALUES (u, e);
```

Because the association end (Person, events) is personal data, SQLSPI rewrites this insertion into the following stored procedure:

```
CREATE PROCEDURE PrivInsert( $\mathcal{P}, i$ ) (IN caller INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN ROLLBACK;
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Insert failed';
  END;
  START TRANSACTION;
  INSERT INTO Event_owner_events_Person (owner, events) VALUES (u, e);
  INSERT INTO PersonalData (resource, subresource, owner)
    VALUES ('Person', 'events', caller);
  SET @pd = LAST_INSERT_ID();
  INSERT INTO Event_owner_events_Person_PD (owner, events, PersonalData_id)
    VALUES (u, e, @pd);
  COMMIT;
END;
```

Calling this stored procedure with u as caller inserts the association link into the database. It also creates a PersonalData row owned by u and links that row to the newly created association link.

SELECT statements. SQLSPI extends the $\text{SecQuery}(\mathcal{S}, q)$ rewriting from Section 2.4.5 to a privacy-aware rewriting $\text{SecPrivQuery}(\mathcal{S}, \mathcal{P}, q)$. In addition to the security predicates, the generated stored procedure $\text{SecPrivQuery}(\mathcal{S}, \mathcal{P}, q)$ realizes the characterization of privacy enforcement on SQL queries in Section 6.1.4 by invoking the privacy predicates from Section 6.2.2. For each accessed personal attribute or association end (C, n) , it performs both the consent checking using the CheckConsent_C_n predicate and the purpose limitation using the $\text{CheckPurposeLimitation_C_n}$ predicate.

Example 6.5. Assume that a user with identifier u and role FREEUSER, whose role identifier is i , accesses an endpoint whose actual purpose is CORE. The endpoint triggers the following query:

```
SELECT name FROM Person WHERE Person_id = 1
```

Since name is personal data, SQLSPI rewrites the query into the following stored procedure $\text{SecPrivQuery}(\mathcal{S}, \mathcal{P}, q)$:

```

CREATE PROCEDURE SecPrivQuery( $\mathcal{S}, \mathcal{P}, q$ ) (IN caller INT, IN role INT, IN purpose
    VARCHAR(255))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Unauthorized access'
    ; END;
    START TRANSACTION;
    IF NOT EXISTS (
        SELECT 1 FROM Person p WHERE p.Person_id = caller AND p.role = role
    ) THEN
        SIGNAL SQLSTATE '45000';
    END IF;
    -- security
    IF EXISTS (
        SELECT 1 FROM Person p
        WHERE p.Person_id = 1
        AND AuthFunc_Person_name(p.Person_id, caller, role) = 0
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Security: Unauthorized';
    END IF;
    -- privacy
    IF EXISTS (
        SELECT 1 FROM Person p
        WHERE p.Person_id = 1
        AND (CheckPurposeLimitation_Person_name(p.Person_id, caller, purpose) = 0
            OR CheckConsent_Person_name(p.Person_id, purpose) = 0)
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Privacy: Unauthorized';
    END IF;
    -- execute the original query
    SELECT p.name FROM Person p WHERE p.Person_id = 1;
    COMMIT;
END;

```

The application then calls $\text{SecPrivQuery}(\mathcal{S}, \mathcal{P}, q)(u, i, \text{CORE})$ instead of executing the original query directly. In addition to the authorization check, the procedure verifies that access to `Person.name` is permitted under the actual purpose `CORE` and that the data owner has consented to this purpose. If any security or privacy check fails, the procedure signals an SQL exception, rolls back the transaction, and returns an unauthorized-access error. If all checks succeed, the procedure executes the original query and returns its result.

The model-based characterization and the SQLSPI tool operate directly on SQL queries over a relational database schema. In modern application development, however, developers rarely write SQL statements. Instead, they define application logic over objects and delegate persistence to an Object-Relational Mapping (ORM) framework, which maps objects to relational tables and automatically issues the corresponding SQL statements. To make SQLSPI more pragmatic, we integrate the tool with the ORM workflow.

6.2.4 The Object-Relational Mapper Interceptor

In this section, we show how SQLSPI can be integrated into applications developed using ORM frameworks through a database interceptor. The interceptor transparently rewrites SQL statements emitted by the ORM into the security- and privacy-aware stored procedures generated by SQLSPI. Developers can therefore continue writing (object-oriented) application code, while fine-grained access control, purpose limitation, and consent enforcement are applied at the database layer.

Background

An ORM maps object-oriented classes to relational tables and translates operations over objects into SQL statements. This allows developers to work in a general-purpose programming language, such as C# or Python, while delegating database persistence and SQL generation to the ORM. ORMs also help support different database systems by translating programmatic queries into the SQL dialect of the target database management system. In this work, we focus on the Entity Framework Core (EF Core), a lightweight, extensible, open-source, and cross-platform ORM developed by Microsoft [88].

Example 6.6. Using EF Core, the EventPlatform data model can be defined directly as the following C# entity classes. Each class maps to a database table and its properties to columns, while navigation properties (together with foreign keys) capture the associations.

```
public class Person {
    [Key] public int Id { get; set; }
    public string? name { get; set; }
    public string? surname { get; set; }
    public ICollection<Event>? events { get; set; }
}

public class Event {
    [Key] public int Id { get; set; }
    public string? title { get; set; }
    public string? description { get; set; }
    public bool? private_ { get; set; }
    public Person? owner { get; set; }
}
```

The listing above shows the Person and Event entities, related through the Ownership association. The Event attribute `private_` carries a trailing underscore because `private` is a reserved keyword in C#.

Code-First approach. EF Core supports a Code-First approach [85], where developers define the application’s data model as C# classes and the corresponding database schema is automatically derived through database migrations. As a result, developers can directly perform common database operations, such as reading, inserting, updating, and deleting records, at the application layer without manually writing SQL queries. EF Core supports such database interactions through the Language-Integrated Query (LINQ) [83].

LINQ to SQL translation. Whenever a LINQ query is issued against the database context, EF Core first represents the query as an expression tree. The database provider then translates this expression tree into the corresponding SQL query for the target database management system.

Example 6.7. Consider a LINQ query over the EF Core context `db` that retrieves the titles of all events named “Pizza Night”:

```
var titles = db.Events.Where(e => e.title == "Pizza Night")
                        .Select(e => e.title)
                        .ToList();
```

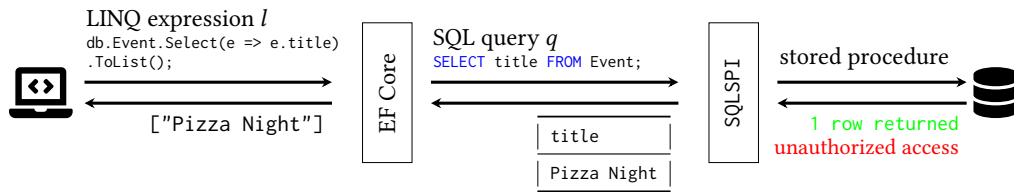
EF Core builds an expression tree from this query, and the database provider translates it into the following SQL, which is then executed on the relational database management system:

```
SELECT e.title
FROM Event AS e
WHERE e.title = 'Pizza Night';
```

We integrate SQLSPI by adding an interceptor that captures the SQL statements generated from LINQ queries and rewrites them into calls to the SQLSPI-generated stored procedures. In this way, fine-grained access control, consent, and purpose limitation are enforced at the database layer, while developers continue writing ordinary LINQ queries. Neither the business logic nor the LINQ code needs to change.

The Interceptor Design

The interceptor relies on two components. First, a database interceptor captures the SQL emitted by EF Core for each query and rewrites it into a call to the corresponding SQLSPI stored procedure. Second, a middleware component supplies the request context required by these stored procedures, namely the current user, their roles, and the active purposes. The application passes this context to the interceptor along with each query. Figure 6.2 gives an overview of the approach at runtime.



In this workflow, the developer writes a LINQ expression l over the EF Core context and eventually receives the result.

EF Core translates the LINQ expression l into an SQL query q , and would normally execute it directly on the database.

Instead, the SQLSPI rewrites the query into a stored procedure. It either returns the authorized result or an exception.

Figure 6.2: Runtime workflow of the SQLSPI ORM interceptor.

To make this integration work in practice, SQLSPI needs a model view of the ORM classes. We therefore translate the C# entity classes into the SQLSPI data model, which the tool uses for query rewriting. For actual purpose tracking, we rely on the aspect-oriented library PostSharp [107] to instrument methods and class properties. For example, when an instrumented method is called, its actual purpose is pushed onto the request context stored in the middleware; when the method exits, the purpose is popped.

6.3 Evaluation

SQLSPI is a stand-alone code generator prototype implemented in Java. We assess the performance overhead incurred by SQLSPI and how this overhead scales with the size of the input artifacts. In this evaluation, we distinguish two costs: the cost of *generating* a stored procedure and the auxiliary functions it calls, and the cost of *executing* the generated procedure against the database.

All experiments were conducted on a machine with an Intel Core i5-11300H CPU, 3.10 GHz, four cores, and 16 GB of RAM. SQLSPI generates SQL stored procedures, which are executed at runtime by the C#/.NET application through the interceptor. Thus, the generator and the evaluated application belong to separate technology stacks.

SPI-RQ1 How much time does SQLSPI take to generate a stored procedure, and how does this cost scale with the size of the models?

To evaluate generation time, we run SQLSPI as a stand-alone tool, i.e., without the ORM interceptor, over synthetically generated data, security, and privacy models of increasing size. For a fixed query, we measure the time required to produce the corresponding

stored procedure and auxiliary functions. To isolate the cost of SQLSPI itself, we subtract the fixed initialization overhead, such as Java Virtual Machine startup time.

Experiment configurations. We consider four configurations that differ in which models are supplied:

- no-model: neither a security nor a privacy model is supplied, so the generated procedure performs no enforcement;
- sec-only: a security model is supplied, so the procedure enforces access control;
- priv-only: a privacy model is supplied, so the procedure enforces privacy;
- sec-priv: both models are supplied, so the procedure enforces both.

In each experiment, we report only the configurations whose generation cost actually depends on the dimension being scaled. For the data model experiment, this includes all four configurations. Even when the supplied security or privacy model is *empty*, SQLSPI still emits enforcement predicates for data resources. For the security and privacy model experiments, by contrast, we report only the two configurations that contain the model being scaled. More specifically, we report sec-only and sec-priv for the security model, priv-only and sec-priv for the privacy model.

Table 6.1: Generation experiments summarized by model size.

Model	Varied component	Range	Components held fixed
Data ($\mathcal{D}$)	classes, attributes, associations (CS = attr = as)	16–4,096	$\mathcal{S}$ and $\mathcal{P}$ are empty
Security ($\mathcal{S}$)	roles, full access ($\mathcal{R}$)	16–1,024	CS = attr = as = 16; empty $\mathcal{P}$
	permission assignments ($\mathcal{PA}$)	1–16	CS = attr = as = 16; $\mathcal{R} = 1$; empty $\mathcal{P}$
Privacy ($\mathcal{P}$)	personal data ($\text{Props}^{\mathcal{P}}$)	16–1,024	CS = as = 16, attr = 1,024; empty $\mathcal{S}$; $\mathcal{PA}^{\mathcal{P}} = \Pi = 16$
	policies ($\mathcal{PA}^{\mathcal{P}}$)	16–1,024	CS = as = 16, attr = 1,024; empty $\mathcal{S}$; $\text{Props}^{\mathcal{P}} = 1,024$, $\Pi = 16$
	purposes (Π)	16–256	CS = attr = as = 16; empty $\mathcal{S}$; $\text{Props}^{\mathcal{P}} = \mathcal{PA}^{\mathcal{P}} = 16$

Model size. To identify which parts of the models drive generation cost, we vary one model at a time, while keeping the other models at their default sizes. We vary the number of classes (CS), attributes (attr), and associations (as) in the data model; the number of roles ($\mathcal{R}$) and authorization permissions ($\mathcal{PA}$) in the security model; and the number of personal data declarations ($\text{Props}^{\mathcal{P}}$), declared privacy policies ($\mathcal{PA}^{\mathcal{P}}$), and purposes (Π) in the privacy model. Table 6.1 summarizes these experiments.

For the data model experiment, each generated class contains one attribute, and the number of associations is set equal to the number of generated classes. Thus, in these experiments, the numbers of classes, attributes, and associations are equal. When a security or privacy model is present, it is empty (no roles, permissions, personal data, purposes, or declared policies).

For the security model experiments, the data model is fixed at 16 classes, 16 attributes, and 16 associations. We then consider two cases. In the first case, we vary the number of roles from 16 to 1,024, and each role is granted full access. In the second case, we fix the number of security roles to one and increase the number of permission assignments from 1 to 16; each assignment grants this role the right to create and delete on one class, and the right to read and update on all attributes of that class.

For the privacy model experiments, we do not distinguish between basic and complex purposes, as the purpose hierarchy affects runtime enforcement rather than generation. When varying the number of personal data declarations, the data model has 16 classes with 1,024 attributes and 16 associations, and we declare an increasing number of attributes as personal data, from 16 to 1,024, while keeping the associations fixed so that only the personal data varies. When varying the number of declared privacy policies, we keep the same data model, declare all 1,024 attributes as personal data, and generate from 16 to 1,024 policies, each assigning a randomly selected purpose to personal data. Finally, when varying the number of purposes, we use a minimal data model of 16 classes, attributes, and associations with 16 personal data declarations and 16 policies, and increase only the number of purposes from 16 to 256.

Query patterns. For each configuration and model size, we report the average generation time over 100 randomly generated queries. These queries are evenly distributed across four query patterns, with 25 queries per pattern:

- `SELECT items FROM C WHERE exp,`
- `SELECT items FROM as WHERE exp,`
- `SELECT items FROM C JOIN C' ON exp1 WHERE exp2,`
- `SELECT items FROM C JOIN as ON exp1 WHERE exp2.`

where C and C' are classes, and as is an association. For each query pattern, we instantiate the placeholders randomly from the generated model. For example, in the first pattern, we randomly choose a class to instantiate C . In what follows, we group the results by the model component being varied and discuss them in turn.

Data model experiment. Figure 6.3 shows how generation time scales as the size of the data model increases. With both security and privacy enforcement enabled, generation time increases from about 0.16 s at 16 units to about 2.3 s at 4,096 units. The difference

between configurations becomes more visible at larger model sizes. At 4,096 units, sec-only generation takes about 1.5 s, priv-only about 2.2 s, and sec-priv about 2.3 s. This increase is expected because larger data models induce more generated auxiliary functions, regardless of the size of the security and privacy models. Privacy enforcement accounts for most of the additional generation costs since it requires SQLSPI to generate substantially more privacy predicates.

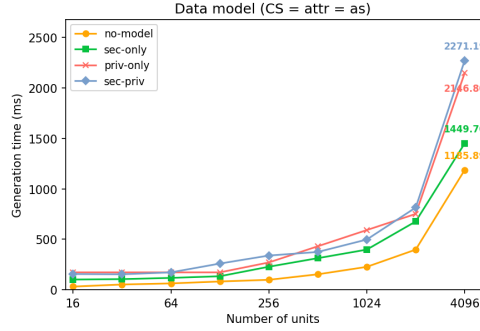


Figure 6.3: Generation time as the data model grows.

Security model experiments. As shown in Figure 6.4, when a security model is present, generation time increases with the number of roles, from about 0.18 s at $R = 16$ to about 0.47 s at $R = 1,024$. The sec-only and sec-priv configurations exhibit similar trends, and the difference between them is attributed to the generated auxiliary artifacts related to the privacy model. The number of permission assignments, in contrast, has little effect on generation time. This is because permissions only extend the body of predicates rather than introducing new ones. The experiments show that, in addition to the data model, generation cost is also affected by the number of roles in the security model, but not substantially by the number of authorization permissions.

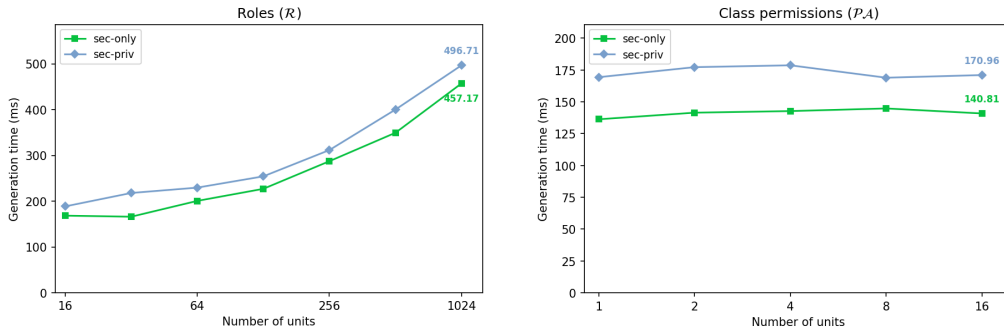


Figure 6.4: Generation time as the security model grows: number of roles (left) and number of permission assignments (right).

Privacy model experiments. As far as the privacy model is concerned, the generation cost is mainly driven by the number of personal data declarations. With the fixed data model, as shown in Figure 6.5, generation time increases with respect to the number of personal data declared, from about 0.3 s at 16 personal data to about 0.8 s at 1,024 personal data in the priv-only configuration. This is expected, since each personal data declaration introduces additional privacy-related auxiliary predicates, such as consent checking predicates. The number of policies and purposes, by contrast, has only a mild effect. These components mainly affect the bodies of existing generated predicates rather than introducing many additional auxiliary functions or stored procedures.

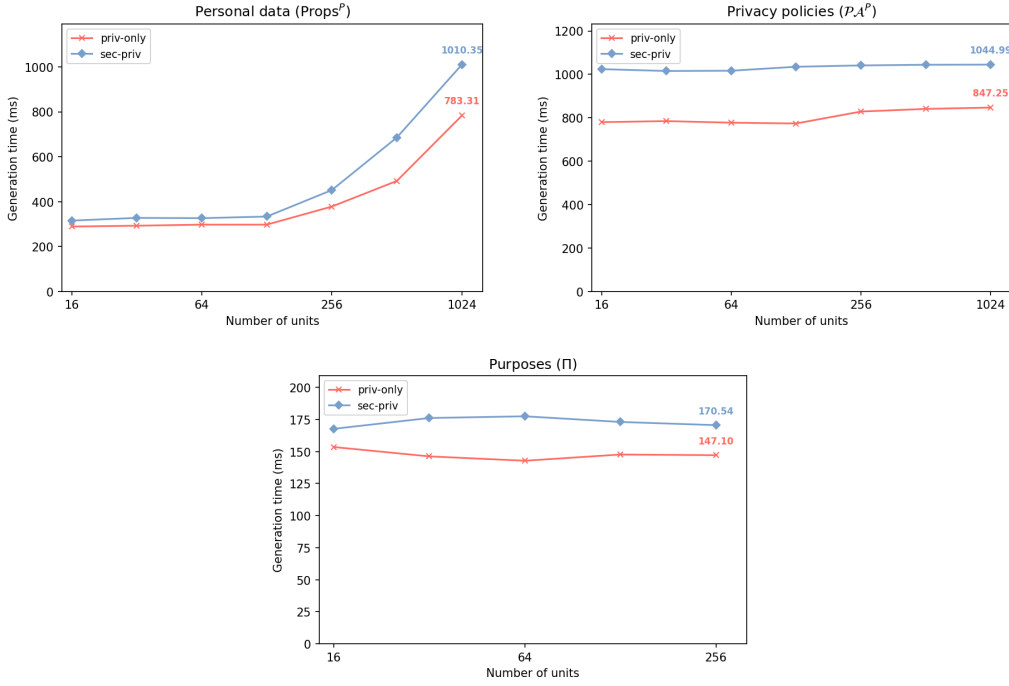


Figure 6.5: Generation time as the privacy model grows: personal data (top left), policies (top right), and purposes (bottom).

SPI-RQ2 How much runtime overhead do the generated stored procedures incur, and how does this overhead scale with the size of the database?

Case study. To evaluate runtime overhead, we compare each generated stored procedure with the corresponding native SQL query on the same database instance of the MiniTweet (for short, X) case study. The MiniTweet case study is described in Appendix E. Its schema contains two classes: a person class and a post class. Persons can author posts and follow other persons.

We scale the database instance by a parameter N . More specifically, the MiniTweet database instance $O_X(N)$ contains N persons and $10N$ posts; each person authors ten posts, and each person follows the $N - 1$ other persons.

Experiment configurations. We compare several configurations, each combining a security and a privacy model.

For security, we consider three cases: *none*, where no security model is provided; *allow-none*, where every access is denied, i.e., an empty security model; and *allow-all*, where every access is granted, i.e., a full-access security model.

For privacy, we consider four cases: *none*, where no privacy model is provided; *no-pd*, where no personal data is declared; *all-pd*, where all attributes and association ends are declared as personal data but no consent is provided; and *all-pd-consent*, where all attributes and association ends are declared as personal data and consent is provided.

Table 6.2 lists the resulting configurations. The top left combination corresponds to the native query and performs no enforcement. The green squares (configurations 1, 5, and 7) are permissive configurations that return authorized results; the red squares (configurations 2, 3, 4, and 6) raise either security or privacy exceptions.

Table 6.2: The runtime experiment configurations.

Security model ($\mathcal{S}$)	Privacy model ($\mathcal{P}$)			
	none	no-pd	all-pd	all-pd-consent
none	1 (native)	–	–	–
allow-none	–	2	3	4
allow-all	–	5	6	7

Query patterns. We evaluate the following three representative queries:

Query #1: (Person attributes)

```
SELECT name, verified FROM Person
```

Query #2: (Follow association)

```
SELECT followers, follows FROM Follow WHERE follows = 1
```

Query #3: (Person-Authorship join)

```
SELECT p.name, a.posts FROM Person p JOIN Authorship a ON p.Person_id = a.author
```


For each query, configuration, and database size $N = 2, 4, \dots, 128$, we report the median execution time over 25 runs. We also checked that each generated procedure behaves as expected. For every query, configuration, and database size, we compared the procedure output with the native query result. Under allowed configurations, such as `allow-all` or `all-pd-consent`, the procedure returns exactly the native result. Under restrictive configurations, such as `allow-none` or `all-pd`, it raises an authorization or consent error and returns no data. All 21 checks passed, corresponding to 3 queries and 7 experiment configurations.

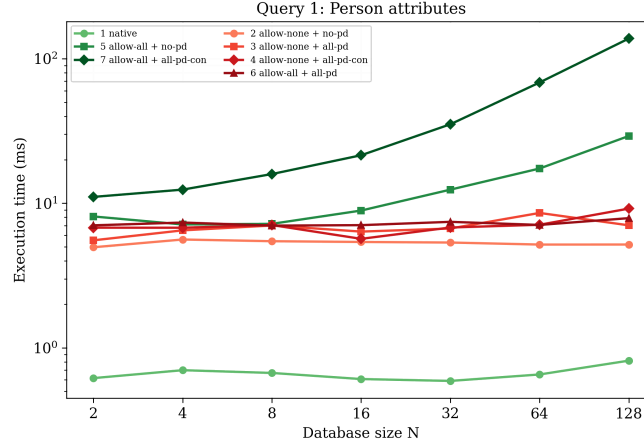


Figure 6.6: Execution time of Query #1.

Query #1. Figure 6.6 reports the execution time for the first query. Native query execution remains essentially flat, at around 0.8 ms, for all database sizes N . The generated stored procedures introduce additional overhead, but this overhead remains modest when the procedure refuses to return the query results. In the restrictive configurations, such as `allow-none` or `all-pd` without consent, execution stays roughly between 5 and 9 ms, since the procedure raises security or privacy exceptions instead of returning the full result.

For configurations in which the generated procedure is allowed to return the query result, the overhead is higher. With access control only, `allow-all + no-pd` increases from about 8 ms at ($N = 2$) to about 30 ms at ($N = 128$). The highest cost appears in `allow-all + all-pd-consent`, which increases from about 10 ms to more than 100 ms. This is expected since these procedures must check the relevant security permissions for each returned Person object, and moreover, when all attributes are declared personal data and consent is provided, the stored procedure must additionally perform consent checks before returning the attributes.

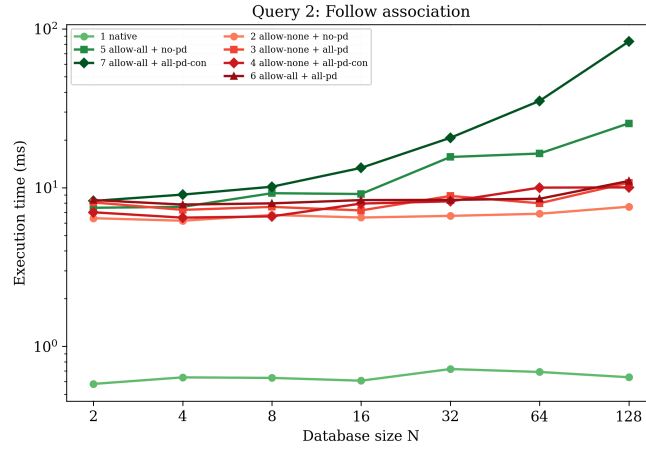


Figure 6.7: Execution time of Query #2.

Query #2. Figure 6.7 shows that queries over associations are far more expensive than single-class queries. To enforce such queries, the generated procedure must check read permissions and consent for each candidate pair in the Cartesian product of the two association ends. If each end contains on the order of N objects, this amounts to $O(N^2)$ pairs to check, explaining the quadratic growth in execution time.

Restrictive configurations, such as allow-none or all-pd without consent, remain roughly between 5 and 11 ms, because the procedure raises an exception as soon as it encounters the first unauthorized access and therefore does not evaluate the full Cartesian product. The overhead is higher when the procedure is allowed to return the association links. The allow-all + no-pd configuration increases from about 8 ms at ($N = 2$) to about 25 ms at ($N = 128$). The highest cost occurs in allow-all + all-pd-consent, which increases from about 8 ms to about 80 ms.

This is expected because SQLSPI produces stored procedures that respect the given security and privacy characterizations, i.e., it must not only evaluate the join but also check the relevant permissions for each returned object and association link; in the consent-enabled privacy configuration, it must additionally perform consent checks before returning the personal data.

Query #3. The generated procedures, in contrast to the first query, show a much larger overhead because the query ranges over a complete join between the two classes of an association. Even under the restrictive configurations, which eventually raise an exception, the execution time increases from around 10 ms at ($N = 2$) to about 2 s at ($N = 128$).

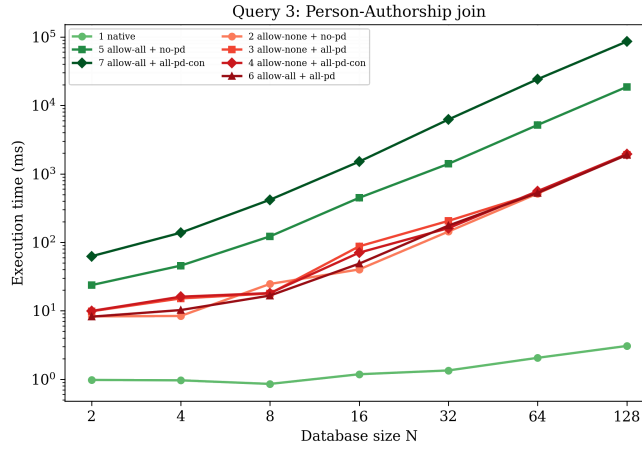


Figure 6.8: Execution time of Query #3.

The overhead is naturally the highest for configurations that are allowed to return the query result. The allow-all + no-pd configuration increases from about 25 ms at ($N = 2$) to roughly 18 s at ($N = 128$). Furthermore, when all data is declared personal data and consent is provided, the allow-all + all-pd-consent configuration is even more expensive, increasing from about 60 ms to roughly 80–90 s.

Summary. Generation time increases with the size of the input models, with the data model being the main cost driver. Within the privacy model, the number of personal-data declarations has the strongest effect. Nevertheless, this generation cost is paid only once per distinct query shape, rather than during every execution, because the interceptor caches the generated stored procedures.

Execution overhead depends strongly on the query pattern. For queries over classes, the overhead remains small. For association-related queries, however, the generated stored procedures must evaluate candidate links between the classes at both ends of the association, rather than only the links that occur in the query result. This is expensive, as the number of pairs to be checked grows rapidly with the database size, causing execution overhead to increase much more quickly. In the worst case we measured, namely association queries under full privacy enforcement with consent at the largest database size ($N = 128$), execution time reaches tens of seconds. This represents a stress-test scenario rather than typical usage.

Part III

Empirical Evaluation

Literature Review

The literature reviewed in this chapter was selected to cover empirical work related to security and privacy in software development. We considered studies that empirically evaluate model-driven and code-centric development, as well as the implementation and evolution of security and privacy requirements.

We organize the literature into three categories, namely (i) comparison between model-driven and code-centric development, (ii) empirical studies on security, and (iii) empirical studies on privacy. At the end of this chapter, Table 7.1 summarizes the literature reviewed in this chapter. Each row lists a related work, the context of the study, and the study participants.

Empirical comparisons of model-driven and code-centric development

In an academic setting, Roussev [114] compares model-driven and code-centric development approaches with undergraduate student participants and shows that model-driven approaches can improve students' overall programming skills and help them manage system complexity as the system evolves.

Domingo et al. [38] conduct a controlled experiment comparing a domain-specific language (i.e., a model-driven approach) for video game development against a traditional programming approach. Their experiment measures the development time, the number of functional requirements correctly implemented, and the conformance to the given formal specification. The results show improvements in both development efficiency and specification conformance when using the model-driven approach. While this study focuses on functional correctness in a specific development domain (i.e., video game development) using a specialized domain-specific language, the work in this thesis focuses on security and privacy requirements in web applications using more general-purpose modeling languages.

Several studies examine the relationship between the quality of the design model and the resulting implementation. Arisholm et al. [9] study the role of design models in maintenance tasks and show that developers who work with models are more efficient and introduce fewer faults when making changes to an existing system. Nugroho and Chaudron [97] later also reach a similar conclusion, showing that higher quality design models tend to produce implementations with fewer defects and better maintainability scores.

Fulton et al. [48] further analyze the relationship between early system design and security outcomes. Although this work does not compare model-driven and code-centric development approaches, it is worth mentioning that their results indicate that development teams investing more effort in the design phase prior to the implementation introduce fewer security errors and are more successful at repairing vulnerabilities identified by others. In their work, access control policies are written and enforced manually within the application code.

Hutchinson et al. [64, 138] examine the organizational and social factors that influence the adoption of model-driven engineering in practice. They conduct interviews with industrial organizations that applied a model-driven approach to their software development and report that successful adoption of a model-driven approach depends not only on tool support, but also on organizational commitment and gradual integration of modeling practices. Kuhn et al. [70] conduct a qualitative case study involving 20 employees working in different roles at a large automotive company and identify several challenges. In particular, they observe that although model-driven approaches provide languages with good abstraction, they do not necessarily help developers solve specific technical problems.

Liu et al. [76] examine how large language models (LLMs) integrate with model-driven development methodology. They analyze more than 1,500 Stack Overflow discussions related to model-driven development with and without LLM assistance. Their results show that LLM-assisted model-driven development can support a wider range of application domains compared to applying model-driven methodology alone, but it also introduces greater uncertainty, as LLMs may generate erroneous code or unsatisfactory solutions.

Empirical studies on security

Parkinson et al. [104] present a systematic survey of empirical assessments of access control mechanisms. Their study shows that relatively few empirical works evaluate access control. Among the studies identified, most works rely on synthetic datasets, and only a small number of them use real-world data. Most importantly, the survey does not include empirical comparisons between model-driven and code-centric approaches for implementing access control, which confirms that the question we study in this thesis remains largely unaddressed.

Acar et al. [2] conduct a controlled experiment with 54 Android developers performing security programming tasks using different information resources, including Stack Overflow, official documentation, and books. Their results show that developers relying on Stack Overflow produce less secure implementations than those using official documentation or books.

Several empirical studies investigate security development practices in academic settings. The multi-year “Build It, Break It, Fix It” contests [115, 135, 103, 48] evaluate secure coding practices through competitive programming exercises. In these contests, teams of undergraduate participants first implement (i.e., *build*) a software system, after which other teams attempt to discover security vulnerabilities (i.e., *break*) in the implementation. The original teams subsequently repair (i.e., *fix*) the identified flaws. These competitions provide valuable insights into the types and frequency of security vulnerabilities introduced by developers.

Industrial evidence with model-driven security has also been reported. Clavel et al. [28] report industrial experience with model-driven security using SecureUML and ComponentUML models that are automatically transformed into C++ code. Their results highlight benefits such as model reuse and support for system evolution. However, the study is primarily qualitative and does not provide controlled quantitative comparisons with code-centric implementations.

Other studies analyze the complexity and evolution of access control policies in real-world systems. Hwang et al. [65] analyze industrial access control datasets and show that policies tend to grow significantly in complexity over time, as new roles and permissions are added incrementally (without systematic review). Similarly, Han et al. [55] analyze datasets from industrial Internet of Things (IIoT) deployments and find that uncontrolled policy growth often leads to configuration errors and overly permissive access rights, which in turn reduce the effective security of the system. However, these studies analyze existing policies rather than examining the development processes that produced them, and do not investigate whether model-driven approaches can reduce such errors.

Model inference has also been explored as a way to support the evolution and maintenance of the system. Gauthier et al. [49] apply model inference to an existing web application and show how reasoning at the model level can help developers maintain and evolve security policies as the system’s requirements change. Specifically, they infer a SecureUML model from an existing codebase and demonstrate that subsequent policy changes can be made at the model level and propagated back to the implementation automatically. However, their evaluation is qualitative and is based on expert experience, rather than quantitative comparisons across multiple developers.

Empirical studies on privacy

Empirical research on privacy has focused primarily on three areas: measuring system compliance with privacy regulations, studying user responses to consent requests, and investigating challenges developers face when implementing privacy requirements.

Several large-scale measurement studies assess the extent to which websites comply with privacy regulations, such as cookie consent compliance [22, 80] and marketing

emails [69]. Our work differs from these studies in that we consider compliance with purpose limitation and user consent to all personal data, rather than focusing solely on cookies.

Other studies investigate how the design of consent interfaces affects user behavior. Singh et al. [125] conduct a comparative study of different consent request page designs, while Utz et al. [133] analyze how such designs influence user decisions and show that interface design can significantly affect consent behavior, often leading to uninformed user consent.

Research has also examined the challenges developers face when implementing privacy requirements. Bednar et al. [20] interview six senior developers to understand their motivations and abilities to comply with privacy regulations. Horstmann et al. [60] interview developers, privacy experts, and team coordinators, highlighting the communication challenges during privacy implementation. Franke et al. [46] survey 56 developers to study the impact of GDPR on open-source software development. Horstmann et al. [59] conduct a five-hour programming assignment in which 30 developers are asked to implement three GDPR-related tasks. Similarly, Senarath and Arachchilage [123] conduct a case study with 36 volunteer developers who manually implement privacy requirements in a healthcare application.

Table 7.1: Overview of empirical studies related to model-driven development, security, and privacy.

Work	Area	Study Participants / Dataset	Application Domain	Context	Method
Roussev [114]	○	112 undergraduate students	E-business applications	Academic	◆
Arisholm et al. [9]	○	Undergrad informatics students	ATM system & vending machine	Academic	◆
Nugroho & Chaudron [97]	○	Industrial legacy project	Healthcare systems	Industry	◆
Domingo et al. [38]	○	44 mobile student developers	Video game development	Academic	◆
Hutchinson et al. [64, 138]	○	Industrial MDE practitioners	Multiple domains	Industry	◇
Kuhn et al. [70]	○	20 developers and managers	Automotive systems	Industry	◇
Liu et al. [76]	○	1,500+ Stack Overflow discussions	Not specified	Industry	◆
Clavel et al. [28]	●	Paper authors	Internal test report system	Industry	◇
Acar et al. [2]	●	54 Android developers	Secure coding problems	Industry	◆
Build It, Break It, Fix It [115, 135, 103, 48]	●	116 development teams	Secure coding problems	Academic	◆
Bednar et al. [20]	●	6 senior developers	Not specified	Industry	◇
Senarath & Arachchilage [123]	●	36 volunteer developers	Healthcare applications	Industry	◇
Matte et al. [80]	●	Large-scale web measurement	Web applications	Industry	◆
Utz et al. [133]	●	82,890 website visitors	E-commerce websites	Industry	◆
Kubicek et al. [69]	●	7,694 (EU) websites	Web applications	Industry	◆
Singh et al. [125]	●	98 internet users	Web applications	Industry	◆
Bouhoula et al. [22]	●	97,000 websites	Web applications	Industry	◆
Franke et al. [46]	●	56 developers	Open-source software	Industry	◇
Horstmann et al. [60]	●	30 employee interviews	Not specified	Industry	◇
Horstmann et al. [59]	●	30 professional developers	GDPR implementation tasks	Academic	◆

Area: ○ Software Engineering ● Security ○ Privacy ● Security & Privacy

Method: ◇ Qualitative ◆ Quantitative ◇ Mixed

Study Design

To evaluate the effectiveness of model-driven approaches, we conducted an empirical study as part of a project in the *Security Engineering* course at ETH Zurich [40]. The study was carried out over four consecutive course editions, spanning 2022 to 2025. Each edition involved two distinct groups of students who attended the course that year. The course focuses on software engineering principles for building secure and privacy-aware systems. As part of the course project, students were required to develop a secure and privacy-compliant web application twice: once using our model-driven approach and once using the code-centric approach.

In the model-driven approaches, students were tasked with analyzing the given access control and privacy requirements and using the respective tools to design the corresponding security or privacy models. Development in this setting takes place during the design phase of the software development cycle.

In contrast, the code-centric approaches required students to implement the requirements directly in the application logic, either in an *ad hoc* manner or with the support of existing libraries. Development in this setting therefore occurs mainly during the implementation phase of the software development cycle.

Table 8.1 provides an overview of the methodologies and tools used across all four editions. Our study focuses on the following classes of security and privacy requirements: *access control*, *purpose limitation*, and *user consent*. We use *vActionGUI* and its predecessor *ActionGUI* for the model-driven approach. For code-centric approaches, students are free to choose any additional libraries to complete the tasks in the targeted Flask web framework. Only in the final edition of the study did we require students to use the state-of-the-art Cedar framework [32] for security specification and enforcement.

Table 8.1: Overview of the empirical study requirements and approaches.

Year	Requirements	Model-driven	Other Approaches
2022	Security	ActionGUI (Sec. 2.3)	Code-centric (Sec. 8.1)
2023	Security	ActionGUI	Code-centric
2024	Security + Privacy	<i>vActionGUI</i> (Chap. 5)	Code-centric
2025	Security + Privacy	<i>vActionGUI</i>	Cedar (Sec. 8.1.4) + Code-centric

In the subsequent sections, we introduce the code-centric approach using the Flask web framework (Section 8.1), the empirical study organization (Section 8.2), the re-

search questions (Section 8.3), our evaluation methodology (Section 8.4), and ethical considerations (Section 8.5).

8.1 Code-Centric Approach

In our study, we exemplify the code-centric approach using Flask [43], a Python-based web framework. Our choice of Flask as the code-centric baseline is representative: Python is amongst the most popular programming languages for web development, and Flask ranks as the second most widely used Python web framework [127], making it a realistic choice for real-world applications.

Flask maps URLs to Python functions, called *endpoints*, which implement the functionality provided by the web application when a user accesses a specific URL. To achieve this, developers decorate (using Python’s @ syntax) the desired function with the `route(url)` function of a Flask application object, where the parameter `url` specifies the target URL string:

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/hello")
6 def hello():
7     return "Hello world!"
```

Here, the initialized object `app` represents the web application. The function `hello()` implements the functionality of the endpoint and is bound to the URL path `/hello` by means of the `@app.route("/hello")` decorator. A visit to the endpoint `/hello` executes the function, and the string `"Hello world!"` is returned as the response displayed in the browser.

8.1.1 SQLAlchemy: The Persistence Library

Flask typically relies on external libraries such as SQLAlchemy [126] to manage application state. SQLAlchemy allows developers to declare Python classes as persistent and synchronizes in-memory objects with their corresponding records in a relational database stored on disk. This allows developers to work with Python objects, while the library transparently issues the appropriate SQL commands in the background.

As an example, the following code listing shows a simple database schema with two classes and two associations. The class `Person` defines two attributes: `id`, an integer representing the primary key, and `name`, a string representing the person’s name

(lines 10–12). The class `Event` defines two attributes: `id`, the primary key, and `title`, a string representing the event’s title (lines 16–18). An event can be associated with exactly one person as its owner (line 13). Conversely, a person may create multiple events; these are the person’s events. In addition, a person may attend events; these are the person’s attends events. Symmetrically, an event may have none or many attendants (lines 5–8).

```

1  from flask_sqlalchemy import SQLAlchemy
2
3  db = SQLAlchemy()
4
5  atts = db.Table('atts',
6                  db.Column('person_id', db.Integer, db.ForeignKey('person.id'), nullable=False),
7                  db.Column('event_id', db.Integer, db.ForeignKey('event.id'), nullable=False)
8  )
9
10 class Person(db.Model):
11     id      = db.Column(db.Integer, primary_key=True)
12     name    = db.Column(db.String(100))
13     events = db.relationship('Event', backref='owner')
14     attends = db.relationship('Event', secondary=atts, backref='attendants')
15
16 class Event(db.Model):
17     id      = db.Column(db.Integer, primary_key=True)
18     title   = db.Column(db.String(80), nullable=False)
19     person_id = db.Column(db.Integer, db.ForeignKey('person.id'))

```

Suppose that the code listing above is stored in a `model.py` file. The following statement fetches the row with the primary key value 3 from the `Event` table and returns it as a Python object of the `Event` class (line 3). The `event.title` accesses the `title` attribute of the event (line 4), while the list comprehension iterates over the associated attendants and extracts their names (line 5).

```

1  from model import Event
2
3  event = Event.query.get(3)
4  title = event.title
5  attendant_names = [p.name for p in event.attendants]

```

8.1.2 Templates

Each endpoint returns an HTML page to be rendered for the user. These pages are organized into *templates*, where static parts are defined once, and dynamic parts are computed and inserted at runtime. Flask relies on the Jinja templating engine [129]. Developers typically use the `render_template(t, ps*)` function, which combines the static HTML template *t* with dynamic parameters *ps*.

Consider the endpoint `/view_event`, which decorates the `view_event()` function. A request to the endpoint `/view_event/3` fetches the event whose identifier is 3 and returns the corresponding `view_event.html` HTML page, with the event object passed to the template for rendering.

```
1 @app.route("/view_event/<int:event_id>")
2 def view_event(event_id):
3     event = Event.query.get(event_id)
4     return render_template("view_event.html", event = event)
```

The `view_event.html` template extends the `root.html` boilerplate, which contains all headers and HTML metadata. By passing the event object to the template, its attributes can be directly accessed through Jinja expressions, e.g., `{{event.title}}`. This allows the page to present event information, including its title (lines 8–9) and the names of its attendants (lines 18–20).

```
1 {% extends 'root.html' %}
2 {% block content %}
3 <h5 class="card-header">View Event</h5>
4 <div class="card-body">
5     <div class="card-text">
6         <div class="form-group">
7             <label for="title" class="form-label">Title</label>
8             <input type="text" name="title" id="title" value="{{ event.title }}"
9                 class="form-control" disabled>
10         </div>
11     </div>
12 </div>
13 <div class="card-header">Attendants:</div>
14 <div class="card-body">
15     <table id="attendantsTbl">
16         <thead><tr><th>Name</th></tr></thead>
17         <tbody>
18             {% for u in event.attendants %}
19             <tr><td>{{ u.name }}</td></tr>
20             {% endfor %}
21         </tbody>
22     </table>
23 </div>
24 {% endblock %}
```

8.1.3 Flask-User: Authentication Library

For user authentication, we rely on the Flask-User library [130]. This library requires that developers annotate one persistent class to represent the application users. When an endpoint is invoked by an authenticated user, the library initializes a global-scope object called `current_user` with the appropriate instance of the annotated user class. If

the endpoint is invoked in the context of a non-authenticated user, the `current_user` object remains uninitialized, and its field `is_authenticated` is set to `False`. Developers can programmatically enforce authentication using an explicit check:

```
1 @app.route("/view_event/<int:event_id>")
2 def view_event(event_id):
3     if not current_user.is_authenticated:
4         return redirect(url_for("login"))
5     event = Event.query.get(event_id)
6     return render_template("view_event.html", event = event)
```

Alternatively, Flask-User provides a `@login_required` decorator, which ensures that the annotated endpoint can only be invoked by authenticated users. This guarantees that the `current_user` object is always properly initialized during the request. When any non-authenticated user accesses the endpoint, the decorating function will redirect to the login page by default.

```
1 @app.route("/view_event/<int:event_id>")
2 @login_required
3 def view_event(event_id):
4     event = Event.query.get(event_id)
5     return render_template("view_event.html", event = event)
```

8.1.4 Access Control

In addition to authentication and credential management features, Flask-User provides role-based access control enforcement through its `@roles_required` decorator. Continuing with the above example, we extend the database schema with a `Role` class and associate it with the `Person` class.

```
1 class Role(db.Model):
2     id = db.Column(db.Integer(), primary_key=True)
3     name = db.Column(db.String(50), unique=True)
4
5 class Person(db.Model):
6     ...
7     roles = db.relationship('Role', secondary='user_roles')
8
9     @property
10    def role(self):
11        return self.roles[0] if self.roles else None
12
13 class UserRoles(db.Model):
14     id = db.Column(db.Integer, primary_key=True)
15     user_id = db.Column(db.Integer, db.ForeignKey('person.id'), unique=True)
16     role_id = db.Column(db.Integer, db.ForeignKey('role.id'))
```

Suppose that there is a role `FREEUSER` in the system and that only users with this `FREEUSER` role can view events. We include the `@roles_required` annotation over the `view_event` endpoint.

```
1 @app.route("/view_event/<int:event_id>")
2 @login_required
3 @roles_required(FREEUSER)
4 def view_event(event_id):
5     ...
```

This approach is convenient for applications in which role-based access control is required. When the access control policy becomes more complex, such as the one in Example 2.8, the developer can decide to define and enforce those policies manually, or rely on additional libraries, or both.

Recall the security policy in Example 2.8: “A `FREEUSER` can read the title of an event if the event is public or if the user is the event’s owner, a manager, or an attendant”. In what follows, we describe an example of each of the proposed solutions. We begin with the most straightforward approach.

Manual Implementation

In this approach, the developer implements the enforcement check directly in the endpoint’s body. More specifically, the application first queries the event from the database and evaluates the authorization constraint before rendering the page. The authorization constraint (lines 6–9) is a direct Python implementation of the OCL constraint from Example 2.8. If the constraint is not satisfied, the request is rejected.

```
1 @app.route("/view_event/<int:event_id>")
2 @login_required
3 @roles_required(FREEUSER)
4 def view_event(event_id):
5     event = Event.query.get(event_id)
6     if not event.private \
7         or event.owner == current_user \
8         or current_user in event.managedBy \
9         or current_user in event.attendants:
10         return render_template("view_event.html", event = event)
11     else:
12         return render_template("error.html",
13                                message="You are not allowed to read this event")
```

Although this approach is simple and requires no additional libraries, the authorization logic is interleaved with the business logic. As the number of endpoints and policies grows, maintaining consistency and correctness of the application’s authorization becomes error-prone.

Flask-Principal

Flask-Principal [109] is an authorization framework for Flask that organizes access control around the notions of *Needs* and *Permissions*.

A *Need* is considered the smallest unit of access control, represented as a tuple, e.g., ('role', Freeuser) or ('read_title', 42). A *Permission* is a set of *Needs*. After a user logs in, the application computes its *Permission*, a set of *Needs*, and associates it with an Identity object representing the user. To obtain access for a certain action, the user's Identity must provide the right *Needs*. The following code snippet shows how to enforce the policy of Example 2.8 using Flask-Principal. First, we define a custom Need and Permission for reading an event's title:

```

1  from flask_principal import Principal, Identity, Permission, \
2      RoleNeed, identity_loaded, identity_changed
3  from collections import namedtuple
4  from functools import partial
5
6  principals = Principal(app)
7
8  EventNeed = namedtuple('Event', ['method', 'value'])
9  ReadTitleNeed = partial(EventNeed, 'read_title')
10
11 class ReadTitlePermission(Permission):
12     def __init__(self, event_id):
13         need = ReadTitleNeed(event_id)
14         super().__init__(need)

```

Whenever a user logs in, we compute all the needs the user is entitled to and add them to the user's Identity object. In this case, we assign for each user their own RoleNeed; and for every event that is public or where the user is the owner, a manager, or an attendant, we add a ReadTitleNeed:

```

1  @identity_loaded.connect_via(app)
2  def on_identity_loaded(sender, identity):
3      identity.user = current_user
4      if hasattr(current_user, 'id'):
5          identity.provides.add(RoleNeed(current_user.role.name))
6          for e in Event.query.all():
7              if not e.private \
8                  or e.owner == current_user \
9                  or current_user in e.managedBy \
10                 or current_user in e.attendants:
11                  identity.provides.add(ReadTitleNeed(e.id))

```

Finally, in the `view_event` endpoint, we implement a conditional statement to check whether the user has the required permissions.

```

1  @app.route("/view_event/<int:event_id>")
2  @login_required
3  def view_event(event_id):
4      event = Event.query.get(event_id)
5      rolePerm = RoleNeed(FREEUSER)
6      readPerm = ReadTitlePermission(event_id)
7      if rolePerm.can() and readPerm.can():
8          return render_template("view_event.html", event = event)
9      else:
10         return render_template("error.html",
11                                message="You are not allowed to read this event")

```

Flask-Principal separates the authorization logic (via the *Need* computation) from the enforcement point (via the *Permission* checks). Nevertheless, the level of abstraction remains low: the developer must manually enumerate Needs for every resource instance at login time, and the authorization constraint is still expressed imperatively via Python code, interleaved with the actual business code.

Flask-Oso

Flask-Oso [92] is a security framework in which access control policies are defined in a declarative language called Polar, rather than as Python code. The policy can be defined in a separate policy file (e.g., `main.polar`). Here, we declare the `Person` class as an **actor** and the `Event` class as a **resource**. The `has_permission` rule encodes the authorization constraint from Example 2.8:

```

1  actor Person {}
2
3  resource Event {
4      permissions = ["read_title"];
5      roles = ["FREEUSER"];
6  }
7
8  allow(actor, action, resource) if has_permission(actor, action, resource);
9
10 has_role(p: Person, "FREEUSER", _: Event) if p.isUser();
11
12 has_permission(p: Person, "read_title", e: Event) if
13     has_role(p, p.role.name, e) and
14     (not e.private or
15      e.owner == p or
16      p in e.managedBy or
17      p in e.attendants);

```

Here, `p.isUser()` holds for any `Person` that is an authenticated application user, so every such user is assigned the `FREEUSER` role.

In the application, first, an `Oso` object is initialized and the relevant classes are registered. The policy file `main.polar` is loaded into the engine:

```
1 from oso import Oso, ForbiddenError
2
3 oso = Oso()
4 oso.register_class(Event)
5 oso.register_class(Person)
6 oso.register_class(Role)
7 oso.load_files(["main.polar"])
```

The authorization check in the endpoint is replaced with a call to `oso.authorize`:

```
1 @app.route("/view_event/<int:event_id>")
2 @login_required
3 def view_event(event_id):
4     event = Event.query.get(event_id)
5     try:
6         oso.authorize(current_user, "read_title", event, check_read=False)
7         return render_template("view_event.html", event = event)
8     except ForbiddenError:
9         return render_template("error.html",
10                                message="You are not allowed to read this event")
```

Compared to the previous approach, Flask-Oso provides a cleaner separation of concerns, i.e., the access control policy is defined declaratively and stored in a separate artifact. Although language expressiveness is equivalent to Flask-Principal, the Polar language allows direct references to object attributes and associations. Finally, in terms of policy enforcement, developers must still manually place `oso.authorize` calls at the appropriate enforcement points.

Cedar

Cedar [32] is an open-source policy language and authorization engine developed by Amazon. Cedar structures each authorization request using the so-called Principal-Action-Resource-Context (PARC) model.

Cedar uses a schema language that describes the structure of the authorization domain. A schema declares the entities, their attributes, and the actions that apply to them. For example, the following schema translates the event platform's data model, stored in a `main.cedarschema` file.

```

1  entity Role;
2
3  entity Person in Role {
4      id: Long,
5      name: String,
6      surname: String,
7      role: Role
8  };
9
10 entity Event {
11     title: String,
12     private: Bool,
13     owner: Person,
14     managedBy: Set<Person>,
15     attendants: Set<Person>
16 };
17
18 action readTitle, createEvent, editEvent, manageEvent appliesTo {
19     principal: Person,
20     resource: Event
21 };

```

Policies are written in `.cedar` files as permit or forbid rules. Given a set of policies and a request, the Cedar engine returns *allow* if and only if at least one permit policy matches and no forbid policy matches; otherwise, it returns *deny*. The following rule encodes the authorization constraint from Example 2.8. The policy is defined in the `main.cedar` file.

```

1  permit (
2      principal in Role::"FREEUSER",
3      action == Action::"readTitle",
4      resource is Event
5  ) when {
6      !resource.private ||
7      resource.owner == principal ||
8      principal in resource.managedBy ||
9      principal in resource.attendants
10 };

```

Principals and resources are represented by unique identifiers of the form `T::"I"` (e.g., `Person::"Alice"`). The Cedar engine resolves references (e.g., `resource.private`) by looking up the corresponding entity in an *entity store*, a JSON structure that maps each entity to its attributes and parent relationships. A membership check is expressed via the `in` keyword (e.g., `principal in Role::"FREEUSER"`), which Cedar resolves through the entity's parent hierarchy.

To bridge the gap between the application's Python objects and Cedar's expected input format, one option is to provide a customized `EntitySerializer` class that converts

application objects into Cedar unique identifiers and JSON entries for the entity store. Each application class may be registered with the serializer, specifying how its attributes and relationships are mapped.

```

1  serializer = EntitySerializer()
2
3  serializer.register(
4      Person, "Person",
5      lambda p: { "role": p.role.name if p.role else "VISITOR" },
6      lambda p: []
7  )
8
9  serializer.register(
10     Event, "Event",
11     lambda e: {
12         "owner": serializer.entity_reference_json(e.owner),
13         "managedBy": [serializer.entity_reference_json(u) for u in e.managedBy],
14         "attendants": [serializer.entity_reference_json(u) for u in e.attendants],
15     },
16     lambda e: []
17 )

```

The last argument of each register call is a lambda returning the entity's parent entities in Cedar's entity hierarchy; here it returns an empty list, as neither Person nor Event belongs to a parent group.

In the Flask application, the developer initializes a Cedar client and calls its authorization function at the appropriate enforcement point:

```

1  from cedar.authz import CedarClient, EntitySerializer
2
3  cedar = CedarClient(serializer=EntitySerializer(),
4                      policy_file="main.cedar",
5                      schema_file="main.cedarschema")
6
7  @app.route("/view_event/<int:event_id>")
8  @login_required
9  def view_event(event_id):
10     event = Event.query.get(event_id)
11     try:
12         cedar.assert_allowed(current_user, "readTitle", event)
13         return render_template("view_event.html", event = event)
14     except SecurityException:
15         return render_template("error.html",
16                                message="You are not allowed to read this event")

```

The CedarClient takes this serializer together with the policy and schema files. It provides the assert_allowed method, which serializes the principal and resource into

Cedar's format, invokes the Cedar engine, and raises a `SecurityException` if the verdict is *deny*.

Cedar provides the clearest separation between the policy and application code among the approaches presented: the policy is fully declarative, resides in a separate file, and is evaluated by an external engine. Similarly to Flask-Oso, the developer must still manually insert authorization calls at each enforcement point and maintain the entity store that bridges the application's data model and the Cedar engine.

The four approaches represent an increasing degree of formality in access control requirements specification, from policies manually hardcoded in application code to fully declarative policies in a separate specification language. As far as enforcement is concerned, all four approaches share a fundamental limitation: enforcement check placement must be done manually. In other words, when using these approaches, the developer must identify and instrument each enforcement point in the application code. A single missed point creates a security vulnerability.

8.1.5 Privacy Requirements

To the best of our knowledge, there is no existing library that supports enforcing purpose limitation and user consent in the Flask framework for building web applications. In this section, we describe our manual setup for defining and enforcing these policies in the code-centric approach used for the project.

In accordance with privacy regulations such as the GDPR and in the context of our project, applications must enforce data privacy principles that ensure that personal data is collected and processed for specified purposes, and only with the informed consent of the data subject. Concretely, the developer must (i) identify which data elements constitute personal data, (ii) define the purposes for which personal data is processed, (iii) declare which personal data is accessed for which purpose, (iv) provide a consent collection mechanism, and (v) verify user consent before processing their personal data.

Personal Data and Ownership

In our project, personal data is defined at the level of class properties, i.e., individual attributes and association ends, rather than at the level of entire classes (see Section 4.2). For example, the name and surname attributes of the `Person` class are personal data, while the `id` attribute is not. In the Flask implementation, personal data is referenced by pairs of class and property names, e.g., ("`Person`", "`name`").

Data ownership links each personal data item to the user to whom it belongs. To make this relationship explicit in the database schema, we add a property reference owner to the personal data class that refers to the data's owner:

```
1 class Person(db.Model):
2     @property
3     def owner(self):
4         return self
```

This allows the application to identify the data owner when performing consent checks on any personal data instance. For example, when displaying a user's name on a page, the application can determine that the owner whose consent must be verified is the person to whom that name belongs.

Purpose and Its Hierarchy

Purposes represent the reasons for which personal data may be processed. We model purposes as a persistent class with a self-referential foreign key that captures the purpose hierarchy. A purpose that includes another is considered more general: consenting to a parent purpose implies consent to all of its children.

```
1 class Purpose(db.Model):
2     id = db.Column(db.Integer, primary_key=True)
3     name = db.Column(db.String(100))
4
5     parent_id = db.Column(db.Integer, db.ForeignKey('purpose.id'), nullable=True)
6     parent = db.relationship('Purpose', remote_side=[id], backref='children', lazy=True)
```

The parent_id column references another row in the same table, forming a tree structure. A purpose with parent_id set to None is a root purpose.

Privacy Policy Declaration

To declare which personal data is accessed for which purpose, we define a dictionary PRIVACY_INPUT that encodes the application's privacy policy. Each entry in the dictionary represents a purpose and contains: (i) a list of (class_name, property_name) pairs identifying the personal data accessed under that purpose; (ii) a dictionary of child purposes, reflecting the hierarchy; and (iii) an optional constraint description displayed in the privacy notice.

As a concrete example, consider the privacy model from Example 5.8, where the name and surname attributes and the events and attends association ends of a Person object are personal data. Three purposes are declared: Core, Analytics, and Any, where Any includes the other two. The corresponding PRIVACY_INPUT dictionary is:

```
1 PRIVACY_INPUT = {
2     ANY: {
3         "data": [],
4         "children": {
5             CORE: {
6                 "data": [("Person", "name"),
7                         ("Person", "events"),
8                         ("Person", "surname")],
9                 "children": {},
10                "constraintDesc": "core functionality of the platform"
11            },
12            ANALYTICS: {
13                "data": [("Person", "attends")],
14                "children": {},
15                "constraintDesc": "compute attendance statistics for events"
16            },
17        },
18    }
19 }
```

Consent Management

To capture user consent, the application maintains a Consent class that records which user has consented to which purpose for which personal data item:

```
1 class Consent(db.Model):
2     id = db.Column(db.Integer, primary_key=True)
3     classname = db.Column(db.String(100))
4     propertyname = db.Column(db.String(100))
5     person_id = db.Column(db.Integer, db.ForeignKey('person.id'), nullable=False)
6     purpose_id = db.Column(db.Integer, db.ForeignKey('purpose.id'), nullable=False)
7
8     user = db.relationship('Person', backref='consents', lazy=True)
9     purpose = db.relationship('Purpose', backref='consents', lazy=True)
```

Next, a dedicated `/policy` endpoint renders a consent form based on the PRIVACY_INPUT dictionary:

```
1 @app.route("/policy")
2 @login_required
3 def policy():
4     consents = current_user.consents
5     return render_template('policy.html',
6                             dpurposes = PRIVACY_INPUT, consents = consents)
```

The endpoint passes the declared purposes and the user's existing consent records to the template, which presents each purpose with its constraint description and a toggle

for granting or revoking consent. Users interact with this form through two endpoints that modify consent records:

```

1 @app.route("/grant_consent", methods=['POST'])
2 @login_required
3 def grant_consent():
4     data = request.get_json()
5     my_purpose = Purpose.query.filter_by(name = data.get('purpose')).first()
6     db.session.add(
7         Consent(classname = data.get('class_name'),
8                 propertyname = data.get('property_name'),
9                 user = current_user,
10                purpose = my_purpose))
11     db.session.commit()
12     return redirect(url_for('policy'))

```

```

1 @app.route("/revoke_consent", methods=['DELETE'])
2 @login_required
3 def revoke_consent():
4     data = request.get_json()
5     my_purpose = Purpose.query.filter_by(name = data.get('purpose')).first()
6     consents = Consent.query.filter_by(
7         person_id = current_user.id,
8         purpose_id = my_purpose.id,
9         classname = data.get('class_name'),
10        propertyname = data.get('property_name')).all()
11     for consent in consents:
12         db.session.delete(consent)
13     db.session.commit()

```

Finally, to verify whether a user has consented to a specific purpose for a given personal data item, the application provides a `has_consent` function that queries the stored consent records:

```

1 def has_consent(owner, data_pair, purpose_name):
2     purpose = Purpose.query.filter_by(name = purpose_name).first()
3     return Consent.query.filter_by(
4         person_id = owner.id, purpose_id = purpose.id,
5         classname = data_pair[0],
6         propertyname = data_pair[1]).first() is not None

```

This function takes as arguments the data owner, a pair (`class_name`, `property_name`) identifying the personal data, and the name of the purpose. It returns `True` if a matching consent record exists. Note that this implementation checks consent by exact purpose name and does not traverse the purpose hierarchy. Thus, consent to a more general parent purpose is not automatically treated as consent to its child purposes. Supporting hierarchical consent must therefore be implemented manually by the developer, illustrating the additional care required in code-centric implementations.

Privacy Policy Enforcement

Enforcing the privacy policy requires the developer to address two concerns at every endpoint that accesses personal data: *consent checking* and *purpose limitation*.

Consent checking. Before accessing any personal data, the developer must verify that the data owner has given consent for the relevant purpose. For example, the `view_event` endpoint displays the full name of the event's owner by accessing the name and surname attributes for the Core purpose (from Example 5.8). The developer must check that the event owner has consented to this usage before rendering the page:

```
1 @app.route("/view_event/<int:event_id>")
2 @login_required
3 def view_event(event_id):
4     event = Event.query.get(event_id)
5     owner = event.owner
6     if has_consent(owner, ("Person", "name"), CORE) \
7         and has_consent(owner, ("Person", "surname"), CORE):
8         fullname = owner.getFullname()
9     else:
10        fullname = "RESTRICTED"
11    return render_template("view_event.html", event = event, owner_fullname = fullname)
```

If the event owner has not consented to the Core purpose for their name or surname, the full name is replaced with "RESTRICTED" before rendering. This check must be performed individually for each data owner. For instance, if the template also renders the names of the event's attendants, the developer must iterate over all attendants and check consent for each one separately.

Purpose limitation. Beyond consent checking, the developer must also reason about the *actual purpose* (see Section 4.2) under which personal data is accessed. In the above example, it may seem straightforward to determine that CORE is the actual purpose of accessing the owner's name within `view_event`. However, in more complex applications, endpoints may invoke shared methods or call other endpoints for different purposes. The developer must ensure that the purpose passed to `has_consent` matches the actual purpose of the data access in the current execution context. A mismatch, for example, checking consent for CORE when data is actually being processed for ANALYTICS, constitutes a privacy violation even if the user has granted consent for both purposes, because the declared and actual purposes are not aligned.

These two concerns must be addressed consistently across all endpoints that access personal data.

8.2 Study Organization

Table 8.2 presents the complete timeline of our study. Each row corresponds to a project phase, while the columns specify the input provided, the expected deliverables (if any), and the duration of that phase. The main project (highlighted in blue) consists of three phases: Development (conducted in two stages, Stage I and Stage II), Repair, and Evolution.

Table 8.2: Empirical study timeline and structure.

Phase	Input	Deliverable	Duration
Training (All groups)	Training materials		6 weeks
Development (Stage I) (Group A) (Group B)	<i>Base Requirements</i> <i>TemplateMDE</i> <i>TemplateFlask</i>	<i>MDE models</i> <i>Flask implementations</i>	2 weeks
Development (Stage II) (Group A) (Group B)	<i>TemplateFlask</i> <i>TemplateMDE</i>	<i>Flask implementations</i> <i>MDE models</i>	2 weeks
Repair (All groups)	<i>Test Summary</i> <i>TestsMDE</i> <i>TestsFlask</i>	<i>MDE models</i> <i>Flask implementations</i>	1 week
Evolution (All groups)	<i>New Requirements</i> <i>PatchMDE</i> <i>PatchFlask</i>	<i>MDE models</i> <i>Flask implementations</i>	1 week
Survey (All groups)	Questionnaire	Anonymous answers	2 weeks

Training Phase. To build a common foundation required for the project, in the first half of the course, students are introduced to key security and privacy concepts, such as requirements engineering, model-driven security, secure coding, purpose limitation, and consent collection. To help them become familiar with the tools used in the project, we provide several exercises focused on the Flask web framework and model-driven development tools. The students are also asked to implement several model-driven and Flask applications as part of the lab exercises. Although these exercises are part of the course, they are optional. Students are encouraged to attend tutorials during the exercise and lab sessions. All solutions are available on the course Moodle webpage. We survey the students to understand to what extent they participated in the exercises.

Development Phase. The Development phase is conducted in two stages, each lasting two weeks. In Stage I, students are randomly assigned to two groups, A and B. At the beginning of this stage, both groups of students receive the same *Base Requirements* document, which describes the functional, security, and privacy requirements of the target application.

In Stage I, Group A secures the application using model-driven approaches, while Group B secures the Flask application. Each group is provided with a corresponding template, i.e., *TemplateMDE* for Group A and *TemplateFlask* for Group B, that implements the application's functional requirements. Students are tasked with eliciting and analyzing the security and privacy requirements described in the document and modifying the provided templates to enforce them.

In Stage II, the two groups switch tasks. Group A now secures the Flask application, while Group B secures the application using model-driven approaches. Each group receives the remaining template corresponding to its new task.

Repair Phase. In the Repair phase, we provide unit tests (*TestsMDE* and *TestsFlask*) designed to evaluate whether the security requirements are correctly implemented and enforced in both applications. A *Test Summary* document provides an overview of the tests and their coverage. Each student is required to study the tests and repair both implementations to pass as many tests as possible. As the Repair phase only occurs in the first edition of the study, no privacy requirements are introduced. Consequently, no privacy tests are provided in this phase.

Evolution Phase. In the Evolution phase, we introduce new functional and security requirements through the *New Requirements* document. We also provide updated versions of the application templates, i.e., *PatchMDE* and *PatchFlask*, that already implement the new functional requirements. Students are again asked to elicit and analyze the new requirements and implement them in both of their applications.

Survey Phase. After the course ends, we conduct an anonymous survey to gather data on the participants' experiences and opinions, as well as the time spent on each project phase. The survey is open for two weeks, and invitations to participate are distributed by email to all students who had participated in at least one project phase.

8.3 Research Questions

The research questions are categorized into two groups: those concerning access control, which apply to all four editions of the study, and those concerning privacy, which apply only to the two most recent editions.

Security Research Questions. Our focus in security is twofold: the access control policy and the access control mechanism that enforces it. In practice, the distinction between policy and mechanism is often blurred, as access control checks are frequently hardcoded directly into the application code. This entanglement makes policies difficult to understand, verify, and maintain.

In this study, we consider two key principles [118] that determine whether an access control policy is correct (*least privilege*) and whether its access control mechanism is properly implemented (*complete mediation*). The principle of *least privilege* requires that each user can only perform actions that are necessary to carry out their business role within the application. The principle of *complete mediation* requires that all actions be checked for authority, ensuring that policy checks cannot be bypassed.

Privacy Research Questions. Similarly to access control, the privacy compliance of an application involves both a correctly declared privacy policy and enforcement mechanisms that ensure that the policy is respected during the execution of the application. The former specifies how personal information may be used, while the latter checks for purpose limitation and user consent, which must be consistently enforced across all actions involving personal data.

In this study, we evaluate the implementation of privacy requirements using three criteria, namely, *privacy policy adequacy*, *purpose limitation*, and *consent checking*. The *privacy policy adequacy* criterion assesses whether the declared purposes in a privacy policy are reasonable. The *purpose limitation* criterion requires that any action performed on personal data be explicitly declared in the privacy policy. The *consent checking* criterion requires obtaining explicit consent from the data owner before performing any action on personal data, according to the GDPR's requirement for explicit user consent.

To evaluate how well model-driven and code-centric approaches enforce these principles and criteria, we pose the following correctness questions.

- RQ1** To what extent does modeling access control improve the enforcement of least privilege?
- RQ2** To what extent does modeling access control help achieve the principle of complete mediation?
- RQ3** To what extent does modeling privacy improve an application's implementation of purpose limitation?
- RQ4** To what extent does modeling privacy improve privacy policy adequacy?
- RQ5** To what extent does modeling privacy improve consent checking?

Using participant submissions and anonymous survey responses, we measure the effort required, in terms of lines of code and time, to implement and evolve security and privacy requirements.

- RQ6** How much coding effort is required to implement and evolve access control and privacy policy?
- RQ7** What effort is required to implement and evolve access control and privacy policy, in terms of time?

The crossover nature of our study design, in which groups switch between model-driven and code-centric approaches across the two stages of the Development phase, further allows us to investigate whether prior experience with one approach benefits performance in the other.

- RQ8** Does prior experience in designing (respectively, implementing) the same security and privacy requirements improve their implementation (respectively, design)?

Finally, we shed light on the specific difficulties that arise when designing or implementing security and privacy requirements, as well as the effects of auxiliary factors such as the choice of authorization library, the availability of regression tests, and the challenge of maintaining correctness as requirements evolve.

- RQ9** Which aspects of access control and privacy are the most challenging to design (respectively, to implement)?
- RQ10** Which aspects of access control and privacy are the most challenging to evolve?
- RQ11** Which Flask authorization library yields the most secure applications?
- RQ12** Can regression tests improve security as an application evolves?

8.4 Evaluation Methodology

We evaluate submissions using two complementary approaches: quantitative analysis and qualitative analysis.

For quantitative analysis, we employ three methods: (i) unit security and privacy tests to assess the correctness of submitted implementations, (ii) static analysis of submitted models and code to measure development effort and privacy policy adequacy, and (iii) self-reported effort estimates collected from survey responses. For qualitative analysis, we analyze the justification answers provided by participants in the survey. Since the study is conducted over four iterations with slight adjustments (which are detailed in Appendix F.1) between each iteration, not all evaluation methods apply uniformly.

The first two editions (Ed’22–23) used ActionGUI. In these editions, participants focused only on security requirements. The later two editions (Ed’24–25) used *v*ActionGUI. Here, participants were tasked with both security and privacy requirements. Consequently, privacy-related research questions can only be answered from Ed’24–25 data. The Repair phase was unique to Ed’22, so findings related to that phase do not generalize to other editions. Table 8.3 summarizes the applicable research questions per evaluation method and edition. When we report the results in Chapter 9, we report them per edition and state which editions are applicable.

Table 8.3: Applicable research questions (RQ) per evaluation method and edition.

	Ed’22	Ed’23	Ed’24	Ed’25
Security tests	RQ1, 2, 8–12	RQ1, 2, 8–11	RQ1, 2, 8–11	RQ1, 2, 8–11
Privacy tests	—	—	RQ3, 5, 8–10	RQ3, 5, 8–10
LoC submissions	RQ6	RQ6	RQ6	RQ6
Privacy model	—	—	RQ4	RQ4
Survey	RQ7	RQ7	RQ7	RQ7

8.4.1 Quantitative Analysis

Security Tests

To assess whether the access control requirements are correctly implemented, we use two sets of unit tests. The first set consists of more than 1,000 manually written tests, designed to exercise the security requirements in both the base and evolved versions of the application. This set of tests is used exclusively in the first edition of the study. The second set, used in the other three editions and kept private, consists of more than

3,500 automatically generated tests, which exhaustively exercise all authorized and unauthorized actions across all roles defined in the security model. The automatic test generator takes the security model of the reference solution as input and produces test cases covering every combination of role, action, and resource.

A test passes if the application correctly allows (respectively, denies) the action being tested. We distinguish between *base tests*, which evaluate the base requirements from the Development phases, and *evolved tests*, which evaluate the newly introduced requirements of the Evolution phase. The upper part of Table 8.4 reports the number of security tests evaluated per phase and edition. The complete list of security tests is provided in Tables F.1 and F.2 in Appendix F.2.

Table 8.4: Number of unit tests (Dev + Evol) per edition.

		Ed'22	Ed'23	Ed'24	Ed'25
Security (Tables F.1–F.2)	ActionGUI	455 + 108	1,970 + 294	—	—
	vActionGUI	—	—	1,019 + 70	1,357 + 71
	Flask	548 + 96	1,408 + 119	1,019 + 70	1,357 + 91
Privacy (Table F.3)	vActionGUI	—	—	280 + 26	307 + 26
	Flask	—	—	280 + 26	307 + 26

Privacy Tests

To assess whether privacy requirements are correctly implemented, we design a set of tests that evaluate compliance with purpose limitation and consent checking. The test set is not revealed to the students. These tests cover the following three scenarios: (i) a user has not consented to the use of their personal data for a declared purpose; (ii) basic consent has been provided; and (iii) consent involving hierarchical purposes was given. In the first scenario, the application must deny access to personal data. In the latter two scenarios, the application must correctly process data according to the consent granted. The lower part of Table 8.4 reports the number of privacy tests evaluated per phase and edition. The complete list of privacy tests is provided in Table F.3 in Appendix F.2.3.

Static Analysis of Submissions

We perform a static analysis on the submitted artifacts to measure two quantities: development effort (in terms of lines of code) and privacy policy adequacy.

Lines of code. We use the number of changed lines of code (LoC) as an approximation of the implementation effort. LoC changes are measured from the standard git diff output, where each added or removed line is counted as one changed line. For the Development phases, we compare the initial template against the student’s submission. For the Evolution phase, we compare the student’s final submission against the one from the previous phase.

In the case of *vActionGUI*, we analyze the models submitted (i.e., the security and privacy models). For Flask, we analyze the main project file and any additional .py files submitted. To normalize for varying levels of correctness, we report the ratio of LoC changed to the number of tests passed.

Privacy policy adequacy. In addition to testing, we statically analyze the privacy models submitted to assess the *adequacy of their policy* [P5], i.e., the extent to which the set of declared purposes aligns with the given privacy requirements. Our starting point is a reference solution: a *vActionGUI* privacy model that adheres to the given project description. The reference solution is both correct (i.e., passing all privacy tests) and adequate (i.e., manually validated by the teaching staff) with respect to the stated privacy requirements.

For *vActionGUI* submissions, we extract the set of personal data, purposes, and declared purposes from the submitted privacy model and compare them against the reference solution. For Flask submissions, we extract the declared purposes from the submitted PRIVACY_INPUT dictionary and compare them analogously. Since our reference solution declares the minimal set of declared purposes satisfying the privacy requirements, any additional declared purpose in a submission is considered *over-declared*. Although over-declared purposes do not affect the outcome of the privacy tests, they could lead to other privacy violations, e.g., violating the data minimization principle (Art. 5 §1 (c) GDPR), which states that personal data should be used only when necessary for a specific purpose.

Survey

At the end of the study, we conducted an anonymous survey to collect the experiences, opinions, and time of the participants in each phase of the project. Participation was voluntary: invitations were sent by email to all enrolled students who had participated in at least one project phase, and the survey remained open for two weeks after the course ended.

From the survey, we extract two types of quantitative data. First, we measure the *development effort* in terms of time. Since project tasks include both security and privacy requirements, we ask participants to report the total hours spent on each phase and

estimate the percentage of that time dedicated specifically to privacy requirements. The privacy development time is then computed as the product of these two values, whereas the security development time is the remainder.

Second, we measure the *perceived clarity of project requirements* using a five-point Likert scale (Strongly Disagree / Disagree / Neither Agree nor Disagree / Agree / Strongly Agree) in response to the statement: “The project requirements were sufficiently clear.”

8.4.2 Qualitative Analysis

Complementing the quantitative results, we use the survey to better understand how the participants perceive the two provided approaches. As part of the survey, we asked participants to choose which approach they would prefer for a similar future project and to justify their choice in a free-text response.

We applied an inductive qualitative coding approach to analyze participants’ free-text responses. The analysis was conducted by the author of this thesis. The responses were coded to develop a codebook. The resulting codebook is presented in Appendix F.4.

8.5 Ethical Considerations

All editions of the empirical study were approved by the ETH Zurich Ethics Review Board under reference numbers 2023-N-63, EK-2024-N-94, 25-ETHICS-048, and 26-ETHICS-088, respectively. Furthermore, they were conducted in accordance with the ethical principles established in the Menlo Report [35]. In the following subsections, we provide more details on our stakeholder-based ethics analysis.

Stakeholders. Our empirical studies involve several stakeholder groups. Primary stakeholders include computer science students participating in the course, the teaching assistant and research team, and ETH Zurich as the host institution. Secondary stakeholders include future students, the software engineering community, web developers who implement security and privacy controls, end users of privacy-aware applications, and the broader academic community.

Potential harms. We identified three primary potential harms. First, students may perceive coercion, i.e., feel that their participation could affect their grades and thus compromise their autonomy, even though there is no actual dependency. Second, code analysis could potentially reveal programming skills, work habits, and effort patterns, raising privacy concerns that may conflict with data protection regulations. Third,

students might experience psychological distress if they believe that their code quality would be scrutinized publicly beyond the normal grading process.

Mitigations. To address these concerns, we implemented the following mitigations.

Temporal separation ensures that all grading tasks were completed and final grades were issued before the research study began. Furthermore, as described in the previous section, participation in the survey occurred only after the course ended, eliminating any actual dependency and substantially reducing perceived coercion. The elective nature of the course further reinforces student autonomy, as students freely choose to enroll without any compulsory requirements.

Anonymization measures prohibit the collection of identities during code analysis. No personal data was collected. The survey responses are anonymous and cannot be linked to individual identities or grades. The results reported in this thesis are only aggregated, and no individual code is published except in anonymized form.

Transparent communication ensures that participants receive clear messaging that the study evaluates tools rather than students, with analysis conducted on anonymized code and results presented in aggregate form only. We provide study participants with the contact details of the study coordinator, the institutional data protection officer, and the ethics commission to address any concerns.

Data Protection. All submitted code and anonymous survey responses are *stored confidentially*. Only course staff had access to the code, and only the authors of this research had access to survey responses; neither dataset was shared externally. Furthermore, participants could request details about their data at any time and could restrict further processing of their data or request its deletion from the study without any negative consequences.

Benefits. We believe that the study makes a valuable contribution to security and privacy engineering while protecting student welfare, that the benefits substantially outweigh the residual risks for all stakeholders, and that the research serves the public interest in evidence-based security and privacy engineering while maintaining ethical integrity. More specifically, students are exposed to different security and privacy implementation paradigms and receive comprehensive feedback through extensive sets of test cases, gaining career-relevant security and privacy engineering skills. The software engineering community receives the first empirical comparison of model-driven and code-centric approaches to security and privacy implementation. The academic community benefits from both a research contribution and access to our open-science artifacts.

Results and Analysis

9.1 Participation and Attrition

Table 9.1 summarizes student participation and attrition across the four study editions. Non-applicable cells are marked with a minus sign (-). Each row corresponds to one edition, and each column corresponds to a project phase.

Within each edition block, the *Students* row reports the number of students with submissions and the corresponding percentage relative to the edition’s enrollment. The indented sub-rows further break down these counts by tool. For each tool, submissions are reported in the format *sec/priv/submitted*, where *sec* and *priv* denote the numbers of *valid* security and privacy submissions, respectively, and *submitted* is the total number of submissions. For example, 56/57/58 indicates 56 valid security submissions and 57 valid privacy submissions out of 58 submitted implementations.

The *Pairs* row reports the number of matching implementations submitted by the same student. For the development phases, pairs are computed across the two stage submissions of the same students. In the Repair and Evolution phases, submissions are paired by construction. No pairs are reported for the Survey phase, as it involves no implementation submissions. The bottom block of the table, *Total*, reports total participation and paired submissions across all editions.

In the first edition, Ed’22, the course consisted of 99 students, with 50 students assigned to Group A and 49 students in Group B. The Repair phase was conducted only in this edition and was removed from the study protocol thereafter; therefore, the corresponding cells for later editions are marked as non-applicable.

In the second edition, Ed’23, the course had 131 students, with 66 in Group A and 65 in Group B. In Ed’22 and Ed’23, ActionGUI was used as the model-driven engineering tool, and the study focused exclusively on the implementation of access control requirements.

The third edition, Ed’24, enrolled 81 students, with 40 in Group A and 41 in Group B. Two changes to the study protocol are worth noting. First, the model-driven tool used in the project was replaced by its successor, *vActionGUI*, while the Flask part remained unchanged. Second, the scope of the study was extended to include privacy requirements.

Table 9.1: Participation and attrition per phase across four study editions.

Edition	(Security valid submissions / Privacy valid submissions / Total submissions)					Survey
	Dev. I	Dev. II	Repair	Evolution		
First 2022 <i>n</i> = 99	Students	75 (75.8%)	71 (71.7%)	69 (69.7%)	68 (68.7%)	16 (16.2%)
	ActionGUI	35 / - / 35	38 / - / 38	69 / - / 69	67 / - / 67	-
	Flask	40 / - / 40	32 / - / 33	68 / - / 69	65 / - / 65	-
	Pairs	71 / -		68 / -	64 / -	-
Second 2023 <i>n</i> = 131	Students	94 (71.8%)	94 (71.8%)	-	90 (68.7%)	18 (13.7%)
	ActionGUI	45 / - / 49	49 / - / 49	-	86 / - / 90	-
	Flask	49 / - / 49	45 / - / 47	-	90 / - / 90	-
	Pairs	89 / -		-	86 / -	-
Third 2024 <i>n</i> = 81	Students	60 (74.1%)	57 (70.4%)	-	59 (72.8%)	23 (28.4%)
	vActionGUI	34 / 34 / 34	25 / 25 / 25	-	56 / 57 / 58	-
	Flask	26 / 26 / 26	32 / 32 / 32	-	55 / 55 / 56	-
	Pairs	57 / 57		-	54 / 54	-
Fourth 2025 <i>n</i> = 113	Students	80 (70.8%)	74 (65.5%)	-	71 (62.8%)	14 (12.4%)
	vActionGUI	42 / 42 / 42	37 / 36 / 37	-	68 / 71 / 71	-
	Flask	38 / 38 / 38	37 / 37 / 37	-	66 / 66 / 67	-
	Pairs	74 / 74		-	65 / 66	-
Total	Students	309	296	69	288	71
	Pairs	291 / 131		68 / -	269 / 120	-

Finally, the fourth edition, Ed'25, enrolled 113 students, with 57 in Group A and 56 in Group B. Compared to Ed'24, the model-driven part remained unchanged in this edition. In the code-centric approach, however, students were asked to implement security requirements using the policy-based authorization tool Cedar [32].

Due to errors that prevented evaluation, 15 submissions were excluded from the analysis. More specifically, six were Flask submissions that failed at runtime: one in Dev. II of Ed'22, one in the Repair phase of Ed'22, two in Dev. II of Ed'23, one in the Evolution phase of Ed'24, and one in the Evolution phase of Ed'25. The remaining nine were model-driven submissions with syntax errors: eight ActionGUI submissions in Ed'23, split evenly between Dev. I and the Evolution phase, and one vActionGUI submission in the Evolution phase of Ed'24. A further five vActionGUI submissions were excluded only from a single analysis dimension, since the generated code for the respective enforcement mechanism failed to compile. More specifically, four were excluded from the security analysis: one in the Evolution phase of Ed'24 and three in the Evolution phase of Ed'25. One submission from Dev. II of Ed'25 was excluded from the privacy analysis.

9.2 Collected and Derived Data

Across all editions, we collected two categories of data: student submissions produced during the project phases and anonymous survey responses collected at the end of the course. To describe these data precisely, this section introduces the notation used throughout the analysis.

9.2.1 Submissions

We denote the set of student submissions by S , as defined in Table 9.2.

Table 9.2: Notation used to identify sets of student submissions.

$S_{[Tech] [Edition] [Phase] [Group]}$	
denotes the set of $[Tech]$ submissions from $[Edition]$, in $[Phase]$, by $[Group]$, where:	
$Tech$	$:= AG \mid vAG \mid Flask$ (ActionGUI, vActionGUI, or Flask)
$Edition$	$:= 22 \mid 23 \mid 24 \mid 25$ (academic years 2022–2025)
$Phase$	$:= DevI \mid DevII \mid Dev \mid Repair \mid Evol$ (Dev. I, Dev. II, both, Repair, or Evolution)
$Group$	$:= A \mid B$ (groups)

Each submission set is indexed along four dimensions: technology, edition, phase, and group. The superscript indices $[Tech]$ and $[Edition]$ identify the *technology* used and the study *edition*, respectively. The subscript indices $[Phase]$ and $[Group]$ denote the project *phase* and the submission *group*. Whenever an index is omitted, the notation ranges over all valid values for that dimension. Table 9.2 also lists the admissible values for each index.

Example 9.1. S^{Flask} denotes the set of Flask submissions collected across all four editions, all phases, and both groups. $S_{DevI}^{AG, 22}$ denotes the set of ActionGUI submissions from Stage I of the Development phase in the first edition. Finally, $S_{Evol, A}^{vAG, 24}$ denotes the set of vActionGUI submissions from Group A in the Evolution phase of the third edition.

9.2.2 Survey Responses

Similarly, we denote the set of anonymous survey responses by R . As defined in Table 9.3, the set is indexed along two dimensions, namely, a subscript index $[Questions]$ denoting the survey question or set of questions, and the superscript index $[Edition]$ denoting the study edition. These question indices correspond to the question numbers in the questionnaires provided in Appendix F.3.

Table 9.3: Notation used to identify sets of survey responses.

$R_{[Questions]}^{[Edition]}$ denotes the set of responses from $[Edition]$ to the questions $[Questions]$, where: $Edition := 22 \mid 23 \mid 24 \mid 25$ (academic years 2022–2025) $Questions := \{1, 2, \dots, 25\}$ (question indices, see Appendix F.3)
--

Example 9.2. $R_{\{1,2,3\}}$ denotes all survey answers to Questions 1–3 concerning participants’ background knowledge of model-driven engineering tools across all editions. $R_{\{12\}}^{22}$ denotes the responses to Question 12 in Ed’22, which asks participants to report the approximate time required to complete Stage I of the Development phase.

9.2.3 Tests

In our study, we consider two categories of tests: one for security requirements and one for privacy requirements. We define these test sets in the following subsections.

Security Tests

In the first edition of the study, student submissions were evaluated using manually written unit tests. In the subsequent three editions, security tests were automatically generated using our model-based approach. We denote the manually written security test set by WST and the automatically generated security test set by GST .

Table 9.4: Notation used to identify subsets of security tests.

WST	$\begin{smallmatrix} [Tech] \\ [Req] [Role] \end{smallmatrix}$	and	GST	$\begin{smallmatrix} [Tech] \\ [Req] [Role] \end{smallmatrix}$
denote, respectively, the set of manually written and automatically generated security tests that target $[Tech]$, exercise the $[Req]$ requirements, for $[Role]$ role, where:				
$Tech \coloneqq AG \mid \nu AG \mid \text{Flask}$ (ActionGUI, ν ActionGUI, or Flask)				
$Req \coloneqq Dev \mid Evol$ (Development or Evolution)				
$Role \coloneqq V \mid F \mid P \mid M \mid A$ (caller role)				

Example 9.3. WST_{Dev}^{AG} denotes the set of manually written security tests for ActionGUI used in the Development phase, i.e., the tests released to students during the Repair phase. $GST_{Dev,A}^{\nu AG}$ represents the set of generated security tests for ν ActionGUI under the ADMIN role in the Development phase.

Privacy Tests

Privacy tests are defined under three scenarios: $\langle \text{No-Consent} \rangle$, $\langle \text{Basic} \rangle$, and $\langle \text{Complex} \rangle$. Table 9.5 defines the notation used to identify the privacy test set PT .

Table 9.5: Notation used to identify subsets of privacy tests.

PT	$\begin{smallmatrix} [Tech] \\ [Req] [Scenario] \end{smallmatrix}$
denotes the set of privacy tests that target $[Tech]$, exercise the $[Req]$ requirements, for $[Scenario]$ scenario, where:	
$Tech \coloneqq \nu AG \mid \text{Flask}$ (ν ActionGUI or Flask)	
$Req \coloneqq Dev \mid Evol$ (Development or Evolution)	
$Scenario \coloneqq \langle \text{No-Consent} \rangle \mid \langle \text{Basic} \rangle \mid \langle \text{Complex} \rangle$ (test scenario)	

Example 9.4. $PT_{\text{Dev}}^{\text{vAG}}$ denotes the set of privacy tests used to evaluate the vActionGUI submissions in the Development phase.

9.2.4 Submission Evaluation

Let S be an n -dimensional column vector of student submissions $S = [s_1 \ s_2 \ \dots \ s_n]^T$ and let T be an m -dimensional row vector of tests $T = [t_1 \ t_2 \ \dots \ t_m]$. We write $s \cdot t \in \{0, 1\}$ for the result of running test $t \in T$ on submission $s \in S$, where (1) denotes a passing test result, and (0) denotes a failing test result. Equivalently, $t \cdot s$ denotes *evaluating* test t on submission s .

Recall that the Kronecker product of an $n \times m$ matrix $A = [a_{ij}]_{1 \leq i \leq n, 1 \leq j \leq m}$ and a $p \times q$ matrix $B = [b_{ij}]_{1 \leq i \leq p, 1 \leq j \leq q}$ is the $np \times mq$ block matrix obtained by multiplying each entry of A by the entire matrix B :

$$A \otimes B = \begin{bmatrix} a_{11} \cdot B & a_{12} \cdot B & \dots & a_{1m} \cdot B \\ a_{21} \cdot B & a_{22} \cdot B & \dots & a_{2m} \cdot B \\ \vdots & \vdots & & \vdots \\ a_{n1} \cdot B & a_{n2} \cdot B & \dots & a_{nm} \cdot B \end{bmatrix}.$$

When applied to the $n \times 1$ submission vector S and the $1 \times m$ test vector T , the Kronecker product $S \otimes T$ yields an $n \times m$ binary-valued matrix

$$[s_1 \cdot T^T \quad s_2 \cdot T^T \quad \dots \quad s_n \cdot T^T]^T$$

that encodes the outcome of running every test (represented by the matrix column) on each submission (represented by the matrix row).

To mark failed tests for each submission, we perform a scalar exclusive disjunction with 1 on the matrix, i.e., $(S \otimes T) \oplus 1$, denoted as $\overline{S \otimes T}$. In this matrix, entry (1) indicates a failed test, and entry (0) indicates a passed test.

9.3 Answers to Research Questions

We now answer the research questions introduced in Section 8.3. For each research question, we first describe the relevant data, then present the analysis, and finally report the statistical evidence and justification supporting our conclusions. All statistical tests are reported together with their corresponding significance levels.

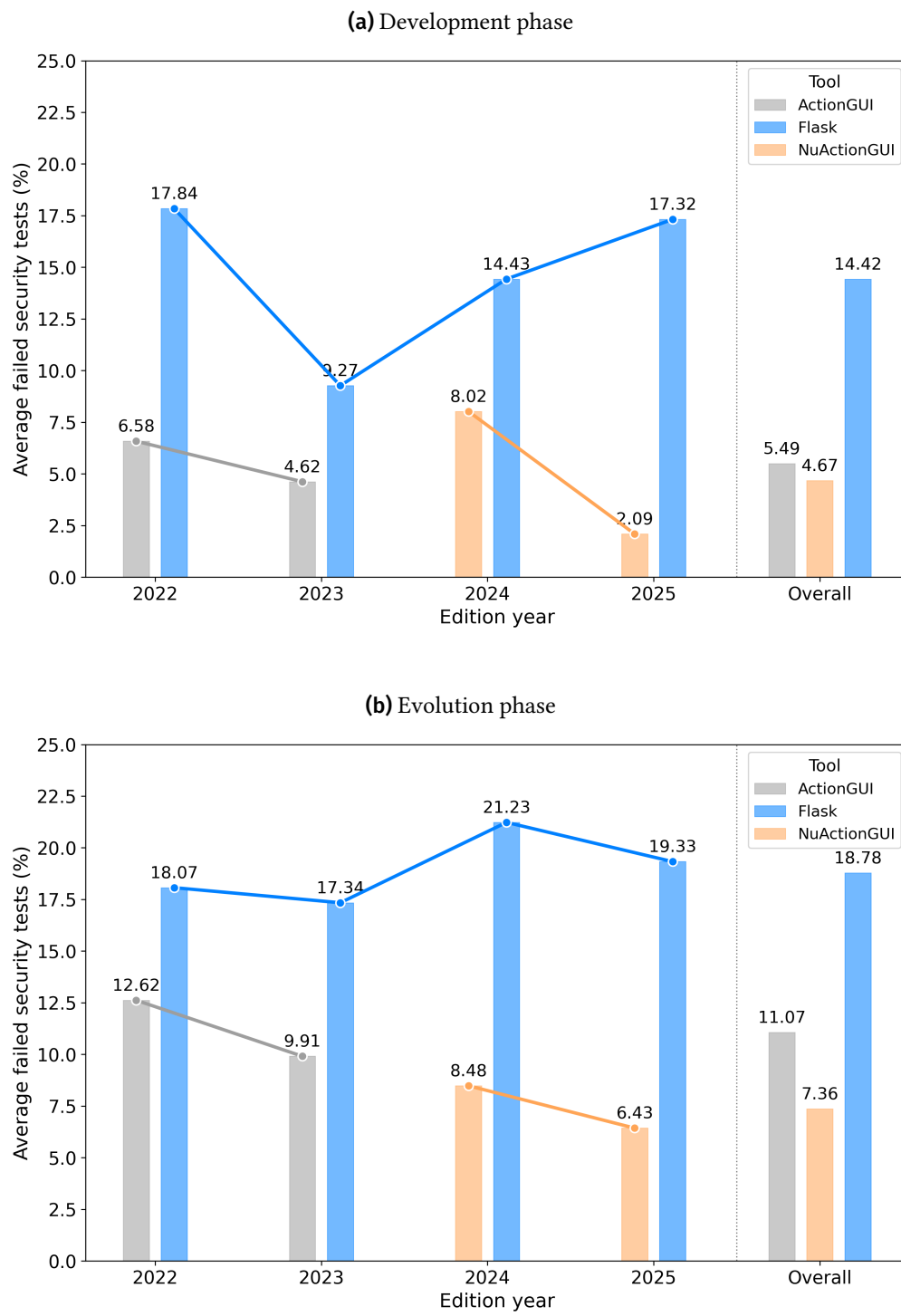


Figure 9.1: Average percentage of failed security tests (by edition, approach).

RQ1 To what extent does modeling access control improve the enforcement of least privilege?

Finding. Modeling access control significantly improves the enforcement of least privilege. Model-driven submissions fail only 4.7–5.5% of the generated security tests on average, compared with 14.4% for code-centric Flask submissions.

Data. To answer this research question, we ran the automatically generated security test sets against the security submissions collected in each edition. Table 9.6 summarizes the evaluation scheme.

For the Development phase, the ActionGUI test set GST_{Dev}^{AG} was applied to the submissions from Ed’22 and Ed’23 because these two editions share the same security requirements, as shown in the first row. Similarly, the corresponding ν ActionGUI test set $GST_{Dev}^{\nu AG}$ was applied to the submissions from Ed’24 and Ed’25, as shown in the second row.

For the Evolution phase, the test sets GST_{Evol}^{AG} and $GST_{Evol}^{\nu AG}$ were additionally run against the corresponding Evolution submissions, as shown in the last two rows. These tests evaluate the evolved security requirements introduced in the Evolution phase.

Table 9.6: Test sets and paired submission sets used to evaluate **RQ1**.

Test set	Submission sets	Pairs
$\{GST_{Dev}^{AG}, GST_{Dev}^{Flask}\}$	$\{S_{DevI,A}^{AG,22}, S_{DevII,A}^{Flask,22}\}, \{S_{DevII,B}^{AG,22}, S_{DevI,B}^{Flask,22}\}$	71
	$\{S_{DevI,A}^{AG,23}, S_{DevII,A}^{Flask,23}\}, \{S_{DevII,B}^{AG,23}, S_{DevI,B}^{Flask,23}\}$	89
$\{GST_{Dev}^{\nu AG}, GST_{Dev}^{Flask}\}$	$\{S_{DevI,A}^{\nu AG,24}, S_{DevII,A}^{Flask,24}\}, \{S_{DevII,B}^{\nu AG,24}, S_{DevI,B}^{Flask,24}\}$	57
	$\{S_{DevI,A}^{\nu AG,25}, S_{DevII,A}^{Flask,25}\}, \{S_{DevII,B}^{\nu AG,25}, S_{DevI,B}^{Flask,25}\}$	74
$\{GST_{Evol}^{AG}, GST_{Evol}^{Flask}\}$	$\{S_{Evol}^{AG,22}, S_{Evol}^{Flask,22}\}$	64
	$\{S_{Evol}^{AG,23}, S_{Evol}^{Flask,23}\}$	86
$\{GST_{Evol}^{\nu AG}, GST_{Evol}^{Flask}\}$	$\{S_{Evol}^{\nu AG,24}, S_{Evol}^{Flask,24}\}$	54
	$\{S_{Evol}^{\nu AG,25}, S_{Evol}^{Flask,25}\}$	65

We further restrict the analysis to the subset of *paired submissions* shown in Table 9.1. A student is included in this evaluation only if they submitted both model-driven and Flask solutions across two stages of the Development phase or in the Evolution phase.

From the evaluation results, we compute the *average percentage of failed tests* for a set of submissions under a given test set. Given a submission vector S and a test vector T ,

the average failure is defined as:

$$\text{AvgFail}(S, T) = \frac{\sum_{i=1}^{|S|} \sum_{j=1}^{|T|} (\overline{S \otimes T})_{ij}}{|S| \times |T|} \quad (9.1)$$

Figure 9.1a reports the average percentage of failed generated security tests in the Development phase, grouped by edition and tool, with the aggregate total on the right side. Figure 9.1b presents these results for the Evolution phase.

Answer. Modeling access control significantly improves the enforcement of least privilege. The overall results, shown in the rightmost group of both plots in Figure 9.1, demonstrate the advantage of the model-driven approaches over the code-centric implementations. The lower failure rates indicate fewer deviations from the expected authorization behavior and, consequently, fewer violations of least privilege.

In the Development phase (Figure 9.1a), ActionGUI submissions fail 5.5% of the generated security tests on average, and ν ActionGUI submissions fail 4.7%, whereas Flask submissions fail 14.4%. Thus, Flask has a failure rate more than 2.6 $\times$ higher than ActionGUI and more than 3 $\times$ higher than ν ActionGUI.

In the Evolution phase (Figure 9.1b), ActionGUI submissions fail 11.1% of the security tests on average, ν ActionGUI submissions fail 7.4%, and Flask submissions fail 18.8%. Here, Flask has a failure rate 1.7 $\times$ higher than AG and 2.6 $\times$ higher than ν AG.

Across all editions, the model-driven submissions consistently outperform the Flask submissions. The strongest result is observed for ν ActionGUI in Ed'25, where it achieves a failure rate of only 2.1% in the Development phase and 6.4% in the Evolution phase, compared to 17.3% and 19.3% for Flask.

Justification. First, we use the percentage of failed tests per submission as a proxy for the degree to which the principle of least privilege is enforced, as fewer authorization failures correspond to fewer violations of least privilege. Second, the automatically generated test sets *GST* are applied instead of the manually written set *WST* because the latter was used and released in Ed'22 and was constructed in an *ad hoc* manner. The generated test sets, in contrast, were not published to students and provided better scenario coverage. Finally, to ensure the comparability of our analysis, we exclude students with partial submissions as well as results from the Repair phase.

We apply the Wilcoxon Signed-Rank (WSR) test to paired submissions, accounting for all editions within each phase. In the Development phase ($n = 291$), the WSR test confirms that Flask submissions have significantly more failed tests than model-driven

submissions ($p = 6.05 \times 10^{-28}$, $r = 0.642$, large effect), with a median difference of +4.79 percentage points in favor of the model-driven approach. In the Evolution phase ($n = 269$), the result is also significant ($p = 2.26 \times 10^{-12}$, $r = 0.428$, medium effect), with a median difference of +4.29 percentage points.

Table 9.7: WSR test results for **RQ1**: paired model-driven vs. Flask failure rates.

Phase	Edition	n	Median Δ	p	r	Effect
Development	All	291	+4.79 pp	6.05×10^{-28}	0.642	large
	Ed'22	71	+3.83 pp	2.81×10^{-5}	0.497	medium
	Ed'23	89	+2.00 pp	1.18×10^{-6}	0.515	large
	Ed'24	57	+3.43 pp	2.36×10^{-7}	0.685	large
	Ed'25	74	+10.50 pp	3.59×10^{-12}	0.808	large
Evolution	All	269	+4.29 pp	2.26×10^{-12}	0.428	medium
	Ed'22	64	-1.39 pp	0.815	0.029	–
	Ed'23	86	+1.68 pp	1.71×10^{-2}	0.257	small
	Ed'24	54	+5.71 pp	7.44×10^{-6}	0.610	large
	Ed'25	65	+10.15 pp	1.05×10^{-9}	0.757	large

Median Δ = median of (Flask – model-driven) per pair, in percentage points (pp).

As a secondary analysis, we perform per-edition WSR tests within each phase to assess the effect consistency. Table 9.7 reports the results. In the Development phase, the effect is significant in all four editions, with large effects in Ed'23 ($r = 0.515$), Ed'24 ($r = 0.685$), and Ed'25 ($r = 0.808$), and a medium effect in Ed'22 ($r = 0.497$). In the Evolution phase, the effect is significant with a large effect in Ed'24 ($r = 0.610$) and Ed'25 ($r = 0.757$), with a small effect in Ed'23 ($p = 1.7 \times 10^{-2}$, $r = 0.257$), and not significant in Ed'22 ($p = 0.815$).

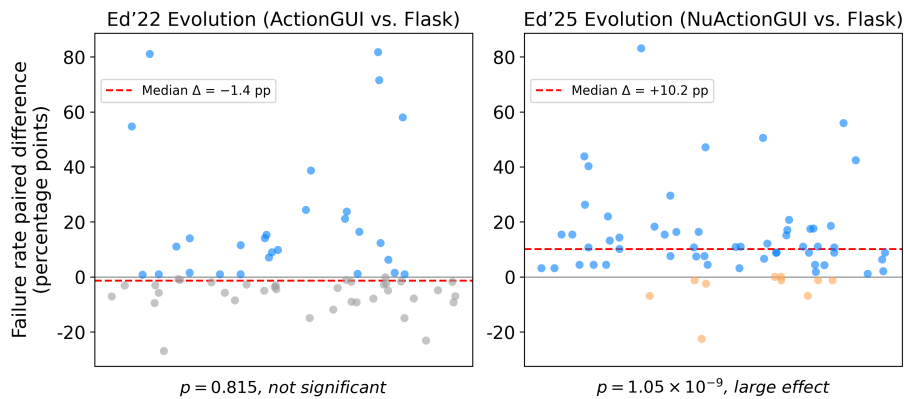


Figure 9.2: Paired failure rate differences per student (Evolution phase, Ed'22 and Ed'25).

The insignificant result for the Evolution phase in Ed’22 is explained by the paired-difference distribution in the left plot of Figure 9.2. Although the mean failure rate favors the model-driven approach, as shown in Figure 9.1b, the paired differences are scattered around zero, with a median difference of $\Delta = -1.4$ percentage points.

RQ2 To what extent does modeling access control help achieve the principle of complete mediation?

Finding. The model-driven approach enforces complete mediation *by construction*: the generated mechanism intercepts every data access, so no authorization check can be omitted. In contrast, code-centric submissions result in between 1.0 and 3.4 complete mediation violations on average, and using a Flask authorization library does not help in closing this gap.

Model-driven approaches such as ActionGUI and ν ActionGUI enforce complete mediation by construction: the generated enforcement mechanism from their security models intercepts every data access and performs authorization checks systematically. For the code-centric approach, although libraries and frameworks separate policy specification from application logic, they do not guarantee complete mediation. The developer must explicitly invoke authorization checks at every relevant endpoint. Omissions in these invocations lead to a violation of complete mediation.

Data. To answer this research question, we use the same evaluation scheme as in RQ1, but restrict the analysis to Flask submissions. Table 9.8 reports the number of Flask submissions in the Development and Evolution phases across all editions.

Table 9.8: Test sets and paired submission sets used to evaluate RQ2.

Test set	Submission sets	Submissions
GST_{Dev}^{Flask}	$\{S_{DevI,\mathbb{B}}^{Flask}, S_{DevII,\mathbb{A}}^{Flask}\}$	300
GST_{Evol}^{Flask}	$\{S_{Evol}^{Flask}\}$	276

We manually inspect each Flask submission and classify it according to its authorization strategy. Submissions that use Flask-Principal [109], Oso [92], or Cedar [5] are classified as *policy-based*, while submissions implementing authorization checks without such libraries are classified as *manual*. Table 9.9 summarizes the resulting distribution across editions and phases.¹

¹Among submissions that appear in both Development and Evolution phases, we observe only two cases where the strategy changes: one Ed’23 submission switches from Flask-Principal to manual, and one Ed’24 submission switches from Oso to manual.

Table 9.9: Policy-based and manual access control specifications in Flask submissions.

Edition	Phase	<i>n</i>	Oso	Principal	Cedar	Policy-based	Manual
Ed'22	Dev.	72	9	4	–	13	59
	Evol.	65	10	4	–	14	51
Ed'23	Dev.	94	10	28	–	38	56
	Evol.	90	10	25	–	35	55
Ed'24	Dev.	58	9	10	–	19	39
	Evol.	55	8	10	–	18	37
Ed'25	Dev.	76	–	–	76	76	–
	Evol.	66	–	–	66	66	–
Total	Dev.	300	28	42	76	146	154
	Evol.	276	28	39	66	133	143

We execute the automatically generated Flask test sets on all Flask submissions across all editions. We then identify complete mediation violations in the evaluation results obtained as follows: given a permission, if all tests associated with some endpoints fail while tests for other endpoints pass, we infer that the authorization logic is implemented correctly but is not consistently enforced across all relevant entry points. Such failure is classified as a complete mediation violation.

Answer. Model-driven approaches achieve complete mediation by construction: the generated enforcement mechanism systematically intercepts every data access, eliminating the possibility of missing authorization checks. In contrast, in the code-centric approach, developers must explicitly invoke authorization checks at every relevant endpoint. Figure 9.3 reports the average number of complete mediation violations per submission, grouped by Flask approach and edition.

No single Flask approach consistently outperforms the others. In the Development phase (Figure 9.3a), manual submissions average between 1.76 and 2.72 violations, Oso between 2.30 and 2.89, and Flask-Principal between 1.30 and 2.29. Cedar, used exclusively in Ed'25, has the highest average, with 3.35 violations.

In the Evolution phase (Figure 9.3b), the number of violations is generally lower. Manual submissions range from 1.76 to 2.16, Oso from 1.40 to 2.20, and Principal from 1.20 to 1.50, while Cedar averages 1.64 violations. Overall, all Flask approaches exhibit between 1 and 3.4 violations per submission on average. These results suggest that policy-based frameworks alone do not improve complete mediation.

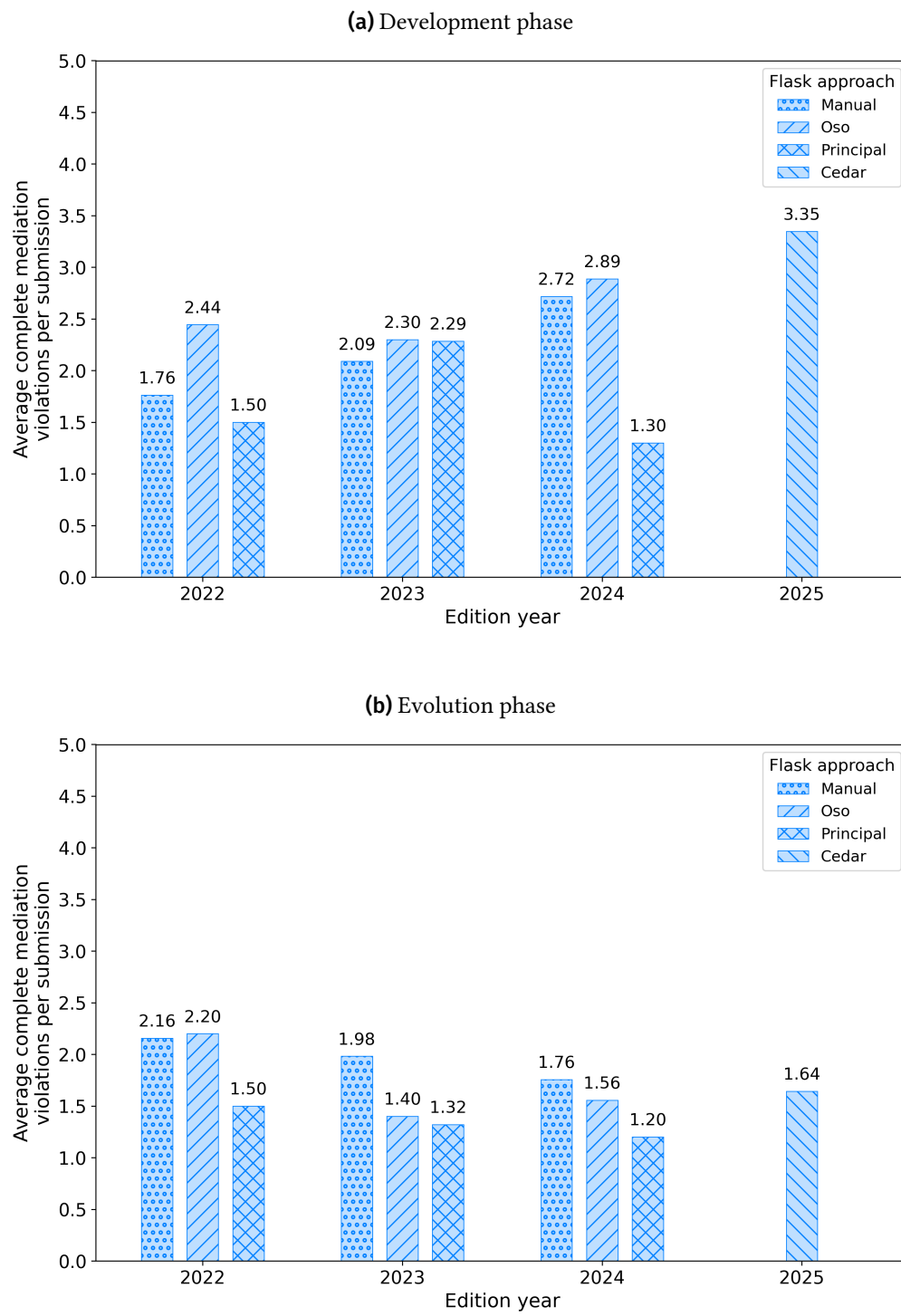
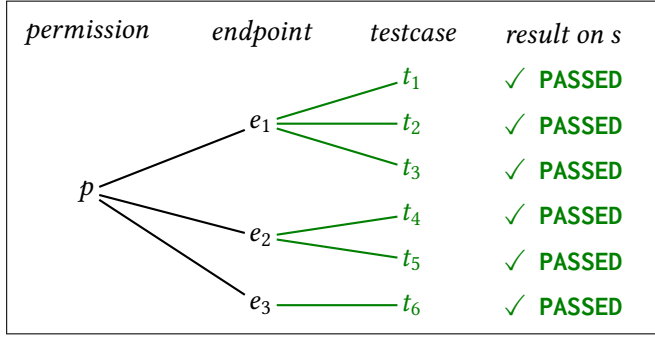
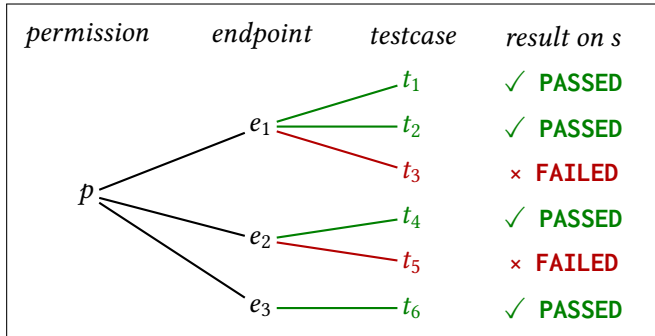


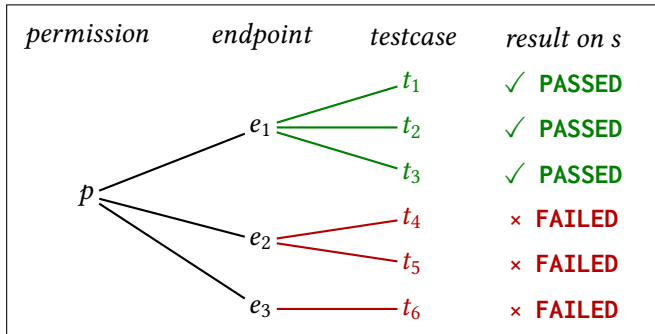
Figure 9.3: Average number of complete mediation violations (by edition, approach).

**(a) No violation.**

All test cases for permission p pass across all endpoints. The security requirements are correctly enforced.

**(b) No violation.**

Some test cases fail, but the failures are scattered across endpoints. This indicates logic errors in the access control policy enforcement.

**(c) Violation.**

All test cases for endpoints e_2 and e_3 fail, while all tests for e_1 pass. Access control policy is enforced at e_1 but neither at e_2 nor e_3 .

Justification. Since the model-driven approach submissions achieve complete mediation by construction, we do not conduct WSR tests on paired data. Instead, we justify the two approximations used in our analysis.

First, we label a submission as *policy-based* if it imports or references an authorization library, and as *manual* otherwise. This is an overapproximation, as a submission may use a library for only a subset of its authorization logic. To validate this classification, we randomly sampled 18 library-classified submission-phases from Ed'23 and 14 from Ed'25, corresponding to 10% of the submissions. For each sampled submission, we computed the *endpoint coverage*, defined as the fraction of authorization-relevant endpoint functions whose body, including decorators, contains a direct or indirect call to the library's

authorization API. Indirect calls include calls through helper functions. We consider a submission correctly classified as *policy-based* if at least 50% of its authorization-relevant endpoints use the library.

Of the 32 inspected submissions, 13 achieved more than 80% endpoint coverage, 11 achieved 50–79% endpoint coverage, and 8 fell below 50%. Of the 8 submissions with low endpoint coverage, 6 are Flask-Principal cases in which students imported the library but implemented authorization manually, and 2 are Cedar cases in which students imported the library but used it for only a small fraction of endpoints. The import-based heuristic therefore correctly classified 24 out of 32 submissions (75.0% accuracy). Since the 8 misclassified submissions are effectively manual implementations counted as *policy-based*, the overapproximation biases *in favor* of the *policy-based* group. The absence of a significant difference despite this bias strengthens the conclusion that library adoption alone does not improve complete mediation.

Second, our criterion for identifying complete mediation violations is also an approximation. We infer a violation when, for a given permission, all tests associated with some endpoints fail while tests for other endpoints exercising the same permission pass. Manual inspection of a 10% random sample of projects from Ed’23 shows that, among cases classified as complete mediation violations by our criterion, 3.02% are estimated to be false positives.

RQ3 To what extent does modeling privacy improve an application’s implementation of purpose limitation?

Finding. Modeling privacy substantially improves the purpose limitation enforcement. *vActionGUI* submissions account for only 22.3% of all failed privacy tests, compared with 77.7% for *Flask*. Code-centric implementations produce about 3.5× as many purpose limitation failures.

Data. Since *ActionGUI* does not support modeling privacy, we answer this research question using submissions from Ed’24 and Ed’25, where *vActionGUI* was introduced. We run the generated privacy test sets against submissions from both the Development and Evolution phases. As in **RQ1**, we restrict the analysis to valid paired submissions, i.e., students who submitted both a *vActionGUI* and a *Flask* solution for the same phase. Table 9.10 summarizes the resulting test sets and paired submission sets.

Recall that privacy tests are defined under three scenarios: *Basic*, *Complex*, and *No-Consent*. For each test set, we run the tests separately for each scenario and report the number of failed tests. Figure 9.4 shows the distribution of failed privacy tests by scenario and tool in the Development phase, for each edition and overall.

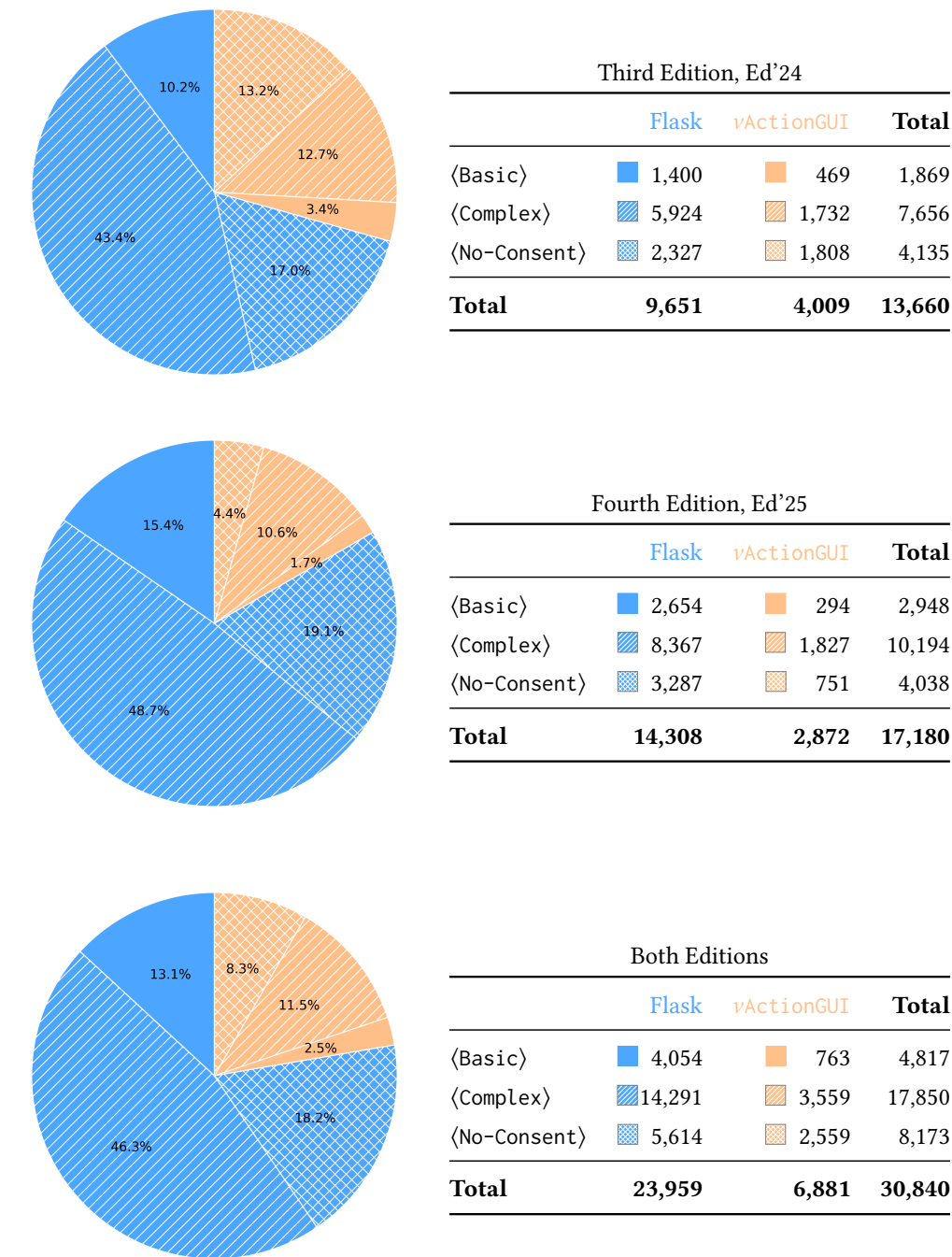


Figure 9.4: Distribution of failed privacy tests (by approach, privacy scenario).

Table 9.10: Test sets and paired submission sets used to evaluate **RQ3**.

Test set	Submission sets	Pairs
$\{PT_{Dev}^{vAG}, PT_{Dev}^{Flask}\}$	$\{S_{DevI,A}^{vAG,24}, S_{DevII,A}^{Flask,24}\}, \{S_{DevII,B}^{vAG,24}, S_{DevI,B}^{Flask,24}\}$	57
	$\{S_{DevI,A}^{vAG,25}, S_{DevII,A}^{Flask,25}\}, \{S_{DevII,B}^{vAG,25}, S_{DevI,B}^{Flask,25}\}$	74
$\{PT_{Evol}^{vAG}, PT_{Evol}^{Flask}\}$	$\{S_{Evol}^{vAG,24}, S_{Evol}^{Flask,24}\}$	54
	$\{S_{Evol}^{vAG,25}, S_{Evol}^{Flask,25}\}$	66

For example, the 13.1% ■ slice in the bottom pie of Figure 9.4 represents the share of all failed privacy tests in that pie plot that corresponds to Flask submissions under the $\langle \text{Basic} \rangle$ scenario. It is calculated as the number of failed Flask tests under the $\langle \text{Basic} \rangle$ scenario divided by the total number of failed privacy tests across all scenarios and tools in the same plot.

$$\frac{\sum_{i=1}^{|S_{Dev,\langle Basic \rangle}^{Flask}|} \sum_{j=1}^{|PT_{Dev,\langle Basic \rangle}^{Flask}|} (S_{Dev,\langle Basic \rangle}^{Flask} \otimes PT_{Dev,\langle Basic \rangle}^{Flask})_{ij}}{\sum_{tool} \left(\sum_{i=1}^{|S_{Dev}^{tool}|} \sum_{j=1}^{|PT_{Dev}^{tool}|} (S_{Dev}^{tool} \otimes PT_{Dev}^{tool})_{ij} \right)} = \frac{4,054}{30,840} = 13.1\% \quad (9.2)$$

Each bar in Figure 9.5 represents $\text{AvgFail}(S, PT)$, as defined in Formula (9.1), applied to the privacy test set PT and the submission set S for the corresponding tool and edition. For the Development phase, we combine submissions from both stages. Figure 9.5a reports the resulting average failure rates for the Development phase, while Figure 9.5b reports the corresponding results for the Evolution phase.

Answer. The analysis shows that modeling privacy substantially improves an application's implementation of purpose limitation. As shown in the bottom pie in Figure 9.4, Flask submissions account for 77.7% of all failed privacy tests in the Development phase (23,959 out of 30,840), while $vActionGUI$ submissions account for only 22.3% (6,881).

The distribution by scenario shows that $\langle \text{Complex} \rangle$ privacy requirements dominate the failures for both tools, accounting for 57.9% of all failures (17,850 out of 30,840). However, $vActionGUI$ produces fewer failures across all three scenarios. Overall, across both editions in the Development phase, Flask submissions produce 3.5× more privacy test failures than $vActionGUI$ submissions.

Figure 9.5 further shows that this advantage holds consistently across both phases and editions. In the Development phase (Figure 9.5a), Flask submissions fail 44.4% and 45.7% of privacy tests in Ed'24 and Ed'25 respectively, whereas $vActionGUI$ submissions

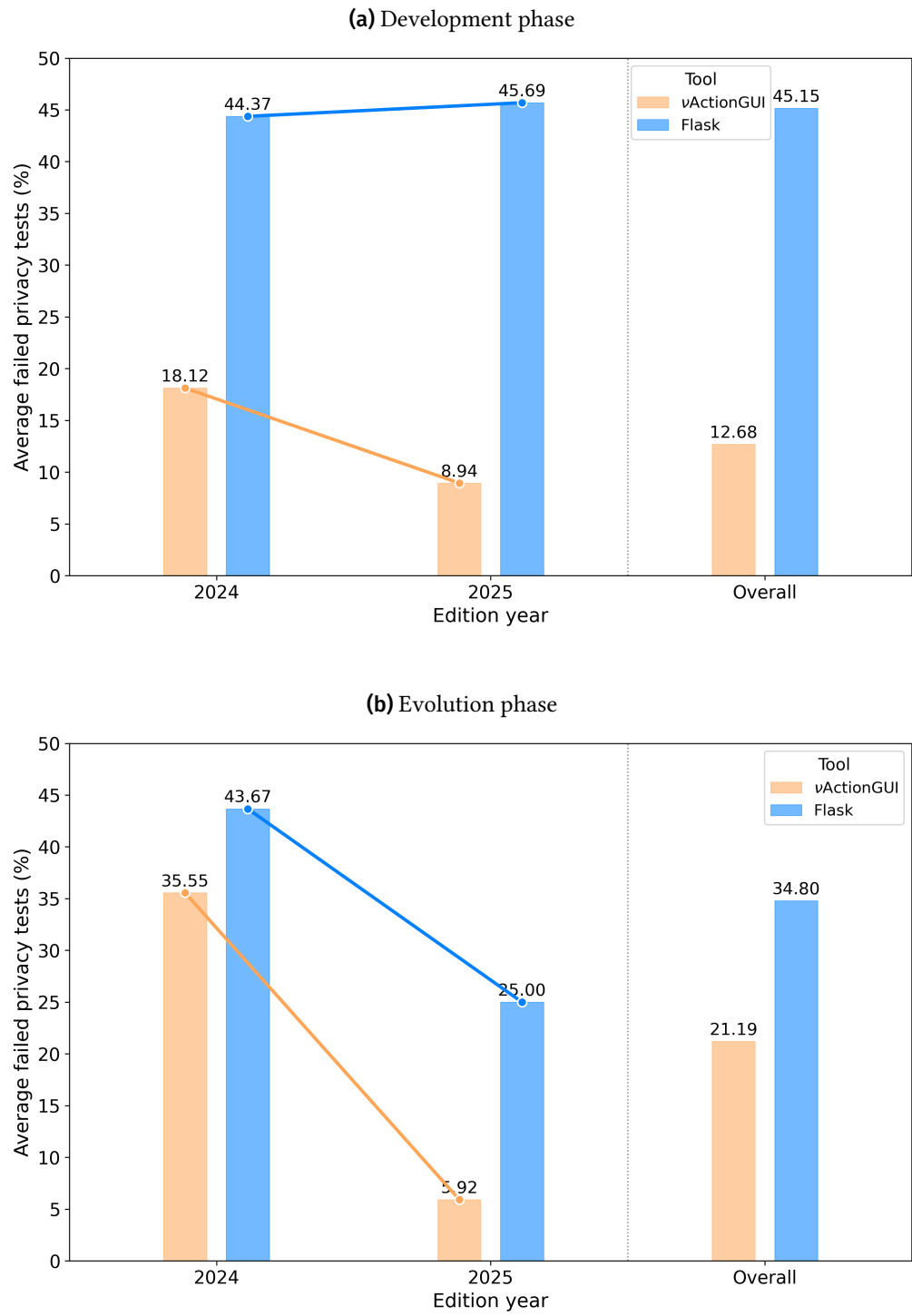


Figure 9.5: Average percentage of failed privacy tests (by edition, approach).

fail only 18.1% and 8.9%. The gap is largest in Ed'25, where the Flask failure rate is $5.1\times$ higher than the ν ActionGUI failure rate. In the Evolution phase (Figure 9.5b), ν ActionGUI maintains its advantage, with failure rates of 35.6% compared to 43.7% for Flask in Ed'24, and 5.9% compared to 25.0% for Flask in Ed'25.

Justification. We compare ν AG and Flask implementations using submissions paired by student and phase. First, we combine the paired submissions from both editions and both phases ($n = 251$). The WSR test confirms that Flask submissions fail significantly more privacy tests than ν AG submissions ($p = 4.74 \times 10^{-31}$, $r = 0.731$, large effect), with a median paired difference of +22.50 percentage points in favor of ν AG.

As a secondary analysis, we perform WSR tests separately by phase and edition. Table 9.11 reports the results. The effect is significant and large in both the Development phase ($n = 131$, $p = 6.41 \times 10^{-19}$, $r = 0.776$) and the Evolution phase ($n = 120$, $p = 1.82 \times 10^{-13}$, $r = 0.672$). The per-edition results confirm the effect in all cases, with the largest effects in Ed'25 (Development: $r = 0.810$, Evolution: $r = 0.809$) and medium-to-large effects in Ed'24 (Development: $r = 0.717$; Evolution: $r = 0.426$).

Table 9.11: WSR test results for **RQ3**: paired ν AG vs. Flask failure rates.

Phase	Edition	n	Median Δ	p	r	Effect
Development	All	131	+38.59 pp	6.41×10^{-19}	0.776	large
	Ed'24	57	+22.93 pp	6.28×10^{-8}	0.717	large
	Ed'25	74	+42.96 pp	3.27×10^{-12}	0.810	large
Evolution	All	120	+16.98 pp	1.82×10^{-13}	0.672	large
	Ed'24	54	+7.55 pp	1.74×10^{-3}	0.426	medium
	Ed'25	66	+22.50 pp	4.83×10^{-11}	0.809	large

Median Δ = median of (Flask – ν AG) per pair, in percentage points (pp).

RQ4 To what extent does modeling privacy improve privacy policy adequacy?

Finding. Modeling privacy helps keep declared purpose policies adequate. Developers are roughly three times more likely to over-declare at least one purpose with Flask than with ν ActionGUI.

Data. To answer this research question, we statically analyze the declared privacy policies in the paired submissions used for **RQ3**. In ν ActionGUI submissions, the privacy policy is explicitly specified in the privacy model, `project.ptm`. For Flask, it is encoded

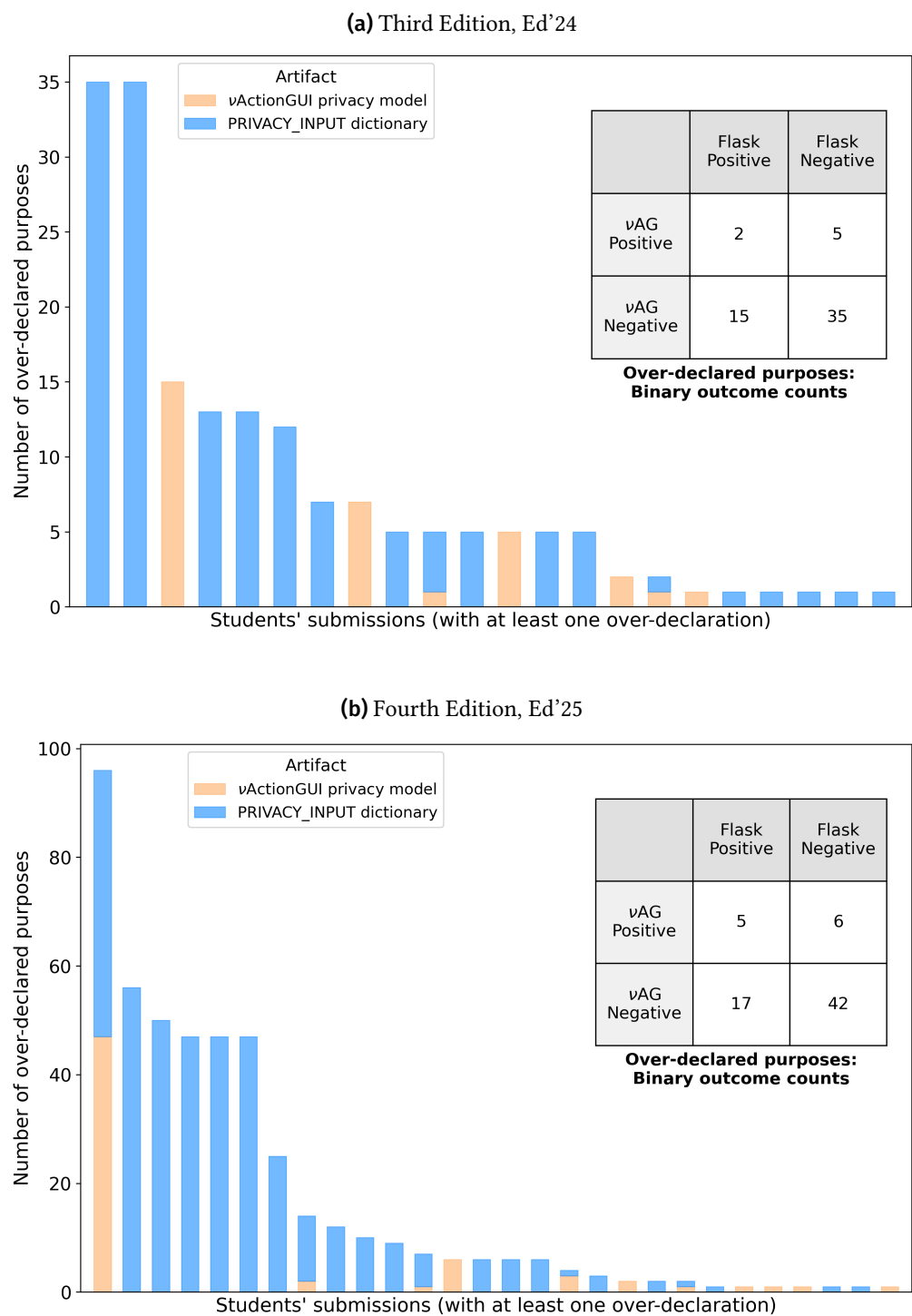


Figure 9.6: Number of over-declared purposes per student, with McNemar contingency tables. Students with zero over-declarations are excluded.

in the `PRIVACY_INPUT` Python dictionary in the source code. As 4 Flask submissions in the Development phase of Ed’25 did not define a valid `PRIVACY_INPUT` dictionary, we exclude these submissions from the analysis.

More specifically, from each privacy policy submission, we extract the set of declared purpose tuples, each consisting of a purpose and a personal data resource, where a resource is represented by a pair of entity and property. We then compare this set against a validated reference solution. A pair declared in a student submission but absent from the reference solution is counted as an *over-declaration*. The reference solution, specified in `vActionGUI`, is treated as the ground truth. This reference solution was validated by the teaching assistants throughout the study editions and passes all generated privacy test cases.

Answer. Figure 9.6 shows the over-declaration counts per student paired submission in the Development phase for both editions. Flask submissions are substantially more prone to over-declaration than `vActionGUI` submissions. Across both editions, students over-declared 39 out of 127 Flask submissions, compared to only 18 in `vActionGUI` (with 7 students making mistakes with both tools). Most mistakes resulted from unnecessarily associating personal data resources with complex purposes, which implicitly propagate data usage to all child purposes.

We also analyze the over-declaration results as a binary outcome, i.e., whether a student over-declared at least one declared purpose. The McNemar contingency tables for each edition are shown in Figure 9.6. On the combined data, McNemar’s test yields an odds ratio of $or = 2.91$ $[1.47, 5.77]$. This means that students are approximately three times more likely to over-declare with Flask than with `vActionGUI`.

Justification. We combine the paired submissions from Ed’24 ($n = 57$) and Ed’25 ($n = 70$) into an overall sample of $n = 127$ students. First, we compare the number of over-declared purpose tuples per student. The WSR test on the combined data shows a statistically significant difference between Flask and `vActionGUI` ($p = 8.70 \times 10^{-5}$, $r = 0.347$), although the median paired difference is $\Delta m = 0.0$. This is because most students do not over-declare with either approach.

RQ5 To what extent does modeling privacy improve consent checking?

Finding. Across all three privacy consent scenarios, `vActionGUI` fails fewer privacy tests than Flask. For example, in the Development phase, `vActionGUI` averages 5.0 compared with 30.5 for Flask for consent checking on basic purposes, and 24.1 compared with 107.1 for complex purposes.

Data. Similar to the evaluation in [RQ3](#), we run the submissions from Ed’24 and Ed’25 against the generated privacy test sets. Table 9.12 reports the average number of failed privacy tests by scenario, phase, and tool.

Table 9.12: Average number of failed privacy tests by scenario, phase, and tool.

Phase	<i>n</i>	⟨No-Consent⟩		⟨Basic⟩		⟨Complex⟩	
		<i>vAG</i>	<i>Flask</i>	<i>vAG</i>	<i>Flask</i>	<i>vAG</i>	<i>Flask</i>
Dev.	131	17.97	41.48	5.05	30.50	24.13	107.05
Evol.	120	4.14	7.08	1.84	3.62	3.65	5.15

Answer. Applications developed using *vAG* consistently exhibit fewer consent checking violations than those implemented with *Flask* across all three privacy scenarios. In the Development phase, *vAG* submissions fail substantially fewer tests than *Flask* submissions: 17.97 vs. 41.48 for ⟨No-Consent⟩, 5.05 vs. 30.50 for ⟨Basic⟩, and 24.13 vs. 107.05 for ⟨Complex⟩. The same pattern holds in the Evolution phase, where the gap is smaller but still consistently favors *vAG*.

These results indicate that consent checking is substantially easier to implement correctly when it is generated from the privacy model. In *vAG*, consent checks are automatically inserted by the generated enforcement code, whereas *Flask* developers must manually identify where personal data is accessed and insert the corresponding checks at the right locations.

Justification. Since consent collection mechanisms are already included in the provided templates, our evaluation focuses on whether implementations correctly check for consent before accessing personal data. We perform a WSR test for each of the three test scenarios in each phase, using a Bonferroni-adjusted significance level of $\alpha = 0.0083$. In all six comparisons, the median paired difference between *Flask* and *vAG* test failures is non-negative, favoring *vAG*. Table 9.13 summarizes the results.

RQ6 How much coding effort is required to implement and evolve access control and privacy policy?

Finding. Modeling reduces the coding effort needed to satisfy both access control and privacy requirements. The median lines of code per passing test are 3.6–5× lower for the model-driven tools. For example, in the Development phase, *vActionGUI* requires 0.13 lines per passing test compared with 0.68 for *Flask*.

Table 9.13: WSR test results for **RQ5**.

Phase	Scenario	Median Δ	95% CI	p	r	Effect
Development ($n = 131$)	$\langle$ No-Consent $\rangle$	+16.0 tests	[11.0, 23.0]	2.19×10^{-12}	0.613	large
	$\langle$ Basic $\rangle$	+21.0 tests	[16.0, 25.0]	1.34×10^{-18}	0.769	large
	$\langle$ Complex $\rangle$	+86.0 tests	[50.0, 113.0]	3.15×10^{-16}	0.714	large
Evolution ($n = 120$)	$\langle$ No-Consent $\rangle$	+3.0 tests	[2.0, 3.0]	2.63×10^{-12}	0.637	large
	$\langle$ Basic $\rangle$	+2.0 tests	[1.0, 2.0]	2.87×10^{-6}	0.425	medium
	$\langle$ Complex $\rangle$	+0.0 tests	[0.0, 0.0]	1.74×10^{-3}	0.286	small

Data. Figure 9.7 reports the distribution of lines of code (LoC) per passed test across project phases and tools. We use this metric as an approximation of implementation effort: lower values indicate that fewer code changes are needed to obtain a passing test.

For each submission in the Development phase, we compute the number of lines changed relative to the provided template and divide it by the number of tests passed. For the Evolution phase, we instead compute the number of lines changed relative to the previous phase and divide it by the number of newly passed Evolution tests. To focus on submissions of sufficient quality, we include only submissions that pass at least 75% of the applicable tests. The resulting LoC values are normalized to account for differences in test set size across tools.

We analyze the first two editions and the last two editions separately, since Ed’22–23 use ActionGUI as the model-driven tool, whereas Ed’24–25 use ν ActionGUI. For Ed’24–25, the metric combines both security and privacy LoC and test results.

AG vs. Flask For Ed’22–23, ActionGUI consistently requires less coding effort than Flask. In the Development phase, the median LoC per passed test is 0.08 for ActionGUI ($n = 131$) compared to 0.29 for Flask ($n = 125$). In the Evolution phase, the corresponding medians are 0.15 for ActionGUI ($n = 130$) and 0.58 for Flask ($n = 116$).

ν AG vs. Flask For Ed’24–25, ν ActionGUI again requires less coding effort than Flask. In the Development phase, the median LoC per passed test is 0.13 for ν ActionGUI ($n = 133$) compared to 0.68 for Flask ($n = 89$). In the Evolution phase, the corresponding medians are 0.48 for ν ActionGUI ($n = 109$) and 2.33 for Flask ($n = 70$).

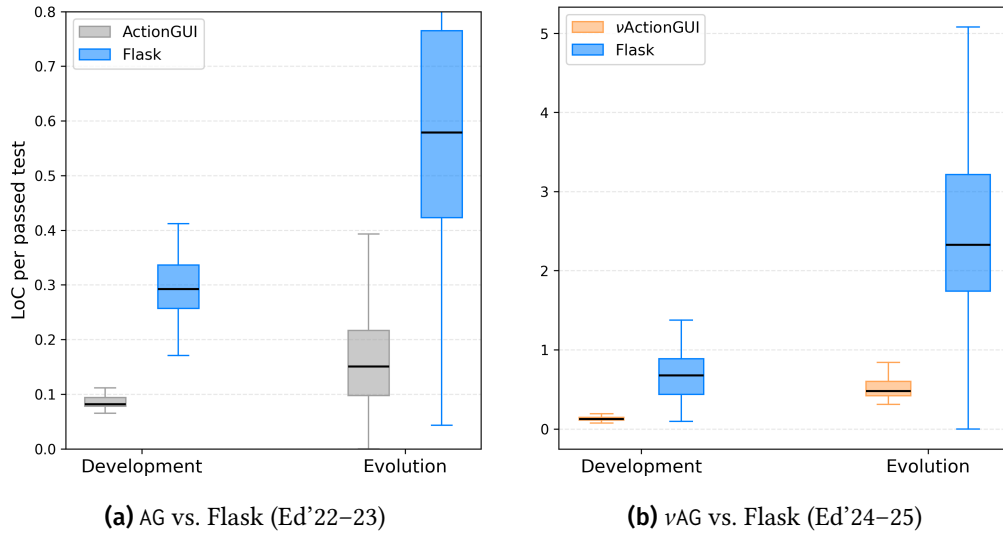


Figure 9.7: Distribution of LoC per passed test.

Answer. Modeling reduces the coding effort needed to implement and evolve both access control and privacy requirements. The model-driven tools consistently yield a lower LoC per passed test than the code-centric approach in both phases.

Justification. For Ed'22–23, the WSR test on paired LoC per passed test confirms that ActionGUI requires significantly less coding effort than Flask in both phases. The effect is significant and large in the Development phase ($n = 125$, $p < 10^{-21}$, $r = 0.868$) and in the Evolution phase ($n = 116$, $p < 10^{-17}$, $r = 0.816$). For Ed'24–25, the same pattern holds for vActionGUI. The WSR test shows a significant and large effect in favor of vActionGUI in both the Development phase ($n = 89$, $p < 10^{-15}$, $r = 0.868$) and the Evolution phase ($n = 68$, $p < 10^{-12}$, $r = 0.867$).

RQ7 What effort is required to implement and evolve access control and privacy policy, in terms of time?

Finding. The model-driven approach also reduces development time. vActionGUI developers spent, on average, 6.0 fewer hours than Flask developers on the same task. ActionGUI developers spent 1.7 fewer hours on average. In the Evolution phase, however, the two approaches required comparable time.

Data. Figure 9.8 shows the self-reported time students spent completing the project. The data is collected from the survey responses $R_{\{12,14,16,17\}}$ in Appendix F.3. Ques-

tions 12 and 14 ask for the hours spent on the two stages of the Development phase, respectively, while Questions 16 and 17 ask for the time spent in the Evolution phase for each approach. Across the four editions, we collected $n = 71$ survey responses, of which $n = 65$ provide complete time data for all phases.

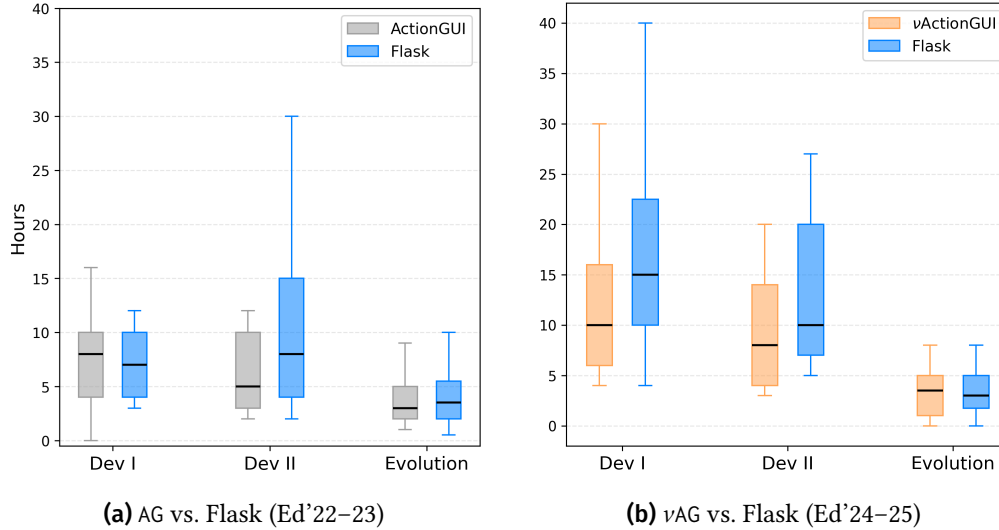


Figure 9.8: Distribution of self-reported time per phase.

AG vs. Flask For Ed'22-23, the overall self-reported time, combining the Development and Evolution phases, averages 17.1 hours for ActionGUI and 18.8 hours for Flask ($n = 30$ responses). The second development stage takes less time than the first stage for both tools. This can be attributed to the fact that the requirements were clarified and became sufficiently clear after the requirements engineering step in the first stage. In the Evolution phase, the reported times are similar, with a median of 3.0 hours for ActionGUI and 3.5 hours for Flask.

vAG vs. Flask For Ed'24-25, the difference is more pronounced. The overall self-reported time averages 14.2 hours for vActionGUI and 20.2 hours for Flask ($n = 35$ responses). Most of this difference comes from the Development phase, while the Evolution phase remains comparable, with a median of 3.5 hours for vActionGUI and 3.0 hours for Flask.

Answer. The model-driven approach requires less overall time than Flask. The effect is modest for ActionGUI where the mean difference is 1.7 hours, but more substantial for vActionGUI, where the mean difference is 6.0 hours. In the Evolution phase, the reported time is comparable across tools in both edition groups.

Justification. For Ed’22–23, the WSR test on overall paired hours shows a significant difference in favor of ActionGUI ($n = 30$, $p = 0.037$, $r = 0.380$, medium effect), with a mean difference of 1.7 hours. However, when considered separately, neither the Development phase ($p = 0.067$) nor the Evolution phase ($p = 0.310$) reaches significance.

For Ed’24–25, the WSR test on overall paired hours shows a stronger effect in favor of ν ActionGUI ($n = 35$, $p = 0.002$, $r = 0.520$, large effect), with a mean difference of 6.0 hours. This difference is driven by the Development phase, which is significant on its own ($n = 35$, $p < 10^{-3}$, $r = 0.584$), while the Evolution phase again shows no significant difference ($p = 0.454$).

RQ8 Does prior experience in designing (resp. implementing) the same security and privacy requirements improve their implementation (resp. design)?

Finding. Implementation correctness is driven by the development approach, not by prior experience with the other approach. The crossover effect δ_S remains close to zero in most editions, ranging from -1.3 to $+7.5$ percentage points across the four security editions, with larger deviations only in the first edition. The model-driven approach yields fewer failures regardless of which approach developers used first.

Data. Recall the two-stage setup of the Development phase (Section 8.2), in which students are divided into two groups. Group $\mathbb{A}$ students first work with the model-driven tool in Stage I and then switch to the code-centric approach in Stage II, whereas Group $\mathbb{B}$ students perform the two stages in the opposite order.

Table 9.14: Test sets and paired submission sets used to evaluate RQ8.

Test sets	Submission sets	Pairs
$\{GST_{Dev}^{AG}, GST_{Dev}^{Flask}\}$	$\{S_{DevI,A}^{AG,22}, S_{DevII,A}^{Flask,22}\}, \{S_{DevII,B}^{AG,22}, S_{DevI,B}^{Flask,22}\}$	71
	$\{S_{DevI,A}^{AG,23}, S_{DevII,A}^{Flask,23}\}, \{S_{DevII,B}^{AG,23}, S_{DevI,B}^{Flask,23}\}$	89
$\{GST_{Dev}^{\nu AG}, GST_{Dev}^{Flask}\}$	$\{S_{DevI,A}^{\nu AG,24}, S_{DevII,A}^{Flask,24}\}, \{S_{DevII,B}^{\nu AG,24}, S_{DevI,B}^{Flask,24}\}$	57
	$\{S_{DevI,A}^{\nu AG,25}, S_{DevII,A}^{Flask,25}\}, \{S_{DevII,B}^{\nu AG,25}, S_{DevI,B}^{Flask,25}\}$	74

This setup allows us to compare, for a given target tool, students who arrive at Stage II with prior experience using the alternate approach against students who used the target tool directly in Stage I. For example, when evaluating the code-centric approach in Stage II, Group $\mathbb{A}$ has prior experience modeling the same requirements in Stage I,

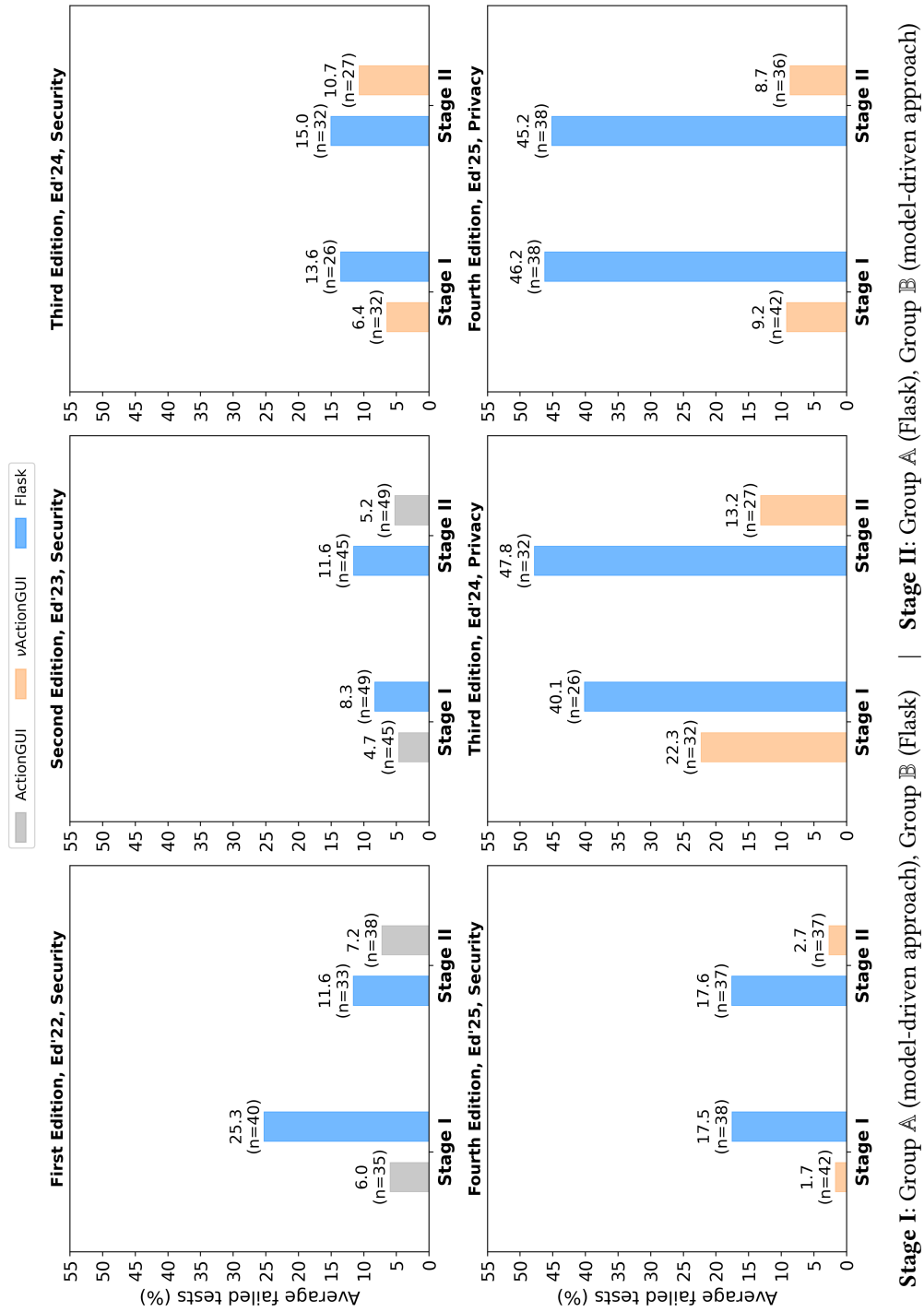


Figure 9.9: Average percentage of failed tests by group across all editions.

whereas Group $\mathbb{B}$ used this code-centric approach without such prior modeling experience in Stage I.

In this research question, we consider only the two stages of the Development phase across all editions. For each edition, we report the average percentage of failed tests separately for each stage, i.e., for each student group, using Equation (9.1). Table 9.14 summarizes the test sets and paired submission sets.

Based on the average percentage of failed tests, we then calculate the *prior-knowledge effect* for each edition of the Development phase:

$$\delta_S = \frac{1}{2} \left(\text{AvgFail}(S_{\text{DevI},\mathbb{B}}^{\text{Flask},yr}, T_{\text{Dev}}^{\text{Flask}}) + \text{AvgFail}(S_{\text{DevII},\mathbb{B}}^{md,yr}, T_{\text{Dev}}^{md}) - \text{AvgFail}(S_{\text{DevI},\mathbb{A}}^{md,yr}, T_{\text{Dev}}^{md}) - \text{AvgFail}(S_{\text{DevII},\mathbb{A}}^{\text{Flask},yr}, T_{\text{Dev}}^{\text{Flask}}) \right) \quad (9.3)$$

Here, $yr \in \{22, 23, 24, 25\}$ denotes the edition year, and md denotes either AG or ν AG, depending on the edition. The test set T is realized as GST for the security test set and, for Ed'24–25, as PT for the privacy test set.

A positive δ_S indicates that the Group $\mathbb{A}$ students produce a lower overall failure rate. This suggests that prior design experience improves subsequent implementation. A negative δ_S indicates otherwise, i.e., prior implementing experience improves subsequent design. Values close to zero indicate no clear advantage for either order.

Answer. Figure 9.9 shows the average percentage of failed tests by development stage, and Table 9.15 reports the prior-knowledge effect δ_S for each edition.

Table 9.15: Prior-knowledge effect δ_S per edition and requirements.

Edition	Requirements	Group $\mathbb{A}$		Group $\mathbb{B}$		δ_S
		Stage I	Stage II	Stage I	Stage II	
Ed'22	Security	6.0	11.6	25.3	7.2	+7.5
Ed'23	Security	4.7	11.6	8.3	5.2	−1.3
Ed'24	Security	6.4	15.0	13.6	10.7	+1.4
	Privacy	22.3	47.8	40.1	13.2	−8.4
Ed'25	Security	1.7	17.6	17.5	2.7	+0.5
	Privacy	9.2	45.2	46.2	8.7	+0.3

Across the four security editions, δ_S ranges from −1.3 to +7.5, with three of the four values near zero. For privacy, Ed'25 is also close to zero ($\delta_S = +0.3$). The two larger deviations occur in the first editions in which the corresponding security and privacy

requirements were introduced. We hypothesize that they may reflect other effects, such as the instructions and setup of the project templates, rather than a genuine prior-experience effect.

Overall, the results show that prior experience with the alternate approach does *not* appear to improve the correctness of subsequent implementations or specifications. Instead, implementation correctness is primarily associated with the tool used: the model-driven approach consistently produces fewer failures than the code-centric approach for both security and privacy requirements across all editions.

Justification. We conduct the Mann–Whitney U (MWU) tests comparing student groups with and without prior experience. For Flask, we compare students of Group A at Stage II (with prior design experience, denoted by n_1) and Group B at Stage I (without prior design experience, denoted by n_2). For ν ActionGUI, we compare students of Group B at Stage II (with prior implementing experience) and Group A at Stage I (without prior implementing experience).

For access control across all four security editions, the combined MWU test for Flask submissions ($n_1 = 153$ and $n_2 = 147$) is not significant ($p = 0.89$). For the model-driven tool ($n_1 = 154$ and $n_2 = 151$), the test yields $p = 0.009$, $r = 0.151$, but the group *without* prior code-centric experience actually performs better. The difference in general proficiency between the groups is a plausible explanation. For privacy, the combined MWU tests yield $p = 0.34$ for Flask ($n_1 = 64$ and $n_2 = 70$) and $p = 0.43$ for ν ActionGUI ($n_1 = 74$ and $n_2 = 63$), neither of which is significant.

RQ9 Which aspects of access control and privacy are the most challenging to design (respectively, to implement)?

Finding. Code-centric submissions fail at similar rates across roles, ranging from 14.1 to 17.5%, whereas model-driven submissions remain below 8%. Within the model-driven tools, the relatively most error-prone role differs by tool: ADMIN for ActionGUI and VISITOR for ν ActionGUI.

Data. We again consider submissions from the Development phase across all editions. To identify which aspects of access control and privacy are most challenging to design and implement, we analyze the evaluation results obtained by running these submissions against our generated security and privacy test sets.

For access control, we consider security submissions from all four editions. We characterize the evaluation results along two dimensions: the *subject role* and the *expected authorization verdict*.

The former characterization refers to the role of the user acting as the subject in the test. For example, the test case “An ADMIN user can update another user’s role from FREEUSER to PREMIUMUSER.” is classified as an ADMIN test case, whereas the test case “A user with the role FREEUSER cannot update their own role to ADMIN.” is classified as a FREEUSER test case.

The latter characterization captures the expected outcome of the test, i.e., whether the action under test should be *permitted* or *denied*. In the examples above, the first test case is a permit test, whereas the second is a deny test. Failing a permit test means that the design (and, respectively, the implementation) is overly restrictive because it blocks a legitimate action. Failing a deny test means that it is insufficiently restrictive because it allows an unauthorized action.

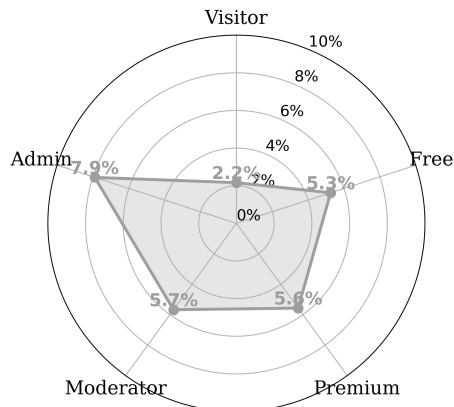
For privacy, we consider the privacy submissions from Ed’24–25. We characterize the evaluation results along two dimensions: the *personal data* referenced by the test, such as name or role, and the *endpoint* of the application in which the data is accessed, such as `manage_event()` or `update_user()`.

Access control by subject role. Figure 9.10 shows the average percentage of failed security tests per subject role in the Development phases. For each tool and subject role, we calculate the average percentage by applying Equation 9.1, where S is the set of submissions in all editions of the respective tool, and T is the generated security test set for the respective role. For example, the highest point in the top plot in Figure 9.10 can be calculated as $\text{AvgFail}(S_{\text{Dev}}^{\text{AG}}, \text{GST}_{\text{Dev}, \text{V}}^{\text{AG}})$.

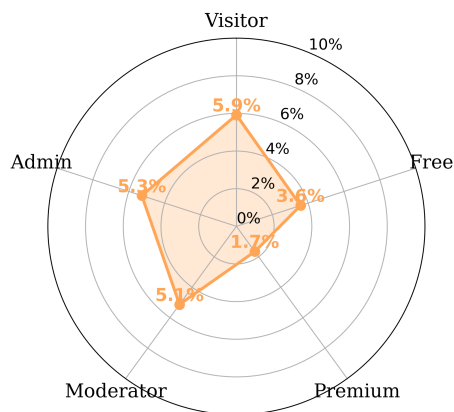
For Flask, the failure rates are uniformly high across all security roles, indicating that students who use the code-centric approach struggle with access control requirements, regardless of the role’s complexity. The VISITOR role exhibits the highest failure rate (17.5%), followed by MODERATOR and ADMIN (both at 15.0%).

For ActionGUI (Ed’22–23), ADMIN is the most challenging role, with a failure rate of 7.9%, whereas VISITOR is the least challenging, with a failure rate of 2.2%. For vActionGUI (Ed’24–25), VISITOR becomes the most challenging role, with a failure rate of 5.9%, followed by ADMIN at 5.3%.

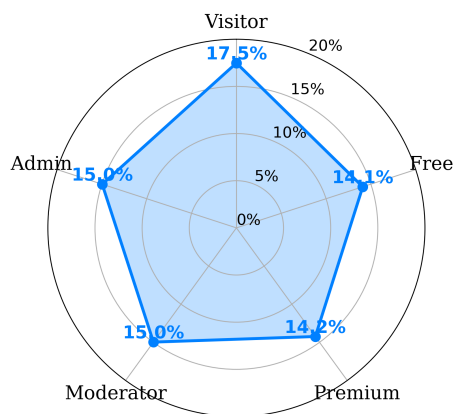
Access control by expected authorization verdict. Table 9.16 breaks down the failed security tests according to the expected authorization verdict, i.e., whether the tested action should have been permitted or denied. The central bar shows the distribution of all security tests, with permit tests on the left and deny tests on the right. Within each portion, the solid segment represents failed tests, while the lighter segment represents passed tests. The failure rate shown at each edge is computed as the number of failed tests divided by the total number of tests with the corresponding verdict.

**ActionGUI — Ed'22–Ed'23 ($n = 164$)**

Role	Avg. failed
VISITOR	2.2%
FREEUSER	5.3%
PREMIUMUSER	5.6%
MODERATOR	5.7%
ADMIN	7.9%

**vActionGUI — Ed'24–Ed'25 ($n = 135$)**

Role	Avg. failed
VISITOR	5.9%
FREEUSER	3.6%
PREMIUMUSER	1.7%
MODERATOR	5.1%
ADMIN	5.3%

**Flask — Ed'22–Ed'25 ($n = 293$)**

Role	Avg. failed
VISITOR	17.5%
FREEUSER	14.1%
PREMIUMUSER	14.2%
MODERATOR	15.0%
ADMIN	15.0%

Figure 9.10: Average percentage of failed security tests per role in the Development phase.

Table 9.16: Failed security tests by expected authorization verdict.

Tool	Permit	Test distribution		Deny
ActionGUI	7,726	7.0%		10,987
vActionGUI	2,222	2.0%		5,153
Flask	35,112	14.3%		23,950

For Flask, 35,112 permit and 23,950 deny tests fail, with comparable failure rates (14.3% vs. 15.9%). This indicates that the code-centric approach is roughly equally prone to over-restriction, where legitimate actions are blocked, and under-restriction, where unauthorized actions are allowed.

For ActionGUI, 7,726 permit and 10,987 deny tests fail. Although more failed tests are deny tests in absolute terms, the permit failure rate is higher than the deny failure rate (7.0% vs. 5.0%), suggesting a slight tendency toward over-restriction.

For vActionGUI, 2,222 permit and 5,153 deny tests fail. Here, the deny failure rate (9.2%) is more than four times the permit failure rate (2.0%), indicating that students using vActionGUI are effective at granting legitimate access but are more likely to omit the constraints needed to reject unauthorized actions.

Privacy. Figure 9.11 shows a heatmap of the most violated privacy policies, broken down by application endpoint and personal data, separately for each tool. Both tools exhibit the same main hotspots: access to name and surname in `manage_event` and `view_category`, and access to role in `update_user`.

For Flask, the three most affected personal data are name (9,511 failures), surname (8,012), and role (3,496). Together, these account for 88% of all privacy test failures. The corresponding vActionGUI counts are 2,730, 2,332, and 990, respectively. Each of the remaining personal data resources contributes fewer than 500 failures.

Justification. Since this is an exploratory research question, we do not conduct statistical tests. Instead, we report descriptive statistics and anonymized examples.

For security, manual inspection of failing Flask submissions in the first two editions shows that inappropriate attribute lookups on unauthenticated users account for 39.56% of unsuccessful tests for the `VISITOR` role. Typically, the `current_user.is_authenticated` returned `false`, but the implementation still dereferenced `current_user`, causing an attribute error. Such attribute errors account for 15.64% of *all* failing Flask tests. This

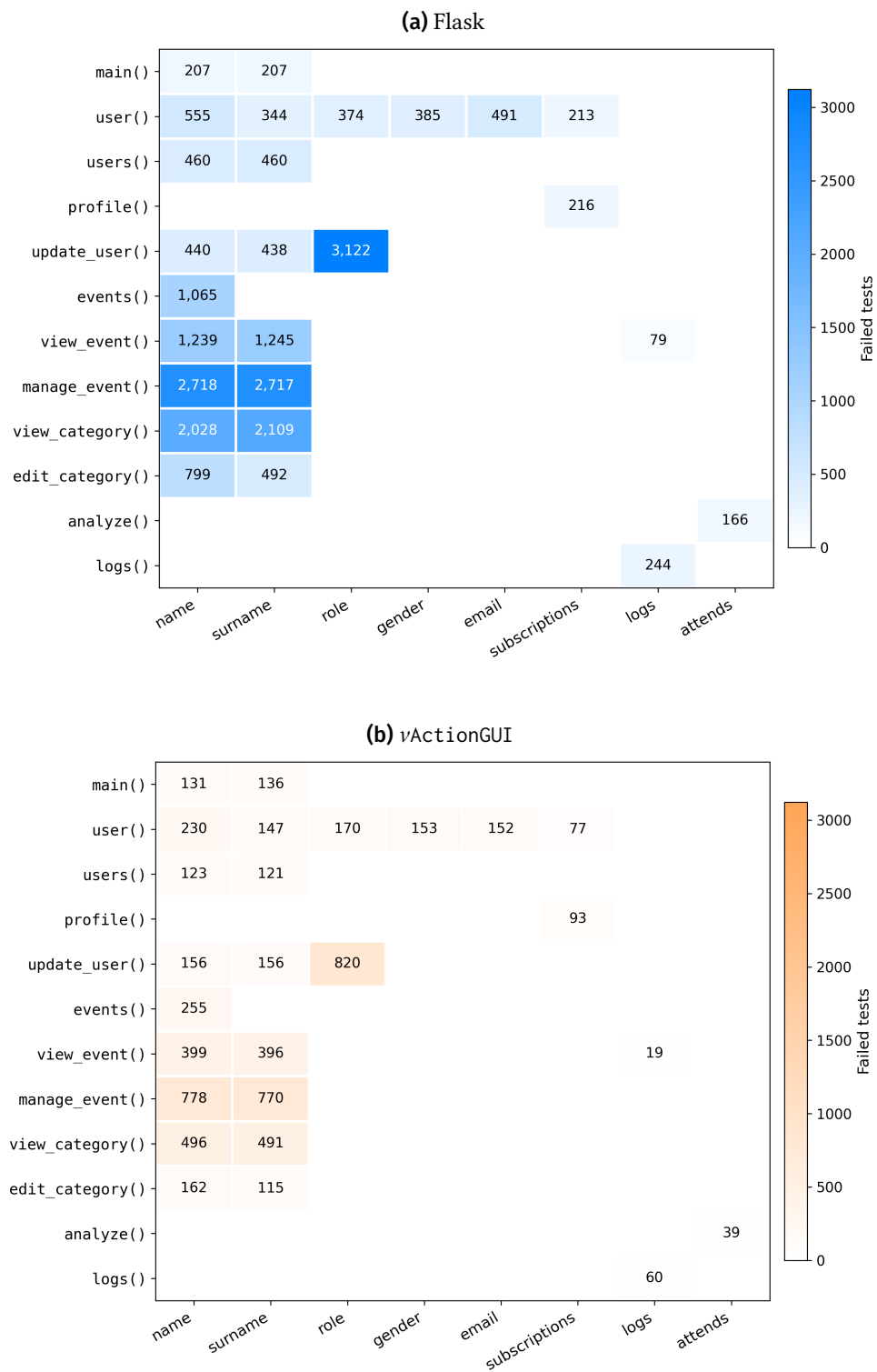


Figure 9.11: Failed privacy tests by endpoint and personal data in the Development phase.

may also explain the nearly balanced permit and deny failure rates for Flask: the same runtime error causes both kinds of tests to fail indiscriminately.

In ActionGUI, the high failure rate for the ADMIN role is mainly due to insufficiently restrictive authorization constraints in permissions for updating user roles and maintaining data model invariants, such as preventing moderators from changing roles while they are currently moderating some events. In particular, ADMIN users may update a user's role to one of the authenticated roles but not to VISITOR, LIB (the system role), or null. A common pitfall was assuming that, because the GUI only offers authenticated user roles as selectable values, no additional authorization constraints were needed. This pattern also helps explain the high deny-test failure rate observed for vActionGUI in Table 9.16, that is, the generated application grants authorized access correctly, but students must still specify the constraints needed to reject unauthorized actions.

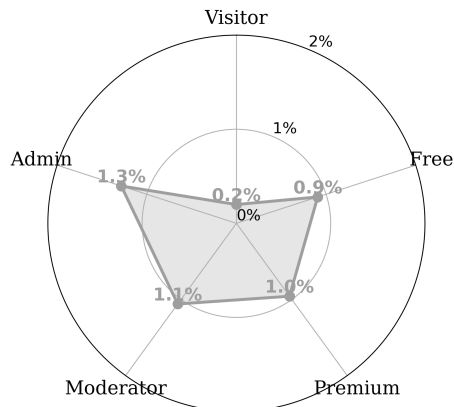
On the privacy side, failures concentrate in `manage_event()` and `view_category()` for name and surname because these endpoints display the names and surnames of several kinds of persons, including attendants, managers, requesters, moderators, and subscribers. As a result, a single missing consent check can affect many test cases. Similarly, the `update_user()` hotspot for role arises because the tests cover many different role transitions.

A common privacy error was checking consent only for the basic purpose, e.g., CORE, rather than also accepting complex purposes that transitively include it, e.g., FUNCTIONAL or ANY. This explains the high proportion of failures in the <Complex> scenario tests shown in Figure 9.4.

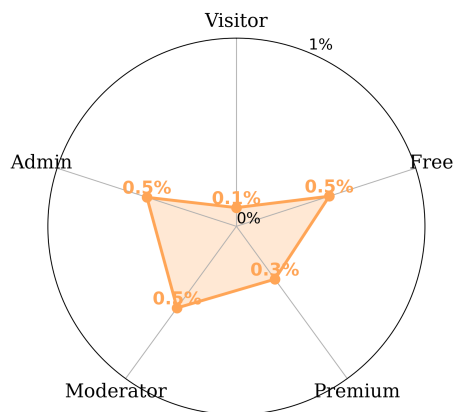
Finally, unlike the more explicit purposes like MARKETING or ANALYTICS, the CORE purpose refers to the platform's main functionality that is not explicitly enumerated in the privacy requirements. To correctly define the actual purposes, participants therefore had to inspect the code manually, identify where personal data is used for core functionality, and associate the corresponding functions with the CORE purpose. This additional reasoning helps explain why accesses governed by CORE were particularly error-prone.

RQ10 Which aspects of access control and privacy are the most challenging to evolve?

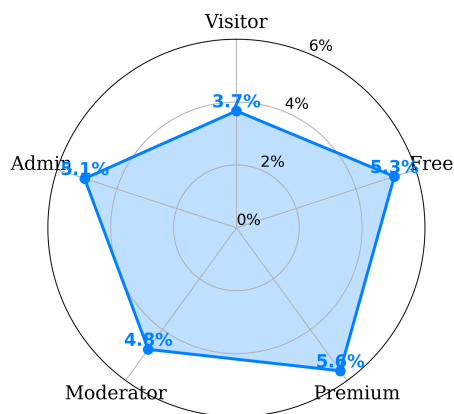
Finding. Modeled policies evolve more safely. Per-role regression rates reach 3.7–5.6% for Flask, but only 0.1–1.3% for the model-driven tools. In both approaches, the few regressions that occur are concentrated on cases related to the newly introduced requirement.

**ActionGUI** — Ed'22–Ed'23 ($n = 152$)

Role	Avg. regressed
VISITOR	0.20%
FREEUSER	0.91%
PREMIUMUSER	0.96%
MODERATOR	1.06%
ADMIN	1.28%

**vActionGUI** — Ed'24–Ed'25 ($n = 122$)

Role	Avg. regressed
VISITOR	0.10%
FREEUSER	0.52%
PREMIUMUSER	0.35%
MODERATOR	0.54%
ADMIN	0.50%

**Flask** — Ed'22–Ed'25 ($n = 265$)

Role	Avg. regressed
VISITOR	3.72%
FREEUSER	5.28%
PREMIUMUSER	5.62%
MODERATOR	4.78%
ADMIN	5.08%

Figure 9.12: Average regression rate (percentage of Development-phase passing tests that fail in the Evolution phase) per caller role.

Data. We consider submissions from the Evolution phase across all editions. In this phase, participants extend their previous submissions with new requirements. To identify which aspects of access control and privacy are most challenging to evolve, we evaluate the evolved submissions against the Development test sets and analyze which previously satisfied requirements regress.

For each submission, we identify tests that passed in the Development phase but failed after the evolution. These tests correspond to requirements that were implemented correctly before the new requirements were introduced but were broken during evolution. We then compute an average *regression rate*, defined as the percentage of previously passing tests that now fail.

As in [RQ9](#), for access control, we characterize the regressions based on the *subject role*. For privacy, we analyze the regressions by endpoint and personal data resource.

Access control by subject role. Figure 9.12 shows the average regression rate by subject role. For Flask, the regression rates are substantially higher than those for the model-driven tools across all roles, ranging from 3.72% for VISITOR to 5.62% for PREMIUMUSER role. For ActionGUI, the regression rate ranges from 0.20% for VISITOR to 1.28% for ADMIN, while ν ActionGUI has the lowest regression rates overall, ranging from 0.10% to 0.54%. Across all three approaches, authenticated roles, from FREEUSER to ADMIN, regress more than the VISITOR role. This is expected because the evolution requirements (e.g., event invitations and personalized statistics) apply mainly to authenticated users.

Privacy. Figure 9.13 shows a heatmap of failed regression privacy tests, broken down by endpoint and personal data. As expected, most cells are zero, and only the Person's attends property exhibits regressions after evolution. For Flask, the regressions concentrate in the `profile()` endpoint, with 89 failures, and `analyze()`, with 41 failures. For ν ActionGUI, `profile()` accounts for 182 failures and `analyze()` for 10. The `profile()` endpoint is the most affected in both tools because the new requirements add attended events to user profiles, and students often fail to update the consent check accordingly.

Justification. On the access control side, the model-driven tools exhibit regression rates roughly $3\times$ – $10\times$ lower than the code-centric approach. The regressions in model-driven submissions are predominantly attributed to the newly introduced requirements. In contrast, Flask submissions exhibit a broader range of regressions, many of which are unrelated to the new requirements.

On the privacy side, the low regression rate across most privacy policies indicates that privacy implementations are mostly stable through evolution. The `attends` attribute is

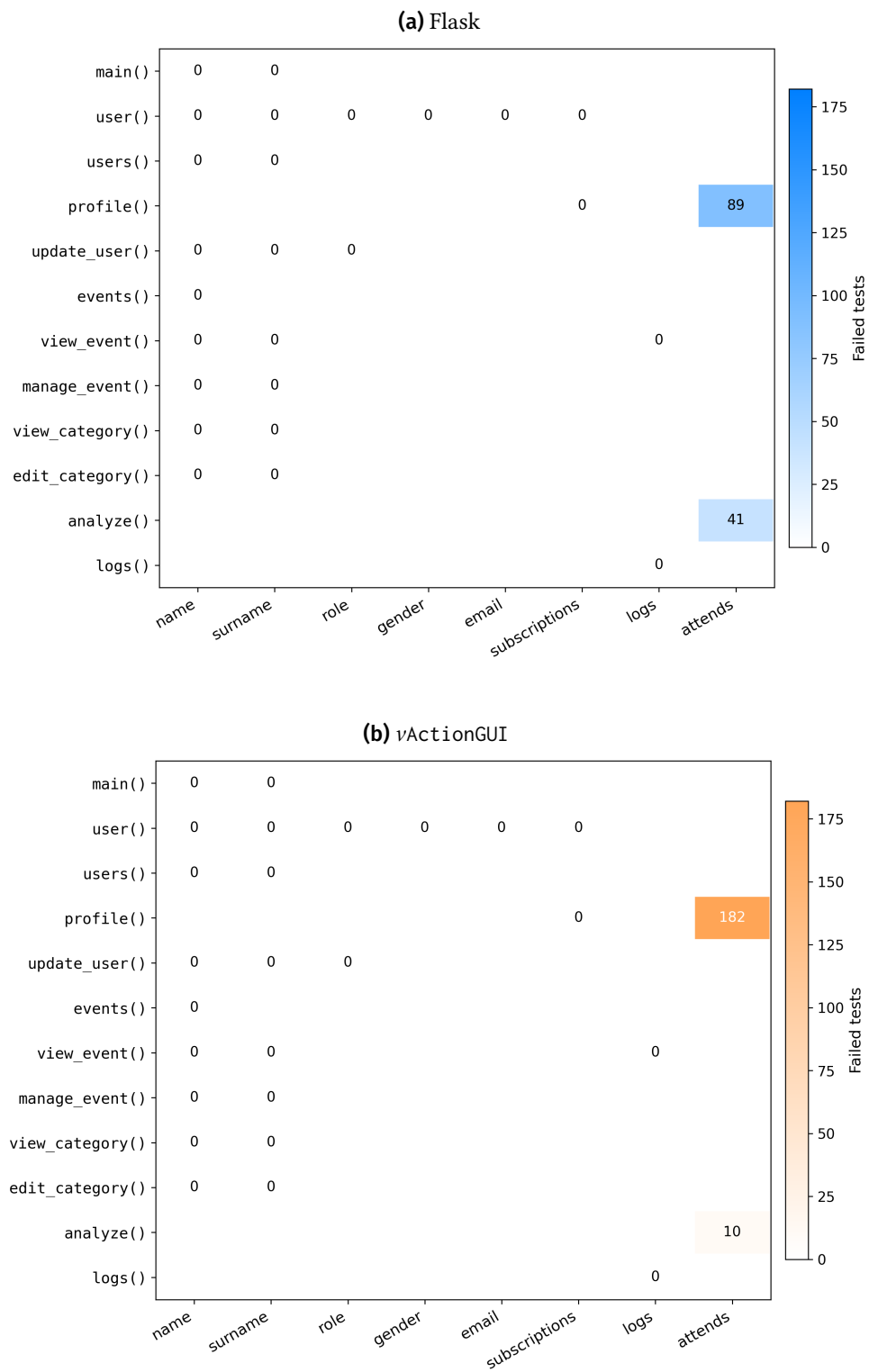


Figure 9.13: Most violated Development privacy policies in the Evolution phase.

the only source of regressions because it was newly introduced in the Evolution phase.

RQ11 Which Flask authorization library yields the most secure applications?

Finding. Authorization libraries do not improve security in the code-centric approach. Oso and Flask-Principal submissions fail at comparable rates, while Cedar performs worst, with 17.3% failures in Development and 19.2% in Evolution. The main reason is complete mediation: developers must still invoke the policy at every relevant endpoint, which the libraries do not enforce automatically.

Data. To answer this research question, we restrict the analysis to Flask submissions and reuse the library usage classification introduced in RQ2. We distinguish between submissions that implement authorization manually and submissions that use one of the supported authorization libraries: Oso, Flask-Principal, or Cedar.

While RQ2 focuses on complete mediation, this research question evaluates the overall security outcome of each approach. We therefore use the average percentage of failed generated security tests as the metric. Figure 9.14 reports the resulting failure rates by library usage and phase.

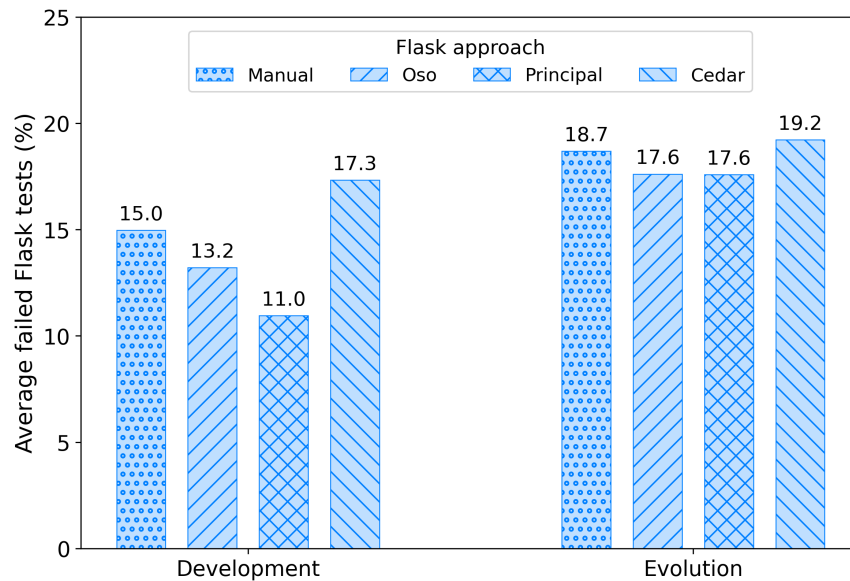


Figure 9.14: Average percentage of failed security tests per Flask authorization approach.

Answer. The choice of Flask authorization library does not improve security. As shown in Figure 9.14, submissions implemented manually, with Oso, or with Flask-Principal achieve comparable average failure rates, with no consistent advantage for any single approach. Cedar, introduced in Ed’25, has failure rates of 17.3% in Development and 19.2% in Evolution, the highest among the observed approaches.

We hypothesize that Flask authorization libraries require specific expertise to be used correctly. Moreover, even when requirements are specified using these libraries, developers must still invoke the corresponding policy checks at every relevant endpoint. As discussed in RQ2, students in our study did not do so consistently.

Justification. We use the Kruskal–Wallis (KW) test, which generalizes the MWU test to compare three or more independent groups on non-normal data. Both when looking at per-phase data (Development: $p = 0.08$; Evolution: $p = 0.39$) and data with all phases combined ($p = 0.28$), the computed differences between the four groups are not significant. To obtain more power, we considered grouping all library-based submissions (Oso, Principal, and Cedar) into a single *policy-based* group, but the resulting MWU test also yields a non-significant difference.

RQ12 Can regression tests improve security as an application evolves?

Finding. Releasing regression tests provides only a fragile, local benefit. The edition given the test suite, Ed’22, passes 3.5–8.4 percentage points more manually written tests than Ed’23. However, this advantage disappears on the more comprehensive generated Evolution tests.

Data. Recall that Ed’22 included a one-week Repair phase. During this phase, students received the manually written Development security test set, WST_{Dev}^{AG} and WST_{Dev}^{Flask} , and were asked to repair both of their Development submissions. In subsequent editions, the Repair phase was removed, and students proceeded directly to the Evolution phase without exposure to regression tests.

Since Ed’22 and Ed’23 used identical security requirements and both used ActionGUI as the model-driven tool, we compare their test results in the Evolution phase. Specifically, we evaluate submissions from both editions on the manually written WST_{Evol} and on the automatically generated GST_{Evol} Evolution test sets. This comparison allows us to assess whether exposure to regression tests during the Repair phase in Ed’22 improved security when the application later evolved.

Table 9.17: Average percentage of passed Evolution tests by edition and test set.

(a) Manually written tests				
Tool	First Edition, Ed'22	Second Edition, Ed'23	Δ (pp)	
ActionGUI	93.78%	90.28%	+3.5	
Flask	83.02%	74.62%	+8.4	
(b) Auto-generated tests				
Tool	First Edition, Ed'22	Second Edition, Ed'23	Δ (pp)	
ActionGUI	87.12%	90.01%	-2.9	
Flask	81.93%	82.53%	-0.6	

Answer. Table 9.17 shows that Ed'22 submissions pass a higher percentage of manually written tests than Ed'23 submissions. The difference is 3.5 percentage points for ActionGUI and 8.4 percentage points for Flask. However, this effect does not carry over to the generated Evolution tests. On these more comprehensive tests, Ed'23 submissions perform slightly better.

A plausible explanation is that exposure to the manually written Development tests during the Repair phase helped Ed'22 participants adapt their implementations to the style and coverage of the released tests. They may also have inferred testing patterns from these regression tests that helped them pass the manually written Evolution tests. Although this strategy can increase the number of tests passed, it does not lead to improved security, as evidenced by the performance with respect to the (more comprehensive) generated tests.

Justification. We compare the performance of the submissions in each edition on the evolution tests using the MWU test. For the manually written test, Ed'22 submissions perform significantly better than Ed'23 submissions for both tools: ActionGUI ($n_1 = 67$, $n_2 = 86$, $p = 1.6 \times 10^{-3}$, $r = 0.255$) and Flask ($n_1 = 64$, $n_2 = 90$, $p = 1.1 \times 10^{-3}$, $r = 0.263$). For the automatically generated tests, Ed'23 submissions perform better than Ed'22 for ActionGUI ($n_1 = 67$, $n_2 = 86$, $p = 6.5 \times 10^{-4}$, $r = 0.276$) whereas for Flask the difference is not statistically significant ($n_1 = 65$, $n_2 = 90$, $p = 0.31$).

9.4 General Remarks

Beyond the findings reported for the main research questions, we provide additional remarks on participants' background (Section 9.4.1) and qualitative feedback on the two approaches (Section 9.4.2).

9.4.1 Participant Background

Among the survey respondents, 36 were initially assigned to the model-driven tools and 35 to Flask. Both groups rated the clarity of the project description similarly, with a median score of 3 out of 5. This suggests that differences in project performance between the two groups are unlikely to be attributed to differences in how clearly they understood the requirements.

Figure 9.15 summarizes participants' self-reported background knowledge from the survey questions (Questions 1 to 8, Appendix F.3). Participants had substantially more prior experience with the code-centric technology than with the model-driven tools. Between 60 and 75% of the respondents reported some familiarity with Python and Flask, whereas 0–17% had used the model-driven tools prior to the project. Therefore, the better performance achieved with the model-driven approach cannot be explained by a greater prior familiarity with the associated tools.

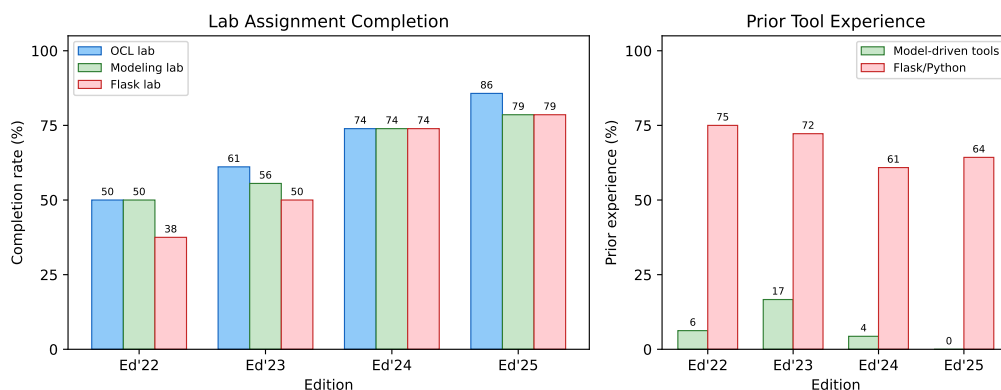


Figure 9.15: Participant background knowledge across editions (survey respondents). Lab assignment completion rates (left) and prior experience (right).

9.4.2 Qualitative Insights

In the survey questionnaire, we asked the participants which approach they would prefer for a similar future project and why (Questions 23 and 25, Appendix F.3). We analyze

these free-text answers using inductive qualitative coding. The resulting codebook and code frequency counts are provided in Appendix F.4. In the discussion below, italicized terms followed by identifiers in parentheses refer to qualitative codes from the codebook, e.g., *efficient* (C1a). Here, we summarize the main findings.

Overall, the results show more positive than negative code counts. This suggests that, when asked which approach they would prefer to implement for a similar project in the future, participants tended to justify their choice by referring to the perceived strengths of that approach rather than to the weaknesses of the alternative. Across different coded aspects, respondents paid particular attention to usability and efficiency.

Participants who preferred the model-driven tools mainly did so because they found the approach and tools more *efficient* (C1a) and *intuitive to use* (C1b). They also valued the abstraction provided by the model-driven approach, which *supports the separation of concerns* (C3b), and perceived the generated enforcement mechanisms as improving *reliability* (C3c) and *maintainability, especially when requirements evolve* (C3d). Compared to its predecessor, *vActionGUI* was also perceived as offering *greater flexibility and control* (C3a). Nevertheless, some participants questioned *whether the approach could be used in developing realistic applications* (C3e) *without better tooling support* (C2b).

In contrast, participants who preferred Flask cited its *stronger ecosystem support* (C2b), including documentation, community resources, and the availability of libraries. Respondents also valued the *full control* that the Flask framework gives developers (C3a).

9.5 Threats to Validity

9.5.1 Conclusion Validity

Statistical power. We assessed the adequacy of our sample size using a power analysis for the Wilcoxon Signed-Rank (WSR), Mann–Whitney U (MWU), Kruskal–Wallis (KW), and McNemar tests. McNemar’s test is used for paired binary outcomes (e.g., whether a student over-declared at least one purpose or not) and assesses whether the marginal proportions differ between two conditions.

For our paired comparisons, a WSR power analysis shows that detecting a medium effect ($r = 0.3$) with 90% power at the $p < 0.05$ level requires approximately 117 paired observations. Across the four editions, we collected more than 500 pairs of model-driven and code-centric implementations developed by the same developer, which is above this threshold.

To make a similar observation on non-paired data with the MWU test, one needs two groups of 134 observations to detect a medium effect ($r = 0.3$). Consequently, the small or non-significant differences we report for some comparisons, namely the limited

influence of prior experience (RQ8) and of the chosen authorization library (RQ11), are unlikely to be due to insufficient statistical power.

Survey responses. The survey response rate is relatively low, representing approximately 12–28% of project participants across editions. We therefore treat survey-based findings as complementary to test-based and code-based metrics, which draw on the full population of submissions.

9.5.2 Construct Validity

Correctness measurement. Our evaluation focuses on the correctness of security and privacy enforcement rather than the general programming or regulatory interpretation abilities of the participants. We evaluate the correctness of the implementations using two key security principles (least privilege and complete mediation) and three privacy criteria (purpose limitation, privacy policy adequacy, and consent checking). These principles and criteria are operationalized through both manually written and automatically generated test suites, as well as static analysis of submissions against a validated reference solution.

Student knowledge. Differences in student motivation and prior knowledge could affect study outcomes. We mitigate this threat by providing extensive learning material during the Training phase before the start of the project. This phase familiarizes students with the tools and teaches them how to implement security and privacy requirements.

Effort measurement. Lines of code (LoC) are an imperfect proxy for developer effort, as they do not capture cognitive load or code quality. Our use of LoC per passed test provides a normalized measure of the coding effort required to deliver the intended security or privacy requirements.

Development time was self-reported retroactively via the survey and may therefore be biased, as previous work has shown that self-reports in security user studies tend to be overly optimistic and produce more extreme effects [111]. In the latter two editions, the project also included both security and privacy tasks. As a result, the time allocation reported by participants may have been influenced by their concurrent work on both types of requirements.

Qualitative coding. The inductive coding of participants' free-text responses (Section 9.4.2) was performed by a single coder, the author of this thesis, without an independent second coder. We therefore did not compute inter-rater reliability, and the coding may reflect individual interpretation. To mitigate this threat, we treat the qualitative findings as complementary to the quantitative test- and code-based results. We also report the full codebook with representative quotes in Appendix F.4 for transparency.

9.5.3 Internal Validity

Collaboration and code sharing. All project work was completed individually. We found no evidence of collaboration or code sharing in discussion forums or student repositories. We also ran a pairwise plagiarism detection tool on all submissions and did not detect any cases of code sharing [121]. Nevertheless, we cannot fully exclude the possibility of informal discussions through side channels.

Authorization library choice. In the code-centric approach, except for Ed'25, students were free to choose which authorization library to use, if any. This flexibility may introduce self-selection bias.

Study replication. Our study spans four editions conducted over consecutive years. Although this longitudinal design increases the robustness of our findings, the editions are not fully independent replications. As shown in Appendix F.1, the project description and the study setup were refined across editions. Although we found no evidence of this, students in later editions may have had access to materials from earlier ones.

AI-assisted development. Students were allowed to use external libraries and development tools, including generative AI tools, but we did not collect direct evidence of their use. We therefore treat AI assistance as an uncontrolled factor. Nevertheless, we do not expect this factor to invalidate the main comparison. AI assistance was available in both approaches. To the extent that such tools were useful, they may have benefited the code-centric approach more, since popular programming languages and frameworks, such as Python and Flask, are likely better represented in their training data than our customized modeling languages.

9.5.4 External Validity

Students as study subjects. Although computer science students are an accepted proxy for developers in software engineering experiments [61, 91], the results may not fully generalize to experienced professionals. Salman et al. [117] show that professionals and

students perform similarly when applying new technologies. To mitigate this threat, we designed training materials to establish baseline knowledge in both approaches. Moreover, our multi-week project simulates real development more closely than a single-session controlled experiment. Svahnberg et al. [128] report that when students have a real commitment to the study, they behave more closely to industry practitioners. A graded project gives students this incentive. We therefore believe that using students in our study is justified, given the general difficulty of recruiting professional developers for controlled experiments [117].

Choice of technologies. Our choice of Flask as the code-centric baseline is representative: Python is amongst the most popular programming languages for web development, and Flask ranks as the second most widely used Python web framework, making it a representative choice for building real-world applications [127]. Cedar is the most recent policy-based tool that supports expressive authorization policies [5]. Although there are alternative model-driven tools (see Chapter 3), none of them generate executable enforcement mechanisms from design models for both security and privacy. Nevertheless, evaluating additional frameworks and tools could further generalize our findings.

Part IV

Epilogue

Conclusions

In this thesis, we present a model-driven methodology for supporting privacy requirements, particularly purpose limitation and user consent, in software systems. Using our approach, developers can design privacy requirements as explicit formal models and rely on predefined model transformations to automatically generate the enforcement mechanisms from these models.

Our first contribution is the formal privacy model that captures privacy notions such as personal data, ownership, purposes, purpose hierarchies, declared privacy policy, and consent. Based on this model definition, we define a domain-specific modeling language that is integrated with predefined data and security modeling languages, providing a comprehensive framework for developers to specify different aspects of the software system under development. From these models, our transformation generates security- and privacy-aware Flask web applications.

The second contribution is the implementation of this methodology in the `vActionGUI` tool, which substantially extends the previous version, `ActionGUI`, in both the modeling language and the generated enforcement infrastructure. It supports fine-grained access control enforcement, purpose tracking, consent collection and checking, and a generated privacy dashboard that allows users to exercise additional data-subject rights, including rectification, erasure, and restriction of processing.

The third contribution concerns enforcement at the database layer. We extend this enforcement with support for association classes and propose a model-based characterization of purpose limitation for SQL queries. Based on this characterization, we develop `SQLSPI`, which generates security- and privacy-aware stored procedures and supports dynamic consent tracking at the database layer. We further integrate this approach by intercepting ORM-generated SQL queries and rewriting them to use the generated stored procedures.

Finally, we evaluate the approach through a series of case studies, performance experiments, and a multi-year empirical study comparing model-driven and code-centric development. The case studies show that the methodology can be applied across different application domains and technology stacks. The performance experiments show that the generated enforcement mechanisms remain practical for common query patterns. The empirical study shows that the model-driven approach reduces developer effort, improves maintainability, and substantially reduces security and privacy violations compared with code-centric implementations.

10.1 Limitations and Future Work

Access control semantics. As discussed in Chapter 3, we adopt *access control semantics* in our approach, i.e., only the initial access to personal data is mediated by the enforcement mechanism. Once developer code legitimately reads a personal data value, such as a patient’s name for the purpose of treatment, the value may be stored in a local variable and later used for another purpose without triggering a further security or privacy check.

We chose this semantics because the stricter alternatives would substantially complicate the generation of usable applications. However, this choice limits the guarantees provided by our approach. It is effective against accidental policy violations by well-intentioned, trustworthy developers, but it does not prevent deliberate circumvention. Extending the approach with lightweight taint tracking or static analysis to detect when personal data escapes its declared scope is a promising direction for future work.

Shared personal data. Our privacy model assumes a single data owner for each personal data instance. This simplification excludes scenarios in which personal data is jointly owned, such as shared photos or communication records involving multiple parties. The treatment of shared personal data remains an open problem in data protection [71, Problem 6]. Extending the model to support multiple data owners, including conflict resolution when owners disagree about consent, is part of the future work.

Further aspects of privacy. In this thesis, we focus on controlling the use of collected personal data through purpose limitation and consent. Other important privacy requirements, including data minimization (e.g., collecting only the data necessary for a specific purpose) and anonymization (e.g., removing identifying information), fall outside the current scope of our models. Extending our model-driven methodology to cover these additional aspects is a promising direction for future work.

Data protection formal reasoning. As our methodology generates enforcement code that is correct by construction with respect to the input models, the guarantees it provides ultimately depend on the correctness of the models per se. Validating these models is therefore critical, but currently remains a manual task. A promising direction for future work is to support this validation through automated formal reasoning. For example, such reasoning could check whether a personal data item can be processed for a declared purpose without valid consent, whether the declared purposes are trivially satisfiable, or whether the security permissions and privacy policies interact in unintended ways. This would allow developers to detect specification errors before any enforcement code is generated.

Appendix

Model-based characterization of enforcing fine-grained access control policies on SQL queries

This appendix chapter presents prior work on a model-based characterization of fine-grained access control (FGAC) enforcement for SQL queries. We first introduce the grammar of the supported SQL fragment and then define the characterization of the seven SQL query patterns (1)–(7). Chapter 6 continues this numbering with patterns (8)–(10) for association classes. The full formal development is available in [94, P1].

Supported SQL Fragment The supported SQL fragment is shown below. An SQL query is a **SELECT** statement.

```

    <stm> ::= SELECT <select_items> <from_clause>?
    <select_items> ::= <select_item> (, <select_item>)*
    <select_item> ::= <expr> (AS <alias>)?
    <from_clause> ::= FROM <from_item> <where_clause>?
    <where_clause> ::= WHERE <expr>
    <from_item> ::= <table_factor> | <joined_table>
    <table_factor> ::= <entity_name> <alias>? | <assoc_name> <alias>? | <subquery> <alias>
    <joined_table> ::= <from_item> JOIN <from_item> <join_spec>?
    <join_spec> ::= ON <expr>
    <subquery> ::= ( <stm> )
    <alias> ::= string
    <expr> ::= <expr> <logic_ops> <expr> | NOT <expr> | <bool_primary>
    <bool_primary> ::= <bool_primary> <comp_ops> <bool_primary> | <simple_expr>
    <simple_expr> ::= <literal> | <col_name> | <string_func>(<expr>)
    | <agg_func>(<col_name>) | EXISTS <subquery>
    <logic_ops> ::= AND | OR | XOR
    <comp_ops> ::= = | <> | != | < | <= | > | >= | LIKE
    <literal> ::= <string_literal> | <integer_literal> | <boolean_value>
    <string_literal> ::= 'string'
    <integer_literal> ::= ... | -1 | 0 | 1 | ...
    <boolean_value> ::= TRUE | FALSE
    <agg_func> ::= MAX | MIN | COUNT
    <string_func> ::= LOWER | UPPER

```

Authorization Characterization The central concept is the *authorization predicate* AuthQuery . Let $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ be a data model, and let $\mathcal{S} = (\mathcal{D}_{\mathcal{S}}, \preceq_{\mathcal{R}}, \mathcal{PA})$ be a security model for $\mathcal{D}$, where $\mathcal{D}_{\mathcal{S}}$ extends $\mathcal{D}$ to $(\text{user}, \text{CS}', \preceq_{\mathcal{D}_{\mathcal{S}}}, \text{attr}', \text{meth}, \text{as})$. Let $\mathcal{O} = (\mathbb{O}, \mathbb{M}, \mathbb{R})$ be an object model of $\mathcal{D}_{\mathcal{S}}$. Let $r \in \mathcal{R}$ be a role, and $u \in \text{dom}(\mathbb{O}(\text{user}))$ be a user object. Our definition of the predicate $\text{AuthQuery}()$ covers the following query *patterns*, where $C \in \text{CS}$ and $as \in \text{dom}(\text{as})$ denote, respectively, a class and an association in the underlying data model $\mathcal{D}$.

For each pattern, we first give an informal description and then provide the formal definition of the corresponding $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ predicate.

Pattern (1): `SELECT items FROM C WHERE exp`. The user must be authorized to read all attributes of C that appear in the filter expression exp , for every row in C . In addition, for every row satisfying exp , the user must be authorized to read the attributes appearing in the projection clause $items$. This first query pattern was already characterized in Section 2.4.4. We repeat the definition here for completeness.

Definition A.1 (AuthQuery: SELECT from class).

Let $q = \text{SELECT items FROM } C \text{ WHERE } \text{boolexp}$ be an SQL query.

Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if:

- for every object $o \in \text{dom}(\mathbb{O}(C))$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the Boolean expression boolexp of q , the user u with role r is allowed to read at of o with respect to the security model $\mathcal{S}$ in the context of $\mathcal{O}$; and
- for every object $o \in \text{dom}(\mathbb{O}(C))$ that satisfies the Boolean expression boolexp , and for every attribute $at \in \text{dom}(\text{attr}(C))$ in the projection clauses $items$ of q , the user u with the role r is allowed to read at of o with respect to the security model $\mathcal{S}$ in the context of $\mathcal{O}$.

In the following, we present the remaining six patterns using the same notation.

Pattern (2): `SELECT items FROM as WHERE exp`. An implementation that only checks rows present in the association table may leak information about the absence of links. To avoid this, $\text{AuthQuery}()$ evaluates authorization over the full Cartesian product of the two classes connected by the association, thereby covering both existing and non-existing links. The user must be authorized to read both ends of the association for every pair of objects in this Cartesian product that satisfies exp .

Definition A.2 (AuthQuery: SELECT from association).

Let $q = \text{SELECT items FROM as WHERE exp}$ be an SQL query.

Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if: for every pair (o_l, o_r) in the Cartesian product $\text{dom}(\odot(C_l)) \times \text{dom}(\odot(C_r))$ that satisfies the expression exp , the user u with role r is allowed to read the association link (o_l, o_r) of as .

Pattern (3): $\text{SELECT items FROM subselect WHERE exp}$. The user must be authorized to execute the subquery $subselect$.

Definition A.3 (AuthQuery: SELECT from subquery).

Let $q = \text{SELECT items FROM subselect WHERE exp}$ be an SQL query.

Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if it holds for the subquery, i.e., $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, subselect)$.

In other words, the authorization of a query over a subquery is reduced to the authorization of the subquery itself.

Pattern (4): $\text{SELECT items FROM C JOIN as ON exp WHERE exp'}$. The user must be authorized to read all information used by the join, the filter, and the projection. More specifically, the user must be authorized to read (i) the attributes of C that appear in the join expression exp , for every object of C ; (ii) both association ends of as , for every pair in the Cartesian product of the associated tables; (iii) the attributes of C that appear in the filter expression exp' , for objects in the join result; and (iv) the projected attributes in $items$, for objects satisfying both exp and exp' .

Definition A.4 (AuthQuery: SELECT from class JOIN association).

Let $q = \text{SELECT items FROM C JOIN as ON exp WHERE exp'}$ be an SQL query. Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if:

- for every object $o \in \text{dom}(\odot(C))$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the join expression exp , the user u with role r is allowed to read at of o ;
- for every pair (o_l, o_r) in the Cartesian product $\text{dom}(\odot(C_l)) \times \text{dom}(\odot(C_r))$, the user u with role r is allowed to read the association link (o_l, o_r) of as ;
- for every object $o \in \text{dom}(\odot(C))$ in the result of $C \text{ JOIN as ON exp}$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the filter expression exp' , the user u with role r is allowed to read at of o ; and
- for every object $o \in \text{dom}(\odot(C))$ in the result of $C \text{ JOIN as ON exp WHERE exp'}$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ in the projection clauses $items$ of q , the user u with role r is allowed to read at of o .

The symmetric pattern `SELECT items FROM as JOIN C ON exp WHERE exp'` is handled analogously.

Pattern (5): `SELECT items FROM C JOIN subselect ON exp WHERE exp'`. The user must be authorized to execute *subselect*, and must be authorized to read the attributes of *C* that are used in the join condition, the filter condition, and the projection. As in the previous patterns, these checks are applied progressively. Attributes appearing in the join expression *exp* are checked for objects of *C* considered by the join. Attributes appearing in *exp'* are checked for objects in the join result. Finally, attributes appearing in *items* are checked for objects that also satisfy the filter *exp'*.

Definition A.5 (AuthQuery: SELECT from class JOIN subquery).

Let $q = \text{SELECT items FROM } C \text{ JOIN subselect ON exp WHERE exp'}$ be an SQL query. Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if it holds for the subquery, i.e., $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, \text{subselect})$, and:

- for every object $o \in \text{dom}(\odot(C))$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the join expression *exp*, the user u with role r is allowed to read *at* of o ;
- for every object $o \in \text{dom}(\odot(C))$ in the result of $C \text{ JOIN subselect ON exp}$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ that appears in the filter expression *exp'*, the user u with role r is allowed to read *at* of o ; and
- for every object $o \in \text{dom}(\odot(C))$ in the result of $\text{SELECT } C. * \text{ FROM } C \text{ JOIN subselect ON exp WHERE exp'}$, and for every attribute $at \in \text{dom}(\text{attr}(C))$ in the projection clauses *items* of q , the user u with role r is allowed to read *at* of o .

The symmetric pattern `SELECT items FROM subselect JOIN C ON exp WHERE exp'` is handled analogously.

Pattern (6): `SELECT items FROM as JOIN subselect ON exp WHERE exp'`. The user must be authorized to execute the subquery *subselect*, and must be authorized to read both association ends of *as*, for every pair in the Cartesian product of the classes connected by the association, so that authorization covers both existing and non-existing links. When the join condition *exp* relates the association to the subquery result, the authorization checks are restricted accordingly to the pairs considered by the join.

Definition A.6 (AuthQuery: SELECT from association JOIN subquery).

Let $q = \text{SELECT items FROM as JOIN subselect ON exp WHERE exp'}$ be an SQL query. Then $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, q)$ holds if and only if it holds for the subquery, i.e., $\text{AuthQuery}(\mathcal{S}, \mathcal{O}, u, r, \text{subselect})$, and for every pair (o_l, o_r) in the Cartesian

product $\text{dom}(\mathbb{O}(C_l)) \times \text{dom}(\mathbb{O}(C_r))$ (or the appropriate join with the subquery result, depending on which association ends appear in the join expression exp), the user u with role r is allowed to read the association link (o_l, o_r) of as .

The symmetric pattern `SELECT items FROM subselect JOIN as ON exp WHERE exp'` is handled analogously.

Pattern (7): `SELECT items FROM subselect1 JOIN subselect2 ON exp WHERE exp'`. The user must be authorized to execute both subqueries $subselect_1$ and $subselect_2$.

Definition A.7 (AuthQuery: SELECT from subquery JOIN subquery).

Let $q = \text{SELECT items FROM subselect}_1 \text{ JOIN subselect}_2 \text{ ON exp WHERE exp'}$ be an SQL query. Then $\text{AuthQuery}()$ holds if and only if it holds for both subqueries, i.e., $\text{AuthQuery}(S, O, u, r, subselect_1) \wedge \text{AuthQuery}(S, O, u, r, subselect_2)$.

Model-driven Privacy: Formal Semantics

We now provide a formal semantics for our models. Conceptually, the models formalize two types of access control decisions [18]: declarative access control decisions, which depend only on static information defined in the models, such as roles in the permission assignment; and programmatic access control decisions, which depend on dynamic information captured by the current system state, such as the satisfaction of authorization constraints. We extend both types of decisions to be *purpose-based*, namely, to depend on the privacy model, such as purposes in the declared purpose policies, and on the extended state of the system, such as the existence of a Consent object.

We use first-order logic (FOL) [84] to formalize the semantics of our security and privacy models. FOL syntax consists of logical symbols, such as $\neg$, $\wedge$, $\exists$, as well as non-logical symbols, such as predicate, function, and constant symbols, defined by a *signature*. FOL semantics is defined with respect to a valuation ν of its free variables and a *first-order structure* σ , which consists of a carrier set and interpretation functions mapping non-logical symbols to predicates, functions, and constants, respectively. We write $\sigma, \nu \models \varphi$ if a FOL formula φ is satisfied by the structure σ and the valuation ν . Instead of having a single homogeneous carrier set, *many-sorted FOL* (MSFOL) extends FOL with multiple carrier sets, each containing elements of the same sort.

Static information, such as the relations $\mathcal{PA}$ and $\mathcal{PA}^P$, is formalized as relations in a first-order structure σ_s . Dynamic information is the content of a system state, i.e., an object model of the extended data model $\mathcal{DP}$, which we also formalize as a first-order structure σ_d . The semantics of our security and privacy models is formalized by MSFOL formulas φ_s and φ_p . An overall system state combines both static and dynamic information into a composite structure (σ_s, σ_d) . Given such a composite structure, the decision to allow an action is equivalent to checking whether $(\sigma_s, \sigma_d), \nu \models \varphi_s \wedge \varphi_p$ for some valuation ν .

To define the two structures and the two formulas, we first define the respective signature Σ , partitioned into two parts, Σ_s and Σ_d , representing static and dynamic information.

Signatures. Let the many-sorted signature for static information be

$$\Sigma_s = (\mathcal{R} \cup \Pi, \preceq_{\mathcal{R}}, \emptyset, \{\text{PA}, \text{PA}^P\}).$$

This signature contains a sort for each role in $\mathcal{R}$ and each purpose in Π , the role hierarchy $\preceq_{\mathcal{R}}$ as an order on the sorts, no function symbols, and two relation symbols, PA and

PA^P . Intuitively, these sorts and relation symbols are interpreted by the roles, purposes, permission assignment relation, and declared purpose relation from our security model (Section 2.2.3) and privacy model (Section 4.2).

The signature for dynamic information, Σ_d , is built from a data model, which naturally induces a many-sorted signature with typed function and relation symbols [18]. For clarity, we further split this signature into two parts: the primitive signature Σ_P and the data model signature $\Sigma_{\mathcal{D}}$. The primitive signature is fixed and independent of any particular data model. Let $\Sigma_P = (\text{PS}, \preceq_P, \Omega_P, \emptyset)$ be a many-sorted signature where PS contains the five primitive sorts (Section 2.2.1). The relation $\preceq_P$ is a partial order on PS such that $N \preceq_P Z \preceq_P F$. The set Ω_P contains common function symbols on the primitive sorts, such as conjunction ($\text{and} : \text{Bool} \times \text{Bool} \rightarrow \text{Bool}$), addition ($+$: $F \times F \rightarrow F$), and string length ($\text{size} : \text{String} \rightarrow N$). The primitive signature has no relation symbols.

Given a finite set of sorts $\text{CS} \subseteq \text{ID}$, where $\text{CS} \cap \text{PS} = \emptyset$, the set of all sorts is the smallest set S such that

- $\text{Any} \in S$;
- $\text{CS} \cup \text{PS} \subseteq S$; and
- if $s \in S$ then $\text{Set}(s), \text{Bag}(s), \text{Sequence}(s), \text{Collection}(s) \in S$.

A data model $\mathcal{D} = (\text{CS}, \preceq_{\mathcal{D}}, \text{attr}, \text{meth}, \text{as})$ induces a data model signature $\Sigma_{\mathcal{D}} = (\text{CS}, \preceq_{\mathcal{D}}, \Omega_{\mathcal{D}}, \text{as})$ by considering all attributes and methods as function symbols ($\Omega_{\mathcal{D}} = \bigcup_{c \in \text{CS}} (\text{attr}(c) \cup \text{meth}(c))$), and associations as relation symbols. Finally, the signature $\Sigma_d = (S, \preceq, \Omega_P \cup \Omega_{\mathcal{D}}, \text{as})$ combines the primitive signature Σ_P and the data model signature $\Sigma_{\mathcal{D}}$ induced by $\mathcal{D}$. The order $x \preceq y$ on S is defined as $x \preceq_P y \vee x \preceq_{\mathcal{D}} y \vee y = \text{Any} \vee \exists s. y = \text{Collection}(s) \wedge x \in \{\text{Set}(s), \text{Bag}(s), \text{Sequence}(s)\}$. Intuitively, this order combines the sort orders from the primitive and data model signatures, and introduces the maximal Any sort and the Collection sorts.

Structures. A first-order structure $\sigma = (\llbracket \cdot \rrbracket, \{\cdot\}, \langle \cdot \rangle)$ over a signature Σ consists of three interpretation functions which interpret the signature's sorts, function symbols, and relation symbols. These functions map sorts, function symbols, and relation symbols to carrier sets, functions, and relations, respectively, such that:

1. unordered sorts have disjoint carrier sets, i.e.,

$$\forall x, y \in S. x \not\preceq y \wedge y \not\preceq x \rightarrow \llbracket x \rrbracket \cap \llbracket y \rrbracket = \emptyset;$$

2. the interpretations of ordered sorts are subsets, i.e.,

$$\forall x, y \in S. x \preceq y \rightarrow \llbracket x \rrbracket \subseteq \llbracket y \rrbracket;$$

3. function and relation symbols are mapped to well-typed functions and relations, respectively, i.e.,

$$\forall f : F \in \Omega. \llbracket f \rrbracket \in \llbracket F \rrbracket \quad \text{and} \quad \forall r : R \in \rho. \llbracket r \rrbracket \in \llbracket R \rrbracket.$$

As indicated above, the static structure σ_s maps each role and purpose sort in Σ_s to a set of strings such that these sets respect $\preceq_R$. The relation symbols in Σ_s are interpreted by the permission assignment relation $\llbracket \text{PA} \rrbracket = \mathcal{PA}$ and the declared purpose relation $\llbracket \text{PA}^P \rrbracket = \mathcal{PA}^P$.

For the dynamic structure σ_d , we first interpret the primitive sorts and then the sorts defined by the data model. Conceptually, an object model induces a first-order structure that interprets the sorts, functions, and relations defined by the data model. The primitive sorts are interpreted by the positive 64-bit integers ($\llbracket \mathbb{N} \rrbracket = \mathbb{N}$), 64-bit integers ($\llbracket \mathbb{Z} \rrbracket = \mathbb{Z}$), IEEE-754 floating-point numbers ($\llbracket \mathbb{F} \rrbracket = \mathbb{F}$), ASCII 8-bit character strings ($\llbracket \text{String} \rrbracket = \mathbb{S}$), and Boolean values ($\llbracket \text{Bool} \rrbracket = \mathbb{B} = \{\top, \perp\}$), respectively. The interpretation of sorts is lifted to compound types in the standard way, i.e., $\llbracket A \times B \rrbracket = \llbracket A \rrbracket \times \llbracket B \rrbracket$, $\llbracket A + B \rrbracket = \llbracket A \rrbracket \cup \llbracket B \rrbracket$, $\llbracket A \rightarrow B \rrbracket = \llbracket B \rrbracket^{\llbracket A \rrbracket}$, and $\llbracket A \multimap B \rrbracket = \bigcup_{a \in \llbracket A \rrbracket} \llbracket B \rrbracket^{\llbracket a \rrbracket}$. The function symbols in Ω_P are interpreted as expected.

We now formally define carrier sets for sorts introduced by the data model. Recall from Section 2.2.1 that an object of sort $c \in \text{CS}$ is a pair $(o \mapsto (c, \text{ats}))$. Since attributes are function symbols, ats is formally a set of attribute interpretations $\{\llbracket n \rrbracket \mid n \in \text{dom}(\text{attr}(c))\}$. Since object names are unique, a finite set of objects forms a partial function from object names to their interpretations. Hence, the carrier set $\llbracket c \rrbracket$ of a sort $c \in \text{CS}$ is the set of all partial functions from object names to possible attribute interpretations, i.e.,

$$\llbracket \text{ID} \multimap (\{c\} \times \prod_{n \in \text{dom}(\text{attr}(c))} \llbracket \text{attr}(c)(n) \rrbracket) \rrbracket.$$

An object model $(\mathbb{O}, \mathbb{M}, \mathbb{R})$ of a data model $\mathcal{D}$ induces a structure in which $\mathbb{O}$ defines the carrier sets for $\mathcal{D}$'s sorts, $\mathbb{M}$ defines the functions for $\mathcal{D}$'s methods, and $\mathbb{R}$ defines the relations for $\mathcal{D}$'s associations. All interpretation functions are as defined above.

Model semantics. To capture the formal semantics of our models, we use standard order-sorted FOL syntax over the signature $\Sigma = \Sigma_s \cup \Sigma_d$ additionally extended with the unary relation symbol CAP modeling the current actual purposes. Since our semantics depend on authorization constraints expressed in OCL, we abuse the notation and use OCL *constraints* as subformulas in FOL formulas. This can be done because OCL expressions can be interpreted as first-order formulas, although they use a specialized concrete syntax. We provide syntax and semantics of OCL for clarity in Section 2.2.2.

The structure σ over which we evaluate the formula combines the static structure σ_s and the dynamic structure σ_d . It is further extended with a sequence $\text{CAP} : \Pi$ of current

actual purposes, which abstracts the state of the purpose tracking mechanism. We write $CAP(p)$ as shorthand for $p \in \text{set}(CAP)$. Thus, formulas can treat CAP as a unary predicate, while the operational semantics maintain it as a stack.

Executing a method pushes its actual purpose annotation onto CAP, while returning from the method removes the same annotation from the top of the stack. Formally, we write ϵ for the empty sequence and $S \cdot T$ for sequence concatenation. We define $\text{push}(S, T) = S \cdot T$ and $\text{pop}(S \cdot T, T) = S$. The exact content of CAP depends on the execution history of the system and is specified later by the labeled transition system.

Given a valuation v that binds the caller subject caller, the object self, and, where applicable, the updated value value, the formal semantics of the security model is captured by the MSFOL formula φ_s :

$$\exists r'. \text{PA}(r', a, \varphi) \wedge r' \preceq_{\mathcal{R}} v(\text{caller}).\text{role} \wedge \varphi[\text{self}/v(\text{self}), \text{caller}/v(\text{caller}), \text{value}/v(\text{value})].$$

Here, a is the action being executed, and the constraint φ is evaluated under the valuation v . In particular, $v(\text{caller})$ is the user executing the action, $v(\text{self})$ is the object on which the action is executed, and $v(\text{value})$ is the updated value when such a value exists.

Given a valuation v that binds the subject caller, the object self, and, where applicable, the updated value value, the formal semantics of the privacy model for an action a on a personal property $(C, n) \in \text{Props}^{\mathcal{P}}$ is captured by the MSFOL formula φ_p :

$$\begin{aligned} & (\forall p. CAP(p) \rightarrow \text{PA}^{\mathcal{P}}(p, a, \varphi) \wedge \varphi[\text{caller}/v(\text{caller}), \text{value}/v(\text{value})]) \wedge \\ & \exists \rho \in \text{pdata}(v(\text{self})), c \in \text{consents}(\rho), \text{entity}(\rho) = C \wedge \text{property}(\rho) = n \wedge \\ & \text{user}(c) = \text{owner}(\rho) \wedge \forall p. CAP(p) \rightarrow p \in \text{purposes}(c) \end{aligned}$$

The first conjunct ensures *purpose limitation*: every actual purpose is declared for the action a with a satisfied constraint φ . The second conjunct ensures *consent checking*: a matching consent exists for the accessed personal property, and this consent must be linked to the data owner.

Access control decisions can be evaluated with respect to the current system state. In contrast, the purpose tracking mechanism, modeled by the CAP relation, depends on execution history. We therefore need an explicit account of system execution over time. We then formalize this execution behavior as a labeled transition system.

System semantics To capture how the system evolves over a sequence of actions, and in particular how the purpose tracking mechanism maintains CAP, we define the system dynamics as a labeled transition system (LTS).

We consider the following set of actions $\mathcal{A}'$, which extends the data action set $\mathcal{A}$ defined in Definition 2.6 with return actions for entity methods and application endpoints:

$$\mathcal{A}' = \mathcal{A} \cup \{\text{return}(C, n) \mid n \in \text{dom}(\text{meth}(C)), C \in \text{CS}\} \cup \{\text{return}(e) \mid e \in E\}$$

Formally, we model a system as an LTS $\Delta = (Q, \mathcal{A}', \delta)$, where Q is the set of all possible composite structures σ , and the edges are labeled by the actions $\mathcal{A}'$. The transition relation $\delta \subseteq Q \times \mathcal{A}' \times Q$ contains only authorized transitions. A transition $(\sigma, a, \sigma') \in \delta$ relates the current structure σ to its successor σ' obtained by executing action a .

For each concrete action $a \in \mathcal{A}'$, we write v_a for the valuation induced by a . This valuation binds the variables used in the security and privacy formulas. For example, for a concrete update action $a = \text{update}(C, n)$, we have $v_a(\text{caller}) = u, v_a(\text{self}) = o, v_a(\text{value}) = v$. For readability, we write $\sigma \models_a \varphi$ if and only if $\sigma, v_a \models \varphi$.

Table B.1 presents the operational semantics of the system. For simplicity, we treat all collections as sets, and we write $\sigma \models_a \varphi$ for the evaluation of φ under the valuation v_a induced by the action a .

Each rule relates a source structure σ to its successor σ' and is enabled only when its action is authorized. The rules READ, UPDATE, ADD, and REMOVE govern access to object properties and therefore require satisfaction from both the security and privacy formulas, $\sigma \models_a \varphi_s \wedge \varphi_p$. The READ rule leaves the state unchanged. The UPDATE rule assigns the value $v = v_a(\text{value})$ to attribute n of the accessed object $o = v_a(\text{self})$. The ADD and REMOVE rules respectively insert and delete the link (o, v) in the association r underlying end n . The rules CREATE and DELETE create and delete an object of class C from its extent $\llbracket C \rrbracket$ and require only the security formula φ_s . Finally, the EXEC and RET rules maintain the stack of current actual purposes CAP, i.e., executing an endpoint (EXEC_E) or entity method (EXEC_M) pushes the action's actual purpose onto CAP, while the matching return (RET_E, RET_M) pops it.

Correctness. By construction, the transition relation δ is *sound*: every transition it admits already satisfies the enforcement conditions with its action because the premise of each transition rule is precisely the corresponding enforcement formula. Concretely, for every transition $(\sigma, a, \sigma') \in \delta$ with $a \in \mathcal{A}$:

1. *Authorized.* The action is authorized, i.e., $\sigma \models_a \varphi_s$, since every data-action rule includes φ_s as a guard.
2. *Purpose limitation and consent.* If a accesses personal property $(C, n) \in \text{Props}^{\mathcal{P}}$, then $\sigma \models_a \varphi_p$. The first conjunct of φ_p establishes purpose limitation: every current actual purpose is declared for a and satisfies its associated constraint. The second conjunct establishes consent checking: there exists consent linked to the data owner for the accessed property.

Both properties follow immediately from the inversion of the rule deriving the transition. Establishing that the generated enforcement code *faithfully implements* δ , i.e., the generated checks and CAP induce a transition system refining this LTS, is left for future work.

Table B.1: Operational semantics of the system.

$\frac{a = \text{read}(C, n) \quad \sigma \models_a \varphi_s \wedge \varphi_p}{\sigma \xrightarrow{\text{read}(C, n)} \sigma} \text{READ}$		$\frac{a = \text{update}(C, n) \quad v_a(\text{self}) = o \quad \text{entity}(o) = C \quad v_a(\text{value}) = v \quad \sigma \models_a \varphi_s \wedge \varphi_p \quad \sigma' = \sigma[\{\{n\}\} \mapsto \{\{n\}\}[o \mapsto v]]}{\sigma \xrightarrow{\text{update}(C, n)} \sigma'} \text{UPDATE}$	
$\frac{a = \text{add}(C, n) \quad v_a(\text{self}) = o \quad \text{entity}(o) = C \quad v_a(\text{value}) = v \quad \sigma \models_a \varphi_s \wedge \varphi_p \quad \sigma' = \sigma[\{\{r\}\} \mapsto \{\{r\}\} \cup (o, v)]}{\sigma \xrightarrow{\text{add}(C, n)} \sigma'} \text{ADD}$		$\frac{a = \text{remove}(C, n) \quad v_a(\text{self}) = o \quad \text{entity}(o) = C \quad v_a(\text{value}) = v \quad (o, v) \in \langle\langle r \rangle\rangle \quad \sigma \models_a \varphi_s \wedge \varphi_p \quad \sigma' = \sigma[\{\{r\}\} \mapsto \{\{r\}\} \setminus (o, v)]}{\sigma \xrightarrow{\text{remove}(C, n)} \sigma'} \text{REMOVE}$	
$\frac{a = \text{create}(C) \quad \sigma \models_a \varphi_s \quad \sigma' = \sigma[\llbracket C \rrbracket \mapsto \llbracket C \rrbracket \cup c']}{\sigma \xrightarrow{\text{create}(C)} \sigma'} \text{CREATE}$		$\frac{a = \text{delete}(C) \quad \sigma \models_a \varphi_s \quad \sigma' = \sigma[\llbracket C \rrbracket \mapsto \llbracket C \rrbracket \setminus c]}{\sigma \xrightarrow{\text{delete}(C)} \sigma'} \text{DELETE}$	
$\frac{a = \text{execute}(e) \quad e \in E \quad \sigma \models_a \varphi_s \quad \sigma' = \sigma[\llbracket \langle\langle CAP \rangle\rangle, \text{annotate}(e) \rrbracket]}{\sigma \xrightarrow{\text{execute}(e)} \sigma'} \text{EXEC_E}$		$\frac{e \in E \quad \sigma' = \sigma[\llbracket \langle\langle CAP \rangle\rangle, \text{pop}(\langle\langle CAP \rangle\rangle, \text{annotate}(e)) \rrbracket]}{\sigma \xrightarrow{\text{return}(e)} \sigma'} \text{RET_E}$	
$\frac{a = \text{execute}(C, n) \quad \sigma \models_a \varphi_s \quad \sigma' = \sigma[\llbracket \langle\langle CAP \rangle\rangle, \text{push}(\langle\langle CAP \rangle\rangle, \text{annotate}(n)) \rrbracket]}{\sigma \xrightarrow{\text{execute}(C, n)} \sigma'} \text{EXEC_M}$		$\frac{\sigma' = \sigma[\llbracket \langle\langle CAP \rangle\rangle, \text{pop}(\langle\langle CAP \rangle\rangle, \text{annotate}(n)) \rrbracket]}{\sigma \xrightarrow{\text{return}(C, n)} \sigma'} \text{RET_M}$	

ASP.NET C# Model Transformation

In addition to the Flask/Python model transformation and the `vActionGUI` tool described in Chapter 5, we have implemented a second model transformation and a tool prototype targeting C# ASP.NET Core [P3]. Similarly to `vActionGUI`, this tool takes three models (data, security, and privacy) as input and generates an ASP.NET web application that enforces the specified policies. In this appendix, we present the transformation and describe the tool usage.

Table C.1: Model-driven privacy methodology targeting ASP.NET C#.

Developers define	ASP.NET C# generates
Step 1 Data model	Data classes & method stubs (EF Core, §C.1.1)
Step 2 Security model	Security enforcement mechanism (§C.1.4)
– user class	Authentication (Core Identity, §C.1.3)
Step 3 Privacy model	Privacy enforcement mechanism (via PostSharp [107])
– personal data denotation	Privacy entity & helper methods (§C.1.5)
– purposes and hierarchy	Database configuration (§C.1.5)
– declared purposes	Consent collection page (§C.1.7)
– actual purpose mapping	Purpose tracking mechanism (via PostSharp [107])
Step 4 Developers implement method stubs & presentation layers	

We now describe the generated artifacts.

C.1 Generated Artifacts

C.1.1 Database Schema and Entity Classes

We target an ASP.NET C# web application with the database layer following the Code-First development approach [85]. Code-First is a development methodology where developers design the application’s logic and data structures directly with code. The supported infrastructures, such as the database schema, are then automatically generated or derived from that code. This approach is part of the Entity Framework Core (EF Core) [88].

From the data model, the transformation generates EF Core C# classes. Each entity sort becomes a class with a primary key and properties for its attributes. The class

is automatically mapped to a database table. Binary associations are generated as classes with foreign keys and navigation properties for both ends, along with a static `create(left, right)` factory method. Entity methods and application endpoints are generated as method stubs that the developer fills in with business logic.

Example C.1. The following shows excerpts of the entity classes generated for the EventPlatform data model (Example 5.6). Each entity sort becomes a C# class. The Person class (lines 1–8) defines properties for its attributes (name, surname). The Event class (lines 10–18) similarly defines its attributes (title, description, private). The Ownership association (lines 20–30) is generated as a class with foreign keys to both Person and Event, and a static create factory method.

Listing C.1: Generated entity classes for the EventPlatform

```

1  public class Person {
2      [Key]
3      public int Id { get; set; }
4      public string? name { get; set; }
5      public string? surname { get; set; }
6      public static Person create() { ... }
7      public static void delete(Person obj) { ... }
8  }
9
10 public class Event {
11     [Key]
12     public int Id { get; set; }
13     public string? title { get; set; }
14     public string? description { get; set; }
15     public bool? private_ { get; set; }
16     public static Event create() { ... }
17     public static void delete(Event obj) { ... }
18 }
19
20 public class Ownership {
21     [Key]
22     public int Id { get; set; }
23     [ForeignKey("Person")]
24     public int? ownerId { get; set; }
25     public virtual Person? owner { get; set; }
26     [ForeignKey("Event")]
27     public int? eventsId { get; set; }
28     public virtual Event? events { get; set; }
29     public static Ownership create(Person left, Event right) { ... }
30 }

```


C.1.2 Application Routes and Method Stubs

From the data model's entity methods, the transformation generates method stubs inside the entity classes. Each entity method defined in the data model becomes a method with a placeholder body that the developer fills in with the business logic. Application endpoints defined in the data model are exposed through ASP.NET MVC controllers, which the developer implements as part of Step 4 in Table C.1. Unlike the Flask transformation where endpoints are automatically generated as routes with a `@secure` decorator (Section 5.2.2), the ASP.NET transformation delegates the controller and view implementation to the developer. The enforcement is applied at the entity level (Section C.1.4), so the developer does not need to write any enforcement code in the controllers.

Example C.2. The following shows the method stubs generated for the EventPlatform. The Person class includes method stubs for the entity methods `getFullname` and `countAttendingEvents` (lines 3–4). The Event class includes a stub for `countPureAttendants` (line 9).

Listing C.2: Generated method stubs

```
1 public class Person {  
2     ...  
3     public string getFullname() { ... }  
4     public int countAttendingEvents() { ... }  
5 }  
6  
7 public class Event {  
8     ...  
9     public int countPureAttendants() { ... }  
10 }
```

C.1.3 Authentication

For authentication, we rely on the ASP.NET Core Identity [87]. More specifically, to handle users, roles, and the authentication and registration process, the user sort (identified by the USER declaration in the security model) extends the `IdentityUser` class from the ASP.NET Core Identity. Similarly to `Flask-User` in `vActionGUI`, ASP.NET Core Identity provides an API for login, user management, passwords, and roles. It does not support role hierarchies natively, so we encode them in the generated access control enforcement mechanism as described later.

Example C.3. The following shows how the Person class from Listing C.1 of Example C.1 is extended with user annotations. The `userId` foreign key (line 5)

links to ASP.NET Core Identity for authentication, and the role field (line 7) stores the user's role.

Listing C.3: Property annotations on the Person class

```

1  public class Person {
2      [Key]
3      public int Id { get; set; }
4      [ForeignKey("ApplicationUser")]
5      public string? userId { get; set; }
6      public virtual ApplicationUser? user { get; set; }
7      public string role { get; set; }
8      ...
9  }
```

C.1.4 Security Enforcement

From the security model, the transformation generates enforcement mechanisms that intercept every data access and evaluate the corresponding authorization constraints. First, we annotate each property with the attribute `[SecuredData]` and each method with the attribute `[SecuredMethod]`.

Example C.4. The following shows how the Person attributes are annotated with `[SecuredData]` (lines 6–8), and each method with `[SecuredMethod]` (lines 10, 12).

Listing C.4: Method annotations on the Person class

```

1  [Class]
2  public class Person
3  {
4      ...
5      [SecuredData]
6      public string? name { get; set; }
7      [SecuredData]
8      public string? surname { get; set; }
9      [SecuredMethod]
10     public static Person create() { ... }
11     [SecuredMethod]
12     public static void delete(Person obj) { ... }
13     ...
14 }
```

Our instrumentation relies on PostSharp [107], a C# aspect-oriented programming (AOP) library, through two aspects:

- `SecuredDataAttribute` extends PostSharp's `LocationInterceptionAspect` and intercepts all `[SecuredData]` property get/set operations. On each access, it

resolves whether the target is a class attribute or an association end, then enforces the corresponding authorization constraint.

- [SecuredMethod] extends OnMethodBoundaryAspect and intercepts the method entry and exit. Upon method entry, it detects whether the method is a create or delete action on an object and performs the appropriate authorization check.

Both aspects invoke the authorization constraint evaluation in the SecurityEnforcer class. This generated SecurityEnforcer class contains one public dispatcher method per action type. Algorithm C.1 presents the enforcement logic for the read action on attributes; the other action types (update, create, delete) follow the same structure.

Algorithm C.1: Security enforcement mechanism for attribute read access

Input: $S = (\mathcal{D}_S, \preceq_{\mathcal{R}}, \mathcal{PA})$, entity name e , attribute name a , object o

Output: permit or deny the read access

Function AuthCheckRead(S, e, a, o):

```

    caller ← GetCurrentUser()
    role ← GetCurrentRole()
    result ← false
    foreach role  $r_i \in \mathcal{R}$  do
        if  $role = r_i$  then
            | result ← result  $\vee$  AuthCheckReadRole( $r_i, e, a, o, caller$ )
        end
    end
    if  $\neg result$  then
        | raise SecurityException
    end

```

Function AuthCheckReadRole($r, e, a, o, caller$):

```

    if  $\exists r' \in \mathcal{R}$  such that  $r' \preceq_{\mathcal{R}} r$  then
        if AuthCheckReadRole( $r', e, a, o, caller$ ) then
            | return true
        end
    end
    foreach policy  $p \in \mathcal{PA}$  where  $p.role = r$  and  $p.action.name = \text{read}$  do
        if  $e = p.action.class$  and  $a = p.action.attribute$  then
            if eval( $p.constraint, self \mapsto o, caller \mapsto caller$ ) then
                | return true
            end
        end
    end
    return false

```

When the application code accesses an attribute of a secured entity, the AOP library triggers the dispatcher procedure. The dispatcher resolves the current caller and role, then iterates over the declared roles to find the matching one. It delegates to the role-specific function, which first checks the parent role if a hierarchy exists, then iterates over the matching policy entries. For each entry whose entity and attribute match the access, the OCL authorization constraint, manually implemented in C# code, is evaluated with the appropriate variable bindings. If no policy grants access, the function returns false, and the dispatcher raises a `SecurityException`.

Note that, unlike `νActionGUI`, which evaluates OCL authorization constraints using the OCL-py evaluator in the Python runtime context, this C# prototype currently requires the developer to translate each OCL constraint into C# executable code by hand. Automatically translating OCL constraints to C# expressions remains future work.

C.1.5 Purpose, Personal Data, and Consent

The transformation generates three infrastructure classes that implement the privacy extension of the data model, corresponding to the green classes in Figure 4.2.

Example C.5. The Purpose class corresponds to the Π sort, with a name attribute. The purposes defined in the privacy model are instantiated in the database at the database initialization. Next, the PersonalData class corresponds to the PersonalData sort. It records which entity and which property hold personal data via the entity and property attributes. Listing C.5 presents these generated classes.

Listing C.5: Generated privacy infrastructure classes

```

1 public class Purpose {
2     [Key]
3     public int id { get; set; }
4     public string? name { get; set; }
5 }
6
7 public class PersonalData {
8     [Key]
9     public int id { get; set; }
10    public string? entity { get; set; }
11    public string? property { get; set; }
12 }
13
14 public class Consent {
15     [Key]
16     public int id { get; set; }
17     [ForeignKey("Purpose")]
18     public int purposeId { get; set; }
19     public virtual Purpose purpose { get; set; }

```

```

20     [ForeignKey("ApplicationUser")]
21     public string? userId { get; set; }
22     public virtual IdentityUser? user { get; set; }
23     [ForeignKey("PersonalData")]
24     public int? dataId { get; set; }
25     public virtual PersonalData? data { get; set; }
26 }

```

Each entity class identified as containing personal data in the privacy model receives navigation properties linking to its `PersonalData` records. The `Consent` class corresponds to the `Consent` sort, linking a `Purpose`, a `Person` (the data subject), and a `PersonalData` record via foreign keys.

Example C.6. The following shows how the `Person` class from Listing C.4 is further extended with privacy annotations. The navigation property `pdata` (line 5) links to the `PersonalData` records associated with this entity, corresponding to the `+pdata` association in Figure 4.2. The `getFullname` method (line 8) is annotated with the `CORE` purpose, indicating that when this method is invoked, the `CORE` purpose is activated.

Listing C.6: Privacy annotations on the `Person` class

```

1  [Class]
2  public class Person
3  {
4      ...
5      public virtual ICollection<PersonalData> pdata { get; set; }
6
7      [SecuredMethod(purpose = "CORE")]
8      public string getFullname() { ... }
9      ...
10 }

```

C.1.6 Purpose Tracking

The `[SecuredMethod]` attribute is extended to include the method's actual purpose as a parameter (e.g., `[SecuredMethod(purpose = "CORE")]`), specified as a constant during code generation based on the `Actual` purposes block of the privacy model. On method entry, the `OnEntry` handler of the aspect pushes the purpose onto a thread-local stack maintained by the `PrivacyEnforcer`; on method exit, the `OnExit` handler pops it.

C.1.7 Consent Collection

Analogously to the Flask privacy notice (Section 5.2.4), a consent collection page is generated that presents the application’s privacy policy and allows users to consent to each declared purpose individually. The generated `CollectConsentController` fetches the current user, builds a list of `Policy` objects from the privacy model, and checks existing consent records. The Razor view renders each policy as a human-readable sentence of the form “If [condition], then we will use your [data] information for the purpose of [purpose].”, accompanied by accept/reject buttons. Users can grant or revoke consent at any time, and their decisions are persisted as `Consent` records in the database and re-evaluated on each data access.

Unlike the Flask transformation, the ASP.NET transformation does not generate a privacy dashboard for data-subject rights (rectify, erase, restrict), which was not part of the original implementation.

C.1.8 Privacy Enforcement

From the privacy model, the transformation generates a `PrivacyEnforcer` class that implements consent checking, analogous to the Flask privacy enforcement (Section 5.2.5). The enforcement operates at the data access level: every access to a property annotated with `[SecuredData]` triggers a consent check. Algorithm C.2 presents the consent checking logic.

Algorithm C.2: Consent checking mechanism for personal data access

Input: entity name e , attribute name a , object o

Output: permit or deny the data access

Function `ConsentCheck( $e, a, o$ ):`

```

     $user \leftarrow \text{GetCurrentUser}()$ 
     $pd \leftarrow \text{LookupPersonalData}(e, a)$ 
    if  $pd = \text{null}$  then
        | return
    end
    foreach  $purpose\ p \in \text{actualPurposes}$  do
        | if  $\neg \exists \text{Consent } c : c.purpose = p \wedge c.user = user \wedge c.data = pd$  then
            | | raise PrivacyException
        | end
    end

```

The procedure resolves the current user and looks up the `PersonalData` record corresponding to the accessed entity and attribute. If the attribute is not marked as personal data in the privacy model, the access is permitted without further checks. Otherwise,

for each currently active purpose on the thread-local stack, the procedure verifies that a Consent record exists linking the purpose, the data subject, and the personal data item. If any active purpose lacks consent, a `PrivacyException` is raised.

This C# prototype implements consent checking in a simplified form. Unlike the complete privacy enforcement mechanism from Chapter 5, Algorithm 5.3, it only checks consent against the exact purpose rather than against the purpose hierarchy. Accordingly, the generated `Purpose` class omits a parent reference. The prototype also does not perform a separate purpose limitation check. Extending the prototype with full support for purpose hierarchies and purpose limitation is left for future work.

APPENDIX D

νActionGUI Tool Usage

νActionGUI is distributed as a Docker container and is publicly available at [96]. The tool is released under the MIT License. In this appendix, we describe how to use the tool. We first present the standard workflow for generating and running applications, and then describe the setup for development mode.

Getting Started. The distribution includes a `docker-compose.yml` file that defines a Linux container with all required Python dependencies pre-installed. To start νActionGUI for the first time, the developer navigates to the directory containing `docker-compose.yml` and runs:

```
$ docker compose up --build --detach
```

This command builds the Docker images and starts the container, named `nuactiongui` by default.¹ The developer can then open an interactive shell inside the running container with:

```
$ docker exec -it nuactiongui bash
```

If the container has been started successfully, the shell prints the νActionGUI welcome message and opens with the Python virtual environment already activated:

```
#####
#                                     #
#               NuActionGUI Tool      #
#                                     #
#####
# Organization: InfSec Group, D-INFK, ETH Zurich      #
# Authors:      Srdan Krstic, Hoang Nguyen            #
# License:      MIT License                          #
#                                     #
# Description:                                       #
# Simple Py generator for privacy enhanced web applications #
#                                     #
#####
```

The shell opens directly in the tool directory because the `docker-compose.yml` configuration mounts the νActionGUI directory as a volume. Listing the directory shows the main structure of the tool:

¹Once the image has been built, the `--build` flag can be omitted to speed up subsequent starts.

```
(venv) $ ls
Makefile  examples/  ocl-py/  output/  pyproject.toml  requirements.txt  src/  tooling/
```

The `src/` directory contains the `vActionGUI` source code, including the code generator. The `examples/` directory is synchronized with the corresponding local directory, so models can be edited locally using any code editor. The `output/` directory is the default location where generated applications are placed.

D.1 Usage Setup

This part of the setup is for developers who wish to use `vActionGUI` to generate web applications.

Create a new project. To create a new project, the developer creates a subdirectory under `examples/`. Inside this project directory, the developer defines the three model files. The data, security, and privacy models use the suffixes `.dtm`, `.stm`, and `.ptm`, respectively.

Generating the project. Assume that the developer has created a model directory `MyProject` and defined all three models. From the root directory of the tool, the developer runs:

```
(venv) $ nuactiongui -p MyProject -i examples -o <output_folder>
```

This command generates a new project named `MyProject` inside `<output_folder>`.² The generated project contains the web application and the enforcement code described in Section 5.2.

Running the project. To execute the generated web application, the developer navigates to the generated project directory and runs:

```
(venv) $ flask --app app.py run --host=0.0.0.0
```

The application is then reachable from the developer's local machine at `localhost:5000/`.

²The `-i` and `-o` options are optional (defaulting to `models/` and `output/`, respectively); paths are resolved relative to the current directory.

Regeneration after model changes. One key advantage of the model-driven approach is that changes to requirements can often be localized to the corresponding models. When a model is modified, `vActionGUI` can regenerate the affected artifacts.

Data requirement changes, such as adding new entities, attributes, or associations, require the developer to update the data model. Since data model changes modify the database structure, the application must be generated from scratch rather than regenerated incrementally. If the new data elements are subject to access control or privacy requirements, the security and privacy models must be updated accordingly.

Security requirement changes, such as adding new roles, modifying access control constraints, or changing the role hierarchy, require an update to the security model. The data model and the developer's business logic may remain unchanged.

Privacy requirement changes, such as declaring additional personal-data items, adding new purposes, or modifying consent conditions, require an update to the privacy model. Again, the data model and the business logic remain unchanged.

To regenerate only the security and/or privacy artifacts, the developer runs:

```
(venv) $ nuactiongui -p <project_name> -i examples -re
```

The `-re` flag instructs `vActionGUI` to regenerate only the artifacts affected by changes to the security or privacy models. This preserves the developer's business logic and presentation layer code.

Note that regeneration may still affect the database schema if the underlying data model has changed. In that case, the existing database instance should be deleted and recreated before running the application again. The developer may also need to update the business logic to account for the new data structures according to their intended application behavior.

D.1.1 Installing the VS Code Extension

`vActionGUI` provides a Visual Studio Code (VSCode) extension for editing the three modeling languages. The extension supports syntax highlighting and model validation for data, security, and privacy models.³

³The extension is released under the MIT License. It is under active development.

Requirements. The extension targets Visual Studio Code version 1.100 or later. Syntax highlighting and model validation features require the *vActionGUI* development environment described in Section D.2, including the generated ANTLR parsers produced by running `make` grammars.

The extension recognizes the following file types: `.dtm` for data model, `.stm` for security model, and `.ptm` for privacy model. The language server reports errors and warnings as the developer edits the model files, including malformed syntax, undefined references, ill-typed expressions, and warnings from the privacy compiler. Since security and privacy models refer to elements declared in the data model, validation of a security or privacy model file also checks certain properties against the data model.

To additionally show the diagnostic message inline, next to the offending line as in Example D.1, the developer may install the Error Lens extension [132].

Example D.1. Consider the Event Management Platform security model. Suppose the developer adds to the read permission of the VISITOR role (line 5) by appending `and value.oclIsUndefined()` to its constraint. As the `value` variable is not available for the read action, the language server flags the constraint and renders the following inline message:

```
1  USER Person
2
3  anonymous Role VISITOR {
4    Event {
5      read(title) [not self.private and value.oclIsUndefined()]
6    }
7  }
```

Constraint for action read must have at most {'self', 'caller'} free variables [Ln 5, Col 22]

Settings. The developer can configure the Python interpreter used by the language server through the `nag.server.pythonPath` setting. When this setting is not provided, the interpreter is auto-detected. After changing the setting, the developer can restart the language server using the command `vActionGUI: Restart language server` from the VSCode command palette.

Running the extension. The extension is located in the `tooling/vscode-nag/` directory. To run it from source, the developer first installs the Node dependencies:

```
$ cd tooling/vscode-nag/
$ npm install
```

The developer then presses F5 in the VSCode window, which launches a second VSCode window with the extension loaded. Opening a `.dtm`, `.stm`, or `.ptm` file in that window activates syntax highlighting and model validation.

D.2 Developer Setup

This part of the setup is intended for developers who want to modify or extend `vActionGUI` itself.

Building the environment. After starting the container and opening an interactive shell, the developer installs the Python dependencies required for development:

```
(venv) $ pip install -r requirements.txt
```

The developer then builds the ANTLR parsers for the modeling languages:

```
(venv) $ make clean && make grammars
```

The `make grammars` command compiles the ANTLR4 grammar files used by the parsers for the data (`.dtm`), security (`.stm`), and privacy (`.ptm`) models.

The developer then installs the local OCL python evaluator `ocl-py` library:

```
(venv) $ pip install ./ocl-py
```

Finally, the developer installs the `vActionGUI` package, which makes the `nuactiongui` command available:

```
(venv) $ pip install -e .
```

Verifying the setup. To check that the development environment works correctly, the developer can generate the bundled example application:

```
(venv) $ nuactiongui -p EventPlatform -i examples -o output
```


Case Study Requirements

This appendix presents the case studies used in this thesis. It includes the functional, security, and privacy requirements of the web applications used to evaluate the `vActionGUI` tool (Section 5.3), as well as the application used in the empirical study (Part III).

E.1 Conference Management System

Data model. Figure E.1 presents the ConfMS data model. The model consists of two entities (Researcher and Paper) and three associations (Authorship, Reviewership, and Advisorship). A Researcher has a name and a Boolean flag `student` indicating whether they are a student. A Paper has a title, a year, and a Boolean flag `published`.

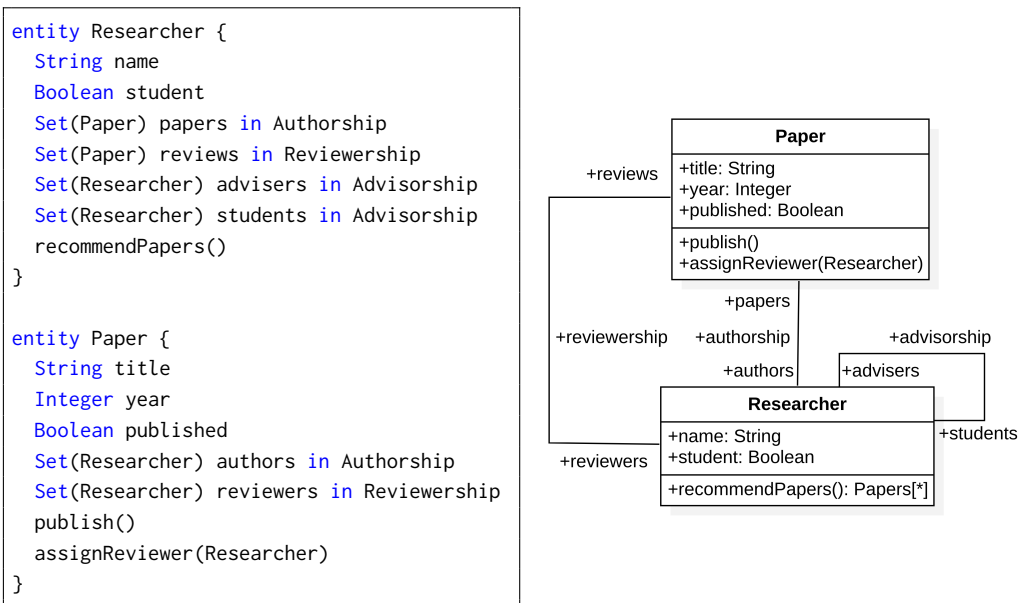


Figure E.1: Data model of the ConfMS: the textual model in our data modeling language (left) and the corresponding UML class diagram (right).

Authorship association links researchers to the papers they have authored, Reviewership links researchers to the papers they review, and Advisorship is a self-association on Researcher linking advisers to students. The Paper entity defines two methods: `publish()`, which marks a paper as published, and `assignReviewer(Researcher)`, which

assigns a reviewer to the paper. The Researcher entity defines `recommendPapers()`, which recommends papers to a student based on their authored papers.

Security requirements. The ConfMS application has three authenticated user roles: Normal, PC, and Chair, in addition to an anonymous Visitor role. Every registered user is assigned the Normal role by default. Using the *vActionGUI* security modeling language, these policies are expressed as:

```

1  USER Researcher
2
3  anonymous Role VISITOR { }
4
5  default Role NORMAL {
6    Paper {
7      create
8      read(title), read(year) [self.published]
9      update(title), update(year) [self.authors->includes(caller)]
10     add(authors), remove(authors) [self.authors->includes(caller)]
11     read(authors) [self.authors->includes(caller)]
12   }
13   Researcher {
14     read(name), read(student) [self = caller]
15     update(name) [self = caller]
16     read(papers) [self = caller]
17     execute(recommendPapers) [self = caller]
18   }
19 }
20
21 Role PC extends NORMAL {
22   Paper {
23     read(title), read(year), read(published)
24     read(reviewers)
25     execute(assignReviewer)
26   }
27 }
28
29 Role CHAIR extends PC {
30   Paper {
31     read(published), update(published)
32     execute(publish)
33   }
34 }

```

Normal users can manage their own profiles and paper submissions. In particular, they may read their own name and student status, update their own name, create new papers, and read the papers they have authored. They may also update the title and year of their own papers, add or remove authors from these papers, and read their papers' authors. In addition, normal users may read the title and year of published papers and

see what papers have been recommended (i.e., execute the `recommendPapers()` function) on themselves.

Program committee members can manage the review process. They may read the title, year, and publication status of all papers, read the reviewers assigned to all papers, and assign reviewers (i.e., execute the `assignReviewer()` function) to any paper.

Chair users can manage publication decisions. They may read and update the publication status of any paper and publish (i.e., execute the `publish()` function) any paper.

Privacy requirements. The personal data in ConfMS is associated with the Researcher entity. It includes researchers' basic information, such as their name and student status, as well as their authored and reviewed papers. The application then uses personal data for three purposes: `PUBLISHPAPER`, `ASSIGNREVIEWER`, and `RECOMMENDPAPERS`. The privacy requirements are as follows:

- A student's authored papers may be used to recommend papers to that student.
- A researcher's authored papers may be used for publishing decisions.
- A researcher's reviewed papers may be used to assign reviewers.

Using the `vActionGUI` privacy modeling language, these policies are expressed as:

```

1  Personal data {
2    Researcher.name
3    Researcher.student
4    Researcher.papers
5    Researcher.reviews
6  }
7
8  Purposes {
9    PUBLISHPAPER
10   ASSIGNREVIEWER
11   RECOMMENDPAPERS
12 }
13
14 Actual purposes {
15   Paper.publish PUBLISHPAPER
16   Paper.assignReviewer ASSIGNREVIEWER
17   Researcher.recommendPapers RECOMMENDPAPERS
18 }
19
20 Declared purposes {
21   Researcher.papers RECOMMENDPAPERS [self.student] <you are a student>
22   Researcher.papers PUBLISHPAPER
23   Researcher.reviews ASSIGNREVIEWER
24 }
```

E.2 Social Networking Application

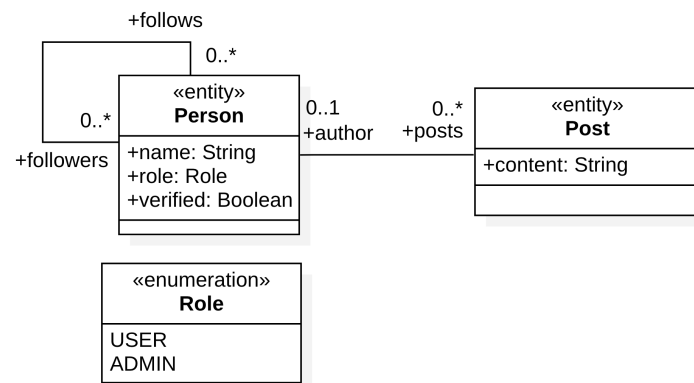


Figure E.2: UML class diagram of MiniTweet.

Data model. Figure E.2 presents the MiniTweet data model. The model consists of two entities, Person and Post, and two associations. A Person has a name and a Boolean flag verified. A Post has a content attribute. The association between Person and Post links users to the posts they authored. The self-association on Person models the follow relationship, linking each user to the users they follow and to their followers. In addition, Person defines the method `generate_ad` for generating personalized advertisements.

```

entity Person {
    String name
    Boolean verified
    Set(Post) posts in Authorship
    Set(Person) follows oppositeTo followers
    Set(Person) followers oppositeTo follows
    generate_ad()
}

entity Post {
    String content
    Person author in Authorship
}
  
```

Security requirements. The application has two authenticated roles: REGULARUSER and ADMIN.

Regular users can use the core functionality. They may create posts, set themselves as the author of posts they create, update or delete their own posts, and read the content and author of any post. They may also read any user's name and posts, read their own

role and verified status, update their own name, manage their follow relationships, and execute generate_ad on themselves.

Administrators should be able to moderate the platform. They may read and delete any post, and they may read and update the role and verified status of any user.

Using our security modeling language, the above policies are expressed as:

```

1  USER Person
2
3  anonymous Role VISITOR { }
4
5  default Role REGULARUSER {
6    Post {
7      create
8      update(author) [self.author.oclIsUndefined() and value = caller]
9      update(content) [self.author = caller]
10     delete [self.author = caller]
11     read(content), read(author) [true]
12   }
13   Person {
14     execute(generate_ad) [self = caller]
15     read(name) [true]
16     read(verified), read(role) [self = caller]
17     read(posts) [true]
18     update(name) [self = caller]
19     read(follows) [self = caller]
20     add(follows), remove(follows) [self = caller]
21     add(followers) [value = caller]
22     remove(followers) [value = caller]
23   }
24 }
25
26 Role ADMIN extends REGULARUSER {
27   Post {
28     read(content), read(author)
29     delete
30   }
31   Person {
32     read(verified), read(role), update(verified), update(role)
33   }
34 }

```

Privacy requirements. The personal data in MiniTweet is associated with the Person entity. It includes users' basic information, namely their name and verified status, as well as their follows list. The application uses personal data for three purposes: $\Pi = \{\text{CORE}, \text{GENERATEDADS}, \text{DISPLAYPOSTS}\}$.

The CORE purpose covers essential functionality, such as profile viewing, posting, and following. Data processing for this purpose is treated as necessary. The remaining two purposes cover optional functionality and require explicit user consent.

For GENERATEDADS, the application may read a user's name and verified status to generate personalized advertisements. For DISPLAYPOSTS, the application may read a user's follows list to populate their timeline with posts.

Using our privacy modeling language, the privacy model is expressed as:

```
1  Personal data {
2    Person.name
3    Person.verified
4    Person.follows
5  }
6
7  Purposes {
8    CORE
9    GENERATEDADS
10   DISPLAYPOSTS
11 }
12
13 Actual purposes {
14   view_timeline DISPLAYPOSTS
15   Person.generate_ad GENERATEDADS
16   public_timeline CORE
17   user_profile CORE
18   create_post CORE
19   edit_post CORE
20   delete_post CORE
21   follow_user CORE
22   unfollow_user CORE
23   update_user CORE
24 }
25
26 Declared purposes {
27   Person.name CORE
28   Person.verified CORE
29   Person.follows CORE
30   Person.name GENERATEDADS
31   Person.verified GENERATEDADS
32   Person.follows DISPLAYPOSTS
33 }
```

E.3 Event Management Platform

In this section, we describe in detail the web application used throughout the four editions of the empirical study. The case study was not identical in every edition.

In what follows, we first present the project description from the third edition and subsequently describe the differences between this and the other editions. We begin by introducing the functional requirements of the application (Section E.3.1), followed by its security requirements (Section E.3.2), and finally its privacy requirements (Section E.3.3). The evolution requirements are described separately in Section E.3.4.

E.3.1 Functional Requirements

The Event Management Platform, hereinafter referred to as EventPlatform or simply the event platform, is a simplified version of typical event management websites, such as Meetup or Facebook Events. Anyone can browse the platform anonymously and register to gain access to basic functionality. Registered users may also purchase a premium subscription to unlock additional features, such as the ability to create private events.

The application is used to create and maintain a set of events curated by its users. Users can create, attend, request to attend, and (help) manage events. Events are organized into categories, which are managed by moderators, and users can subscribe to categories of interest. Administrators have additional management capabilities, including managing user profiles and viewing event statistics. The platform also displays advertisements and uses personal information to send marketing messages and recommend relevant events to users.

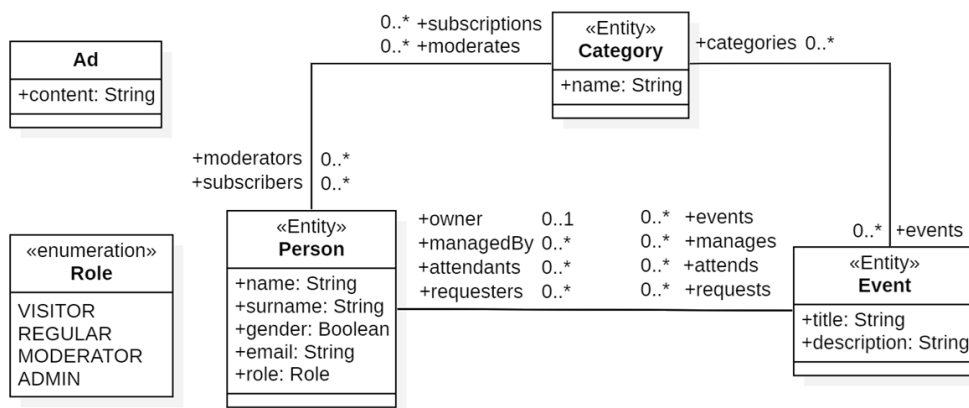


Figure E.3: UML class diagram of EventPlatform in the Development phases.

The complete data model of the EventPlatform for the development phases is shown in Figure E.3. It contains four classes, namely Person, Event, Category, and Ad, and seven binary associations.

The Person class represents the authenticated users of the platform and stores attributes such as name, surname, gender, email, and role. The Event class includes a title,

description, and a Boolean flag private. The Category class contains only a name attribute, while the Ad class contains only the content of the advertisement.

There are four associations between the Person and Event classes. A person can create, manage, attend, or request to attend multiple events; these correspond to the person's events, manages, attends, and requests association ends. Dually, an event is always owned by one person but can be managed or attended by multiple users, or have its attendance requested by them; these correspond to the event's owner, managedBy, attendants, and requesters association ends.

Each event can belong to multiple categories, represented by the event's categories association end. Conversely, each category can contain multiple events, which are referred to as categories' events. A person can moderate or subscribe to multiple categories; hence, they are the category's moderators and subscribers. Correspondingly, the Person class includes the moderates and subscriptions association ends.

E.3.2 Security Requirements

In this section, we define the application's access control permissions. The permissions are organized into two groups, namely, data-based and role-based. In the requirements that follow, whenever we refer to *everyone*, we include both authenticated and non-authenticated users. Otherwise, we make an explicit distinction. Non-authenticated users are assigned the VISITOR role. Authenticated users, those who have previously registered with the application, are assigned one of the following three roles: FREEUSER, MODERATOR, or ADMIN.

Data-Based Permissions. We define access control permissions for all attributes and association ends of each class in the application.

Authenticated users can create public events, and everyone can read the core information of a public event, including its title, description, owner, and associated categories. For private events, only attendants may access this core information. Event managers can edit the core information of a public or private event, with the exception of its owner, which is immutable once assigned. Additionally, only event managers can modify the event's categories, i.e., adding or removing categories from the event.

For a public event, authenticated users can view the list of attendants and managers (i.e., the event's attendants and managedBy association ends). For a private event, this information is restricted to the event's attendants. The event owner has the permission to promote and demote users between being the event's attendant and manager. Managers may remove attendants from events they manage, provided that the removed user is not also a manager of the event. Managers may also accept new attendance requests

to join their managed event. Any authenticated user, except for event managers, can withdraw from attending an event. The event owner cannot remove themselves from managing the event.

Authenticated users can request to join a public event and withdraw their requests. Managers can view and process (i.e., accept or deny) outstanding requests for the events they manage.

Within the Person class, authenticated users can view the core information (name, surname, and role) of any user. They may also update their own core information, with the exception of role. In addition, users can access the list of events they own, manage, attend, or have requested to attend, as well as the categories to which they have subscribed. The list of a category's moderators is visible to everyone.

Everyone can read the name, associated events, and moderators of a category. However, only its moderators can view the list of subscribers. Any user may remove themselves from being a category's moderator.

Role-Based Permissions. The roles are organized in a hierarchy that determines the set of permissions available to each user.

- Non-authenticated users may perform all actions that are allowed to *everyone*.
- `FREEUSER` users may perform all actions available to any authenticated user.
- `MODERATOR` users inherit all `FREEUSER` permissions and may additionally remove events from categories they moderate.
- Finally, `ADMIN` users inherit all `FREEUSER` permissions and may additionally delete users, edit any user's role, create and delete categories, edit category names, and assign or remove users with the `MODERATOR` role as moderators of a category.

E.3.3 Privacy Requirements

To ensure that the platform complies with data protection principles, such as the purpose limitation and data-subject consent requirements, we define (i) a structured hierarchy of data processing purposes, (ii) personal data in terms of properties associated with the user class, and (iii) the application's privacy policy notice.

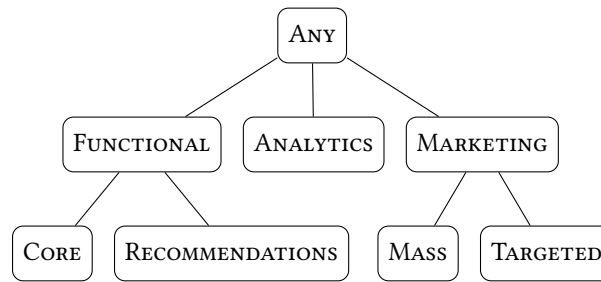


Figure E.4: Purpose hierarchy of EventPlatform in the Development phases.

Purposes. Figure E.4 presents the purpose hierarchy of the EventPlatform in the development phases. At the root of the hierarchy is the *ANY* purpose, which denotes the most general, top-level purpose. We then distinguish three different purpose categories related to *marketing*, *analytics*, and platform *functionality*.

The *MARKETING* purpose governs all marketing-related data processing. This is further subdivided into *TARGETED* (marketing) and *MASS* (marketing) purposes. The former denotes the platform functionality to display personalized advertisements on a user's profile, while the latter allows moderators of a category to send general advertisements to all users subscribed to that category.

The *ANALYTICS* purpose allows the platform to analyze users' data to compute usage statistics and other aggregate insights.

The *FUNCTIONAL* purpose captures the basic functionality of the platform. This is further divided into the *RECOMMENDATIONS* purpose, which denotes recommending relevant events to users based on their subscriptions, and the *CORE* purpose, which includes all remaining functional features that are not covered by the aforementioned purposes, such as profile viewing and management.

Personal Data. We define personal data in terms of attributes and association ends of the *Person* class. Specifically, this includes the user's name, surname, role, gender, and email, as well as the subscriptions association end, which represents the set of categories to which the user has subscribed.

Privacy Policy. The privacy policy presents users with a list of declared purposes and asks for their consent. Each declared purpose specifies a set of personal data, a processing purpose, and an optional condition. Users may grant or revoke consent to each declared purpose independently. Where no condition is specified, the declared purpose applies unconditionally. The EventPlatform's privacy policy for the Development phase is as follows:

Privacy Policy Notice

We process your personal data for the purposes listed below. Please indicate your consent for each purpose:

- ☐ Your name, surname, role, subscriptions, gender, and email for CORE purpose.
- ☐ Your subscriptions for RECOMMENDATIONS purpose, if you are a regular user.
- ☐ Your gender for ANALYTICS purpose.
- ☐ Your name for MARKETING purpose.
- ☐ Your gender for TARGETED purpose.
- ☐ Your email for MASS purpose.

E.3.4 Evolution Requirements

The Evolution phase introduces new functional, security, and privacy requirements.

Evolved Data Model. The data model is extended to support event invitations. An Invite entity relates an event to two users: the one who created the invitation (invitedBy) and the one who is invited (invitee). Figure E.5 shows the extended class diagram, with additions marked in red. Upon creation, the association ends of an Invite object are initialized in the order: event, invitedBy, invitee. Invitations are read-only once created and can only be removed by deletion.

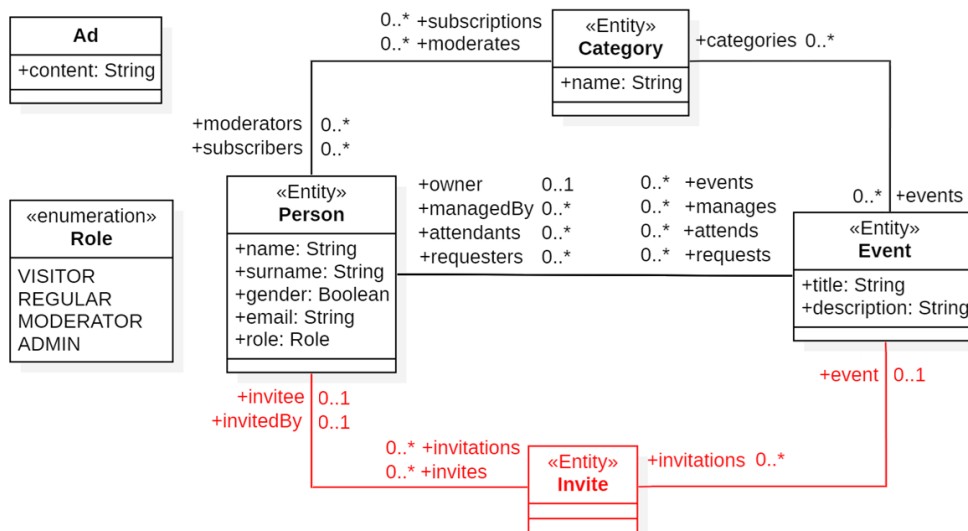


Figure E.5: UML class diagram of EventPlatform (Evolution extensions marked in red).

The application is also extended with a personalized insights page, allowing authenticated users to view statistics about events they attend, manage, or are invited to, as well as categories to which they subscribe.

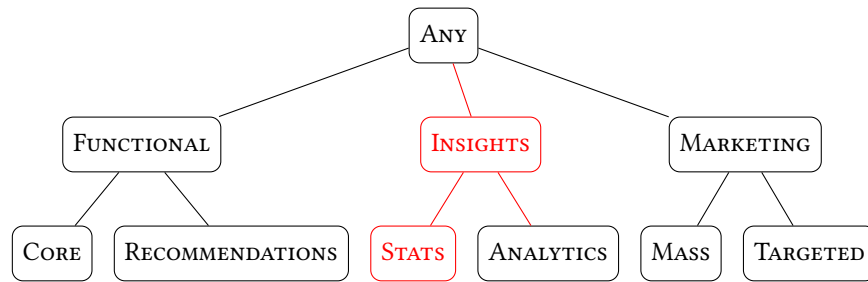


Figure E.6: Purpose hierarchy of EventPlatform (Evolution extensions marked in red).

Evolved Security Requirements. The Evolution phase introduces additional access control permissions related to invitations. An event *manager* can read the invitations association end of the events they manage and read each Invite entity in that list. Similarly, a *user* can read the lists of invitations they received (invitations) or sent (invites), and read each Invite entity therein. Any authenticated user can create an empty Invite object; event managers can then associate it with their event, assign themselves as invitedBy, and designate an invitee. A user can delete invitations they sent, and event managers can delete any invitation for their events. An invited user may accept or decline the invitation, in both cases resulting in the Invite being deleted; upon acceptance, the user is added to the event's attendants before deletion. Finally, a user who has already requested to join or is already attending an event cannot be invited to it, and an invited user cannot request to join the same event.

Evolved Privacy Requirements. The attends association end of the Person entity is added to the set of personal data. Then, the purpose hierarchy is restructured: the ANALYTICS purpose is repositioned to a child of the new INSIGHTS purpose, which also contains the newly introduced STATS purpose. STATS provides personalized insights to users on their profile page. The functions that manage invitations are considered to implement CORE functionality. Figures E.5 and E.6 present the evolved class diagram and the extended purpose hierarchy, with the extensions marked in red. Finally, we add several new policies related to the new purposes.

Evolved Privacy Policy Notice (additions)

The following declared purposes are added to the privacy policy:

- ☐ Your attends for FUNCTIONAL purpose.
- ☐ Your name, subscriptions, and attends for STATS purpose, if you attended more than two events.

Empirical Study Artifacts

This appendix presents material relating to the empirical study (Part III): the variations across the four study editions (Section F.1), a summary of the security and privacy tests used to evaluate the participants' submissions (Section F.2), the survey questionnaire (Section F.3), the codebook for the qualitative analysis of survey responses (Section F.4), and the `vActionGUI` reference solution (Section F.5) used at the end of the course.

F.1 Variations Across Study Editions

All four editions share the same core functional and security requirements (described for the `EventPlatform` in Appendix E) and the same study design for the Development and Evolution phases (Chapter 8). Taking the third edition as the baseline, we describe in the following subsections only the differences between each edition and the third edition.

F.1.1 First Edition

In the first edition of the study, we rely on `ActionGUI` as the model-driven development tool for comparison and focus only on the security requirements.

Study design

Training Phase. Students are trained using the `ActionGUI` training materials.

Repair Phase. This is the only edition that includes a Repair phase. After the two Development stages, students receive a set of regression tests (*TestsMDE* and *TestsFlask*) and are asked to repair both of their implementations to pass as many tests as possible. The inclusion of this phase allows us to assess whether access to regression tests improves the quality of the subsequent evolution of the application (RQ12).

Survey. Additional questions related to the development effort in the Repair phase are included in this edition.

Project description

Functional requirements. Authentication is handled within the `ActionGUI` application. Therefore, `Person` includes additional authentication attributes such as username and password. Events, in this edition, can be configured as private through a Boolean attribute `private`. The `Ad` entity is not introduced in this edition.

Security requirements. In addition to the roles described for the third edition, this edition introduces a `PREMIUMUSER` role for authenticated users of the event platform. Similar to the third edition, a `FREEUSER` has the permissions typically granted to event platform users. However, here, they cannot freely subscribe to categories.

`PREMIUMUSER` users have all the permissions of the `FREEUSER`. In addition, they may subscribe to and unsubscribe from categories, create private events, and request to join private events. The permissions of the `MODERATOR` and `ADMIN` roles are inherited from `PREMIUMUSER`.

To support the authentication process, in this edition, we introduce an additional role `LIB`, which is responsible for authentication-related data actions. Users who are not logged in may register for an account (create new `Person` instances), but only with a `FREEUSER` role. When a new user registers, they may only specify their username and password. Additionally, users who are not logged in yet may log in. To do so, they require permissions to read username, password, and role. These permissions are granted exclusively to the `LIB` role.

Privacy requirements. The privacy requirements (described in Section [E.3.3](#)) were not part of the project specification for this edition.

Conducting the study

Stage I extension. As this was the first edition of the study, during Stage I of the Development phase, students raised many questions due to the ambiguous nature of the informal project description and several minor technical issues (e.g., problems compiling the templates). To address these concerns, we updated the project description to better reflect the intended requirements and extended the submission deadline for this phase by four days. The requirements were frozen once all students submitted their deliverables in Stage I.

Manually written tests. The evaluation infrastructure consisted of manually written security tests covering security requirements in all phases. In total, 455 tests were used for ActionGUI submissions and 548 for Flask submissions, with an additional 108 and 96 tests, respectively, for the evolved requirements. The difference in test counts reflects that some ActionGUI actions correspond to multiple Flask endpoints that must each be tested independently.

F.1.2 Second Edition

In the second edition of the study, we continue to rely on ActionGUI and focus only on the security requirements.

Study design The study setup is similar to that of the third edition, i.e., it does not include a Repair phase. Consequently, the questionnaire does not contain additional questions related to the Repair phase.

Project description The project description follows the same specifications as in the first edition.

Conducting the study

Automatically generated tests. We replaced the manually written tests used in the first edition with tests generated automatically using our test generator. Since this edition did not include a Repair phase, these tests were not released to the students.

Because the tests were released to students in the first edition, withholding them in this edition helps prevent students from overfitting their implementations to the (supposedly) known test cases. This also allows us to better evaluate the effect studied in [RQ12](#).

In this edition, both Flask and ActionGUI tests are generated automatically. The model-based test generation approach is described in [Appendix F.2](#).

F.1.3 Fourth Edition

The fourth edition shares the same scope as the third, covering both security and privacy requirements, and uses *v*ActionGUI as the model-driven tool.

Study design

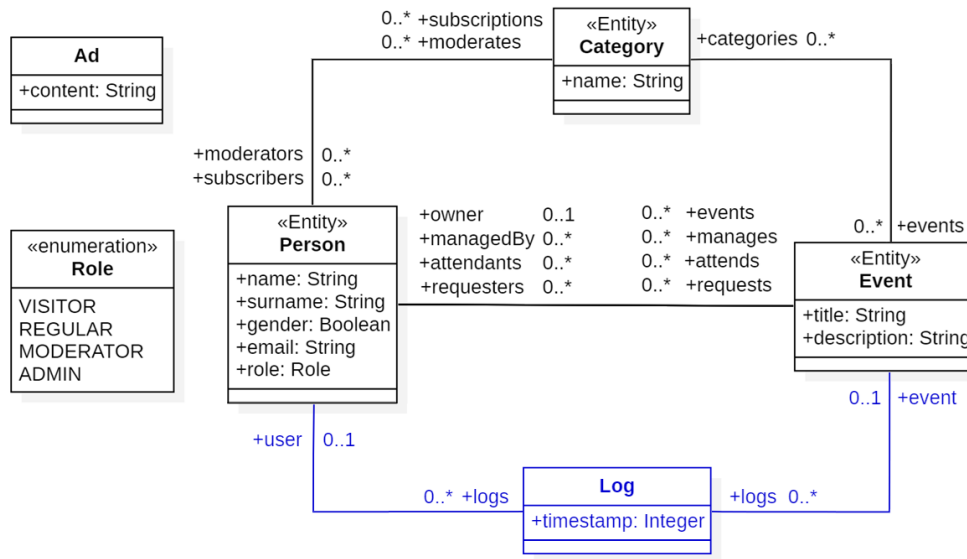


Figure F.1: UML class diagram of EventPlatform in the Development phases of the fourth edition (differences marked in blue).

Using Cedar for access control. Instead of implementing access control checks in an *ad hoc* manner in Flask, students were required to use Cedar [32]. This enables a comparison between model-driven, *ad hoc* code-centric, and the state-of-the-art policy-based code-centric approaches for implementing security requirements (RQ11).

Survey. The questionnaire includes additional questions asking students to reflect on their experience using Cedar.

Project description

Functional requirements. The platform records a viewing history of events. Each log entry contains a timestamp and is linked to at most one user and to exactly one event. Correspondingly, these logs are the person’s and the event’s logs. These logs are used for event recommendations. To support this feature, we introduce a new Log entity (as shown in Figure F.1).

Security requirements. Each user may read only their own logs. Log entries are created whenever an event is viewed. Any user may create a log entry, set its timestamp, and initialize the event association end. The timestamp must be greater than the largest timestamp among all existing log entries.

Only the user who viewed the event may associate the log with themselves. If the event is viewed by an unauthenticated user, the user association remains uninitialized. Log entries cannot be updated or deleted.

Authenticated users may read their own log entries and their associated information (i.e., timestamp, event, and user). Administrators may read all log entries.

Privacy requirements. We consider the logs association end in the Person entity and the event association end in the Log entity to be personal data. In this case, the user who created the Log instance is considered the owner of the log.

Privacy Policy Notice (Fourth edition)

The privacy notice in this edition has two additional policies:

- ☐ Your log's event for FUNCTIONAL purpose.
- ☐ Your logs for RECOMMENDATIONS purpose.

Conducting the study The study was conducted in the same way as it was in the third edition.

F.2 Test Generation

In the first edition of the study, which considered only access control requirements, we evaluated the submissions using a set of more than 1,000 manually written security unit tests. From the second edition onward, we evaluated the submissions using automatically generated tests. In this appendix section, we describe the generation of the security and privacy tests. Figure F.2 gives an overview of our test generator.

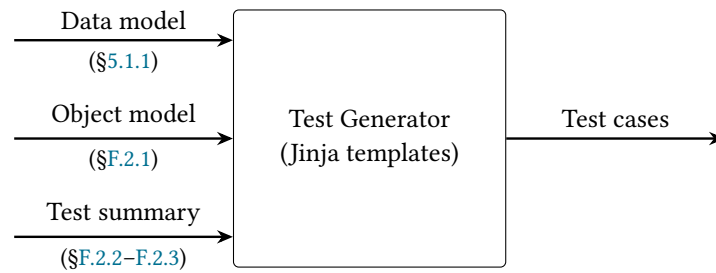


Figure F.2: Overview of the security and privacy test generator

Our generator takes three inputs. First, the data model of the targeted system. Secondly, an object model defines a concrete application state that provides the common scenario in which the generated test cases are executed. Finally, a test summary specifies the test cases to be generated.

Each test case contains a unique identifier, a textual description, the action to be tested (in the case of ActionGUI and vActionGUI) or the endpoint to be invoked (in Flask), the caller role, the values of relevant variables (e.g., self, caller, target, and value), and the expected verdict (i.e., whether the test should pass, fail, or not applicable).

As far as privacy tests are concerned, the test summary additionally specifies the consent given by the owner of the accessed data. We consider three different scenarios, namely (i) the owner has not given consent for the data to be used, (ii) the owner has given consent for the data to be used for a basic purpose, and (iii) the owner has given consent for the data to be used for a complex purpose.

Internally, the generator parses the object model and the test summary and instantiates predefined Jinja templates with the corresponding scenario configuration. It produces a database configuration that instantiates the scenario defined by the object model and a collection of executable test cases corresponding to the test summary. The following sections provide examples of the security and privacy test cases, respectively.

F.2.1 Object modeling language

The object modeling language provides a textual syntax for specifying an *object model*, i.e., an application state over which the generated test cases are executed. As defined in Section 2.2.1, an object model interprets each class of the data model as a finite set of objects and each association as a relation over those objects. Accordingly, our object modeling language declares each object along with its class, the values of its attributes, and its association ends. Its concrete syntax is defined by the grammar shown below.

Object modeling language

```

<InstanceModel> ::= <Statement>+
<Statement> ::= create <ObjectName> as <EntityName> { <Properties>? }
<Properties> ::= <Property> (, <Property>)*
<Property> ::= <PropertyName> : <Value>
<Value> ::= <Boolean> | <String> | <Identifier> | <Number> | <Identifiers>
<Identifiers> ::= [ <Identifier> (, <Identifier>)* ]
<Boolean> ::= true | false
<Identifier> ::= [a-zA-Z_][a-zA-Z_0-9]*
<Number> ::= -?[0-9]+([.][0-9]+)?
<String> ::= " (~["\] | \. ) * "

```

Example F1. The following listing presents the textual representation of the object model O_2 from Example 2.3.

```

1  create Person as p2 {
2      name: "Bob",
3      surname: "Brown"
4  }
5
6  create Person as p3 {
7      name: "Charlie",
8      surname: "Clark"
9  }
10
11 create Event as e2 {
12     title: "Study Group",
13     description: "Exam Preparation",
14     private: false,
15     owner: p2,
16     managedBy: [p2],
17     attendants: [p2, p3]
18 }

```

F.2.2 Security Test Summary

We summarize the security tests used to systematically evaluate the participants' submissions. The tests are organized into groups according to the `vActionGUI/ActionGUI` atomic actions and, for each action, the corresponding collection of Flask endpoints.

Each test includes a textual description and an expected authorization decision. A checkmark (✓) indicates that the scenario is authorized, a cross (✗) indicates that it is unauthorized, and a hyphen (–) indicates that it is not applicable. The six rightmost columns report the expected decision for each caller role. The roles considered are LIB, VISITOR, FREEUSER, PREMIUMUSER, MODERATOR, and ADMIN.

Development Phases

Table F.1: Security tests summary in the Development phases.

L = LIB role; V = VISITOR role; F = FREEUSER role; P = PREMIUMUSER role; M = MODERATOR role; A = ADMIN role.						
vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
<code>create(Person)</code>						
Create a Person instance	✓	✗	✗	✗	✗	✗
<code>delete(Person)</code>						
Delete a Person instance	✗	✗	✗	✗	✗	✓
<code>read(Person, name)</code> , <code>read(Person, surname)</code> , <code>events()</code> , <code>profile()</code> , <code>users()</code> , <code>user()</code>						
Read name and surname of other users	✗	✗	✓	✓	✓	✓
Read one's own name and surname	–	–	✓	✓	✓	✓
<code>read(Person, username)</code> , <code>user()</code>						
Read a Person's username field	✓	✗	✓	✓	✓	✓
Read one's own username field	–	–	✓	✓	✓	✓
<code>read(Person, password)</code>						
Read a Person's password field	✓	✗	✗	✗	✗	✗
Read one's own password field	–	–	✗	✗	✗	✗
<code>read(Person, role)</code> , <code>user()</code>						
Read a Person's role field	✓	✗	✓	✓	✓	✓
Read one's own role field	–	–	✓	✓	✓	✓
<code>update(Person, name)</code> , <code>update(Person, surname)</code> , <code>update_user()</code>						
Update a Person's name and surname from null to something	✗	✗	✗	✗	✗	✗
Update one's own name and surname from null to something	–	–	✓	✓	✓	✓
Update a Person's name and surname from something to something else	✗	✗	✗	✗	✗	✗
Update one's own name and surname from something to something else	–	–	✓	✓	✓	✓
Update a Person's name and surname from something to null	✗	✗	✗	✗	✗	✗
Update one's own name and surname from something to null	–	–	✓	✓	✓	✓
<code>update(Person, username)</code>						
Update a Person's username from null to something	✓	✗	✗	✗	✗	✗
Update a Person's username from something to something else	✗	✗	✗	✗	✗	✗
Update one's own username from something to something else	–	–	✓	✓	✓	✓
Update a Person's username from something to null	✗	✗	✗	✗	✗	✗

Continued on next page

Table F.1 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
update(Person, password)						
Update a Person's password from null to something	✓	×	×	×	×	✓
Update a Person's password from something to something else	×	×	×	×	×	✓
Update one's own password from something to something else	–	–	✓	✓	✓	✓
update(Person, role), update_user()						
Update a Person's role from null to FREEUSER	✓	×	×	×	×	✓
Update a Person's role from null to LIB	×	×	×	×	×	×
Update a Person's role from null to ADMIN	×	×	×	×	×	✓
Update a Person's role from FREEUSER to LIB	×	×	×	×	×	×
Update one's own role (whatever it is) to LIB	–	–	×	×	×	×
Update a Person's role from FREEUSER to PREMIUMUSER	×	×	×	×	×	✓
Update one's own role (whatever it is) to PREMIUMUSER	–	–	×	✓	×	✓
Update a Person's role from FREEUSER to ADMIN	×	×	×	×	×	✓
Update one's own role (whatever it is) to ADMIN	–	–	×	×	×	✓
Modify the role of a user who moderates a Category	×	×	×	×	×	×
Modify one's own role while moderating a Category	–	–	–	–	×	–
Update a Person's role from FREEUSER to VISITOR	×	×	×	×	×	×
Update one's own role (whatever it is) to VISITOR	–	–	×	×	×	×
read(Person, requests)						
View a Person's attendance requests	×	×	×	×	×	×
View one's own attendance requests	–	–	✓	✓	✓	✓
read(Person, events)						
View a Person's owned events	×	×	×	×	×	×
View the events one owns	–	–	✓	✓	✓	✓
read(Person, manages), profile()						
View a Person's managed events	×	×	×	×	×	×
View the events one manages	–	–	✓	✓	✓	✓
read(Person, attends), profile()						
View a Person's attending events	×	×	×	×	×	×
View the events one attends	–	–	✓	✓	✓	✓
read(Person, subscriptions), profile(), user()						
View a Person's category subscriptions	×	×	×	×	×	×
View one's own subscribed categories	–	–	✓	✓	✓	✓
read(Person, moderates)						
Read a Person's moderates field	×	✓	✓	✓	✓	✓
Read one's own moderates field	–	–	✓	✓	✓	✓
create(Event), create_event()						
Create an Event instance	–	×	✓	✓	✓	✓
delete(Event)						
Delete an Event instance	×	×	×	×	×	×
read(Event, title), profile(), events(), view_event(), edit_event()						
Read title of a public Event (not owned, not attending, not requested)	×	✓	✓	✓	✓	✓
Read title of a public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read title of a public Event (not owned, attending)	–	–	✓	✓	✓	✓
Read title of a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read title of a public Event (owned)	–	–	✓	✓	✓	✓
Read title of a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read title of a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Read title of a private Event (not owned, attending)	–	–	✓	✓	✓	✓
Read title of a private Event (not owned, managing)	–	–	✓	✓	✓	✓

Continued on next page

Table F.1 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
Read title of a private Event (owned)	–	–	✓	✓	✓	✓
<code>read(Event, description)</code> , <code>view_event()</code> , <code>edit_event()</code>						
Read description of a public Event (not owned, not attending, not requested)	×	✓	✓	✓	✓	✓
Read description of a public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read description of a public Event (not owned, attending)	–	–	✓	✓	✓	✓
Read description of a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read description of a public Event (owned)	–	–	✓	✓	✓	✓
Read description of a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read description of a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Read description of a private Event (not owned, attending)	–	–	✓	✓	✓	✓
Read description of a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Read description of a private Event (owned)	–	–	✓	✓	✓	✓
<code>read(Event, private)</code> , <code>view_event()</code> , <code>edit_event()</code>						
Read private field on a public Event (not owned, not attending, not requested)	×	✓	✓	✓	✓	✓
Read private field on a public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read private field on a public Event (not owned, attending)	–	–	✓	✓	✓	✓
Read private field on a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read private field on a public Event (owned)	–	–	✓	✓	✓	✓
Read private field on a private Event (not owned, not attending, not requested)	×	✓	✓	✓	✓	✓
Read private field on a private Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read private field on a private Event (not owned, attending)	–	–	✓	✓	✓	✓
Read private field on a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Read private field on a private Event (owned)	–	–	✓	✓	✓	✓
<code>update(Event, title)</code> , <code>update_event()</code>						
Update title field on a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update title field on a public Event (not owned, not attending, requested)	–	–	×	×	×	×
Update title field on a public Event (not owned, attending)	–	–	×	×	×	×
Update title field on a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Update title field on a public Event (owned)	–	–	✓	✓	✓	✓
Update title field on a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update title field on a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Update title field on a private Event (not owned, attending)	–	–	×	×	×	×
Update title field on a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Update title field on a private Event (owned)	–	–	✓	✓	✓	✓
<code>update(Event, description)</code> , <code>update_event()</code>						
Update description of public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update description of public Event (not owned, not attending, requested)	–	–	×	×	×	×
Update description of public Event (not owned, attending)	–	–	×	×	×	×
Update description of public Event (not owned, managing)	–	–	✓	✓	✓	✓
Update description of public Event (owned)	–	–	✓	✓	✓	✓
Update description of private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update description of private Event (not owned, not attending, requested)	–	–	×	×	×	×
Update description of private Event (not owned, attending)	–	–	×	×	×	×
Update description of private Event (not owned, managing)	–	–	✓	✓	✓	✓
Update description of private Event (owned)	–	–	✓	✓	✓	✓
<code>read(Event, owner)</code> , <code>profile()</code> , <code>events()</code>						
Read owner of public Event (not owned, not attending, not requested)	×	✓	✓	✓	✓	✓
Read owner of public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read owner of public Event (not owned, attending)	–	–	✓	✓	✓	✓
Read owner of public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read owner of public Event (owned)	–	–	✓	✓	✓	✓
Read owner of private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read owner of private Event (not owned, not attending, requested)	–	–	×	×	×	×

Continued on next page

Table F.1 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
Read owner of private Event (not owned, attending)	–	–	✓	✓	✓	✓
Read owner of private Event (not owned, managing)	–	–	✓	✓	✓	✓
Read owner of private Event (owned)	–	–	✓	✓	✓	✓
<code>read(Event, attendants)</code> , <code>view_event()</code> , <code>manage_event()</code>						
Read attendants of public Event (not owned, not attending, not requested)	×	×	✓	✓	✓	✓
Read attendants of public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read attendants of public Event (not owned, attending)	–	–	✓	✓	✓	✓
Read attendants of public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read attendants of public Event (owned)	–	–	✓	✓	✓	✓
Read attendants of private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read attendants of private Event (not owned, not attending, requested)	–	–	×	×	×	×
Read attendants of private Event (not owned, attending)	–	–	✓	✓	✓	✓
Read attendants of private Event (not owned, managing)	–	–	✓	✓	✓	✓
Read attendants of private Event (owned)	–	–	✓	✓	✓	✓
<code>read(Event, managedBy)</code> , <code>manage_event()</code>						
Read managedBy field on a public Event (not owned, not attending, not requested)	×	×	✓	✓	✓	✓
Read managedBy field on a public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Read managedBy field on a public Event (not owned, attending)	–	–	✓	✓	✓	✓
Read managedBy field on a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read managedBy field on a public Event (owned)	–	–	✓	✓	✓	✓
Read managedBy field on a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read managedBy field on a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Read managedBy field on a private Event (not owned, attending)	–	–	✓	✓	✓	✓
Read managedBy field on a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Read managedBy field on a private Event (owned)	–	–	✓	✓	✓	✓
<code>read(Event, requesters)</code> , <code>manage_event()</code>						
Read requesters field on a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read requesters field on a public Event (not owned, not attending, requested)	–	–	×	×	×	×
Read requesters field on a public Event (not owned, attending)	–	–	×	×	×	×
Read requesters field on a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Read requesters field on a public Event (owned)	–	–	✓	✓	✓	✓
Read requesters field on a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Read requesters field on a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Read requesters field on a private Event (not owned, attending)	–	–	×	×	×	×
Read requesters field on a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Read requesters field on a private Event (owned)	–	–	✓	✓	✓	✓
<code>update(Event, private)</code> , <code>update_event()</code>						
Initialize an Event's private flag from null to FALSE (not owned, not attending, not requested)	×	×	×	×	×	×
Initialize an Event's private flag from null to FALSE (not owned, attending)	–	–	×	×	×	×
Initialize an Event's private flag from null to FALSE (not owned, managing)	–	–	×	×	×	×
Initialize an Event's private flag from null to FALSE (owned)	–	–	✓	✓	✓	✓
Initialize an Event's private flag from null to TRUE (not owned, not attending, not requested)	×	×	×	×	×	×
Initialize an Event's private flag from null to TRUE (not owned, attending)	–	–	×	×	×	×
Initialize an Event's private flag from null to TRUE (not owned, managing)	–	–	×	×	×	×
Initialize an Event's private flag from null to TRUE (owned)	–	–	×	✓	✓	✓
Update a private flag from TRUE to FALSE (not owned, not attending, not requested)	×	×	×	×	×	×
Update a private flag from TRUE to FALSE (not owned, attending, requested)	×	×	×	×	×	×
Update a private flag from TRUE to FALSE (not owned, attending)	–	–	×	×	×	×
Update a private flag from TRUE to FALSE (not owned, managing)	–	–	×	×	×	×
Update a private flag from TRUE to FALSE (owned)	–	–	×	×	×	×
<code>update(Event, owner)</code> , <code>remove(Person, events)</code>						
Update the owner (to null) of a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update the owner (to null) of a public Event (not owned, not attending, requested)	–	–	×	×	×	×

Continued on next page

Appendix F Empirical Study Artifacts

Table F.1 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
Update the owner (to null) of a public Event (not owned, attending)	–	–	×	×	×	×
Update the owner (to null) of a public Event (not owned, managing)	–	–	×	×	×	×
Update the owner (to null) of a public Event (owned)	–	–	×	×	×	×
Update the owner (to null) of a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update the owner (to null) of a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Update the owner (to null) of a private Event (not owned, attending)	–	–	×	×	×	×
Update the owner (to null) of a private Event (not owned, managing)	–	–	×	×	×	×
Update the owner (to null) of a private Event (owned)	–	–	×	×	×	×
add(Event, requesters), add(Person, requests), join()						
Request to join a public Event for oneself (not owned, not attending, not requested)	–	–	✓	✓	✓	✓
Request to join a public Event for someone else (not owned, not attending, not requested)	×	×	×	×	×	×
Request to join a public Event for someone else (not owned, not attending, requested)	–	–	×	×	×	×
Request to join a public Event for someone else (not owned, attending)	–	–	×	×	×	×
Request to join a public Event for someone else (not owned, managing)	–	–	×	×	×	×
Request to join a public Event for someone else (owned)	–	–	×	×	×	×
Request to join a private Event for oneself (not owned, not attending, not requested)	–	–	×	✓	✓	✓
Request to join a private Event for someone else (not owned, not attending, not requested)	×	×	×	×	×	×
Request to join a private Event for someone else (not owned, not attending, requested)	–	–	×	×	×	×
Request to join a private Event for someone else (not owned, attending)	–	–	×	×	×	×
Request to join a private Event for someone else (not owned, managing)	–	–	×	×	×	×
Request to join a private Event for someone else (owned)	–	–	×	×	×	×
remove(Event, requesters), remove(Person, requests), accept_request(), reject_request()						
Remove one's own request from a public Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Remove one's own request from a private Event (not owned, not attending, requested)	–	–	✓	✓	✓	✓
Remove someone else's request from a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Remove someone else's request from a public Event (not owned, not attending, requested)	–	–	×	×	×	×
Remove someone else's request from a public Event (not owned, attending)	–	–	×	×	×	×
Remove someone else's request from a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Remove someone else's request from a public Event (owned)	–	–	✓	✓	✓	✓
Remove someone else's request from a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Remove someone else's request from a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Remove someone else's request from a private Event (not owned, attending)	–	–	×	×	×	×
Remove someone else's request from a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Remove someone else's request from a private Event (owned)	–	–	✓	✓	✓	✓
add(Event, managedBy), add(Person, manages), promote_manager()						
Promote a stranger to a manager in a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Promote a stranger to a manager in a public Event (not owned, attending)	–	–	×	×	×	×
Promote a stranger to a manager in a public Event (not owned, managing)	–	–	×	×	×	×
Promote a stranger to a manager in a public Event (owned)	–	–	×	×	×	×
Promote a stranger to a manager in a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Promote a stranger to a manager in a private Event (not owned, attending)	–	–	×	×	×	×
Promote a stranger to a manager in a private Event (not owned, managing)	–	–	×	×	×	×
Promote a stranger to a manager in a private Event (owned)	–	–	×	×	×	×
Promote an attendant to a manager in a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Promote an attendant to a manager in a public Event (not owned, attending)	–	–	×	×	×	×
Promote an attendant to a manager in a public Event (not owned, managing)	–	–	×	×	×	×
Promote an attendant to a manager in a public Event (owned)	–	–	✓	✓	✓	✓
Promote an attendant to a manager in a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Promote an attendant to a manager in a private Event (not owned, attending)	–	–	×	×	×	×
Promote oneself to a manager in a private Event (not owned, attending)	–	–	×	×	×	×
Promote an attendant to a manager in a private Event (not owned, managing)	–	–	×	×	×	×
Promote an attendant to a manager in a private Event (owned)	–	–	✓	✓	✓	✓

Continued on next page

Table F.1 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
<code>remove(Event, managedBy)</code> , <code>remove(Person, manages)</code> , <code>demote_manager()</code>						
Demote a manager in a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Demote a manager in a public Event (not owned, attending)	–	–	×	×	×	×
Demote a manager in a public Event (not owned, managing)	–	–	×	×	×	×
Demote oneself in a public Event (not owned, managing)	–	–	×	×	×	×
Demote a manager in a public Event (owned)	–	–	✓	✓	✓	✓
Demote an owner in a public Event (owned)	–	–	×	×	×	×
Demote a manager in a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Demote a manager in a private Event (not owned, attending)	–	–	×	×	×	×
Demote oneself to a manager in a private Event (not owned, attending)	–	–	×	×	×	×
Demote a manager in a private Event (not owned, managing)	–	–	×	×	×	×
Demote oneself in a private Event (not owned, managing)	–	–	×	×	×	×
Demote a manager in a private Event (owned)	–	–	✓	✓	✓	✓
Demote an owner in a private Event (owned)	–	–	×	×	×	×
<code>add(Event, attendants)</code> , <code>add(Person, attends)</code> , <code>accept_request()</code>						
Add attendant (not requested) to a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Add attendant (not requested) to a public Event (not owned, attending)	–	–	×	×	×	×
Add attendant (not requested) to a public Event (not owned, managing)	–	–	×	×	×	×
Add attendant (not requested) to a public Event (owned)	–	–	×	×	×	×
Add attendant (not requested) to a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Add attendant (not requested) to a private Event (not owned, attending)	–	–	×	×	×	×
Add attendant (not requested) to a private Event (not owned, managing)	–	–	×	×	×	×
Add attendant (not requested) to a private Event (owned)	–	–	×	×	×	×
Add attendant (requested) to a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Add oneself as an attendant to a public Event (not owned, not attending, requested)	–	–	×	×	×	×
Add attendant (requested) to a public Event (not owned, attending)	–	–	×	×	×	×
Add attendant (requested) to a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Add attendant (requested) to a public Event (owned)	–	–	✓	✓	✓	✓
Add attendant (requested) to a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Add oneself as an attendant to a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Add attendant (requested) to a private Event (not owned, attending)	–	–	×	×	×	×
Add attendant (requested) to a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Add attendant (requested) to a private Event (owned)	–	–	✓	✓	✓	✓
<code>update(Event, owner)</code> , <code>add(Person, events)</code>						
Initialize the owner (from null) of the Event with oneself	–	–	✓	✓	✓	✓
Initialize the owner (from null) of the Event with someone else	×	×	×	×	×	×
Update the owner (to someone else) of a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update the owner (to someone else) of a public Event (not owned, not attending, requested)	–	–	×	×	×	×
Update the owner (to someone else) of a public Event (not owned, attending)	–	–	×	×	×	×
Update the owner (to someone else) of a public Event (not owned, managing)	–	–	×	×	×	×
Update the owner (to someone else) of a public Event (owned)	–	–	×	×	×	×
Update the owner (to someone else) of a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Update the owner (to someone else) of a private Event (not owned, not attending, requested)	–	–	×	×	×	×
Update the owner (to someone else) of a private Event (not owned, attending)	–	–	×	×	×	×
Update the owner (to someone else) of a private Event (not owned, managing)	–	–	×	×	×	×
Update the owner (to someone else) of a private Event (owned)	–	–	×	×	×	×
<code>remove(Event, categories)</code> , <code>remove(Category, events)</code> , <code>update_event()</code> , <code>remove_category()</code>						
Remove a Category from an Event (not owned, not attending, not requested, not moderated)	×	×	×	×	×	×
Remove a Category from an Event (not owned, not attending, requested, not moderated)	–	–	×	×	×	×
Remove a Category from an Event (not owned, attending, not moderated)	–	–	×	×	×	×
Remove a Category from an Event (not owned, managing, not moderated)	–	–	✓	✓	✓	✓
Remove a Category from an Event (owned, not moderated)	–	–	✓	✓	✓	✓
Remove a Category from an Event (moderated)	–	–	–	–	✓	–

Continued on next page

Table F.1 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
Remove a Category from an Event (subscribed)	–	–	×	×	×	×
<code>create(Category)</code> , <code>create_category()</code>						
Create a Category instance	×	×	×	×	×	✓
<code>delete(Category)</code>						
Delete a Category instance	×	×	×	×	×	✓
<code>read(Category, name)</code> , <code>edit_category()</code> , <code>events()</code> , <code>categories()</code> , <code>profile()</code>						
Read a Category's name field (moderated)	–	–	–	–	✓	–
Read a Category's name field (not moderated nor subscribed)	×	✓	✓	✓	✓	✓
Read a Category's name field (subscribed)	–	–	✓	✓	✓	✓
<code>remove(Category, subscribers)</code> , <code>remove(Person, subscriptions)</code> , <code>unsubscribe()</code>						
Unsubscribe someone else from a Category	×	×	×	×	×	×
Unsubscribe oneself from a Category	–	–	×	✓	✓	✓
<code>read(Category, subscribers)</code> , <code>profile()</code> , <code>view_category()</code>						
Read a Category's subscribers (moderated)	–	–	–	–	✓	–
Read a Category's subscribers (not moderated nor subscribed)	×	×	×	×	×	×
Read a Category's subscribers (subscribed)	–	–	×	×	×	×
<code>remove(Event, attendants)</code> , <code>remove(Person, attends)</code> , <code>remove_attendee()</code> , <code>leave()</code>						
Remove oneself from attending a public Event (not owned, attending)	–	–	✓	✓	✓	✓
Remove oneself from attending a public Event (not owned, managing)	–	–	×	×	×	×
Remove oneself from attending a public Event (owned)	–	–	×	×	×	×
Remove oneself from attending a private Event (not owned, attending)	–	–	✓	✓	✓	✓
Remove oneself from attending a private Event (not owned, managing)	–	–	×	×	×	×
Remove oneself from attending a private Event (owned)	–	–	×	×	×	×
Remove others from attending a public Event (not owned, not attending, not requested)	×	×	×	×	×	×
Remove others from attending a public Event (not owned, attending)	–	–	×	×	×	×
Remove others (attending) from attending a public Event (not owned, managing)	–	–	✓	✓	✓	✓
Remove others (attending) from attending a public Event (owned)	–	–	✓	✓	✓	✓
Remove others (managing) from attending a public Event (not owned, managing)	–	–	×	×	×	×
Remove others (managing) from attending a public Event (owned)	–	–	×	×	×	×
Remove others from attending a private Event (not owned, not attending, not requested)	×	×	×	×	×	×
Remove others from attending a private Event (not owned, attending)	–	–	×	×	×	×
Remove others (attending) from attending a private Event (not owned, managing)	–	–	✓	✓	✓	✓
Remove others (attending) from attending a private Event (owned)	–	–	✓	✓	✓	✓
Remove others (managing) from attending a private Event (not owned, managing)	–	–	×	×	×	×
Remove others (managing) from attending a private Event (owned)	–	–	×	×	×	×
<code>add(Event, categories)</code> , <code>add(Category, events)</code> , <code>update_event()</code>						
Add a Category to an Event (not owned, not attending, not requested, not moderated)	×	×	×	×	×	×
Add a Category to an Event (not owned, not attending, requested, not moderated)	–	–	×	×	×	×
Add a Category to an Event (not owned, attending, not moderated)	–	–	×	×	×	×
Add a Category to an Event (not owned, managing, not moderated)	–	–	✓	✓	✓	✓
Add a Category to an Event (owned, not moderated)	–	–	✓	✓	✓	✓
Add a Category to an Event (moderated)	–	–	–	–	×	–
Add a Category to an Event (subscribed)	–	–	×	×	×	×
<code>read(Category, moderators)</code> , <code>edit_category()</code>						
Read a Category's moderators (moderated)	–	–	–	–	✓	–
Read a Category's moderators (not moderated nor subscribed)	×	✓	✓	✓	✓	✓
Read a Category's moderators (subscribed)	–	–	✓	✓	✓	✓
<code>read(Category, events)</code> , <code>profile()</code> , <code>categories()</code> , <code>view_category()</code>						
Read a Category's events (moderated)	–	–	–	–	✓	–
Read a Category's events (not moderated nor subscribed)	×	✓	✓	✓	✓	✓

Continued on next page

Table F.1 – continued from previous page

<i>vAG</i> atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
Read a Category's events (subscribed)	–	–	✓	✓	✓	✓
<i>add(Category, moderators)</i> , <i>add(Person, moderates)</i> , <i>add_moderator()</i>						
Add a FREEUSER user as a moderator of a Category	×	×	×	×	×	×
Add a PREMIUMUSER user as a moderator of a Category	×	×	×	×	×	×
Add a MODERATOR user as a moderator of a Category	×	×	×	×	×	✓
Add oneself (with MODERATOR role) as a moderator of a Category	–	–	–	–	×	–
Add an ADMIN user as a moderator of a Category	×	×	×	×	×	×
Add oneself (with ADMIN role) as a moderator of a Category	–	–	–	–	–	×
<i>remove(Category, moderators)</i> , <i>remove(Person, moderates)</i> , <i>remove_moderator()</i>						
Remove someone else from being a moderator of a Category	×	×	×	×	×	✓
Remove oneself as a moderator of a Category	–	–	–	–	✓	–
<i>add(Category, subscribers)</i> , <i>add(Person, subscriptions)</i> , <i>subscribe()</i>						
Subscribe someone else to a Category	×	×	×	×	×	×
Subscribe oneself to a Category	–	–	×	✓	✓	✓

Evolution Phase

Table F.2: Security tests summary in the Evolution phase.

L = LIB role; V = VISITOR role; F = FREEUSER role; P = PREMIUMUSER role; M = MODERATOR role; A = ADMIN role.

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
<code>create(Invite), send_invite()</code>						
Create an invitation	×	×	✓	✓	✓	✓
<code>delete(Invite), decline_invitation(), accept_invitation()</code>						
Delete someone else's invitation to an Event (not owned, not attending, not requested)	×	×	×	×	×	×
Delete someone else's invitation to an Event (not owned, not attending, requested)	–	–	×	×	×	×
Delete someone else's invitation to an Event (not owned, attending)	–	–	×	×	×	×
Delete someone else's invitation to an Event (not owned, managing)	–	–	✓	✓	✓	✓
Delete someone else's invitation to an Event (owned)	–	–	✓	✓	✓	✓
Delete one's own sent invitation to an Event	–	–	✓	✓	✓	✓
Delete one's own received invitation to an Event	–	–	✓	✓	✓	✓
<code>read(Invite, event), profile()</code>						
View someone else's invitation event (not owned, not attending, not requested)	×	×	×	×	×	×
View someone else's invitation event (not owned, not attending, requested)	–	–	×	×	×	×
View someone else's invitation event (not owned, attending)	–	–	×	×	×	×
View someone else's invitation event (not owned, managing)	–	–	✓	✓	✓	✓
View someone else's invitation event (owned)	–	–	✓	✓	✓	✓
View an invitation event (sent)	–	–	✓	✓	✓	✓
View an invitation event (received)	–	–	✓	✓	✓	✓
<code>read(Invite, invitee)</code>						
View someone else's invitation invitee (not owned, not attending, not requested)	×	×	×	×	×	×
View someone else's invitation invitee (not owned, not attending, requested)	–	–	×	×	×	×
View someone else's invitation invitee (not owned, attending)	–	–	×	×	×	×
View someone else's invitation invitee (not owned, managing)	–	–	✓	✓	✓	✓
View someone else's invitation invitee (owned)	–	–	✓	✓	✓	✓
View an invitation invitee (sent)	–	–	✓	✓	✓	✓
View an invitation invitee (received)	–	–	✓	✓	✓	✓
<code>update(Invite, event), update(Invite, invitedBy), update(Invite, invitee), add(Person, invitations), add(Person, invites), add(Event, invitations), send_invite()</code>						
Invite a user to join an Event (not owned, not attending, not requested)	–	–	×	×	×	×
Invite a user to join an Event (not owned, not attending, requested)	–	–	×	×	×	×
Invite a user to join an Event (not owned, attending)	–	–	×	×	×	×
Invite a user to join an Event (not owned, managing)	–	–	✓	✓	✓	✓
Invite a user to join an Event (owned)	–	–	✓	✓	✓	✓
Invite a user who already requested to join, while being the manager	–	–	×	×	×	×
Invite a user who is already in the attendant list, while being the owner	–	–	×	×	×	×
Invite a user who already requested to join, while being the owner	–	–	×	×	×	×
<code>update(Invite, event), update(Invite, invitedBy), update(Invite, invitee), remove(Person, invitations), remove(Person, invites), remove(Event, invitations)</code>						
Change an Invite's invitee field (to null)	×	×	×	×	×	×
Change an Invite's invitee field (from someone to someone else)	×	×	×	×	×	×
Change an Invite's invitedBy field (to null)	×	×	×	×	×	×
Change an Invite's invitedBy field (from someone to someone else)	×	×	×	×	×	×
Change an Invite's event field (to null)	×	×	×	×	×	×
Change an Invite's event field (from one event to another)	×	×	×	×	×	×
<code>read(Person, invitations), profile()</code>						
View other's received invitations	×	×	×	×	×	×

Continued on next page

Table F.2 – continued from previous page

vAG atomic actions, Flask endpoints, and Textual description	L	V	F	P	M	A
View one's own received invitations	–	–	✓	✓	✓	✓
<code>read(Person, invites)</code>						
View other's sent invitations	×	×	×	×	×	×
View one's own sent invitations	–	–	✓	✓	✓	✓
Auxiliaries						
Read the title of a public Event (invited)	–	–	✓	✓	✓	✓
Read the title of a private Event (invited)	–	–	✓	✓	✓	✓
Read the description of a public Event (invited)	–	–	✓	✓	✓	✓
Read the description of a private Event (invited)	–	–	✓	✓	✓	✓
Read the attendants of a public Event (invited)	–	–	✓	✓	✓	✓
Read the attendants of a private Event (invited)	–	–	✓	✓	✓	✓
Read the managers of a public Event (invited)	–	–	✓	✓	✓	✓
Read the managers of a private Event (invited)	–	–	✓	✓	✓	✓
Request to join an Event to which one is already invited	–	–	×	×	×	×

For each cell in Tables F.1 and F.2, we generate a corresponding test that checks whether the given action, when executed by a user with the specified role, is correctly authorized or denied.

Listings F.1 and F.2 show a simplified event handler in ActionGUI's GUI model, generated for a user with the FREEUSER role. Each handler contains the test code directly, with authorization assertions embedded in try/catch blocks. The test case 33 in Listing F.1 checks that reading the title field of a public event is permitted: a successful read increments the passed counter (line 5), while a `SecurityException` marks the test as failed (line 7).

Listing F.1: Generated ActionGUI test for authorized scenario

```

1 // Set up: FREEUSER, public and private event (not owned, attending nor requested)
2 // Test Case 33: read(Event, title), Role: FREEUSER, Expected outcome: ALLOW
3 try {
4     var33 := public_event.title
5     passed := passed + 1
6 } catch (SecurityException) {
7     failedTests := failedTests + " 33"
8 }
```

The test case 34 in Listing F.2 checks the logic for a private event: a successful read indicates an unauthorized disclosure and is counted as a failure (line 4), while a caught `SecurityException` confirms correct enforcement and is counted as passed (line 6).

Listing F.2: Generated ActionGUI test for unauthorized scenario

```

1 // Test Case 34: read(Event, title), Role: FREEUSER, Expected outcome: DENY
2 try {
3     var34 := private_event.title
4     failedTests := failedTests + " 34"
5 } catch (SecurityException) {
6     passed := passed + 1
7 }

```

vActionGUI generates Flask web applications. We produce Python tests that target Flask endpoints directly. These tests can also be used for code-centric approaches. The Listings F.3 and F.4 show the generated tests for the authorized and unauthorized scenarios that correspond to the ActionGUI tests above. They evaluate the returned response. For a public event (Listing F.3), the title must be accessible and equal to the stored value (line 8).

Listing F.3: Generated security test for authorized scenario

```

1 def test_case_33_FREEUSER_view_event(client, app_with_data):
2     _self = Event.query.filter(Event.testid == 'e33').one()
3     caller = Person.query.filter(Person.testid == 'FREEUSER0').one()
4     with app_with_data.test_request_context():
5         login_user(caller)
6         try:
7             result = view_event(_self.id)
8             assert result.get("event").title == _self.title
9         except SecurityException:
10             db.session.rollback()
11             assert False

```

For a private event (Listing F.4), access must return a restricted result (line 8), or raise a SecurityException (lines 9–10).

Listing F.4: Generated security test for unauthorized scenario

```

1 def test_case_34_FREEUSER_view_event(client, app_with_data):
2     _self = Event.query.filter(Event.testid == 'e34').one()
3     caller = Person.query.filter(Person.testid == 'FREEUSER0').one()
4     with app_with_data.test_request_context():
5         login_user(caller)
6         try:
7             result = view_event(_self.id)
8             assert is_response(result) or result.get("event").title is RESTRICTED
9         except SecurityException:
10             assert True
11         finally:
12             db.session.rollback()

```

F.2.3 Privacy Test Summary

Table F.3 presents the set of privacy tests, organized into groups according to the application endpoints. Each row specifies the action on personal data, a textual description, and the basic and complex purposes to which the declared purpose refers. For example, the first two rows refer to reading the Person's name and surname when the `main()` endpoint is invoked. Since `main()` is associated with the CORE purpose, this purpose and the higher-level complex purposes that include it are considered during testing.

Table F.3: Privacy tests summary in all phases.

Action, Textual description	Purposes
<code>main()</code>	
<code>read(Person, name)</code> , Read the caller's name	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read the caller's surname	{CORE, FUNCTIONAL, ANY}
<code>users()</code>	
<code>read(Person, name)</code> , Read the caller's name	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read the caller's surname	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read other user's name	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read other user's surname	{CORE, FUNCTIONAL, ANY}
<code>user()</code>	
<code>read(Person, name)</code> , Read the caller's name	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read the caller's surname	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read other user's name	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read other user's surname	{CORE, FUNCTIONAL, ANY}
<code>read(Person, gender)</code> , Read the caller's gender	{CORE, FUNCTIONAL, ANY}
<code>read(Person, gender)</code> , Read other user's gender	{CORE, FUNCTIONAL, ANY}
<code>read(Person, email)</code> , Read the caller's email	{CORE, FUNCTIONAL, ANY}
<code>read(Person, email)</code> , Read other user's email	{CORE, FUNCTIONAL, ANY}
<code>read(Person, subscriptions)</code> , Read the caller's subscriptions	{CORE, FUNCTIONAL, ANY}
<code>read(Person, subscriptions)</code> , Read other user's subscriptions	{CORE, FUNCTIONAL, ANY}
<code>profile()</code>	
<code>read(Person, name)</code> , Read the owner's name of events the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, attends)</code> , Read events the caller attends	{FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read the owner's name of events the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, subscriptions)</code> , Read the caller's subscriptions	{CORE, FUNCTIONAL, ANY}
<code>leave()</code>	
<code>remove(Person, attends)</code> , Removes the caller from an attended event	{FUNCTIONAL, ANY}
<code>events()</code>	
<code>read(Person, name)</code> , Read the owner's name of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read the owner's name of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read the owner's name of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read the owner's name of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read the owner's name of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>remove_attendee()</code>	
<code>remove(Person, attends)</code> , Removes the caller from an attended event	{FUNCTIONAL, ANY}
<code>remove(Person, attends)</code> , Removes other user from an event the caller owns	{FUNCTIONAL, ANY}
<code>remove(Person, attends)</code> , Removes other user from an event the caller manages	{FUNCTIONAL, ANY}
<code>send_mass_advertisement()</code>	

Continued on next page

Table F.3 – continued from previous page

Action, Textual description	Purposes
<code>read(Person, email)</code> , Read attendants' email addresses of an event the caller owns	{MASS, MARKETING, ANY}
<code>update_user()</code>	
<code>update(Person, name)</code> , Initializes the caller's name to a value	{CORE, FUNCTIONAL, ANY}
<code>update(Person, name)</code> , Edits the caller's name from one value to another	{CORE, FUNCTIONAL, ANY}
<code>update(Person, surname)</code> , Initializes the caller's surname to a value	{CORE, FUNCTIONAL, ANY}
<code>update(Person, surname)</code> , Edits the caller's surname from one value to another	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits the caller's role from ADMIN to FREEUSER	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits the caller's role from ADMIN to MODERATOR	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits other user's role from FREEUSER to MODERATOR	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits other user's role from FREEUSER to ADMIN	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits other user's role from MODERATOR to FREEUSER	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits other user's role from MODERATOR to ADMIN	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits other user's role from ADMIN to FREEUSER	{CORE, FUNCTIONAL, ANY}
<code>update(Person, role)</code> , Edits other user's role from ADMIN to MODERATOR	{CORE, FUNCTIONAL, ANY}
<code>update(Person, gender)</code> , Initializes the caller's gender to a value	{CORE, FUNCTIONAL, ANY}
<code>update(Person, gender)</code> , Edits the caller's gender from one value to another	{CORE, FUNCTIONAL, ANY}
<code>update(Person, email)</code> , Initializes the caller's email to a value	{CORE, FUNCTIONAL, ANY}
<code>update(Person, email)</code> , Edits the caller's email from one value to another	{CORE, FUNCTIONAL, ANY}
<code>view_event()</code>	
<code>read(Person, name)</code> , Read attendants' names of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>manage_event()</code>	
<code>read(Person, name)</code> , Read attendants' names of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read attendants' names of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read attendants' surnames of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read managers' names of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read managers' names of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read managers' names of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read managers' names of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read managers' names of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read managers' surnames of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read managers' surnames of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read managers' surnames of an event the caller attends	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read managers' surnames of an event the caller requests	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read managers' surnames of an unrelated event	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read requesters' names of an event the caller owns	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read requesters' names of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read requesters' surnames of an event the caller owns	{CORE, FUNCTIONAL, ANY}

Continued on next page

Table F.3 – continued from previous page

Action, Textual description	Purposes
<code>read(Person, surname)</code> , Read requesters' surnames of an event the caller manages	{CORE, FUNCTIONAL, ANY}
<code>personalized_stats()</code>	
<code>read(Person, attends)</code> , Read events the caller attends for statistics	{STATS, INSIGHTS, ANY}
<code>read(Person, subscriptions)</code> , Read the caller's subscriptions for statistics	{STATS, INSIGHTS, ANY}
<code>accept_invitation()</code>	
<code>add(Person, attends)</code> , Accepts an invitation received by the caller	{FUNCTIONAL, ANY}
<code>edit_category()</code>	
<code>read(Person, name)</code> , Read moderator's name when the user is a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read moderator's name when the user is not a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read moderator's surname when the user is a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read moderator's surname when the user is not a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read candidate moderator's name when the user is a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read candidate's name when the user is not a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read candidate's surname when the user is a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read candidate's surname when the user is not a moderator	{CORE, FUNCTIONAL, ANY}
<code>analyze()</code>	
<code>read(Person, gender)</code> , Analyzes attendants' gender when the user is the owner	{ANALYTICS, INSIGHTS, ANY}
<code>read(Person, gender)</code> , Analyzes attendants' gender when the user is the manager	{ANALYTICS, INSIGHTS, ANY}
<code>read(Person, gender)</code> , Analyzes attendants' gender when the user is an attendant	{ANALYTICS, INSIGHTS, ANY}
<code>read(Person, gender)</code> , Analyzes attendants' gender when the user is a requester	{ANALYTICS, INSIGHTS, ANY}
<code>read(Person, gender)</code> , Analyzes attendants' gender when the user is unrelated	{ANALYTICS, INSIGHTS, ANY}
<code>view_category()</code>	
<code>read(Person, name)</code> , Read subscriber's name when the user is a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read subscriber's surname when the user is a moderator	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read event owner's name (moderator)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read event owner's name (non-moderator, owner)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read event owner's name (non-moderator, manager)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read event owner's name (non-moderator, attendant)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read event owner's name (non-moderator, requester)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, name)</code> , Read event owner's name (non-moderator, unrelated)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read event owner's surname (moderator)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read event owner's surname (non-moderator, owner)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read event owner's surname (non-moderator, manager)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read event owner's surname (non-moderator, attendant)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read event owner's surname (non-moderator, requester)	{CORE, FUNCTIONAL, ANY}
<code>read(Person, surname)</code> , Read event owner's surname (non-moderator, unrelated)	{CORE, FUNCTIONAL, ANY}

For each row in Table F.3, multiple test cases are generated in the following scenarios: $\langle$ No-Consent $\rangle$, $\langle$ Basic $\rangle$, and $\langle$ Complex $\rangle$.

$\langle$ No-Consent $\rangle$ scenario asserts that the system denies access when no consent has been granted for the requested personal data. Listing F.5 shows the test generated for the first row of the summary table, where the application attempts to access the caller's name when running the main() endpoint.

Listing F.5: Generated privacy test for $\langle$ No-Consent $\rangle$ scenario

```
1 def test_case_main_no_consent(client, app_with_data):
2     caller = Person.query.filter(Person.testid == 'user0').one()
3     with app_with_data.test_request_context():
4         login_user(caller)
5         try:
6             from project import main
7             result = main()
8             testing_user = result['user']
9             assert testing_user.name is RESTRICTED
10        except PrivacyException:
11            assert True
```

$\langle$ Basic $\rangle$ scenario In this scenario, access is permitted as the user has explicitly granted consent for the declared purpose. Listing F.6 shows the corresponding generated test, in which consent for the CORE purpose (line 5) allows the main() endpoint to display the user's name.

Listing F.6: Generated privacy test for $\langle$ Basic $\rangle$ scenario

```
1 def test_case_main_basic_consent(client, app_with_data):
2     caller = Person.query.filter(Person.testid == 'user0').one()
3     with app_with_data.test_request_context():
4         login_user(caller)
5         grant_consent(caller, CORE, 'Person', 'name')
6         try:
7             from project import main
8             result = main()
9             testing_user = result['user']
10            assert testing_user.name == caller.name
11        except PrivacyException:
12            assert False
13        finally:
14            revoke_consent(caller, CORE, 'Person', 'name')
```


⟨Complex⟩ scenario asserts that access is also granted when consent is provided for a complex purpose in the hierarchy. Listing F.7 shows the generated test in which consent for FUNCTIONAL or ANY purposes, both of which include CORE (lines 5–6), allows the main() endpoint to return the user’s name.

Listing F.7: Generated privacy test for ⟨Complex⟩ scenario

```

1  def test_case_main_complex_consent(client, app_with_data):
2      caller = Person.query.filter(Person.testid == 'user0').one()
3      with app_with_data.test_request_context():
4          login_user(caller)
5          for purpose in [FUNCTIONAL, ANY]:
6              grant_consent(caller, purpose, 'Person', 'name')
7              try:
8                  from project import main
9                  result = main()
10                 testing_user = result['user']
11                 assert testing_user.name == caller.name
12             except PrivacyException:
13                 assert False
14             finally:
15                 revoke_consent(caller, purpose, 'Person', 'name')

```

F.3 Survey Questionnaire

We provide the survey questionnaire used to collect participants' feedback on the project at the end of the course. The questionnaire was administered through Google Forms. The questionnaire is from the final edition of the study, as it is the most comprehensive version. Earlier editions used subsets of these questions. More specifically, Ed'22–Ed'23 used ActionGUI instead of vActionGUI and therefore did not include questions or segments of questions related to privacy. Ed'24 did not include questions about students' experience with Cedar. Each question lists its answer format in parentheses.

Background knowledge *This section collects information on your previous experience with the technologies and tools used in the course project. This helps us to account for differences in participants' background knowledge when analyzing the results.*

- Q1. Have you completed the Flask tutorial lab assignments before starting the project?
These assignments cover basics of Flask, including routes, endpoints, canonical URLs, HTTP methods, requests, sessions, templates, and adding persistent storage using SQLAlchemy.
(Yes / No)
- Q2. Have you completed the Flask authorization and handling personal data lab assignment before starting the project?
This assignment focuses on implementing security and privacy requirements in Flask.
(Yes / No)
- Q3. Have you completed the Object Constraint Language (OCL) exercise assignment before starting the project?
This assignment covers the syntax and semantics of the Object Constraint Language and demonstrates how to use it to define data invariants and authorization constraints.
(Yes / No)
- Q4. Have you completed the Object Constraint Language tutorial lab assignment before starting the project?
This assignment shows how to create a simple data model and use the OCL-py library to write and evaluate OCL queries and constraints on an instance of the data model.
(Yes / No)
- Q5. Have you completed the vActionGUI/ActionGUI exercise and lab assignments before starting the project?
This assignment introduces the vActionGUI tool using an example web application.
(Yes / No)
- Q6. Before the project, how would you best describe your level of experience using Flask and Python?
Novice: Had limited exposure, built simple applications with guidance from provided tutorials.

Intermediate: Had developed and extended applications, made use of common libraries.

Advanced: Had designed, built and maintained non-trivial enterprise-level applications.

(Novice / Intermediate / Advanced)

Q7. Prior to this course, did you have experience using Cedar?

(Yes / No)

Q8. Prior to this course, did you have experience using vActionGUI or ActionGUI?

(Yes / No)

Requirements engineering *This section evaluates your perception of the project requirements clarity and your approach to implementing them during the project.*

Q9. The project requirements were sufficiently clear.

(Strongly disagree / Disagree / Neither agree nor disagree / Agree / Strongly agree)

Q10. In Part I of the project, which approach did you choose to develop first?

(Model-driven approach (vActionGUI/ActionGUI) / Code-centric approach (Flask))

Q11. In Part III of the project, which approach were you assigned to?

(Model-driven approach (vActionGUI/ActionGUI) / Code-centric approach (Flask))

Development effort *This section collects information about the time you spent on different parts of the project and how you allocated your effort between implementing security and privacy requirements.*

Q12. Approximately how many hours did you spend completing Part I of the project?

(in hours)

Q13. What percentage of the total time in Part I of the project was roughly allocated for implementing privacy requirements (as opposed to security requirements)?

(in %)

Q14. Approximately how many hours did you spend completing Part II of the project?

(in hours)

Q15. What percentage of the total time in Part II of the project was roughly allocated for implementing privacy requirements (as opposed to security requirements)?

(in %)

Q16. Approximately how many hours did you spend completing Part III of the project in vActionGUI/ActionGUI?

(in hours)

- Q17. Approximately how many hours did you spend completing Part III of the project in Flask?
(in hours)

Approach references This section gathers information about the tools and approaches you used during the project, as well as your preferences for future projects involving similar security and privacy requirements.

- Q18. For the Flask implementation, how did you implement security requirements?
(Implemented manually in application code / Using Cedar / Using other library or framework; multiple selections allowed)
- Q19. If you chose “Using other library or framework” to implement security requirements, which library or framework did you use?
(free text)
- Q20. For the Flask implementation, how did you implement privacy requirements?
(Implemented manually in application code / Using an existing library or framework; multiple selections allowed)
- Q21. If you chose “Using an existing library or framework” to implement privacy requirements, which library or framework did you use?
(free text)
- Q22. If you had to enforce data protection requirements in a similar future project using one technology of your choice, which approach would you choose?
(Model-driven approach (vActionGUI) / Code-centric approach (Flask) / Other)
- Q23. Justify your choice in the previous answer.
(free text)
- Q24. If you had to enforce access control requirements in a similar future project using one technology of your choice, which approach would you choose?
(Model-driven approach (vActionGUI) / Code-centric approach (Flask) / Other)
- Q25. Justify your choice in the previous answer.
(free text)

F.4 Codebook and Qualitative Analysis

This appendix presents the codebook and the per-edition frequency analysis that underlie the qualitative insights discussed in Section 9.4.2. We applied inductive qualitative coding to participants' free-text justifications for their approach preference (Questions 23 and 25, Section F.3).

F.4.1 Codebook

The coding was performed by the author of the thesis. To develop the codebook, we drew a stratified 15% random sample from each edition and performed open coding to identify recurring themes. This produced an initial codebook with 3 categories and 9 subcodes. We then applied the codebook to the remaining responses and made minor adjustments.

In what follows, we display our final codebook with examples taken from the survey responses. Each coded segment is additionally tagged with a *direction*: *positive* (+) when the segment describes a strength of the respondent's chosen approach, or *negative* (-) when it describes a weakness of the rejected approach.

C1: Approach usability

C1a – Development efficiency

The approach is described as easier, faster, or simpler to implement.

(+) “vActionGUI *was much easier to implement.*”

(-) “Model driven [approach] *is annoying to write.*”

C1b – Intuitiveness

The approach is described as intuitive, natural, straightforward, or “making sense.”

(+) “I found it easier to translate the requirements into vActionGUI.”

(-) “ActionGUI setup *seems a bit complicated.*”

C2: Approach support

C2a – Documentation and Community

Availability, quality, or clarity of documentation.

(+) “There are plenty of [Flask] resources and guides online.”

(-) “ActionGUI’s documentation *was not clear to me.*”

C2b – Development tooling

IDE support, debugging and testing tools, and library/framework.

(+) *“More documentation is available for Python libraries.”*

(-) *“The lack of [vActionGUI] intuitive testing support and editor tooling.”*

C3: Approach characteristics

C3a – Flexibility and control

Degree of implementation freedom, customization, and control.

(+) *“Flask allows more fine-grained and adaptable enforcement.”*

(-) *“ActionGUI is too walled off.”*

C3b – Separation of concerns

Clean separation between security/privacy and application logic.

(+) *“It [vActionGUI] clearly separates the security aspects from the application logic.”*

C3c – Reliability

Systematic enforcement, consistency, with fewer errors.

(+) *“Privacy requirements can be enforced consistently across the application.”*

(-) *“It [Flask] does move the risk from the tool to the programmer.”*

C3d – Maintainability

Ease of maintenance and editing of the solution.

(+) *“The security model [in vActionGUI] was easier to maintain.”*

C3e – Production viability

Real-world applicability, industry adoption.

(+) *“Flask is much closer to a production environment.”*

(-) *“ActionGUI does not seem to be applicable outside ETH.”*

F.4.2 Qualitative Analysis

Across the four editions, we collected 71 free-text response units (16 in Ed’22, 18 in Ed’23, 23 in Ed’24, and 14 in Ed’25). In Ed’25, we merged the two separate justifications of each respondent for security and privacy into a single response unit.

Frequency analysis. Table F.4 reports the number of response units mentioning each code, broken down by edition and direction. The direction marker indicates whether a code reflects a strength of the chosen approach (+) or a weakness of the rejected approach (-). Among the 71 free-text response units, 67 contained substantive justifications suitable for qualitative coding. Because response units may be assigned multiple codes, some column totals exceed the number of responses.

Table F.4: Survey response code frequency counts by edition.

		By edition				Total	AG		vAG		Flask	
		'22	'23	'24	'25		+	-	+	-	+	-
Code	Name											
Number of substantive responses		15	17	13	22	67						
C1: Approach usability												
C1a	Development efficiency	6	4	5	14	29	4	1	15	2	6	1
C1b	Intuitiveness	0	5	4	5	14	2	2	8	0	2	0
C2: Approach support												
C2a	Documentation & community	6	8	1	0	15	0	6	0	0	8	1
C2b	Development tooling	3	5	1	1	10	0	4	0	2	4	0
C3: Approach characteristics												
C3a	Flexibility and control	2	7	1	3	13	0	4	0	0	7	2
C3b	Separation of concerns	1	1	1	5	8	2	0	6	0	0	0
C3c	Reliability	1	3	1	4	9	1	1	3	0	0	4
C3d	Maintainability	1	0	1	1	3	1	0	2	0	0	0
C3e	Production viability	5	4	5	2	16	0	7	1	4	2	2
Total							10	25	35	8	29	10

F.5 Reference Solution

The following listings show the reference solutions to the vActionGUI security and privacy model. These reference solutions refer to the project requirements in the final edition of the study.

EventPlatform vActionGUI Security Model Reference Solution

```

USER Person

anonymous Role VISITOR {
  Person {
    read(moderates)
  }
  Event {
    read(title)
    read(description)
    read(owner)
    read(categories)
    add(logs) [value.event.oclIsUndefined()]
  }
  Category {
    read(moderators)
    read(name)
    read(events)
  }
  Ad {
    read(content)
  }
  Log {
    create
    update(timestamp) [self.timestamp.oclIsUndefined()
                      and Log.allInstances()->forAll(
                        l|l.timestamp.oclIsUndefined() or l.timestamp < value)]
    update(event) [self.event.oclIsUndefined()]
  }
}

default Role FREEUSER extends VISITOR {
  Person {
    read(name)
    read(surname)
    read(role)
    read(gender) [self = caller]
    read(email) [self = caller]
    read(events) [self = caller]
    read(manages) [self = caller]
    read(attends) [self = caller]
    read(subscriptions) [self = caller]
    read(requests) [self = caller]
    update(name) [self = caller]
  }
}

```



```

update(surname)      [self = caller]
add(events)          [self = caller
                      and value.owner = null
                      and not(self.events->includes(value))]

add(manages)         [value.owner = caller
                      and not(self.manages->includes(value))
                      and (self.attends->includes(value) or self = value.owner)]

remove(manages)      [value.owner = caller
                      and self.manages->includes(value)
                      and not(self = value.owner)]

add(attends)         [((value.managedBy->includes(caller)
                      and value.requesters->includes(self))
                      or (value.owner = caller and self = caller)
                      or (value.invitations.event->includes(caller)))
                      and not(self.attends->includes(value))]

remove(attends)      [(value.managedBy->includes(caller) or self = caller)
                      and self.attends->includes(value)
                      and not(value.managedBy->includes(self))]

add(requests)        [self = caller
                      and not(self.requests->includes(value))
                      and not(self.attends->includes(value))
                      and not(self.invitations.event->includes(value))]

remove(requests)     [(self = caller or value.managedBy->includes(caller))
                      and self.requests->includes(value)]

add(subscriptions)   [self = caller and not(self.subscriptions->includes(value))]
remove(subscriptions) [self = caller and self.subscriptions->includes(value)]
add(logs)            [value.user.oclIsUndefined() and self = caller]
read(invitations)    [self = caller]
read(invites)        [self = caller]
add(invites)         [self = caller
                      and value.event.managedBy->includes(caller)
                      and value.invitedBy.oclIsUndefined()]

add(invitations)     [value.event.managedBy->includes(caller)
                      and value.invitee.oclIsUndefined()
                      and value.event.requests->excludes(self)
                      and value.event.attends->excludes(self)]
}

Event {
  create             [self.events->size() < 5]
  read(attendants)
  read(managedBy)
  read(requesters)   [self.managedBy->includes(caller)]
  update(title)      [self.managedBy->includes(caller)]
  update(description) [self.managedBy->includes(caller)]
  update(owner)      [self.owner = null and value = caller]
  add(categories)    [self.managedBy->includes(caller)
                      and not(self.categories->includes(value))]
  remove(categories) [self.managedBy->includes(caller)
                      and self.categories->includes(value)]
  add(managedBy)     [self.owner = caller
                      and not(self.managedBy->includes(value))]

```

```

                                and (self.attendants->includes(value)
                                or value = self.owner)]
remove(managedBy) [self.owner = caller
                    and self.managedBy->includes(value)
                    and not(value = self.owner)]
add(attendants)    [((self.managedBy->includes(caller)
                    and self.requesters->includes(value))
                    or (self.owner = caller and value = caller)
                    or (self.invitations.user->includes(caller)))
                    and not(self.attendants->includes(value))]
remove(attendants) [(self.managedBy->includes(caller) or value = caller)
                    and self.attendants->includes(value)
                    and not(self.managedBy->includes(value))]
add(requesters)   [value = caller
                    and not(self.requesters->includes(value))
                    and not(self.attendants->includes(value))
                    and not(self.invitations.user->includes(value))]
remove(requesters) [(value = caller or self.managedBy->includes(caller))
                    and self.requesters->includes(value)]
read(invitations) [self.managedBy->includes(caller)]
add(invitations)  [self.managedBy->includes(caller)
                    and value.event.oclIsUndefined()]
}
Category {
  add(events)      [value.managedBy->includes(caller)
                    and not(self.events->includes(value))]
  remove(events)   [value.managedBy->includes(caller)
                    and self.events->includes(value)]
  add(subscribers) [value = caller
                    and not(self.subscribers->includes(value))]
  remove(subscribers) [value = caller
                        and self.subscribers->includes(value)]
}
Log {
  read              [self.user = caller]
  update(user)      [self.user.oclIsUndefined() and value = caller]
}
Invite {
  read              [self.event.managedBy->includes(caller)
                    or self.invitee = caller
                    or self.invitedBy = caller]

  create
  delete            [self.event.managedBy->includes(caller)
                    or self.invitee = caller]

  update(event)     [value.managedBy->includes(caller)
                    and self.event.oclIsUndefined()]

  update(invitedBy) [value = caller
                    and self.event.managedBy->includes(caller)
                    and self.invitedBy.oclIsUndefined()]

  update(invitee)   [self.event.managedBy->includes(caller)
                    and self.invitee.oclIsUndefined()]
}

```

```

        and self.event.requests->excludes(value)
        and self.event.attends->excludes(value)]
    }
}

Role PREMIUMUSER extends FREEUSER {
    Event {
        create
    }
}

Role MODERATOR extends PREMIUMUSER {
    Person {
        read(email) [caller.moderates->exists(c|c.subscribers->includes(self))]
        remove(moderates) [self = caller and self.moderates->includes(value)]
    }
    Event {
        remove(categories) [caller.moderates->includes(value)
                             and self.categories->includes(value)]
    }
    Category {
        read(subscribers) [self.moderators->includes(caller)]
        remove(moderators) [value = caller and self.moderators->includes(value)]
        remove(events) [self.moderators->includes(caller)
                         and self.events->includes(value)]
    }
}

Role ADMIN extends PREMIUMUSER {
    Person {
        read(gender)
        update(role)
        add(moderates) [self.role = Role.MODERATOR
                        and not(self.moderates->includes(value))]
        remove(moderates) [self.moderates->includes(value)]
    }
    Category {
        create
        delete
        update(name)
        add(moderators) [value.role = Role.MODERATOR
                         and not(self.moderators->includes(value))]
        remove(moderators) [self.moderators->includes(value)]
    }
    Ad {
        create
        update
        delete
    }
}

```

F.5.1 vActionGUI Privacy Model

EventPlatform vActionGUI Privacy Model Reference Solution

```
Personal data {
  Person.name,
  Person.surname,
  Person.role,
  Person.gender,
  Person.email,
  Person.subscriptions,
  Person.attends,
  Log.event
}

Purposes {
  ANY includes MARKETING INSIGHTS FUNCTIONAL,
  MARKETING includes TARGETED MASS,
  TARGETED,
  MASS,
  INSIGHTS includes ANALYTICS STATS,
  ANALYTICS,
  STATS,
  FUNCTIONAL includes RECOMMENDATIONS CORE,
  RECOMMENDATIONS,
  CORE
}

Actual purposes {
  Event.analyze ANALYTICS,
  Ad.get_personalize_ad TARGETED,
  Category.send_mass_advertisement MASS,
  Person.send_advertisement_to_user MASS,
  Person.recommend_events RECOMMENDATIONS,
  main CORE,
  Person.profile CORE,
  Person.update_user CORE,
  Event.events CORE,
  Event.view_event CORE,
  Event.manage_event CORE,
  Category.view_category CORE,
  Category.edit_category CORE,
  Person.subscribe CORE,
  Person.unsubscribe CORE,
  Person.users CORE,
  Person.user CORE,
  Category.get_candidates CORE,
  Person.personalized_stats STATS,
  Invite.send_invite CORE,
  Invite.accept_invitation CORE,
  Invite.decline_invitation CORE,
```

```
Log.logs CORE
}

Declared purposes {
  Person.name MARKETING,
  Person.name CORE,
  Person.surname CORE,
  Person.role CORE,
  Person.gender CORE,
  Person.email CORE,
  Person.subscriptions CORE,
  Log.event FUNCTIONAL,
  Person.subscriptions RECOMMENDATIONS [self.role=Role::FREEUSER]
                                     <you are a regular user>,

  Person.gender TARGETED,
  Person.email MASS,
  Person.gender ANALYTICS,
  Person.attends FUNCTIONAL,
  Person.name STATS [self.attends→size()>2]
                  <you attended more than two events>,
  Person.subscriptions STATS [self.attends→size()>2]
                  <you attended more than two events>,
  Person.attends STATS [self.attends→size()>2]
                  <you attended more than two events>,
}
```


Bibliography

- [1] *A privacy notice lets people know what information a website has and what the website will do with it.* Information Commissioner's Office, 2024. URL: <https://ico.org.uk/for-organisations/advice-for-small-organisations/privacy-notices-and-cookies/create-your-own-privacy-notice/>.
- [2] Yasemin Acar, Michael Backes, Sascha Fahl, Doowon Kim, Michelle L. Mazurek, and Christian Stransky. "You Get Where You're Looking For: The Impact of Information Sources on Code Security." In: *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2016, pp. 289–305. DOI: [10.1109/SP.2016.25](https://doi.org/10.1109/SP.2016.25).
- [3] Amir Shayan Ahmadian, Daniel Strüder, and Jan Jürjens. "Privacy-enhanced system design modeling based on privacy features." In: *34th ACM/SIGAPP Symposium on Applied Computing, (SAC)*. Ed. by Chih-Cheng Hung and George A. Papadopoulos. ACM, 2019, pp. 1492–1499. DOI: [10.1145/3297280.3297431](https://doi.org/10.1145/3297280.3297431). URL: <https://doi.org/10.1145/3297280.3297431>.
- [4] Atheer Aljeraisy, Masoud Barati, Omer F. Rana, and Charith Perera. "Privacy Laws and Privacy by Design Schemes for the Internet of Things: A Developer's Perspective." In: *ACM Comput. Surv.* 54.5 (2021), 102:1–102:38. DOI: [10.1145/3450965](https://doi.org/10.1145/3450965). URL: <https://doi.org/10.1145/3450965>.
- [5] Amazon Web Services. *Cedar Authorization Language*. 2025. URL: <https://www.cedarpolicy.com/>.
- [6] Thibaud Antignac and Daniel Le Métayer. "Privacy Architectures: Reasoning about Data Minimisation and Integrity." In: *Security and Trust Management (STM)*. Ed. by Sjouke Mauw and Christian Damsgaard Jensen. Vol. 8743. LNCS. Springer, 2014, pp. 17–32. DOI: [10.1007/978-3-319-11851-2_2](https://doi.org/10.1007/978-3-319-11851-2_2). URL: https://doi.org/10.1007/978-3-319-11851-2_2.
- [7] Thibaud Antignac, Riccardo Scandariato, and Gerardo Schneider. "Privacy Compliance Via Model Transformations." In: *IEEE European Symposium on Security and Privacy Workshops (EuroS&P Workshops)*. IEEE, 2018, pp. 120–126. DOI: [10.1109/EuroSPW.2018.00024](https://doi.org/10.1109/EuroSPW.2018.00024). URL: <https://doi.org/10.1109/EuroSPW.2018.00024>.
- [8] Claudio A. Ardagna et al. "Primelife policy language." In: *W3C Workshop on Access Control Application Scenarios*. W3C, 2009.
- [9] Erik Arisholm, Lionel C. Briand, Siw Elisabeth Hove, and Yvan Labiche. "The Impact of UML Documentation on Software Maintenance: An Experimental Evaluation." In: *IEEE Trans. Software Eng.* 32.6 (2006), pp. 365–381. DOI: [10.1109/TSE.2006.59](https://doi.org/10.1109/TSE.2006.59). URL: <https://doi.org/10.1109/TSE.2006.59>.
- [10] Paul Ashley, Satoshi Hada, Günter Karjoth, Calvin Powers, and Matthias Schunter. *Enterprise privacy authorization language (EPAL)*. Tech. rep. IBM Research, 2003. URL: <https://www.w3.org/2003/p3p-ws/pp/ibm3.html>.
- [11] Thomas H. Austin, Jean Yang, Cormac Flanagan, and Armando Solar-Lezama. "Faceted execution of policy-agnostic programs." In: *8th ACM SIGPLAN Workshop on Programming Languages and Analysis for Security (PLAS)*. 2013, pp. 15–26.

Bibliography

- [12] Monir Azraoui, Kaoutar Elkhayaoui, Melek Önen, Karin Bernsmed, Anderson Santana De Oliveira, and Jakub Sendor. “A-PPL: an accountability policy language.” In: *Data Privacy Management, Autonomous Spontaneous Security, and Security Assurance (DP-M/SETOP/QASA)*. Springer, 2014, pp. 319–326.
- [13] Chinmayi Prabhu Baramashetru, Silvia Lizeth Tapia Tarifa, Olaf Owe, and Nils Gruschka. “A Policy Language to Capture Compliance of Data Protection Requirements.” In: *17th International Conference on Integrated Formal Methods (IFM)*. Ed. by Maurice H. ter Beek and Rosemary Monahan. Vol. 13274. LNCS. Springer, 2022, pp. 289–309. DOI: [10.1007/978-3-031-07727-2_16](https://doi.org/10.1007/978-3-031-07727-2_16). URL: https://doi.org/10.1007/978-3-031-07727-2_16.
- [14] David Basin, Samuel J. Burri, and Günter Karjoth. “Optimal workflow-aware authorizations.” In: *17th ACM Symposium on Access Control Models and Technologies, SACMAT ’12, Newark, NJ, USA - June 20 - 22, 2012*. Ed. by Vijay Atluri, Jaideep Vaidya, Axel Kern, and Murat Kantarcioglu. ACM, 2012, pp. 93–102. DOI: [10.1145/2295136.2295154](https://doi.org/10.1145/2295136.2295154). URL: <https://doi.org/10.1145/2295136.2295154>.
- [15] David Basin, Manuel Clavel, Marina Egea, Miguel Angel García de Dios, and Carolina Dania. “A Model-Driven Methodology for Developing Secure Data-Management Applications.” In: *IEEE Trans. Software Eng.* 40.4 (2014), pp. 324–337. DOI: [10.1109/TSE.2013.2297116](https://doi.org/10.1109/TSE.2013.2297116). URL: <https://doi.org/10.1109/TSE.2013.2297116>.
- [16] David Basin, Manuel Clavel, Marina Egea, Miguel Angel García de Dios, and Carolina Dania. *ActionGUI semantics*. Tech. rep. IMDEA Software Institute, 2013. URL: <https://software.imdea.org/~dania/papers/techreport13.pdf>.
- [17] David Basin, Manuel Clavel, Marina Egea, and Michael Schläpfer. “Automatic Generation of Smart, Security-Aware GUI Models.” In: *2nd International Symposium on Engineering Secure Software and Systems (ESSoS)*. Ed. by Fabio Massacci, Dan S. Wallach, and Nicola Zannone. LNCS. Springer, 2010, pp. 201–217. DOI: [10.1007/978-3-642-11747-3_16](https://doi.org/10.1007/978-3-642-11747-3_16). URL: https://doi.org/10.1007/978-3-642-11747-3_16.
- [18] David Basin, Jürgen Doser, and Torsten Lodderstedt. “Model driven security: From UML models to access control infrastructures.” In: *ACM Trans. Softw. Eng. Methodol.* 15.1 (2006), pp. 39–91. DOI: [10.1145/1125808.1125810](https://doi.org/10.1145/1125808.1125810). URL: <https://doi.org/10.1145/1125808.1125810>.
- [P2] David Basin, Juan Guarnizo, Srđan Krstić, Hoang Nguyen, and Martín Ochoa. “Is Modeling Access Control Worth It?” In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Ed. by Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda. ACM, 2023, pp. 2830–2844. DOI: [10.1145/3576915.3623196](https://doi.org/10.1145/3576915.3623196). URL: <https://doi.org/10.1145/3576915.3623196>.
- [P4] David Basin, François Hublet, Srđan Krstić, and Hoang Nguyen. “Mechanizing Privacy by Design.” In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Ed. by Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier. ACM, 2025, pp. 2–5. DOI: [10.1145/3719027.3748271](https://doi.org/10.1145/3719027.3748271). URL: <https://doi.org/10.1145/3719027.3748271>.

- [19] David Basin et al. “Model-Driven Development of Security-Aware GUIs for Data-Centric Applications.” In: *Foundations of Security Analysis and Design VI - FOSAD Tutorial Lectures*. Ed. by Alessandro Aldini and Roberto Gorrieri. Lecture Notes in Computer Science. Springer, 2011, pp. 101–124. DOI: [10.1007/978-3-642-23082-0_4](https://doi.org/10.1007/978-3-642-23082-0_4). URL: https://doi.org/10.1007/978-3-642-23082-0_4.
- [20] Kathrin Bednar, Sarah Spiekermann, and Marc Langheinrich. “Engineering Privacy by Design: Are engineers ready to live up to the challenge?” In: *Inf. Soc.* 35.3 (2019), pp. 122–142. DOI: [10.1080/01972243.2019.1583296](https://doi.org/10.1080/01972243.2019.1583296). URL: <https://doi.org/10.1080/01972243.2019.1583296>.
- [21] Abhishek Bichhawat, Akash Trehan, Jean Yang, and Matt Fredrikson. “ESTRELA: Automated Policy Enforcement Across Remote APIs.” In: *CoRR* abs/1811.08234 (2018). arXiv: [1811.08234](http://arxiv.org/abs/1811.08234). URL: <http://arxiv.org/abs/1811.08234>.
- [22] Ahmed Bouhoula, Karel Kubicek, Amit Zac, Carlos Cotrini, and David Basin. “Automated Large-Scale Analysis of Cookie Notice Compliance.” In: *33rd USENIX Security Symposium*. USENIX Association, 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/bouhoula>.
- [23] Ji-Won Byun, Elisa Bertino, and Ninghui Li. “Purpose based access control of complex data for privacy protection.” In: *10th ACM Symposium on Access Control Models and Technologies*. Ed. by Elena Ferrari and Gail-Joon Ahn. SACMAT. ACM, 2005, pp. 102–110. DOI: [10.1145/1063979.1063998](https://doi.org/10.1145/1063979.1063998). URL: <https://doi.org/10.1145/1063979.1063998>.
- [24] Ji-Won Byun and Ninghui Li. “Purpose based access control for privacy protection in relational database systems.” In: *VLDB J.* 17.4 (2008), pp. 603–619. DOI: [10.1007/s00778-006-0023-0](https://doi.org/10.1007/s00778-006-0023-0). URL: <https://doi.org/10.1007/s00778-006-0023-0>.
- [25] Ann Cavoukian. *Privacy by Design: The 7 Foundational Principles*. Tech. rep. Information and Privacy Commissioner of Ontario, Canada, 2009.
- [26] *California Consumer Privacy Act*. 2018. URL: <https://oag.ca.gov/privacy/ccpa>.
- [27] Omar Chowdhury, Limin Jia, Deepak Garg, and Anupam Datta. “Temporal Mode-Checking for Runtime Monitoring of Privacy Policies.” In: *26th International Conference on Computer Aided Verification (CAV)*. Ed. by Armin Biere and Roderick Bloem. Vol. 8559. LNCS. Springer, 2014, pp. 131–149. DOI: [10.1007/978-3-319-08867-9_9](https://doi.org/10.1007/978-3-319-08867-9_9). URL: https://doi.org/10.1007/978-3-319-08867-9_9.
- [28] Manuel Clavel, Viviane Torres da Silva, Christiano Braga, and Marina Egea. “Model-Driven Security in Practice: An Industrial Experience.” In: *4th European Conference on Model Driven Architecture – Foundations and Applications (ECMDA-FA)*. Vol. 5095. LNCS. Germany: Springer, 2008, pp. 326–337. DOI: [10.1007/978-3-540-69100-6_22](https://doi.org/10.1007/978-3-540-69100-6_22). URL: https://doi.org/10.1007/978-3-540-69100-6_22.
- [29] Lorrie Cranor. *Web privacy with P3P - the platform for privacy preferences*. O’Reilly, 2002. ISBN: 978-0-596-00371-5. URL: <https://www.w3.org/P3P/>.
- [30] Lorrie Cranor, Marc Langheinrich, Massimo Marchiori, and Joseph Reagle. *The Platform for Privacy Preferences 1.0 (P3P1.0) Specification*. Obsoleted on 30 August 2018. 2002. URL: <https://www.w3.org/TR/P3P/>.

- [31] CrowdStrike. *External Technical Root Cause Analysis, Channel File 291*. Tech. rep. Accessed: 2026-05-04. CrowdStrike, Aug. 2024. URL: <https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf>.
- [32] Joseph W. Cutler et al. “Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization.” In: *Proc. ACM Program. Lang.* 8.OOPSLA1 (2024), pp. 670–697. DOI: [10.1145/3649835](https://doi.org/10.1145/3649835). URL: <https://doi.org/10.1145/3649835>.
- [33] Nicodemos Damianou, Naranker Dulay, Emil Lupu, and Morris Sloman. “The Ponder Policy Specification Language.” In: *International Workshop on Policies for Distributed Systems and Networks (POLICY)*. Ed. by Morris Sloman, Jorge Lobo, and Emil Lupu. Vol. 1995. LNCS. Springer, 2001, pp. 18–38. DOI: [10.1007/3-540-44569-2_2](https://doi.org/10.1007/3-540-44569-2_2). URL: https://doi.org/10.1007/3-540-44569-2_2.
- [S4] Sofie Daniëls. *Secure and Privacy-aware Object Relational Mappings*. ETH Zurich. Master’s Thesis. 2025.
- [34] Birgit Demuth and Heinrich Hußmann. “Using UML/OCL Constraints for Relational Database Design.” In: *«UML»’99: The Unified Modeling Language - Beyond the Standard, Second International Conference, Fort Collins, CO, USA, October 28-30, 1999, Proceedings*. Ed. by Robert B. France and Bernhard Rumpe. Lecture Notes in Computer Science. Springer, 1999, pp. 598–613. DOI: [10.1007/3-540-46852-8_42](https://doi.org/10.1007/3-540-46852-8_42). URL: https://doi.org/10.1007/3-540-46852-8_42.
- [35] Department of Homeland Security of the United States. *The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research*. Tech. rep. Department of Homeland Security of the United States, 2012. URL: <https://www.dhs.gov/publication/menlo-report>.
- [36] Henry DeYoung, Deepak Garg, Limin Jia, Dilsun Kirli Kaynar, and Anupam Datta. “Experiences in the logical specification of the HIPAA and GLBA privacy laws.” In: *Workshop on Privacy in the Electronic Society (WPES)*. Ed. by Ehab Al-Shaer and Keith B. Frikken. ACM, 2010, pp. 73–82. DOI: [10.1145/1866919.1866930](https://doi.org/10.1145/1866919.1866930). URL: <https://doi.org/10.1145/1866919.1866930>.
- [37] Miguel Angel García de Dios, Carolina Dania, and Manuel Clavel. “Formal Reasoning about Fine-Grained Access Control Policies.” In: *11th Asia-Pacific Conference on Conceptual Modelling (APCCM)*. Vol. 165. CRPIT. Australian Computer Society, 2015, pp. 91–100.
- [38] África Domingo, Jorge Echeverría, Oscar Pastor, and Carlos Cetina. “Evaluating the Benefits of Model-Driven Development: Empirical Evaluation Paper.” In: *32nd International Conference on Advanced Information Systems Engineering (CAiSE)*. Ed. by Schahram Dustdar, Eric Yu, Camille Salinesi, Dominique Rieu, and Vik Pant. Vol. 12127. LNCS. Springer, 2020, pp. 353–367. DOI: [10.1007/978-3-030-49435-3_22](https://doi.org/10.1007/978-3-030-49435-3_22). URL: https://doi.org/10.1007/978-3-030-49435-3_22.
- [39] Marina Egea and Carolina Dania. “SQL-PL4OCL: an automatic code generator from OCL to SQL procedural language.” In: *Softw. Syst. Model.* 18.1 (2019), pp. 769–791. DOI: [10.1007/s10270-017-0597-6](https://doi.org/10.1007/s10270-017-0597-6). URL: <https://doi.org/10.1007/s10270-017-0597-6>.

- [40] Information Security Group at ETH Zurich. *Security Engineering, Master's-level course on engineering techniques for developing secure systems across the software lifecycle, covering requirements engineering, system modeling, secure coding, risk analysis, threat modeling, security design, and testing*. Autumn Semester. 2022–2025. 2025. URL: <https://infsec.ethz.ch/education/as2025/seceng.html>.
- [41] David Ferraiolo and Richard Kuhn. “Role-Based Access Control.” In: *15th NIST-NCSC National Computer Security Conference*. 1992, pp. 554–563.
- [42] David F. Ferraiolo, Ravi S. Sandhu, Serban I. Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. “Proposed NIST Standard for Role-Based Access Control.” In: *ACM Trans. Inf. Syst. Secur.* 4.3 (2001), pp. 224–274. DOI: [10.1145/501978.501980](https://doi.org/10.1145/501978.501980).
- [43] Flask. *Flask Web Framework*. 2023. URL: <https://flask.palletsprojects.com/en/2.3.x/>.
- [44] Flask-RBAC Contributors. *Flask-RBAC: Role Based Access Control for Flask*. Accessed: 2026-04-04. 2024. URL: <https://flask-rbac.readthedocs.io/en/latest/>.
- [45] Eclipse Foundation. *Eclipse AspectJ*. 2021. URL: <https://www.eclipse.org/aspectj/>.
- [46] Lucas Franke, Huayu Liang, Sahar Farzanehpour, Aaron Brantly, James C. Davis, and Chris Brown. “An Exploratory Mixed-methods Study on General Data Protection Regulation (GDPR) Compliance in Open-Source Software.” In: *18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2024, pp. 325–336. DOI: [10.1145/3674805.3686692](https://doi.org/10.1145/3674805.3686692). URL: <https://doi.org/10.1145/3674805.3686692>.
- [47] Andreas Frömer, Alexander Lisachenko, and Kirill Nesmeyanov. *Go! Aspect-Oriented Framework for PHP*. 2021. URL: <https://github.com/goaop/framework>.
- [48] Kelsey R. Fulton, Daniel Votipka, Desiree Abrokwa, Michelle L. Mazurek, Michael Hicks, and James Parker. “Understanding the How and the Why: Exploring Secure Development Practices through a Course Competition.” In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. USA: ACM, 2022, pp. 1141–1155. DOI: [10.1145/3548606.3560569](https://doi.org/10.1145/3548606.3560569). URL: <https://doi.org/10.1145/3548606.3560569>.
- [49] François Gauthier, Ettore Merlo, Eleni Stroulia, and David Turner. “Supporting Maintenance and Evolution of Access Control Models in Web Applications.” In: *30th IEEE International Conference on Software Maintenance and Evolution (ICSME)*. USA: IEEE, 2014, pp. 506–510. DOI: [10.1109/ICSME.2014.83](https://doi.org/10.1109/ICSME.2014.83). URL: <https://doi.org/10.1109/ICSME.2014.83>.
- [50] *Gramm-Leach-Bliley Act*. 1999. URL: <https://www.ftc.gov/legal-library/browse/statutes/gramm-leach-bliley-act>.
- [51] Hassan Gomaa. *Software modeling and design: UML, use cases, patterns, and software architectures*. Cambridge University Press, 2011. ISBN: 978-0521764148.
- [52] Marco Guarnieri and David Basin. “Optimal Security-Aware Query Processing.” In: *Proc. VLDB Endow.* 11.10 (2018), pp. 1307–1320. DOI: [10.14778/3231751.3231770](https://doi.org/10.14778/3231751.3231770).

- [53] Michele Guerriero, Damian Andrew Tamburri, and Elisabetta Di Nitto. “Defining, enforcing and checking privacy policies in data-intensive applications.” In: *International Conference on Software Engineering for Adaptive and Self-Managing Systems, (SEAMS)*. Ed. by Jesper Andersson and Danny Weyns. ACM, 2018, pp. 172–182. DOI: [10.1145/3194133.3194140](https://doi.org/10.1145/3194133.3194140). URL: <https://doi.org/10.1145/3194133.3194140>.
- [54] Seda Gürses, Carmela Troncoso, and Claudia Diaz. “Engineering privacy by design.” In: *Computers, Privacy & Data Protection* 14.3 (2011), p. 25.
- [55] Zhuobing Han, Xiaohong Li, Guangquan Xu, Naixue Xiong, Ettore Merlo, and Eleni Stroulia. “An Effective Evolutionary Analysis Scheme for Industrial Software Access Control Models.” In: *IEEE Trans. Ind. Informatics* 16.2 (2020), pp. 1024–1034. DOI: [10.1109/TII.2019.2925422](https://doi.org/10.1109/TII.2019.2925422). URL: <https://doi.org/10.1109/TII.2019.2925422>.
- [56] Michael A. Harrison, Walter L. Ruzzo, and Jeffrey D. Ullman. “Protection in Operating Systems.” In: *Commun. ACM* 19.8 (1976), pp. 461–471. DOI: [10.1145/360303.360333](https://doi.org/10.1145/360303.360333). URL: <https://doi.org/10.1145/360303.360333>.
- [57] *Health Insurance Portability and Accountability Act*. 1996. URL: <https://www.hhs.gov/hipaa/>.
- [58] Gerard J. Holzmann. “The Model Checker SPIN.” In: *IEEE Trans. Software Eng.* 23.5 (1997), pp. 279–295. DOI: [10.1109/32.588521](https://doi.org/10.1109/32.588521). URL: <https://doi.org/10.1109/32.588521>.
- [59] Stefan Albert Horstmann et al. ““Sorry for Bugging You So Much.” Exploring Developers’ Behavior Towards Privacy-Compliant Implementation.” In: *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2025, pp. 1215–1233. DOI: [10.1109/SP61157.2025.00146](https://doi.org/10.1109/SP61157.2025.00146). URL: <https://doi.org/10.1109/SP61157.2025.00146>.
- [60] Stefan Albert Horstmann et al. ““Those things are written by lawyers, and programmers are reading that.” Mapping the Communication Gap Between Software Developers and Privacy Experts.” In: *Proc. Priv. Enhancing Technol.* 2024.1 (2024), pp. 151–170. DOI: [10.56553/POPETS-2024-0010](https://doi.org/10.56553/POPETS-2024-0010). URL: <https://doi.org/10.56553/popets-2024-0010>.
- [61] Martin Höst, Björn Regnell, and Claes Wohlin. “Using Students as Subjects - A Comparative Study of Students and Professionals in Lead-Time Impact Assessment.” In: *Empir. Softw. Eng.* 5.3 (2000), pp. 201–214.
- [62] François Hublet, David Basin, and Srđan Krstić. “User-Controlled Privacy: Taint, Track, and Control.” In: *Proc. Priv. Enhancing Technol.* 2024.1 (2024), pp. 597–616.
- [63] John Hunt. “Decorators.” In: *A Beginners Guide to Python 3 Programming*. Springer, 2019, pp. 337–351.
- [64] John Hutchinson, Jon Whittle, and Mark Rouncefield. “Model-Driven Engineering Practices in Industry: Social, Organizational and Managerial Factors that Lead to Success or Failure.” In: *Sci. Comput. Program.* 89 (2014), pp. 144–161. DOI: [10.1016/j.scico.2013.03.017](https://doi.org/10.1016/j.scico.2013.03.017).
- [65] JeeHyun Hwang, Laurie A. Williams, and Mladen A. Vouk. “Access control policy evolution: an empirical study.” In: *Symposium and Bootcamp on the Science of Security (HotSoS)*. USA: ACM, 2014, p. 28. DOI: [10.1145/2600176.2600204](https://doi.org/10.1145/2600176.2600204). URL: <https://doi.org/10.1145/2600176.2600204>.

- [66] Jan Jürjens. “UMLsec: Extending UML for Secure Systems Development.” In: *5th International Conference on The Unified Modeling Language (UML)*. Vol. 2460. LNCS. Springer, 2002, pp. 412–425. DOI: [10.1007/3-540-45800-X_32](https://doi.org/10.1007/3-540-45800-X_32).
- [67] Govind Kabra, Ravishankar Ramamurthy, and S. Sudarshan. “Redundancy and Information Leakage in Fine-Grained Access Control.” In: *ACM SIGMOD International Conference on Management of Data*. ACM, 2006, pp. 133–144. DOI: [10.1145/1142473.1142489](https://doi.org/10.1145/1142473.1142489).
- [68] Farzane Karami, David Basin, and Einar Broch Johnsen. “DPL: A Language for GDPR Enforcement.” In: *35th IEEE Computer Security Foundations Symposium, (CSF)*. IEEE, 2022, pp. 112–129. DOI: [10.1109/CSF54842.2022.9919687](https://doi.org/10.1109/CSF54842.2022.9919687). URL: <https://doi.org/10.1109/CSF54842.2022.9919687>.
- [P3] Srđan Krstić, Hoang Nguyen, and David Basin. “Model-driven Privacy.” In: *Proc. Priv. Enhancing Technol.* 2024.1 (2024), pp. 314–329. DOI: [10.56553/POPETS-2024-0018](https://doi.org/10.56553/POPETS-2024-0018). URL: <https://doi.org/10.56553/popets-2024-0018>.
- [69] Karel Kubicek, Jakob Merane, Carlos Cotrini, Alexander Stremitzer, Stefan Bechtold, and David Basin. “Checking Websites’ GDPR Consent Compliance for Marketing Emails.” In: *Proc. Priv. Enhancing Technol.* 2022.2 (2022), pp. 282–303. DOI: [10.2478/POPETS-2022-0046](https://doi.org/10.2478/POPETS-2022-0046). URL: <https://doi.org/10.2478/popets-2022-0046>.
- [70] Adrian Kuhn, Gail C. Murphy, and C. Albert Thompson. “An Exploratory Study of Forces and Frictions Affecting Large-Scale Model-Driven Development.” In: *15th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. Vol. 7590. LNCS. Springer, 2012, pp. 352–367. DOI: [10.1007/978-3-642-33666-9_23](https://doi.org/10.1007/978-3-642-33666-9_23). URL: <https://arxiv.org/abs/1207.0855>.
- [71] Mirosław Kutylowski, Anna Lauks-Dutka, and Moti Yung. “GDPR - Challenges for Reconciling Legal Rules with Technical Reality.” In: *25th European Symposium on Research in Computer Security, (ESORICS)*. Ed. by Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider. Vol. 12308. LNCS. Springer, 2020, pp. 736–755. DOI: [10.1007/978-3-030-58951-6_36](https://doi.org/10.1007/978-3-030-58951-6_36). URL: https://doi.org/10.1007/978-3-030-58951-6_36.
- [72] Peifung E. Lam, John C. Mitchell, and Sharada Sundaram. “A Formalization of HIPAA for a Medical Messaging System.” In: *6th International Conference on Trust, Privacy and Security in Digital Business (TrustBus)*. Ed. by Simone Fischer-Hübner, Costas Lambrinoudakis, and Günther Pernul. Vol. 5695. LNCS. Springer, 2009, pp. 73–85. DOI: [10.1007/978-3-642-03748-1_8](https://doi.org/10.1007/978-3-642-03748-1_8). URL: https://doi.org/10.1007/978-3-642-03748-1_8.
- [73] CMS Law. *GDPR Enforcement Tracker*. Accessed: 2026-06-15. 2026. URL: <https://www.enforcementtracker.com/>.
- [74] Kristen LeFevre, Rakesh Agrawal, Vuk Ercegovic, Raghu Ramakrishnan, Yirong Xu, and David J. DeWitt. “Limiting Disclosure in Hippocratic Databases.” In: *30th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2004, pp. 108–119.
- [75] Jens Leicht and Maritta Heisel. “A survey on privacy policy languages: Expressiveness concerning data protection regulations.” In: *12th CMI Conference on Cybersecurity and Privacy (CMI)*. IEEE, 2019, pp. 1–6.
- [76] Yongkun Liu et al. *An Empirical Study on Low Code Programming using Traditional vs Large Language Model Support*. Manuscript submitted for publication. 2025. URL: https://nzjohng.github.io/publications/papers/jss2025_3.pdf.

Bibliography

- [77] Torsten Lodderstedt, David Basin, and Jürgen Doser. “SecureUML: A UML-Based Modeling Language for Model-Driven Security.” In: *5th International Conference on The Unified Modeling Language (UML)*. Ed. by Jean-Marc Jézéquel, Heinrich Hußmann, and Stephen Cook. Vol. 2460. LNCS. Springer, 2002, pp. 426–441. DOI: [10.1007/3-540-45800-X_33](https://doi.org/10.1007/3-540-45800-X_33). URL: https://doi.org/10.1007/3-540-45800-X_33.
- [78] David Maier, K. Tuncay Tekle, Michael Kifer, and David Scott Warren. “Datalog: concepts, history, and outlook.” In: *Declarative Logic Programming: Theory, Systems, and Applications*. Ed. by Michael Kifer and Yanhong Annie Liu. Vol. 20. ACM Books. ACM / Morgan & Claypool, 2018, pp. 3–100. DOI: [10.1145/3191315.3191317](https://doi.org/10.1145/3191315.3191317). URL: <https://doi.org/10.1145/3191315.3191317>.
- [79] Maryam Majedi and Ken Barker. “The Privacy Policy Permission Model: A Unified View of Privacy Policies.” In: *Trans. Data Priv.* 14.1 (2021), pp. 1–36. URL: <http://www.tdp.cat/issues21/tdp.a417a21.pdf>.
- [80] Célestin Matte, Nataliia Bielova, and Cristiana Santos. “Do Cookie Banners Respect my Choice? Measuring Legal Compliance of Banners from IAB Europe’s Transparency and Consent Framework.” In: *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2020, pp. 791–809. DOI: [10.1109/SP40000.2020.00076](https://doi.org/10.1109/SP40000.2020.00076). URL: <https://doi.org/10.1109/SP40000.2020.00076>.
- [81] Michael J. May, Carl A. Gunter, and Insup Lee. “Privacy APIs: Access Control Techniques to Analyze and Verify Legal Privacy Policies.” In: *19th IEEE Computer Security Foundations Workshop, (CSFW)*. IEEE Computer Society, 2006, pp. 85–97. DOI: [10.1109/CSFW.2006.24](https://doi.org/10.1109/CSFW.2006.24). URL: <https://doi.org/10.1109/CSFW.2006.24>.
- [82] Aastha Mehta, Eslam Elnikety, Katura Harvey, Deepak Garg, and Peter Druschel. “Qapla: Policy Compliance for Database-backed Systems.” In: *26th USENIX Security Symposium*. USENIX Association, 2017, pp. 1463–1479.
- [83] Erik Meijer, Brian Beckman, and Gavin M. Bierman. “LINQ: reconciling object, relations and XML in the .NET framework.” In: *Proceedings of the ACM SIGMOD International Conference on Management of Data, Chicago, Illinois, USA, June 27-29, 2006*. ACM, 2006, p. 706. DOI: [10.1145/1142473.1142552](https://doi.org/10.1145/1142473.1142552). URL: <https://doi.org/10.1145/1142473.1142552>.
- [84] Elliott Mendelson. *Introduction to mathematical logic* (3. ed.) Chapman and Hall, 1987. ISBN: 978-0-412-06971-0.
- [85] Microsoft. *Code First to a New Database – EF6*. 2023. URL: <https://learn.microsoft.com/en-us/ef/ef6/modeling/code-first/workflows/new-database>.
- [86] Microsoft. *Database-level roles*. Accessed: 2026-04-04. 2023. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/security/authentication-access/database-level-roles>.
- [87] Microsoft. *MS ASP.NET Core Identity*. 2022. URL: <https://learn.microsoft.com/en-us/aspnet/identity/>.
- [88] Microsoft. *MS Entity Framework Core*. 2021. URL: <https://learn.microsoft.com/en-us/ef/core/>.

- [89] Abdirazak K. Y. S. Mohamed, Daniel Auer, Daniel Hofer, and Josef Küng. “Authorization Strategies and Classification of Access Control Models.” In: *8th International Conference on Future Data and Security Engineering (FDSE)*. Vol. 13076. LNCS. Springer, 2021, pp. 155–174. DOI: [10.1007/978-3-030-91387-8_11](https://doi.org/10.1007/978-3-030-91387-8_11).
- [90] MySQL. *Security in MySQL v 8.0: Using roles*. Accessed: 2026-04-04. 2023. URL: <https://dev.mysql.com/doc/mysql-security-excerpt/8.0/en/roles.html>.
- [91] Alena Naiakshina, Anastasia Danilova, Eva Gerlitz, and Matthew Smith. “On Conducting Security Developer Studies with CS Students: Examining a Password-Storage Study with CS Students, Freelancers, and Company Developers.” In: *CHI Conference on Human Factors in Computing Systems (CHI)*. Ed. by Regina Bernhaupt et al. USA: ACM, 2020, pp. 1–13. DOI: [10.1145/3313831.3376791](https://doi.org/10.1145/3313831.3376791). URL: <https://doi.org/10.1145/3313831.3376791>.
- [92] Graham Neray and Gabe Jackson. *Oso is a batteries-included framework for building authorization in your application*. 2023. URL: <https://pypi.org/project/flask-oso/>.
- [93] Hoang Nguyen and Manuel Clavel. “A Model-Driven Approach for Enforcing Fine-Grained Access Control for SQL Queries.” In: *SN Comput. Sci.* 2.5 (2021), p. 370. DOI: [10.1007/s42979-021-00712-7](https://doi.org/10.1007/s42979-021-00712-7). URL: <https://doi.org/10.1007/s42979-021-00712-7>.
- [P1] Hoang Nguyen and Manuel Clavel. “An extended model-based characterization of fine-grained access control for SQL queries.” In: *26th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 95–105. DOI: [10.1109/MODELS58315.2023.00028](https://doi.org/10.1109/MODELS58315.2023.00028). URL: <https://doi.org/10.1109/MODELS58315.2023.00028>.
- [94] Hoang Nguyen and Manuel Clavel. “Model-based Characterization of fine-grained Access Control Authorization for SQL Queries.” In: *J. Object Technol.* 19.3 (2020), 3:1–13. DOI: [10.5381/JOT.2020.19.3.A15](https://doi.org/10.5381/JOT.2020.19.3.A15). URL: <https://doi.org/10.5381/jot.2020.19.3.a15>.
- [95] Hoang Nguyen and Manuel Clavel. “OCL2PSQL: An OCL-to-SQL Code-Generator for Model-Driven Engineering.” In: *6th International Conference on Future Data and Security Engineering (FDSE)*. Ed. by Tran Khanh Dang, Josef Küng, Makoto Takizawa, and Son Ha Bui. LNCS. Springer, 2019, pp. 185–203. DOI: [10.1007/978-3-030-35653-8_13](https://doi.org/10.1007/978-3-030-35653-8_13). URL: https://doi.org/10.1007/978-3-030-35653-8_13.
- [P7] Hoang Nguyen and Manuel Clavel. “Optimising Fine-Grained Access Control Policy Enforcement for Database Queries. A Model-Driven Approach.” In: *CoRR abs/2209.05561* (2022). DOI: [10.48550/ARXIV.2209.05561](https://doi.org/10.48550/ARXIV.2209.05561). arXiv: [2209.05561](https://arxiv.org/abs/2209.05561). URL: <https://doi.org/10.48550/arXiv.2209.05561>.
- [P8] Hoang Nguyen and Manuel Clavel. “Proving correctness for SQL implementations of OCL constraints.” In: *CoRR abs/2403.18599* (2024). DOI: [10.48550/ARXIV.2403.18599](https://doi.org/10.48550/ARXIV.2403.18599). arXiv: [2403.18599](https://arxiv.org/abs/2403.18599). URL: <https://doi.org/10.48550/arXiv.2403.18599>.
- [96] Hoang Nguyen, Srđan Krstić, and David Basin. *vActionGUI: Model-driven Security and Privacy*. 2024. URL: <https://nuactiongui.github.io/>.
- [P5] Hoang Nguyen, Srđan Krstić, and David Basin. *Models Trump Code: An Empirical Analysis of GDPR Compliance*. 2026. DOI: [10.1145/3830454.3832656](https://doi.org/10.1145/3830454.3832656). URL: <https://doi.org/10.1145/3830454.3832656>.

Bibliography

- [97] Ariadi Nugroho and Michel R. V. Chaudron. “Evaluating the Impact of UML Modeling on Software Quality: An Industrial Case Study.” In: *12th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. Ed. by Andy Schürr and Bran Selic. Vol. 5795. LNCS. Springer, 2009, pp. 181–195. DOI: [10.1007/978-3-642-04425-0_14](https://doi.org/10.1007/978-3-642-04425-0_14). URL: https://doi.org/10.1007/978-3-642-04425-0_14.
- [98] OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard. 2013.
- [99] Object Management Group. *Unified Modeling Language Specification, Version 2.5*. Accessed: 2025-07-27. 2015. URL: <https://www.omg.org/spec/UML/2.5>.
- [100] OneTrust. *Cookie Consent*. <https://www.onetrust.com/products/cookie-consent/>. Accessed: 2026-05-07. 2026.
- [101] Open Worldwide Application Security Project. *OWASP Top 10*. 2025. URL: <https://owasp.org/Top10/2025/>.
- [102] Raúl Pardo and Daniel Le Métayer. “Analysis of Privacy Policies to Enhance Informed Consent.” In: *33rd Conference on Data and Applications Security and Privacy (DBSec)*. Ed. by Simon N. Foley. Vol. 11559. LNCS. Springer, 2019, pp. 177–198. DOI: [10.1007/978-3-030-22479-0_10](https://doi.org/10.1007/978-3-030-22479-0_10). URL: https://doi.org/10.1007/978-3-030-22479-0_10.
- [103] James Parker et al. “Build It, Break It, Fix It: Contesting Secure Development.” In: *ACM Trans. Priv. Secur.* 23.2 (2020), 10:1–10:36. DOI: [10.1145/3383773](https://doi.org/10.1145/3383773). URL: <https://doi.org/10.1145/3383773>.
- [104] Simon Parkinson and Saad Khan. “A Survey on Empirical Security Analysis of Access-control Systems: A Real-world Perspective.” In: *ACM Comput. Surv.* 55.6 (2023), 123:1–123:28. DOI: [10.1145/3533703](https://doi.org/10.1145/3533703). URL: <https://doi.org/10.1145/3533703>.
- [105] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC*. Enacted: 25 May 2018; Currently in force. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [S1] Karlo Piskor. *Secure Object-Relational Mappers*. ETH Zurich. Bachelor’s Thesis. 2023.
- [106] PostgreSQL. *PostgreSQL Documentation: Chapter 22. Database Roles*. Accessed: 2026-04-04. 2023. URL: <https://www.postgresql.org/docs/current/database-roles.html>.
- [107] PostSharp. *PostSharp Documentation*. 2023. URL: <https://doc.postsharp.net/>.
- [108] Alexander Pretschner, Manuel Hilty, and David Basin. “Distributed usage control.” In: *Commun. ACM* 49.9 (2006), pp. 39–44. DOI: [10.1145/1151030.1151053](https://doi.org/10.1145/1151030.1151053). URL: <https://doi.org/10.1145/1151030.1151053>.
- [109] Principal. *Flask Principal Authorization library*. 2023. URL: <https://pythonhosted.org/Flask-Principal/>.
- [110] Óscar Sánchez Ramón, Fernando Molina, Jesús García Molina, and José Ambrosio Toval Álvarez. “ModelSec: A Generative Architecture for Model-Driven Security.” In: *J. Univers. Comput. Sci.* 15.15 (2009), pp. 2957–2980. DOI: [10.3217/jucs-015-15-2957](https://doi.org/10.3217/jucs-015-15-2957). URL: <https://doi.org/10.3217/jucs-015-15-2957>.

- [111] Elissa M. Redmiles, Ziyun Zhu, Sean Kross, Dhruv Kuchhal, Tudor Dumitras, and Michelle L. Mazurek. "Asking for a Friend: Evaluating Response Biases in Security User Studies." In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. Canada: ACM, 2018, pp. 1238–1255. DOI: [10.1145/3243734.3243740](https://doi.org/10.1145/3243734.3243740). URL: <https://doi.org/10.1145/3243734.3243740>.
- [112] Mark Richters and Martin Gogolla. "OCL: Syntax, Semantics, and Tools." In: *Object Modeling with the OCL, The Rationale behind the Object Constraint Language*. Ed. by Tony Clark and Jos Warmer. Vol. 2263. LNCS. Springer, 2002, pp. 42–68. DOI: [10.1007/3-540-45669-4_4](https://doi.org/10.1007/3-540-45669-4_4). URL: https://doi.org/10.1007/3-540-45669-4_4.
- [113] Shariq Rizvi, Alberto Mendelzon, S. Sudarshan, and Prasan Roy. "Extending Query Rewriting Techniques for Fine-Grained Access Control." In: *ACM SIGMOD International Conference on Management of Data*. ACM, 2004, pp. 551–562. DOI: [10.1145/1007568.1007631](https://doi.org/10.1145/1007568.1007631).
- [114] Borislav Roussev. "Empirical Evidence Justifying the Adoption of a Model-Based Approach in the Course Web Applications Development." In: *J. Inf. Technol. Educ.* 2 (2003), pp. 73–90. DOI: [10.28945/314](https://doi.org/10.28945/314). URL: <https://doi.org/10.28945/314>.
- [115] Andrew Ruef, Michael Hicks, James Parker, Dave Levin, Michelle L. Mazurek, and Piotr Mardziel. "Build It, Break It, Fix It: Contesting Secure Development." In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. USA: ACM, 2016, pp. 690–703. DOI: [10.1145/2976749.2978382](https://doi.org/10.1145/2976749.2978382). URL: <https://doi.org/10.1145/2976749.2978382>.
- [116] Andrei Sabelfeld and Andrew C. Myers. "Language-based information-flow security." In: *IEEE J. Sel. Areas Commun.* 21.1 (2003), pp. 5–19. DOI: [10.1109/JSAC.2002.806121](https://doi.org/10.1109/JSAC.2002.806121). URL: <https://doi.org/10.1109/JSAC.2002.806121>.
- [117] Iflaah Salman, Ayse Tosun Misirli, and Natalia Juristo Juzgado. "Are Students Representatives of Professionals in Software Engineering Experiments?" In: *37th IEEE/ACM International Conference on Software Engineering, ICSE 2015, Florence, Italy, May 16-24, 2015, Volume 1*. Ed. by Antonia Bertolino, Gerardo Canfora, and Sebastian G. Elbaum. IEEE Computer Society, 2015, pp. 666–676. DOI: [10.1109/ICSE.2015.82](https://doi.org/10.1109/ICSE.2015.82). URL: <https://doi.org/10.1109/ICSE.2015.82>.
- [118] Jerome H. Saltzer and Michael D. Schroeder. "The protection of information in computer systems." In: *Proc. IEEE* 63.9 (1975), pp. 1278–1308. DOI: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939). URL: <https://doi.org/10.1109/PROC.1975.9939>.
- [119] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. "Role-Based Access Control Models." In: *IEEE Computer* 29.2 (1996), pp. 38–47. DOI: [10.1109/2.485845](https://doi.org/10.1109/2.485845).
- [120] Ravi S. Sandhu and Pierangela Samarati. "Access control: principles and practice." In: *IEEE Commun. Mag.* 32.9 (1994), pp. 40–48. DOI: [10.1109/35.312842](https://doi.org/10.1109/35.312842). URL: <https://doi.org/10.1109/35.312842>.

Bibliography

- [121] Saul Schleimer, Daniel Shawcross Wilkerson, and Alex Aiken. “Winnowing: Local Algorithms for Document Fingerprinting.” In: *ACM SIGMOD International Conference on Management of Data*. Ed. by Alon Y. Halevy, Zachary G. Ives, and AnHai Doan. ACM, 2003, pp. 76–85. DOI: [10.1145/872757.872770](https://doi.org/10.1145/872757.872770). URL: <https://doi.org/10.1145/872757.872770>.
- [122] Daniel Schoepe, Musard Balliu, Benjamin C. Pierce, and Andrei Sabelfeld. “Explicit Secrecy: A Policy for Taint Tracking.” In: *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*. IEEE, 2016, pp. 15–30. DOI: [10.1109/EUROSP.2016.14](https://doi.org/10.1109/EUROSP.2016.14). URL: <https://doi.org/10.1109/EuroSP.2016.14>.
- [123] Awanthika Senarath and Nalin A. G. Arachchilage. “Why Developers cannot embed Privacy into Software Systems? An empirical investigation.” In: *22nd International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2018, pp. 211–216. DOI: [10.1145/3210459.3210484](https://doi.org/10.1145/3210459.3210484). URL: <https://doi.org/10.1145/3210459.3210484>.
- [124] Mark Sidell, David Kavanagh, and Stephan Kemper. *MiniTwit: A tiny sample Flask app featuring a database and user accounts*. 2022. URL: <https://github.com/Viasat/minitwit>.
- [125] Ashutosh Kumar Singh, Nisarg Upadhyaya, Arka Seth, Xuehui Hu, Nishanth Sastry, and Mainack Mondal. “What Cookie Consent Notices Do Users Prefer: A Study In The Wild.” In: *European Symposium on Usable Security (EuroUSEC)*. ACM, 2022, pp. 28–39. URL: <https://doi.org/10.1145/3549015.3555675>.
- [126] SQLAlchemy. *Flask-SQLAlchemy Library*. 2023. URL: <https://flask-sqlalchemy.palletsprojects.com/en/2.x/>.
- [127] Stack Overflow. *Stack Overflow Developer Survey 2025*. 2025. URL: <https://survey.stackoverflow.co/2025/technology>.
- [128] Mikael Svahnberg, Aybüke Aurum, and Claes Wohlin. “Using Students as Subjects - An Empirical Evaluation.” In: *Proceedings of the Second International Symposium on Empirical Software Engineering and Measurement, ESEM 2008, October 9-10, 2008, Kaiserslautern, Germany*. Ed. by H. Dieter Rombach, Sebastian G. Elbaum, and Jürgen Münch. ACM, 2008, pp. 288–290. DOI: [10.1145/1414004.1414055](https://doi.org/10.1145/1414004.1414055). URL: <https://doi.org/10.1145/1414004.1414055>.
- [129] Pallets Team. *Jinja Documentation*. 2025. URL: <https://jinja.palletsprojects.com/>.
- [130] User. *Flask User Authentication Library*. 2023. URL: <https://flask-user.readthedocs.io>.
- [131] Usercentrics. *Cookiebot CMP by Usercentrics*. <https://www.cookiebot.com/>. Accessed: 2026-05-07. 2026.
- [132] usernamehw. *Error Lens*. Visual Studio Code Marketplace. 2024. URL: <https://marketplace.visualstudio.com/items?itemName=usernamehw.errorlens>.
- [133] Christine Utz, Martin Degeling, Sascha Fahl, Florian Schaub, and Thorsten Holz. “(Un)informed Consent: studying GDPR Consent Notices in the field.” In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Ed. by Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz. ACM, 2019, pp. 973–990. DOI: [10.1145/3319535.3354212](https://doi.org/10.1145/3319535.3354212). URL: <https://doi.org/10.1145/3319535.3354212>.

- [134] Vaadin. *Vaadin Web Framework*. 2023. URL: <https://vaadin.com/>.
- [135] Daniel Votipka, Kelsey R. Fulton, James Parker, Matthew Hou, Michelle L. Mazurek, and Michael Hicks. “Understanding Security Mistakes Developers Make: Qualitative Analysis from Build It, Break It, Fix It.” In: *29th USENIX Security Symposium*. USA: USENIX Association, 2020, pp. 109–126.
- [136] Frank Wang, Ronny Ko, and James Mickens. “Riverbed: Enforcing User-defined Privacy Constraints in Distributed Web Services.” In: *16th USENIX Symposium on Networked Systems Design and Implementation*. NSDI. Boston, MA: USENIX Association, Feb. 2019, pp. 615–630. ISBN: 978-1-931971-49-2. URL: <https://www.usenix.org/conference/nsdi19/presentation/wang-frank>.
- [137] Qihua Wang et al. “On the Correctness Criteria of Fine-Grained Access Control in Relational Databases.” In: *Proceedings of the 33rd International Conference on Very Large Data Bases, University of Vienna, Austria, September 23-27, 2007*. Ed. by Christoph Koch et al. ACM, 2007, pp. 555–566. URL: <http://www.vldb.org/conf/2007/papers/research/p555-wang.pdf>.
- [138] Jon Whittle, John Hutchinson, Mark Rouncefield, Håkan Burden, and Rogardt Heldal. “Industrial Adoption of Model-Driven Engineering: Are the Tools Really the Problem?” In: *16th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. Vol. 8107. LNCS. Springer, 2013, pp. 1–17. DOI: [10.1007/978-3-642-41533-3_1](https://doi.org/10.1007/978-3-642-41533-3_1).
- [139] WIRED. *Hack Brief: 885 Million Sensitive Financial Records Exposed Online*. 2019. URL: <https://www.wired.com/story/first-american-data-exposed/>.
- [140] Jean Yang, Travis Hance, Thomas H. Austin, Armando Solar-Lezama, Cormac Flanagan, and Stephen Chong. “Precise, dynamic information flow for database-backed applications.” In: *37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Ed. by Chandra Krintz and Emery D. Berger. ACM, 2016, pp. 631–647. DOI: [10.1145/2908080.2908098](https://doi.org/10.1145/2908080.2908098). URL: <https://doi.org/10.1145/2908080.2908098>.
- [141] Jean Yang, Kuat Yessenov, and Armando Solar-Lezama. “A language for automatically enforcing privacy policies.” In: *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*. Ed. by John Field and Michael Hicks. ACM, 2012, pp. 85–96. DOI: [10.1145/2103656.2103669](https://doi.org/10.1145/2103656.2103669). URL: <https://doi.org/10.1145/2103656.2103669>.
- [142] Jie Zhou, Qingguo Shen, and Yudong Xu. “Research and improvement of Ponder2 policy language.” In: *IEEE International Conference on Computer Science and Automation Engineering (CSAE)*. Vol. 2. IEEE, 2012, pp. 455–458.

Curriculum Vitae

Hoang Nguyen Phuoc Bao
Born on 14.06.1996 in Ho Chi Minh City, Vietnam
Citizen of Vietnam

Education

<i>Doctorate in Computer Science</i> ETH Zurich, Switzerland Supervisors: Prof. Dr. David Basin and Assoc. Prof. Dr. Srđan Krstić	07/2022–08/2026
<i>Master of Science in Computer Science</i> Universidad Autónoma de Madrid, Spain Universidad Complutense de Madrid, Spain Universidad Politécnica de Madrid, Spain Thesis: “Intelligent Enforcement of FGAC Policies for SQL Queries”	09/2020–09/2021
<i>Bachelor of Science in Computer Science</i> Vietnamese-German University, Vietnam Frankfurt University of Applied Sciences, Germany Thesis: “On the Critical Configurations of the Chip Firing Game”	09/2014–09/2018

Employment History

<i>Doctoral assistant</i> , ETH Zurich Research in Information Security and Privacy	07/2022–08/2026
<i>Research intern</i> , IMDEA Software Institute Automated Synthesis of Hardware-Software Contracts	02/2022–06/2022
<i>Research assistant</i> , DAAD	09/2021–01/2022
<i>Research assistant</i> , Vietnamese-German University	02/2019–08/2021
<i>Software engineer</i> , Axon Active, Vietnam	02/2018–01/2019

